Corso di Machine Learning (654AA) — Università di Pisa

Machine Learning

Note riorganizzate per lo studio

Fabio Piscitelli

Dal corso del Prof. Alessio Micheli, a.a. 2026/27

16 ottobre 2026

Indice

Guida alla lettura **v**

I Fondamenti **1**

1 Strumenti matematici **2**
1.1 Vettori, prodotto scalare e norme . . . 2
1.2 Derivate parziali e gradiente . . . 3
1.3 Probabilità . . . 4

2 Il problema dell'apprendimento **6**
2.1 Apprendere funzioni da esempi . . . 6
2.2 Gli ingredienti di un sistema di ML . . . 7
2.3 Funzioni di loss ed errore . . . 12
2.4 Generalizzazione . . . 14
2.5 Il ciclo di progettazione . . . 14

3 Spazio delle ipotesi e bias induttivo **16**
3.1 Un problema mal posto . . . 16
3.2 Imparare una funzione booleana . . . 16
3.3 Restringere lo spazio: le regole congiuntive . . . 17
3.4 L'unbiased learner . . . 17
3.5 Sistemi induttivi e deduttivi equivalenti . . . 18
3.6 Language bias o search bias? . . . 18

II Primi modelli **20**

4 Modelli lineari **21**
4.1 Regressione lineare univariata . . . 21
4.2 Input multidimensionali . . . 22
4.3 Soluzione diretta: le equazioni normali . . . 23
4.4 Soluzione iterativa: la discesa del gradiente . . . 24
4.5 Classificazione con modelli lineari . . . 26
4.6 Limiti: la separabilità lineare . . . 29
4.7 Espansione in basi lineare . . . 30

5 Overfitting e regolarizzazione **32**
5.1 Generalizzazione e overfitting . . . 32
5.2 Un caso di studio: il fitting polinomiale . . . 32
5.3 Regolarizzazione di Tikhonov . . . 35
5.4 Altre forme di regolarizzazione . . . 36

6 K-nearest neighbors **39**
6.1 Apprendimento eager e lazy . . . 39
6.2 L'algoritmo . . . 39
6.3 Il comportamento al variare di k . . . 41
6.4 K-NN contro modello lineare . . . 41
6.5 Il classificatore ottimo di Bayes . . . 42
6.6 Bias induttivo e limiti del K-NN . . . 43
6.7 Scelte di progetto . . . 44

III Generalizzazione: teoria e stima **46**

7 Statistical Learning Theory **47**
7.1 Rischio e rischio empirico . . . 47
7.2 La VC-dimension . . . 47
7.3 Il bound sul rischio . . . 50
7.4 Structural Risk Minimization . . . 51

8 Bias, varianza ed ensemble **53**
8.1 L'idea . . . 53
8.2 La decomposizione . . . 54
8.3 Bias, varianza e regolarizzazione . . . 55
8.4 Ensemble learning . . . 56
8.5 Feature selection (cenni) . . . 57

9 Validazione **59**
9.1 Perché validare . . . 59
9.2 Model selection e model assessment . . . 59
9.3 Un controesempio istruttivo . . . 60
9.4 K-fold cross-validation . . . 61
9.5 Ricerca degli iperparametri . . . 61
9.6 Schemi formali . . . 62
9.7 Errori frequenti e buone pratiche . . . 66
9.8 Misure d'errore . . . 67

IV Reti neurali **70**

10 Dal neurone al Perceptron **71**
10.1 Le reti neurali come strumento di ML . . . 71
10.2 Il neurone artificiale . . . 72
10.3 Reti di unità a soglia e funzioni booleane . . . 72
10.4 Apprendere con una singola unità . . . 73
10.5 Il teorema di convergenza del Perceptron . . . 75
10.6 Perceptron contro LMS . . . 76
10.7 Unità sigmoidali . . . 77

11 Multi-Layer Perceptron e backpropagation **81**
11.1 Il Multi-Layer Perceptron . . . 81
11.2 Perché le reti neurali sono flessibili . . . 82
11.3 Verso l'algoritmo di apprendimento . . . 84
11.4 La backpropagation . . . 84

12 Addestrare una rete neurale **89**
12.1 L'addestramento come percorso . . . 89
12.2 Inizializzazione e minimi multipli . . . 89
12.3 On-line, batch e mini-batch . . . 90
12.4 Il learning rate . . . 91
12.5 Criteri di arresto . . . 93
12.6 Overfitting e regolarizzazione . . . 93
12.7 Numero di unità nascoste . . . 95
12.8 Rappresentazione di input e output . . . 96
12.9 Validazione di una rete neurale . . . 97
12.10 Pregi e difetti delle reti neurali . . . 98
12.11 Verificare un'implementazione: il benchmark MONK . . . 99

V Support Vector Machines **101**

13 Support Vector Machines e metodi kernel **102**
13.1 SVM lineare: il margine massimo . . . 102
13.2 Soft margin: dati non separabili . . . 106
13.3 Kernel: SVM non lineari . . . 107
13.4 SVM per la regressione . . . 110
13.5 Le SVM nella pratica . . . 111
13.6 Metodi kernel . . . 112

VI Modelli avanzati **115**

14 Reti convoluzionali **116**
14.1 Il problema e l'idea . . . 116
14.2 La convoluzione . . . 116
14.3 Pooling . . . 118
14.4 La CNN nel complesso . . . 118
14.5 Risultati . . . 119

15 Deep learning **121**
15.1 Che cos'è il deep learning . . . 121
15.2 Servono davvero molti strati? . . . 122
15.3 Representation learning . . . 124
15.4 Rappresentazioni distribuite . . . 126
15.5 Altri risultati recenti . . . 127
15.6 Tecniche . . . 127

16 Reti neurali randomizzate **131**
16.1 La casualità nel Machine Learning . . . 131
16.2 Reti a pesi casuali . . . 131

17 Apprendimento non supervisionato: K-means e SOM **134**
17.1 Apprendimento non supervisionato . . . 134
17.2 Quantizzazione vettoriale . . . 134
17.3 K-means . . . 135
17.4 Self-Organizing Map . . . 137
17.5 Valutare un clustering . . . 139

18 Reti neurali ricorrenti **140**
18.1 Dati sequenziali e trasduzioni . . . 140
18.2 La memoria nelle reti neurali . . . 140
18.3 Apprendimento: l'unfolding . . . 142
18.4 Approcci collegati . . . 142

19 Apprendimento su dati strutturati e grafi **145**
19.1 Perché i grafi . . . 145
19.2 Deep Graph Networks . . . 146
19.3 Problemi aperti . . . 148
19.4 Kernel per strutture . . . 149

Guida alla lettura

Organizzazione

Le note seguono il programma del corso, ma gli argomenti sono disposti in modo che ogni concetto venga **definito una sola volta**, nel punto in cui serve per la prima volta, e poi soltanto richiamato.

- **Parte I — Fondamenti.** Strumenti matematici, formulazione del problema dell'apprendimento (dati, task, modello, algoritmo, loss) e ruolo del bias induttivo.
- **Parte II — Primi modelli.** Modelli lineari (con la discesa del gradiente), overfitting e regolarizzazione, K-nearest neighbors: i due estremi *rigido* e *flessibile*.
- **Parte III — Generalizzazione.** La teoria (VC-dimension, bound, SRM), la decomposizione bias-varianza con gli ensemble, e la stima empirica dell'errore (validazione).
- **Parte IV — Reti neurali.** Perceptron, MLP, backpropagation e addestramento in pratica.
- **Parte V — SVM.** Margine massimo, soft margin, kernel, regressione, metodi kernel.
- **Parte VI — Modelli avanzati.** CNN, deep learning, reti randomizzate, apprendimento non supervisionato, reti ricorrenti, grafi.

Ogni capitolo si apre con un breve riquadro che ne descrive il contenuto e si chiude con i *punti chiave* e alcune *domande di verifica* (tratte dalle domande d'esame indicate nel corso).

Convenzioni grafiche

Definizione — nome. Definizioni formali dei concetti.

Teorema — nome. *Enunciati di teoremi e risultati; la dimostrazione, quando prevista, segue subito dopo il riquadro.*

Algoritmo — nome.

Procedure passo per passo.

Idea chiave. Il concetto che bisogna aver capito di una sezione.

Attenzione. Errori frequenti e affermazioni da non fraintendere. È l'unico riquadro colorato.

Esempio. Esempi ed esercizi svolti.

Nel testo il **grassetto** segnala un termine definito per la prima volta, il *corsivo* i termini inglesi e le enfasi, il sottolineato le poche affermazioni da ricordare alla lettera. Le formule riquadrate sono quelle da saper scrivere a memoria.

Notazione

Simbolo	Significato
x, $\mathbf{x}$, X	scalare, vettore (colonna, in grassetto), matrice
$\mathbf{x}^T\mathbf{y}$	prodotto scalare; ^{T} indica la trasposta
l	numero di esempi del dataset (nel capitolo sulle SVM: N)
n	dimensione dell'input, cioè numero di feature (nel capitolo sulle SVM: m)
$\mathbf{x}_p$, $x_{p,i}$	p-esimo esempio (riga del dataset), sua componente i
d_p (o y_p, t_p)	target dell'esempio p
h, H	ipotesi, spazio delle ipotesi
f	funzione target (sconosciuta); nelle reti: funzione di attivazione
$\mathbf{w}$, w_0	pesi (parametri liberi), bias (indicato anche con b)
L, E	loss sul singolo esempio, errore (media o somma delle loss)
R, R_{emp}	rischio (errore vero), rischio empirico
η, λ, α	learning rate, coefficiente di regolarizzazione, momentum
TR, VL, TS	training set, validation set, test set

Parte I

Fondamenti

Capitolo 1

Strumenti matematici

Questo capitolo raccoglie gli strumenti usati in tutto il resto delle note: prodotto scalare e norme, derivate parziali e gradiente (la base della discesa del gradiente), regole di derivazione per funzioni composte, e i richiami di probabilità necessari (gaussiana, valore atteso, varianza).

1.1 Vettori, prodotto scalare e norme

Si usa la notazione: x scalare, $\mathbf{x}$ vettore (colonna), X matrice.

Definizione — Prodotto scalare. Dati $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, il **prodotto scalare** (*inner* o *dot product*) è

$$\mathbf{a} \cdot \mathbf{b} \;=\; a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \;=\; \sum_{i=1}^{n} a_i b_i \;=\; \|\mathbf{a}\|\,\|\mathbf{b}\| \cos\theta, \tag{1.1}$$

dove θ è l'angolo tra i due vettori. Notazioni equivalenti: $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T\mathbf{b} = \mathbf{a}^t\mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle$, e anche solo **ab** quando il contesto è chiaro.

La forma con il coseno dà la lettura geometrica: il prodotto scalare misura **quanto due vettori puntano nella stessa direzione**. È massimo quando sono paralleli e concordi ($\cos\theta = 1$), nullo quando sono ortogonali ($\cos\theta = 0$), negativo quando puntano in versi opposti. Nei modelli lineari, $\mathbf{w}^T\mathbf{x}$ misura l'*allineamento* dell'input $\mathbf{x}$ con il vettore dei pesi $\mathbf{w}$.

Definizione — Norme. La **norma euclidea** (modulo, lunghezza) di $\mathbf{x}$ è

$$\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T\mathbf{x}} = \sqrt{\textstyle\sum_i x_i^2} = d(\mathbf{x}, \mathbf{0}),$$

scritta spesso semplicemente $\|\mathbf{x}\|$ o $|\mathbf{x}|$. Altre norme utili sono la **norma** L^1, $\|\mathbf{x}\|_1 = \sum_i |x_i|$, e la **norma del massimo**, $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

Più in generale, un **prodotto interno** $\langle \cdot, \cdot \rangle$ su uno spazio vettoriale V è una forma bilineare simmetrica che associa a due vettori uno scalare, con le proprietà

$$\langle v, w \rangle = \langle w, v \rangle, \qquad \langle v + w, u \rangle = \langle v, u \rangle + \langle w, u \rangle, \qquad \langle kv, w \rangle = k\,\langle v, w \rangle.$$

Teorema — Disuguaglianza di Cauchy-Schwarz. *Per ogni $x, y \in V$ vale $|\langle x, y \rangle| \le \|x\| \cdot \|y\|$, cioè, elevando al quadrato, $\|x\|^2\|y\|^2 \ge \langle x, y \rangle^2$.*

Sarà usata nella dimostrazione del teorema di convergenza del Perceptron (§ 10.5). Un'altra identità che serve spesso è lo sviluppo del quadrato di una norma:

$$\|\mathbf{a} + \mathbf{b}\|^2 = (\mathbf{a} + \mathbf{b})^T(\mathbf{a} + \mathbf{b}) = \|\mathbf{a}\|^2 + 2\,\mathbf{a}^T\mathbf{b} + \|\mathbf{b}\|^2. \tag{1.2}$$

Definizione — Tensore. Nella visione semplificata usata nel ML, un **tensore** è un array multidimensionale di numeri disposti su una griglia regolare con un numero variabile di assi: uno

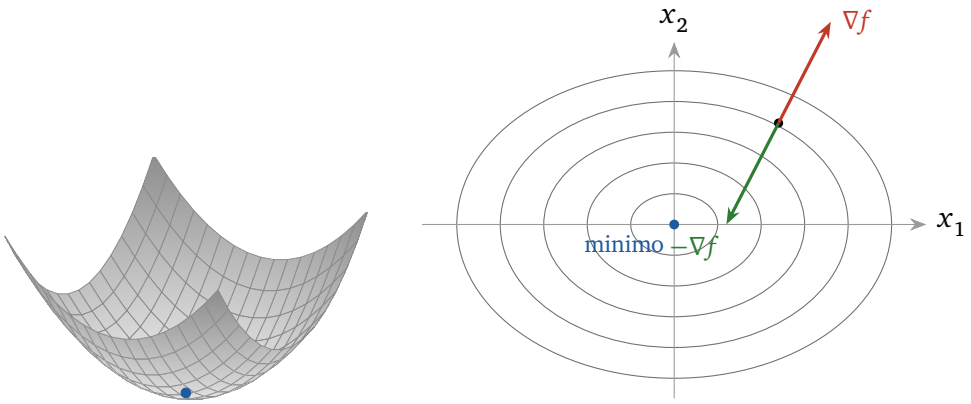


Figura 1.1. La funzione $f(x_1, x_2) = x_1^2 + 2x_2^2$. A sinistra la superficie, con il minimo (gradiente nullo); a destra le curve di livello: in un punto ∇f (rosso) indica la crescita più rapida, $-\nabla f$ (verde) la decrescita più rapida; entrambi sono perpendicolari alla curva di livello che passa per il punto.

scalare ha 0 assi, un vettore 1, una matrice 2, e ad esempio $X_{i,j,k}$ ha 3 indici. Nel ML il termine è soprattutto una comodità di notazione, utile per descrivere il calcolo su GPU; in matematica i tensori hanno proprietà più ricche, legate ai cambi di base.

1.2 Derivate parziali e gradiente

La **derivata parziale** generalizza la derivata alle funzioni di più variabili: è la derivata rispetto a una sola variabile, tenendo fisse tutte le altre. Per $f(x_1, x_2, x_3)$ si calcolano $\partial f/\partial x_1$, $\partial f/\partial x_2$, $\partial f/\partial x_3$.

Definizione — Gradiente. Il **gradiente** di $f : \mathbb{R}^n \to \mathbb{R}$ è il vettore delle sue derivate parziali:

$$\nabla f = \operatorname{grad} f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right) = \sum_{i=1}^{n} \frac{\partial f}{\partial x_i} \, \mathbf{e}_i, \tag{1.3}$$

dove gli $\mathbf{e}_i$ sono i versori ortogonali lungo le direzioni coordinate.

Significato geometrico. In un punto, il gradiente **punta nella direzione di massima pendenza** (massima crescita di f), e la sua norma dice **quanto è ripida** la pendenza. Il suo opposto $-\nabla f$ indica la direzione in cui f **decresce più rapidamente**: è l'idea alla base della *discesa del gradiente*, usata per addestrare quasi tutti i modelli del corso minimizzando una funzione di errore.

Il motivo si vede con il prodotto scalare. La variazione di f spostandosi di poco lungo una direzione unitaria $\mathbf{u}$ (la derivata direzionale) è

$$D_{\mathbf{u}} f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \, \|\mathbf{u}\| \cos\theta = \|\nabla f\| \cos\theta,$$

con θ angolo tra $\mathbf{u}$ e ∇f. Il valore è massimo per $\theta = 0$ (direzione di ∇f), vale $-\|\nabla f\|$, il minimo possibile, per $\theta = \pi$ (direzione di $-\nabla f$), ed è nullo per $\theta = \pi/2$: lungo le *curve di livello* f non cambia, quindi il gradiente è perpendicolare a esse (Figura 1.1).

Definizione — Punto stazionario. Un punto in cui il gradiente è nullo, $\nabla f = \mathbf{0}$, si dice **punto stazionario**. Può essere un minimo locale, un massimo locale o un **punto di sella**: il gradiente

nullo, da solo, non basta a dire di quale caso si tratta.

1.2.1 Regole di derivazione

Il calcolo dei gradienti delle funzioni di errore usa sempre le stesse regole:

$$\frac{\partial k}{\partial w} = 0 \ (k \text{ costante}), \qquad \frac{\partial w}{\partial w} = 1, \qquad \frac{\partial f(w)^2}{\partial w} = 2f(w)\frac{\partial f(w)}{\partial w}, \qquad \frac{\partial}{\partial w}\sum_i f_i = \sum_i \frac{\partial f_i}{\partial w}, \tag{1.4}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\cdot\frac{\partial g}{\partial x} \text{ (regola della catena)}, \qquad D[f(g(x))] = f'(g(x))\,g'(x), \qquad D[f\cdot g] = f'g + fg'. \tag{1.5}$$

La regola della catena è la più importante: **scompone** il calcolo della derivata di una funzione composta in fattori semplici, ciascuno calcolabile localmente. Su di essa si basa la backpropagation (§ 11.4).

1.3 Probabilità

Definizione — Valore atteso e varianza. Per una variabile aleatoria discreta Z che assume valori z_i con probabilità $P(z_i)$:

$$\bar{Z} = \mathbb{E}_P[Z] = \sum_i z_i\,P(z_i), \qquad \mathrm{Var}[Z] = \mathbb{E}\big[(Z-\bar{Z})^2\big].$$

Su un campione di l valori si usano la media $\bar{x} = \frac{1}{l}\sum_p x_p$, la varianza $\mathrm{Var}[x] = \frac{1}{l}\sum_p (x_p-\bar{x})^2$ e la covarianza $\mathrm{Cov}[x,y] = \frac{1}{l}\sum_p (x_p-\bar{x})(y_p-\bar{y})$.

Teorema — Lemma della varianza.

$$\mathrm{Var}[Z] = \mathbb{E}[Z^2] - \bar{Z}^2 \qquad\Longleftrightarrow\qquad \mathbb{E}[Z^2] = \bar{Z}^2 + \mathrm{Var}[Z].$$

Dimostrazione. Si sviluppa il quadrato e si usa $\sum_i P(z_i) = 1$:

$$\begin{aligned}\mathrm{Var}[Z] &= \sum_i (z_i-\bar{Z})^2 P(z_i) = \sum_i z_i^2 P(z_i) - 2\bar{Z}\sum_i z_i P(z_i) + \bar{Z}^2\sum_i P(z_i)\\ &= \mathbb{E}[Z^2] - 2\bar{Z}\cdot\bar{Z} + \bar{Z}^2 = \mathbb{E}[Z^2] - \bar{Z}^2.\end{aligned}$$

■

Il lemma è lo strumento chiave della decomposizione bias-varianza (§ 8.2). Il valore atteso è **lineare**: $\mathbb{E}[aX+bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$; se X e Y sono indipendenti vale anche $\mathbb{E}[XY] = \mathbb{E}[X]\,\mathbb{E}[Y]$. Un campione si dice **i.i.d.** (indipendente e identicamente distribuito) quando i suoi elementi sono estratti indipendentemente l'uno dall'altro dalla stessa distribuzione.

Definizione — Distribuzione normale. La **distribuzione normale** (gaussiana) con media μ e varianza σ^2 ha densità

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\,e^{-(x-\mu)^2/(2\sigma^2)}. \tag{1.6}$$

Con $\mu = 0$ e $\sigma^2 = 1$ si ottiene la **normale standard**.

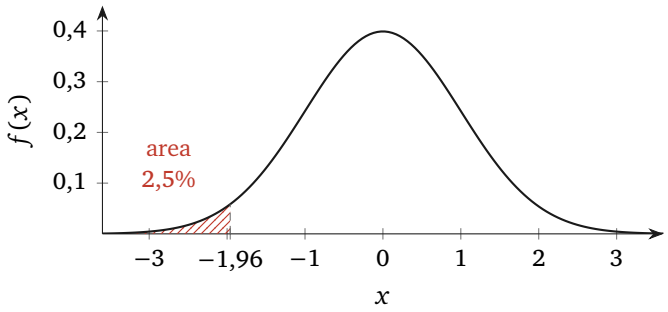


Figura 1.2. Densità della normale standard ($\mu = 0$, $\sigma = 1$): l'area a sinistra di $-1{,}96$ vale il 2,5%.

Punti chiave

- $\mathbf{a}^T\mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$: il prodotto scalare misura l'allineamento.
- ∇f indica la crescita più rapida, $-\nabla f$ la decrescita più rapida; $\nabla f = \mathbf{0}$ individua i punti stazionari (minimi, massimi o selle).
- La regola della catena scompone le derivate di funzioni composte in fattori locali.
- $\mathbb{E}[Z^2] = \bar{Z}^2 + \mathrm{Var}[Z]$.

Domande di verifica

1. Che cos'è il gradiente e perché $-\nabla f$ indica la direzione di massima decrescita?
2. Che cos'è un punto stazionario? È sempre un minimo?

Capitolo 2

Il problema dell'apprendimento

Questo capitolo formula il problema che tutto il corso affronta: inferire una funzione sconosciuta a partire da esempi, in modo che il risultato sia accurato su dati nuovi. Si descrivono gli ingredienti di un sistema di ML (dati, task, modello, algoritmo di apprendimento, validazione), le funzioni di loss associate ai diversi task e il ciclo di progettazione di un'applicazione.

2.1 Apprendere funzioni da esempi

> **Definizione — Machine Learning.** Il **Machine Learning** (ML, apprendimento automatico) studia e propone metodi per costruire (**inferire**) dipendenze, funzioni o ipotesi a partire da esempi di dati osservati, in modo che esse
>
> - si **adattino** agli esempi noti (*fit*);
> - siano in grado di **generalizzare**, cioè di dare risposte con accuratezza ragionevole su dati nuovi;
>
> secondo risultati **verificabili**, sotto condizioni e criteri statistici e computazionali, tenendo conto dell'**espressività** e della **complessità algoritmica** dei modelli e degli algoritmi.

Operativamente il sistema apprende dall'**esperienza** (un insieme di esempi di cui si conosce il risultato) per risolvere un compito computazionale, costruendo un **modello** (o **ipotesi**) da usare per fare predizioni. Il corso adotta il quadro dell'**apprendimento come approssimazione di una funzione sconosciuta a partire da esempi**. Il vantaggio è che task diversi (classificazione, regressione, …) vengono trattati in modo uniforme — matematicamente, nel quadro degli spazi di funzioni (spazi di Hilbert) — e quindi con una formulazione rigorosa.

2.1.1 Quando serve il ML

Il ML va usato con **opportunità** (quando serve davvero) e **consapevolezza** (dei suoi requisiti e limiti). È la scelta giusta quando:

- c'è **poca o nessuna conoscenza a priori**: non esiste una teoria del fenomeno, oppure la conoscenza è **troppo difficile da formalizzare** a mano (le persone riconoscono i volti o il parlato, ma non sanno descrivere *come* lo fanno), oppure la conoscenza umana **non basta** (prevedere la forza di legame di una molecola con una proteina);
- i dati sono **incerti, rumorosi o incompleti**, e questo ostacola una soluzione esatta;
- serve un **comportamento personalizzato** (filtrare email o pagine web secondo le preferenze di un utente);
- ed è relativamente **facile disporre di esperienza**, cioè di dati di cui si conosce la risposta corretta.

In cambio il ML richiede una **fonte di esperienza rappresentativa** per l'addestramento e una certa **tolleranza sulla precisione**: i risultati non sono mai garantiti esatti.

Esempio — Riconoscimento di cifre scritte a mano. Ogni cifra è un'immagine di 8×8 pixel. Scrivere a mano le regole che distinguono un "3" da un "8" per tutte le possibili calligrafie è di fatto impossibile, mentre è facile raccogliere molti esempi di cifre già etichettate. Si lascia allora che il sistema **apprenda un classificatore** f che associa a ogni immagine la classe corretta $0, 1, \ldots, 9$: un problema di classificazione con dieci classi.

In tutti i casi seguenti si vuole inferire una funzione generale a partire da un **training set** (TR) di coppie $\langle x, f(x) \rangle$ note. Gli ultimi due esempi mostrano che input e output possono essere anche **dati complessi**: stringhe, sequenze, grafi.

Tabella 2.1. Esempi di funzioni da apprendere.

Problema	Input x	Output $f(x)$
Riconoscimento della scrittura	dati del movimento della penna	lettera dell'alfabeto
Diagnosi medica	proprietà del paziente (sintomi, esami)	malattia o terapia consigliata
Riconoscimento facciale	bitmap del volto	nome della persona
Rilevamento di spam	messaggio email	spam / non spam
Protein folding	sequenza di amminoacidi (stringa di lunghezza variabile)	coordinate 3D degli atomi (sequenza di vettori)
Drug design	molecola (grafo di atomi e legami)	forza di legame con la proteasi dell'HIV (numero reale)

2.2 Gli ingredienti di un sistema di ML

Un sistema di ML predittivo si descrive attraverso cinque componenti, che sono anche una guida alle scelte di progetto (Figura 2.1):

- **dati**: le osservazioni del mondo, cioè l'esperienza disponibile;
- **task**: lo scopo dell'applicazione, cioè cosa si vuole ottenere;
- **modello**: l'ipotesi che viene costruita o migliorata apprendendo dai dati;
- **algoritmo di apprendimento**: guida la costruzione del modello regolandone i parametri sul problema;
- **validazione**: valuta la qualità del modello ottenuto.

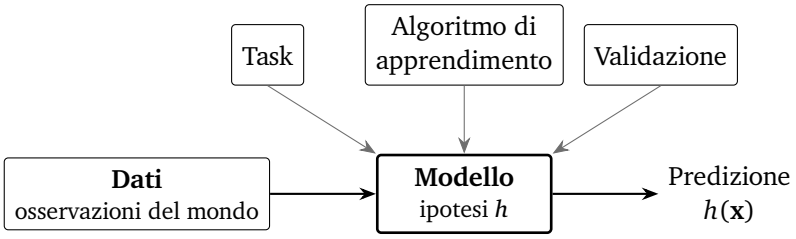


Figura 2.1. Gli ingredienti di un sistema di ML predittivo: il modello è costruito dai dati in base al task, all'algoritmo di apprendimento e alla validazione.

2.2.1 Dati

Il primo problema è la **rappresentazione**: come catturare la struttura degli oggetti analizzati. Nel caso **piatto** (*flat*, linguaggio attributo-valore) ogni oggetto è un **vettore di dimensione fissa** di proprietà (**feature**) e il dataset è una tabella. Gli attributi possono essere **categorici** o discreti (colore: giallo, rosso) o **continui** (peso, costo), e possono esserci **dati mancanti** (indicati con "?").

> **Definizione — Notazione sui dati.** I dati sono organizzati in una matrice X di dimensione $l \times n$: l righe, gli **esempi** ($p = 1, \dots, l$), e n colonne, le feature ($i = 1, \dots, n$).
>
> - $\mathbf{x}$ è un generico esempio (*pattern*, istanza, campione), cioè una riga della tabella; l è la **dimensione del dataset**, n la **dimensione dell'input**;
> - x_i (o x_j) è la componente i di un esempio $\mathbf{x}$ (omettendo l'indice dell'esempio);
> - $\mathbf{x}_p$ è il p-esimo esempio e $x_{p,i}$, scritto anche $(\mathbf{x}_p)_i$, la sua componente i;
> - il target dell'esempio p si indica con d_p (anche y_p o t_p).
>
> Quando il contesto è chiaro alcuni indici vengono omessi.

I dati possono essere **pre-elaborati**: scalatura delle variabili, codifica, selezione delle feature.

Codifica delle variabili categoriche. Per usare variabili categoriche in modelli numerici bisogna codificarle.

- Con **due classi** si usa $0/1$ oppure $-1/+1$.
- Con **più classi** si può usare $1, 2, 3, \dots$, ma questa codifica introduce una somiglianza artificiale (la classe 1 risulta "più vicina" alla 2 che alla 3). È adatta solo a variabili **categoriche ordinate** (piccolo, medio, grande).
- Per simboli senza ordine si usa la codifica **1-of-k** o **one-hot**: ogni simbolo diventa un vettore con un solo 1, ad esempio $A = (1, 0, 0)$, $B = (0, 1, 0)$, $C = (0, 0, 1)$. Tutti i simboli risultano equidistanti (a distanza $\sqrt{2}$). Si usa sia per l'input sia per l'output.

Dati strutturati. Molti dati non sono naturalmente vettori: **sequenze**, **alberi**, **grafi**, dati multi-relazionali (immagini, serie temporali, stringhe, DNA e proteine, molecole, la rete dei link tra pagine web; Figura 2.2). Il loro trattamento diretto è l'argomento dei capitoli 18 e 19.

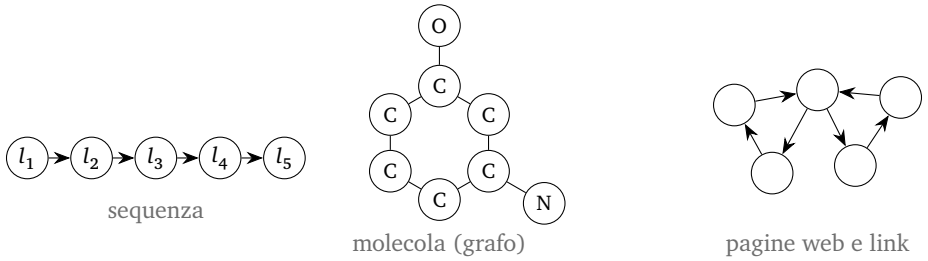


Figura 2.2. Dati strutturati: una sequenza, una molecola rappresentata come grafo, una rete di pagine web.

Rumore, outlier, feature selection.

- Il **rumore** è l'aggiunta di fattori esterni al segnale informativo, dovuta alla casualità delle misure e non alla legge sottostante (ad esempio rumore gaussiano).

- Gli **outlier** sono valori insoliti, incoerenti con la maggior parte delle osservazioni (ad esempio errori di misura anomali). Si gestiscono rilevandoli e rimuovendoli in pre-elaborazione, oppure usando metodi di modellazione **robusti**.
- La **feature selection** seleziona un piccolo numero di feature informative, per ottenere una rappresentazione dell'input adatta al problema (§ 8.5).

2.2.2 Task

Il task definisce quale conoscenza si vuole ottenere e quali informazioni sono disponibili. Si distinguono **task predittivi** (classificazione, regressione: approssimazione di funzioni, il fuoco del corso) e **task descrittivi** (cluster analysis, regole di associazione: trovare gruppi in dati non classificati).

> **Definizione — Apprendimento supervisionato. Dati**: esempi di training nella forma $\langle \text{input}, \text{output} \rangle = \langle \mathbf{x}, d \rangle$ (esempi **etichettati**) di una funzione sconosciuta f, nota solo nei punti degli esempi. Il **target** d (indicato anche con t o y) è fornito da un "insegnante" secondo $f(\mathbf{x})$.
> **Obiettivo**: trovare una buona approssimazione di f, cioè un'**ipotesi** h utilizzabile per predire su dati mai visti $\mathbf{x}'$: un'ipotesi che **generalizzi**.

Il target può essere

- **categorico** ⇒ **classificazione**: $f(\mathbf{x}) \in \{1, 2, \ldots, K\}$ (funzione a valori discreti);
- **numerico** ⇒ **regressione**: $f(\mathbf{x}) \in \mathbb{R}$ o $\mathbb{R}^K$ (funzione a valori reali).

Il formalismo dell'approssimazione di funzioni dà ai due casi una **visione unificata**. In terminologia statistica gli input sono le **variabili indipendenti**, gli output le **variabili dipendenti** o risposte.

> **Definizione — Apprendimento non supervisionato.** Non c'è insegnante: il training set contiene solo dati **non etichettati** $\langle \mathbf{x} \rangle$. Gli obiettivi tipici sono il **clustering** (partizionare i dati in sottoinsiemi di elementi "simili", Figura 2.3), la **riduzione di dimensionalità**, la visualizzazione o la pre-elaborazione, la **modellazione della densità** dei dati.

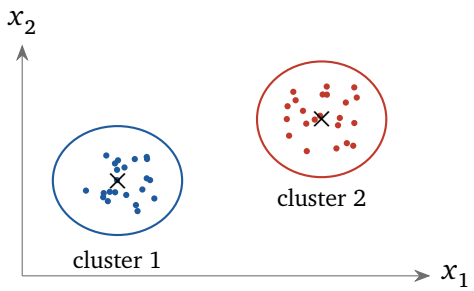


Figura 2.3. Clustering: i dati sono partizionati in gruppi, ciascuno rappresentato dal suo centroide (×).

Classificazione. I pattern sono visti come membri di una classe e l'obiettivo è assegnare a ciascuno la classe corretta. Con **due** classi $f(\mathbf{x})$ è una funzione booleana: si parla di **classificazione binaria** o **concept learning** (vero/falso, 0/1, −1/+1, negativo/positivo); con più classi il problema è **multiclasse**. Geometricamente, classificare significa **suddividere lo spazio degli input in regioni di**

decisione. L'esempio più semplice è il separatore lineare in $\mathbb{R}^2$ (Figura 2.4): una retta, in generale un iperpiano, che divide i punti delle due classi. L'iperpiano è l'insieme dei punti $\mathbf{x}$ con

$$\mathbf{w}^T\mathbf{x} + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0,$$

e il classificatore corrispondente è

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w}^T\mathbf{x} + w_0 \geq 0 \\ 0 & \text{altrimenti} \end{cases} \qquad \text{oppure} \qquad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + w_0).$$

Questo modello si chiama **Linear Threshold Unit** (LTU) ed è studiato nel capitolo 4. Funzioni a valori in $\{0, 1\}$ come questa si dicono **funzioni indicatrici**; l'insieme H di tutte le **dicotomie** (divisioni in due classi) indotte dagli iperpiani è lo spazio delle ipotesi della LTU.

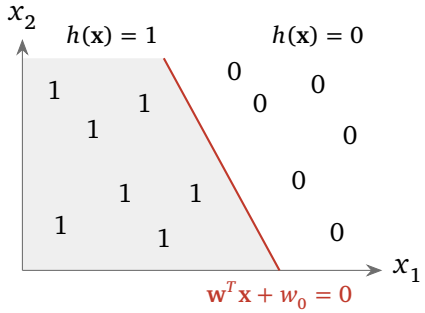


Figura 2.4. Un separatore lineare nel piano: la retta divide lo spazio degli input in due regioni di decisione.

Regressione. La **regressione** è la stima di una funzione a valori reali sulla base di un insieme finito di **campioni rumorosi**: si conoscono coppie $(\mathbf{x}, f(\mathbf{x}) + \text{rumore})$. È una forma di *curve fitting*.

> **Esempio — Quale funzione?.** Dati i punti (1; 2,1), (2; 3,9), (3; 6,1), (4; 8,4), (5; 9,8), viene spontaneo ipotizzare $f(x) = 2x$, che ha errori piccoli su tutti i punti. Ma come si è scelta questa funzione, e come lo farebbe una macchina? È esattamente ciò che l'algoritmo di apprendimento deve formalizzare (la soluzione sistematica è nel § 4.1).

Tra le infinite funzioni compatibili con un insieme di punti, quale scegliere (Figura 2.5)? Una curva che passa esattamente per tutti i punti può oscillare in modo innaturale; una retta commette piccoli errori ma è molto più semplice. Questo dilemma è il cuore del problema della **generalizzazione**.

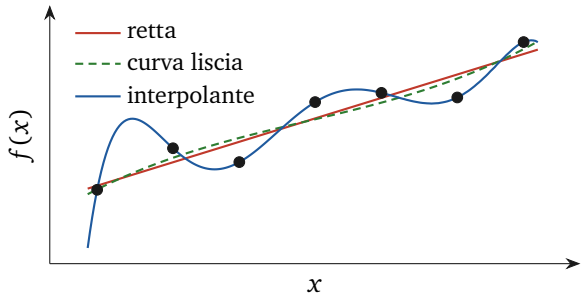


Figura 2.5. Tre ipotesi compatibili con gli stessi punti: la retta (rosso), una curva liscia (verde) e il polinomio di grado 6 che li interpola esattamente (blu).

Altri paradigmi.

- **Semi-supervised learning**: combina esempi etichettati e non etichettati.
- **Self-supervised learning**: apprendimento supervisionato in cui le etichette sono **generate automaticamente dalla struttura stessa dei dati**, senza etichettatura manuale. Il modello è **pre-addestrato** su grandi quantità di dati non etichettati, imparando rappresentazioni generali, poi adattate a task specifici con il **fine-tuning**. Esempi alla base dei grandi modelli linguistici: la predizione della **parola successiva** (apprendimento **autoregressivo** della famiglia GPT, *Generative Pre-trained Transformer*), la ricostruzione di **parole mascherate** (BERT) e, per le immagini, i **masked autoencoder**, che ricostruiscono porzioni nascoste dell'immagine.
- **Reinforcement learning**: apprendimento con un "critico" che dice giusto o sbagliato. L'algoritmo apprende una **politica** su come agire date le osservazioni del mondo; ogni azione ha un impatto sull'ambiente, che restituisce un feedback. Non ci sono esempi passo per passo: l'obiettivo è il *decision-making*.

2.2.3 Modello

Il modello cattura le relazioni tra i dati attraverso un "linguaggio" (numerico, simbolico, …) e soprattutto **definisce la classe di funzioni che la macchina può implementare**.

> **Definizione — Terminologia di base.**
>
> - **Esempio di training** (supervisionato): coppia $(\mathbf{x}, f(\mathbf{x}) + \text{rumore})$, con $\mathbf{x}$ vettore delle feature e $d = f(\mathbf{x}) + \text{rumore}$ target.
> - **Funzione target** f: la vera funzione, sconosciuta.
> - **Ipotesi** h: una funzione proposta, che si ritiene simile a f; un'espressione in un dato linguaggio che descrive le relazioni tra i dati.
> - **Spazio delle ipotesi** H: l'insieme di tutte le ipotesi che l'algoritmo di apprendimento può, in linea di principio, produrre; ad esempio l'insieme delle funzioni $h(\mathbf{x}, \mathbf{w})$ al variare del parametro $\mathbf{w}$.

Esempi di rappresentazioni diverse:

- **modelli lineari**: H è parametrizzato in modo **continuo** e ogni valore di $\mathbf{w}$ è un'ipotesi diversa, ad esempio $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + w_0)$ o $h_{\mathbf{w}}(x) = w_1 x + w_0$ (come $2x + 150$);
- **regole simboliche**: H è **discreto**, ad esempio "se $x_1 = 0$ e $x_2 = 1$ allora $h(\mathbf{x}) = 1$, altrimenti 0";
- **modelli probabilistici**: stimano $p(\mathbf{x}, y)$;
- **K-nearest neighbors**: (in regressione) predice la media dei target dei vicini più prossimi, un modello basato sulla memoria;
- **reti neurali**: modelli computazionali capaci di approssimare relazioni complesse e **non lineari** tra input e output; anch'esse definiscono una classe di funzioni.

Teorema — No Free Lunch. *Non esiste un metodo di apprendimento universalmente migliore (senza alcuna conoscenza, su tutti i problemi): se un algoritmo ottiene risultati superiori su alcuni problemi, deve pagarlo con risultati inferiori su altri.*

Per questo servono sia un **insieme di modelli** sia **strumenti critici per confrontarli**. I modelli però non sono tutti equivalenti: differiscono per **flessibilità** (capacità di approssimare funzioni arbitrarie, non solo lineari) e per gli strumenti di **controllo della complessità**.

Tabella 2.2. I principali paradigmi (linguaggi per H). Alcuni modelli si possono esprimere in più linguaggi.

Paradigma	Esempi
Simbolici / a regole (H discreto)	congiunzioni di letterali, alberi di decisione, grammatiche induttive, algoritmi evolutivi, Inductive Logic Programming
Sub-simbolici (H continuo)	analisi discriminante lineare, regressione lineare multipla, LTU, reti neurali, metodi kernel (SVM)
Probabilistici / generativi	modelli parametrici tradizionali, reti bayesiane, Naïve Bayes, modelli di Markov, HMM
Instance-based	nearest neighbor

Il filo conduttore del corso. Modelli **flessibili** e principi per **controllarne la complessità**: dal compromesso tra i due dipende la capacità di **generalizzazione**. Lo stesso compromesso si ritrova, in forme diverse, in tutti i capitoli.

2.2.4 Algoritmo di apprendimento

Sulla base di dati, task e modello, l'algoritmo di apprendimento esegue una **ricerca (euristica) nello spazio delle ipotesi** H della migliore approssimazione della funzione target: tipicamente l'ipotesi con **errore minimo** sui dati. In pratica adatta i **parametri liberi** del modello al task (il miglior $\mathbf{w}$ nei modelli lineari, le migliori regole nei modelli simbolici, …). Nell'esempio di regressione, la scelta $w_1 = 2$, $w_0 = 0$ è il risultato di questa ricerca (Figura 2.6).

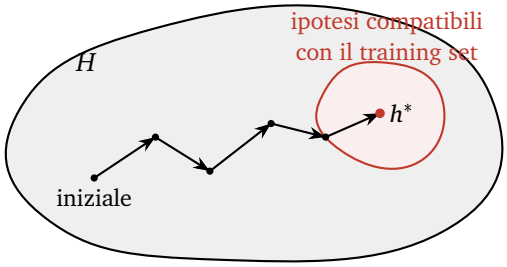


Figura 2.6. L'apprendimento come ricerca nello spazio delle ipotesi: ogni punto è una funzione diversa; si parte da un'ipotesi iniziale e si procede, tipicamente con una ricerca locale, verso quella a errore minimo.

A seconda della disciplina, "apprendimento" prende nomi diversi: **inferenza** (statistica), **abduzione/induzione** (logica), **adattamento** (biologia, sistemi), **ottimizzazione** (matematica), **addestramento** o *training* (reti neurali), **approssimazione di funzioni**, regressione, *curve fitting*. Poiché H può non contenere tutte le funzioni possibili e la ricerca non può essere esaustiva, servono delle **assunzioni**: è il ruolo del bias induttivo (capitolo 3).

2.3 Funzioni di loss ed errore

Per dire che un'approssimazione è "buona" serve una misura. Il modello produce $h(\mathbf{x})$ e si misura la "distanza" tra $h(\mathbf{x})$ e il target d con una **funzione di loss** $L\big(h_{\mathbf{w}}(\mathbf{x}), d\big)$, calcolata sul singolo esempio: un valore alto indica un'approssimazione scarsa.

Definizione — Errore, rischio, loss. L'**errore** è un valore atteso della loss, ad esempio la sua media sugli l esempi:

$$E(\mathbf{w}) = \frac{1}{l}\sum_{p=1}^{l} L\big(h_{\mathbf{w}}(\mathbf{x}_p), d_p\big). \tag{2.1}$$

Serve come **funzione obiettivo** da minimizzare durante il training e come misura dell'errore in fase di test. Per ora errore, rischio e loss sono sinonimi; dal capitolo 5 *loss* indicherà la funzione obiettivo complessiva (errore più eventuale penalità) ed *errore* solo il termine sui dati.

La loss dipende dal task. I quattro casi del corso sono i seguenti.

Regressione: errore quadratico. Il target è $d_p = f(\mathbf{x}_p) + e$ (funzione reale più errore casuale), H è un insieme di funzioni a valori reali e la loss è l'**errore quadratico**

$$L\big(h_{\mathbf{w}}(\mathbf{x}_p), d_p\big) = \big(d_p - h_{\mathbf{w}}(\mathbf{x}_p)\big)^2.$$

La sua media sul dataset è il **Mean Squared Error** (MSE), cioè la media dei quadrati delle distanze verticali tra i dati e il modello (Figura 2.7):

$$E(\mathbf{w}) = \frac{1}{l}\sum_{p=1}^{l} \big(d_p - h_{\mathbf{w}}(\mathbf{x}_p)\big)^2.$$

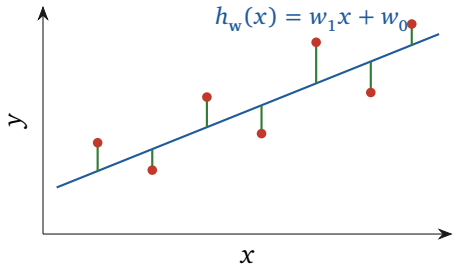


Figura 2.7. L'MSE è la media dei quadrati dei segmenti verdi: le distanze verticali tra i dati (rosso) e la retta $h_{\mathbf{w}}(x)$ (blu).

Classificazione: loss 0/1. Gli output sono ad esempio $\{0, 1\}$, H è un insieme di funzioni indicatrici e la loss è

$$L\big(h_{\mathbf{w}}(\mathbf{x}_p), d_p\big) = \begin{cases} 0 & \text{se } h_{\mathbf{w}}(\mathbf{x}_p) = d_p \\ 1 & \text{altrimenti.} \end{cases}$$

La sua media è la **percentuale di esempi classificati male**: con 20 errori su 100 esempi si ha un errore del 20%, cioè un'**accuratezza** dell'80%.

Clustering e quantizzazione vettoriale. L'obiettivo è partizionare in modo ottimo una distribuzione sconosciuta in regioni (cluster), ciascuna approssimata da un **centro** o **prototipo**. H è un insieme di *vector quantizer* $\mathbf{x} \mapsto c(\mathbf{x})$, che mappano lo spazio continuo in uno discreto, e la loss è la **distorsione quadratica**, la distanza al quadrato del pattern dal centroide del suo cluster:

$$L\big(h(\mathbf{x}_p)\big) = \big(\mathbf{x}_p - h(\mathbf{x}_p)\big)^T\big(\mathbf{x}_p - h(\mathbf{x}_p)\big) = \|\mathbf{x}_p - h(\mathbf{x}_p)\|^2.$$

Il caso è sviluppato nel capitolo 17.

Stima di densità. L'obiettivo è stimare una densità all'interno di una classe assunta (metodi generativi, parametrici), ad esempio una normale $p(\mathbf{x} \mid \mu, \sigma^2)$ con μ e σ^2 parametri sconosciuti. La loss è $L\big(h(\mathbf{x}_p)\big) = -\ln h(\mathbf{x}_p)$: minimizzare la somma dei $-\ln$ equivale a **massimizzare la (log-)verosimiglianza**, cioè il prodotto delle probabilità dei dati osservati, perché $-\sum_p \ln h(\mathbf{x}_p) = -\ln \prod_p h(\mathbf{x}_p)$ e il logaritmo è crescente.

2.4 Generalizzazione

Questo è il **concetto fondamentale del corso**. L'apprendimento è la ricerca di una buona funzione in uno spazio di funzioni a partire da dati noti, tipicamente minimizzando un errore; ma "buona" va inteso rispetto all'**errore di generalizzazione**, cioè a quanto accuratamente il modello predice su **dati nuovi**. Si distinguono due fasi:

1. **fase di apprendimento** (*training*, *fitting*): si costruisce il modello dai dati noti (i dati di training più il bias);
2. **fase predittiva o di test** (*deployment*, inferenza): si applica il modello a nuovi esempi $\mathbf{x}'$ e si confronta la risposta con il target d', che il modello **non ha mai visto**.

> **Prestazione nel ML.** Nel ML "prestazione" significa **accuratezza di generalizzazione**, stimata dall'errore su un **test set** tenuto da parte (*hold out*). È facile *usare* strumenti di ML; è molto più difficile usarli *bene*: la differenza sta tutta nella generalizzazione. La teoria (capitoli 7 e 8) ne studia le condizioni, la validazione (capitolo 9) la stima e la gestisce.

2.5 Il ciclo di progettazione

Un'applicazione di ML si sviluppa con un processo iterativo, guidato dalla conoscenza a priori del dominio (Figura 2.8).

1. **Raccolta dei dati**: selezione, integrazione, pulizia. Serve un insieme di esempi abbastanza **grande e rappresentativo** per training e test.
2. **Rappresentazione dei dati**: dipende dal dominio e sfrutta la conoscenza dell'esperto; comprende feature selection, rilevamento degli outlier, scalatura delle variabili, gestione dei dati mancanti. Spesso è la fase più critica per il successo complessivo.
3. **Scelta del modello**: formulazione del problema e delle ipotesi; bisogna conoscere i **limiti di applicabilità** del modello e controllarne la complessità.
4. **Costruzione del modello** (il cuore del ML): l'algoritmo di apprendimento sui dati di training.
5. **Valutazione**: la prestazione è l'accuratezza predittiva; si aggiungono l'interpretazione dei risultati e l'estrazione di conoscenza.
6. **Deployment**.

2.5.1 Errori di interpretazione

Per ogni modello statistico, ML compreso:

- si assume (spesso) una relazione tra input e output e serve un dataset **rappresentativo** del fenomeno: il ML non funziona con variabili scorrelate o fenomeni casuali (lotterie), e input non informativi portano a modelli scadenti;

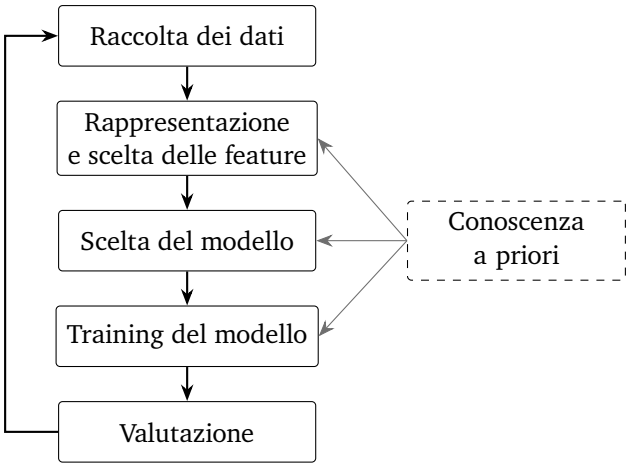


Figura 2.8. Il ciclo di progettazione: i risultati della valutazione possono richiedere di tornare alle fasi precedenti; la conoscenza a priori guida le fasi centrali.

- **la causalità non si può inferire dalla sola analisi dei dati**: una dipendenza statistica può avere ragioni esterne ai dati (in Florida l'età media è più alta che negli altri stati USA, ma non è il clima a far vivere più a lungo: ci si trasferisce lì da pensionati).

Specifico del ML: **modelli potenti aumentano il rischio**, perché si adattano anche a dati privi di significato, e risultati **non validati bene** portano a predizioni e interpretazioni fuorvianti.

Punti chiave

- Apprendere = inferire un'ipotesi $h \in H$ che approssimi la funzione sconosciuta f e **generalizzi**.
- Ingredienti: dati (rappresentazione, codifica one-hot, rumore), task (supervisionato: classificazione e regressione; non supervisionato), modello (H), algoritmo (ricerca in H), validazione.
- Loss: quadratica (regressione), 0/1 (classificazione), distorsione (clustering), $-\ln$ (densità).
- Conta l'errore su dati nuovi, non quello sui dati di training.

Domande di verifica

1. In quali condizioni è appropriato usare il ML? Che cosa richiede in cambio?
2. Differenza tra classificazione e regressione; tra apprendimento supervisionato e non supervisionato.
3. Perché la codifica $1, 2, 3, \ldots$ di una variabile categorica può essere inadatta? Che cos'è la codifica one-hot?
4. Quali loss si usano per regressione, classificazione, clustering e stima di densità?
5. Cosa si intende per generalizzazione e come si stima?

Capitolo 3

Spazio delle ipotesi e bias induttivo

Inferire una funzione da un numero finito di esempi è un problema mal posto: senza assunzioni sulla funzione cercata, gli esempi non dicono nulla sui casi non visti. Il capitolo lo mostra in uno spazio di ipotesi discreto (funzioni booleane), dimostra che un learner senza bias non può generalizzare e distingue i due modi di introdurre un bias: vincolare il linguaggio delle ipotesi o vincolare la strategia di ricerca.

3.1 Un problema mal posto

Inferire funzioni generali da dati noti è un **problema (inverso) mal posto**: la soluzione può violare **esistenza**, **unicità** o **stabilità**. In linea di principio la soluzione non è unica, e con dati finiti non ci si può aspettare di trovare quella esatta. Per questo si lavora con uno spazio delle ipotesi **ristretto**, e lo studio di ogni modello ruota attorno a due domande:

- **cosa si può rappresentare?** Una funzione non rappresentabile in H non può nemmeno essere appresa;
- **cosa si può apprendere?** Domanda che viene dopo la prima.

Per costruire un modello e un algoritmo bisogna fare **assunzioni** sulla natura della funzione target.

> **Definizione — Bias induttivo.** Il **bias induttivo** è l'insieme delle assunzioni fatte sulla funzione target per poter generalizzare. Può riguardare
> - i **vincoli sul modello**, cioè su quali ipotesi H può esprimere: **language bias**;
> - i **vincoli o le preferenze dell'algoritmo**, cioè la strategia di ricerca in H: **search bias**;
> - entrambi.

Il ruolo del bias si vede con chiarezza imparando un **concetto**, cioè una funzione booleana (ad esempio $h_{gatto}(\mathbf{x}) = 1$ se l'animale descritto da $\mathbf{x}$ è un gatto, 0 altrimenti).

3.2 Imparare una funzione booleana

Si vuole imparare una funzione booleana sconosciuta $y = f(x_1, x_2, x_3, x_4)$ dai 7 esempi della Tabella 3.1.

Quante sono le ipotesi possibili. Con 4 input binari esistono $2^4 = 16$ istanze di input distinte. Una funzione booleana assegna a ciascuna un output in $\{0, 1\}$, quindi le funzioni possibili sono $2^{16} = 65\,536$. Sapere quale sia quella corretta richiede di vedere **tutte** le coppie input-output: dopo 7 esempi le 9 istanze non viste possono avere output qualsiasi, e restano $2^9 = 512$ funzioni compatibili con i dati. In generale, con input e output binari e dimensione dell'input n:

$$|H| = 2^{\#\text{istanze di input}} = 2^{2^n}. \tag{3.1}$$

Un modello che usa tutto questo spazio è una **lookup table**: un *rote learner* che memorizza gli esempi e classifica $\mathbf{x}$ solo se coincide con un esempio già visto (altrimenti non risponde). Nessun bias induttivo, nessuna generalizzazione.

Tabella 3.1. Sette esempi di una funzione booleana sconosciuta di quattro variabili.

Esempio	x_1	x_2	x_3	x_4	y
1	0	0	1	0	0
2	0	1	0	0	0
3	0	0	1	1	1
4	1	0	0	1	1
5	0	1	1	0	0
6	1	1	0	0	0
7	0	1	0	1	0

3.3 Restringere lo spazio: le regole congiuntive

Si introduce un **language bias**: si considerano solo **regole congiuntive**, cioè congiunzioni (AND) di letterali, come $h_1 = l_2$, $h_2 = l_1 \wedge l_2$, $h_3 = \mathit{true}$, $h_4 = \neg l_1 \wedge l_2$. Ad esempio h_1 significa "se $x_2 = 1$ allora $h(\mathbf{x}) = 1$, altrimenti 0".

Numero di regole congiuntive. Con n variabili binarie le ipotesi congiuntive semanticamente distinte sono $3^n + 1$.

Dimostrazione. In una congiunzione, per ciascuna delle n variabili ci sono tre possibilità: compare come l_i, compare come $\neg l_i$, oppure non compare ("non importa"). Le scelte sono indipendenti, quindi si ottengono 3^n congiunzioni. Tutte le congiunzioni che contengono una coppia $l_i \wedge \neg l_i$ valgono sempre 0: sono equivalenti tra loro e alla funzione *false*, che va aggiunta come una sola ipotesi in più. Totale $3^n + 1$. ■

Con $n = 4$ si passa da 65 536 a $3^4 + 1 = 82$ ipotesi.

> **Definizione — Consistenza e version space.** Un'ipotesi h è **consistente** con il training set TR se $h(\mathbf{x}) = d(\mathbf{x})$ per ogni esempio $\langle \mathbf{x}, d(\mathbf{x}) \rangle \in TR$. Il **version space** $VS_{H,TR}$ è il sottoinsieme delle ipotesi di H consistenti con tutti gli esempi di training.

In questo spazio ridotto esistono algoritmi efficienti che trovano il version space senza enumerare tutte le ipotesi. Ma l'assunzione congiuntiva è **troppo restrittiva**: se il concetto target non è in H non può essere rappresentato. "Se $x_1 = 1$ *oppure* $x_2 = 1$ allora 1" non è una congiunzione.

3.4 L'unbiased learner

L'idea opposta è scegliere un H capace di esprimere **ogni concetto insegnabile**: l'insieme di tutti i sottoinsiemi dello spazio degli input X, cioè l'insieme delle parti $\mathscr{P}(X)$ (disgiunzioni, congiunzioni, negazioni). Con $n = 10$ input binari, $|X| = 2^{10} = 1024$ e $|\mathscr{P}(X)| = 2^{1024} \approx 10^{308}$ concetti. H contiene sicuramente il concetto target. Che cosa succede alla generalizzazione?

Teorema — Un learner senza bias non può generalizzare. *Con un unbiased learner le uniche istanze classificate in modo non ambiguo dal version space sono gli esempi di training stessi: si ottiene di nuovo una lookup table.*

Dimostrazione. Sia x_i un'istanza non presente nel training set e sia $h \in VS$.

1. Si consideri la funzione h' identica a h ovunque tranne che in x_i, dove $h'(x_i) \neq h(x_i)$.
2. Poiché H contiene *tutte* le funzioni su X, anche $h' \in H$.
3. h e h' coincidono su tutti gli esempi di training (x_i non è tra questi), quindi anche h' è consistente: $h' \in VS$.
4. La corrispondenza $h \mapsto h'$ accoppia ogni ipotesi del VS che classifica x_i come positivo con una che lo classifica negativo, e viceversa.

Quindi ogni istanza non vista è classificata 1 da esattamente metà delle ipotesi del VS e 0 dall'altra metà: il version space non può dare alcuna risposta su x_i. ■

Esempio — Un solo input. Con un input binario x, le funzioni possibili sono $H = \{x,\ \neg x,\ 0,\ 1\}$. Il TR contiene solo $x = 0 \mapsto d = 0$. Il version space è $\{x, 0\}$: entrambe valgono 0 in $x = 0$. Sull'istanza di test $x = 1$, però, x risponde 1 e 0 risponde 0. Non c'è modo di decidere, a meno di usare tutto X come training set.

Futilità dell'apprendimento senza bias. Un learner che non fa **alcuna assunzione a priori** sull'identità del concetto target **non ha alcuna base razionale** per classificare istanze non viste: dovrebbe vedere come esempio ogni singola istanza di X. Il bias, di restrizione o di preferenza, non serve solo all'efficienza: è necessario per la generalizzazione. Il bias però non dice ancora *quale* soluzione generalizza meglio.

3.5 Sistemi induttivi e deduttivi equivalenti

Un sistema induttivo (un algoritmo di apprendimento che, usando lo spazio H, riceve gli esempi e una nuova istanza e produce una classificazione o "non so") è **equivalente a un sistema deduttivo**, cioè a un dimostratore di teoremi che riceve gli stessi esempi e la stessa istanza **più il bias induttivo come assioma aggiuntivo** (Figura 3.1). Il bias induttivo è quindi esattamente l'insieme di assunzioni che rende *logicamente deducibile* la classificazione prodotta dal learner.

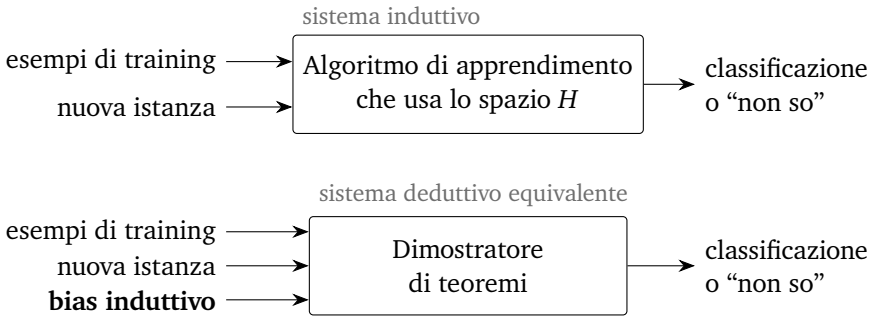


Figura 3.1. Il bias induttivo rende un sistema induttivo equivalente a un sistema deduttivo.

3.6 Language bias o search bias?

Nel ML moderno si preferisce in genere il **search bias**. Si usano modelli flessibili, con spazi di ipotesi molto espressivi e capacità di approssimazione universale (reti neurali, alberi di decisione), evitando il language bias e quindi **senza escludere a priori** la funzione target. Il bias induttivo resta, ma è affidato alla **strategia di ricerca**, cioè a una ricerca **incompleta** che privilegia certe soluzioni rispetto ad altre.

Punti chiave

- Con n input binari le funzioni booleane sono 2^{2^n}; le congiunzioni di letterali $3^n + 1$.
- Il version space contiene le ipotesi consistenti con tutti gli esempi.
- Senza bias induttivo il version space non classifica nessuna istanza nuova: lookup table.
- Language bias = vincoli su H; search bias = preferenze della ricerca. Il ML moderno usa H molto espressivi e search bias. **Ogni approccio di ML ha un bias induttivo**; il problema è caratterizzarlo per ciascun modello.

Domande di verifica

1. Che cos'è il bias induttivo? Differenza tra language bias e search bias.
2. Dimostrare che un *unbiased learner* non può generalizzare.
3. Quante sono le ipotesi con n input binari senza vincoli? E con sole regole congiuntive?
4. Che cos'è il version space?

Parte II

Primi modelli

Capitolo 4

Modelli lineari

Il modello lineare è il primo modello con uno spazio delle ipotesi continuo*, parametrizzato da numeri reali. Il capitolo ricava i parametri ottimi ai minimi quadrati in due modi — in forma chiusa (equazioni normali) e iterativamente (discesa del gradiente,* delta rule*) — prima per la regressione, poi per la classificazione. Si chiude con i limiti del modello (separabilità lineare, XOR) e con la sua estensione tramite espansione in basi.*

4.1 Regressione lineare univariata

Nel caso univariato (una variabile di input x, una di output y) si assume il modello

$$h_{\mathbf{w}}(x) = w_1 x + w_0,$$

dove $w_0, w_1 \in \mathbb{R}$ sono i **pesi** (*weights*), cioè i parametri liberi: si adatta ai dati una retta. Lo spazio delle ipotesi è infinito, perché $\mathbf{w}$ varia con continuità.

4.1.1 Il criterio dei minimi quadrati

Addestrare significa trovare il $\mathbf{w}$ che **minimizza l'errore empirico** sui dati di training (il controllo della complessità è l'argomento del capitolo 5). Dati l esempi (x_p, y_p), si cerca la retta che minimizza la **somma dei quadrati dei residui**:

$$E(\mathbf{w}) = \sum_{p=1}^{l} \big(y_p - h_{\mathbf{w}}(x_p)\big)^2 = \sum_{p=1}^{l} \big(y_p - (w_1 x_p + w_0)\big)^2. \tag{4.1}$$

Dividendo per l si ottiene la media. È il metodo dei **minimi quadrati** (*Least (Mean) Squares*, LMS): $\arg\min_{\mathbf{w}} E(\mathbf{w})$ in norma L^2.

> **Perché i minimi quadrati.** Ogni retta candidata produce dei **residui**, le distanze verticali tra i punti (x_p, y_p) e la retta (Figura 2.7). Minimizzare la somma dei loro quadrati è un modo naturale di trovare la retta che meglio approssima i dati. In generale, i minimi quadrati sono il metodo standard per risolvere in modo approssimato i **sistemi sovradeterminati**, con più equazioni (gli esempi) che incognite (i parametri).

4.1.2 Soluzione in forma chiusa

Un minimo è un punto stazionario, dove il gradiente è nullo. Poiché E è **convessa** nei parametri (un paraboloide), non ci sono minimi locali spuri e basta imporre $\partial E/\partial w_0 = 0$ e $\partial E/\partial w_1 = 0$.

Teorema — Retta ai minimi quadrati. *Il minimo di* (4.1) *si ha per*

$$w_1 = \frac{\sum_p x_p y_p - \frac{1}{l}\sum_p x_p \sum_p y_p}{\sum_p x_p^2 - \frac{1}{l}\big(\sum_p x_p\big)^2} = \frac{\mathrm{Cov}[x, y]}{\mathrm{Var}[x]}, \qquad w_0 = \bar{y} - w_1 \bar{x}, \tag{4.2}$$

con $\bar{x} = \frac{1}{l}\sum_p x_p$ *e* $\bar{y} = \frac{1}{l}\sum_p y_p$.

Dimostrazione. Si deriva ogni termine con la regola $\partial f^2/\partial w = 2f\,\partial f/\partial w$ (§ 1.2.1).

1. **Derivata rispetto a** w_0: $\dfrac{\partial E}{\partial w_0} = -2\sum_p \big(y_p - w_1 x_p - w_0\big) = 0 \iff \sum_p y_p - w_1\sum_p x_p - l\,w_0 = 0.$ Dividendo per l: $\boxed{w_0 = \bar{y} - w_1\bar{x}}$.
2. **Derivata rispetto a** w_1: $\dfrac{\partial E}{\partial w_1} = -2\sum_p \big(y_p - w_1 x_p - w_0\big)x_p = 0 \iff \sum_p x_p y_p - w_1\sum_p x_p^2 - w_0\sum_p x_p = 0.$
3. **Sostituzione** di w_0 dal passo 1: $\sum_p x_p y_p - w_1\sum_p x_p^2 - (\bar{y} - w_1\bar{x})\sum_p x_p = 0$, cioè
$$\sum_p x_p y_p - \bar{y}\sum_p x_p = w_1\Big(\sum_p x_p^2 - \bar{x}\sum_p x_p\Big).$$
4. Poiché $\bar{y}\sum_p x_p = \frac{1}{l}\sum_p x_p \sum_p y_p$ e $\bar{x}\sum_p x_p = \frac{1}{l}(\sum_p x_p)^2$, si ottiene la prima forma di w_1 in (4.2).
5. Dividendo numeratore e denominatore per l: il numeratore diventa $\frac{1}{l}\sum_p x_p y_p - \bar{x}\bar{y} = \mathrm{Cov}[x,y]$ e il denominatore $\frac{1}{l}\sum_p x_p^2 - \bar{x}^2 = \mathrm{Var}[x]$ (lemma della varianza, § 1.3). ■

Attenzione. La formula non va memorizzata: serve sapere che la soluzione esiste in forma chiusa (grazie alla convessità) e saperla **ricavare** imponendo il gradiente nullo, come sopra.

Esempio — Esercizio svolto. Con i dati (1; 2,1), (2; 3,9), (3; 6,1), (4; 8,4), (5; 9,8): $l = 5$, $\bar{x} = 3$, $\bar{y} = 6{,}06$, $\sum_p x_p y_p = 2{,}1 + 7{,}8 + 18{,}3 + 33{,}6 + 49 = 110{,}8$, $\sum_p x_p^2 = 55$. Quindi

$$w_1 = \frac{110{,}8 - 5\cdot 3\cdot 6{,}06}{55 - 5\cdot 3^2} = \frac{110{,}8 - 90{,}9}{10} = 1{,}99, \qquad w_0 = 6{,}06 - 1{,}99\cdot 3 = 0{,}09.$$

La retta $h(x) \approx 1{,}99x + 0{,}09$ conferma l'intuizione $h(x) = 2x$.

4.1.3 Il gradiente su un singolo esempio

Per un solo esempio (x, y), con $E = (y - h_{\mathbf{w}}(x))^2$:

$$\frac{\partial E}{\partial w_i} = 2\big(y - h_{\mathbf{w}}(x)\big)\frac{\partial\big(y - (w_1 x + w_0)\big)}{\partial w_i} \quad\Longrightarrow\quad \frac{\partial E}{\partial w_0} = -2\big(y - h_{\mathbf{w}}(x)\big), \quad \frac{\partial E}{\partial w_1} = -2\big(y - h_{\mathbf{w}}(x)\big)\,x.$$

La quantità $\delta = y - h(x)$, l'errore sull'esempio, si chiama **delta**. Con l esempi basta sommare.

4.2 Input multidimensionali

Con $\mathbf{x}, \mathbf{w}$ vettori colonna il modello lineare è

$$\mathbf{w}^T\mathbf{x} + w_0 = w_0 + w_1 x_1 + \cdots + w_n x_n = w_0 + \sum_{i=1}^{n} w_i x_i,$$

dove w_0 si chiama **intercetta**, soglia, **bias** o offset. Conviene aggiungere all'input una componente costante $x_0 = 1$:

$$\mathbf{x}^T = [1, x_1, \dots, x_n], \qquad \mathbf{w}^T = [w_0, w_1, \dots, w_n], \qquad h(\mathbf{x}_p) = \mathbf{x}_p^T \mathbf{w} = \sum_{i=0}^{n} x_{p,i}\, w_i. \tag{4.3}$$

Il modello lineare è quindi un **prodotto scalare** tra input e pesi. Da qui in poi, salvo avviso, il bias è incluso in $\mathbf{w}$ in questo modo.

4.3 Soluzione diretta: le equazioni normali

Con X la matrice $l \times (n+1)$ che ha per righe gli $\mathbf{x}_p^T$ e $\mathbf{y}$ il vettore dei target, l'errore quadratico è

$$E(\mathbf{w}) = \sum_{p=1}^{l} \left(y_p - \mathbf{x}_p^T \mathbf{w}\right)^2 = \|\mathbf{y} - X\mathbf{w}\|^2.$$

Il gradiente. Derivando rispetto a una componente w_j:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = \sum_{p=1}^{l} 2\left(y_p - \mathbf{x}_p^T \mathbf{w}\right) \frac{\partial \left(y_p - \mathbf{x}_p^T \mathbf{w}\right)}{\partial w_j}.$$

Nel prodotto scalare $\mathbf{x}_p^T \mathbf{w} = x_{p,0} w_0 + x_{p,1} w_1 + \dots$ l'unico termine che dipende da w_j è $x_{p,j} w_j$, la cui derivata è $x_{p,j}$. Quindi

$$\boxed{\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^{l} \left(y_p - \mathbf{x}_p^T \mathbf{w}\right) x_{p,j} = -2 \sum_{p=1}^{l} \delta_p\, x_{p,j}} \qquad \delta_p = y_p - \mathbf{x}_p^T \mathbf{w}. \tag{4.4}$$

Questa forma (errore dell'esempio per input del peso) tornerà identica nelle reti neurali. In forma vettoriale, raccogliendo le componenti: $\nabla E(\mathbf{w}) = -2X^T(\mathbf{y} - X\mathbf{w})$.

Le equazioni normali. Imponendo $\nabla E(\mathbf{w}) = \mathbf{0}$: $X^T\mathbf{y} - X^T X\mathbf{w} = \mathbf{0}$, cioè

$$\boxed{(X^T X)\, \mathbf{w} = X^T \mathbf{y}} \tag{4.5}$$

Teorema — Soluzione ai minimi quadrati. *Se $X^T X$ è non singolare la soluzione è unica:*

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y} = X^+ \mathbf{y},$$

*dove X^+ è la **pseudoinversa di Moore-Penrose** (definita anche quando X non è invertibile). Se $X^T X$ è singolare le soluzioni sono infinite, e si può scegliere quella a **norma minima** di $\mathbf{w}$.*

In pratica si usa la **decomposizione ai valori singolari** (SVD): $X = U\Sigma V^T$ implica $X^+ = V\Sigma^+ U^T$, dove Σ^+ si ottiene sostituendo ogni elemento diagonale non nullo di Σ con il suo reciproco (e trasponendo). Calcolare $\mathbf{w} = X^+\mathbf{y}$ tramite SVD dà direttamente la soluzione a norma minima: **è questo l'algoritmo di apprendimento** dell'approccio diretto, presente in tutte le librerie numeriche con varianti per efficienza e stabilità.

4.4 Soluzione iterativa: la discesa del gradiente

Esistendo una soluzione diretta, perché cercarne una iterativa? Perché permette di ottenere

- soluzioni **più efficienti** (quella diretta ha costo cubico nella dimensione della matrice);
- la **regolarizzazione** (capitolo 5);
- una migliore approssimazione con dati rumorosi, fermando la ricerca **prima del minimo**;
- soprattutto, un metodo applicabile ai **modelli non lineari**: è la base dell'addestramento delle reti neurali.

4.4.1 L'idea

Il gradiente (4.4) indica la direzione di *salita* dell'errore. Muovendosi nel verso opposto, $\Delta\mathbf{w} = -\nabla E(\mathbf{w})$, si scende verso il minimo (§ 1.2). È una **ricerca locale** (*steepest descent*): si parte da un vettore di pesi iniziale e lo si modifica iterativamente per diminuire l'errore (Figura 4.1). In generale

$$\Delta\mathbf{w} = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \left(-\frac{\partial E}{\partial w_0}, -\frac{\partial E}{\partial w_1}, \dots, -\frac{\partial E}{\partial w_n} \right)^T,$$

con una componente per ogni peso: si lavora in spazi di qualsiasi dimensione senza doverli visualizzare. Con due pesi la superficie d'errore $E([w_0, w_1]^T)$ è un paraboloide (una funzione quadratica convessa), e ogni punto del piano (w_0, w_1) è un'ipotesi.

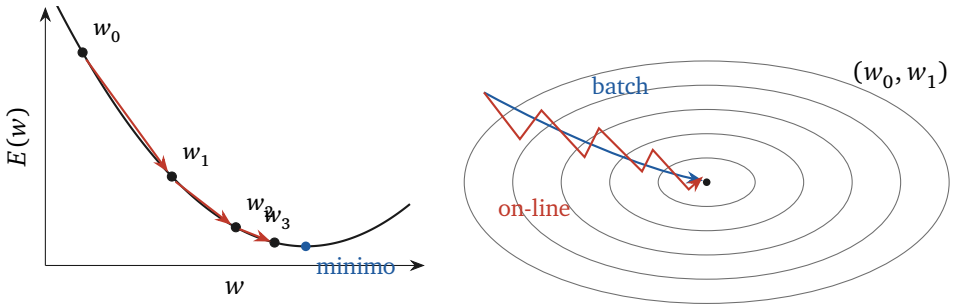


Figura 4.1. A sinistra la discesa del gradiente in una dimensione: a ogni passo il peso si sposta nel verso opposto alla pendenza. A destra le curve di livello di un errore quadratico con due pesi: la discesa batch (blu) procede in modo regolare, quella on-line (rosso) a zig-zag.

4.4.2 La regola di apprendimento

Ci si muove iterativamente con la regola

$$\boxed{\mathbf{w}_{new} = \mathbf{w} + \eta\,\Delta\mathbf{w}} \tag{4.6}$$

dove η è il **learning rate** (*step size*), che regola l'ampiezza dei passi.

Algoritmo — Discesa del gradiente.

1. Inizializzare $\mathbf{w}$ con valori piccoli; fissare $0 < \eta < 1$.
2. Calcolare $\Delta\mathbf{w} = -\partial E(\mathbf{w})/\partial\mathbf{w}$.
3. Aggiornare $\mathbf{w}_{new} = \mathbf{w} + \eta\,\Delta\mathbf{w}$.
4. Ripetere dal passo 2 fino a convergenza, o finché $E(\mathbf{w})$ è abbastanza piccolo.

- Usare $\Delta\mathbf{w}/l$ (gradiente medio) corrisponde al *least mean squares*; dividere per l è il caso standard.
- η regola un compromesso **velocità/stabilità**; può essere ridotto gradualmente verso zero per garantire la convergenza evitando oscillazioni attorno al minimo (varianti nel § 12.4).

4.4.3 Batch e on-line

- **Batch**: il gradiente è la somma su **tutti** gli l esempi e i pesi si aggiornano dopo aver visto tutto il training set, cioè dopo un'**epoca**. La stima del gradiente è più precisa.
- **On-line** (o **stocastica**, SGD): i pesi si aggiornano dopo **ogni** esempio, con il gradiente del singolo esempio

$$\frac{\partial E_p(\mathbf{w})}{\partial w_j} = -2\big(y_p - \mathbf{x}_p^T\mathbf{w}\big)\, x_{p,j} = -\Delta_p w_j.$$

  L'uscita sul secondo esempio è calcolata con i pesi già aggiornati dal primo, e così via. Fa progressi a ogni esempio e può essere più veloce, ma richiede un η più piccolo.

I casi intermedi (*mini-batch*) sono trattati nel § 12.3.

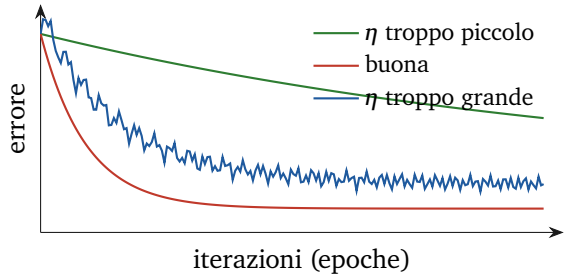


Figura 4.2. Curve di apprendimento (errore in funzione delle iterazioni): una curva che scende molto lentamente indica un learning rate troppo piccolo, una con picchi e oscillazioni un learning rate troppo grande; la curva buona scende rapidamente e si stabilizza (andamenti qualitativi).

4.4.4 Interpretazione: la delta rule

Ricapitolando, nella versione batch

$$\Delta w_j = 2\sum_{p=1}^{l} \underbrace{\big(y_p - \mathbf{x}_p^T\mathbf{w}\big)}_{\delta_p} x_{p,j}, \qquad \mathbf{w}_{new} = \mathbf{w} + \eta\,\Delta\mathbf{w} \tag{4.7}$$

(la costante 2 si può assorbire in η). È una regola di **correzione dell'errore**, detta **regola di Widrow-Hoff** o **delta rule**: ogni w_j cambia **in proporzione all'errore** (target meno output).

- Se l'errore è 0, non c'è correzione.
- Con input $x_j > 0$ e errore positivo (output troppo basso): $\delta > 0$, w_j aumenta, l'output aumenta, l'errore diminuisce.
- Con input $x_j > 0$ e errore negativo (output troppo alto): $\delta < 0$, w_j diminuisce, l'output diminuisce, l'errore diminuisce.
- Con input negativo i segni si invertono; con input nullo il peso non cambia. L'input "decide" quanto e in che verso ogni peso è responsabile dell'errore.

Esempio — La delta rule all'opera. Input $\mathbf{x} = (1, 0, 1)$, pesi $(w_1, w_2, w_3) = (-0{,}2;\ 0{,}5;\ -0{,}2)$, $w_0 = 0$: $\mathbf{w}^T\mathbf{x} = -0{,}4$, quindi l'uscita (con soglia) è -1, ma il target è $+1$. Il delta è positivo: w_1 e w_3 (input pari a 1) aumentano, w_2 (input 0) resta invariato. Dopo la correzione, con $w_1 = w_3 = +0{,}1$, si ha $\mathbf{w}^T\mathbf{x} = 0{,}2 > 0$ e l'uscita è corretta.

Perché la discesa del gradiente è così importante. È un metodo di ricerca locale semplice ed efficace che esplora uno spazio delle ipotesi **infinito e continuo**. Si applica ogni volta che H è continuo e la loss è differenziabile, **non solo ai modelli lineari**: è la base dell'addestramento di reti neurali e deep learning. Esistono molti miglioramenti (metodi di Newton e quasi-Newton, gradiente coniugato, …).

4.5 Classificazione con modelli lineari

4.5.1 Iperpiani e regioni di decisione

Gli stessi modelli si usano per la classificazione, con target categorici (0/1 o $-1/+1$). L'equazione $\mathbf{w}^T\mathbf{x} + w_0 = 0$ definisce un **iperpiano**; poiché $\mathbf{w}^T\mathbf{x} + w_0$ è positivo da un lato e negativo dall'altro, il segno decide la classe. In due dimensioni $w_0 + w_1x_1 + w_2x_2 = 0$ è una retta: il **confine di decisione** (*decision boundary*). Applicando una funzione soglia si ottiene la **Linear Threshold Unit** (LTU):

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w}^T\mathbf{x} + w_0 \geq 0 \\ 0 & \text{altrimenti} \end{cases} \qquad \text{oppure} \qquad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + w_0) \in \{-1, +1\}, \tag{4.8}$$

e, con il bias incluso in $\mathbf{w}$, $h(\mathbf{x}_p) = \text{sign}(\mathbf{x}_p^T\mathbf{w}) = \text{sign}\left(\sum_{i=0}^{n} x_{p,i}w_i\right)$.

Dire $\mathbf{w}^T\mathbf{x} + w_0 \geq 0$ equivale a dire $\mathbf{w}^T\mathbf{x} \geq -w_0$: le due forme individuano la stessa regione, ma la seconda mostra il ruolo del bias. $-w_0$ è il **valore di soglia** che la combinazione pesata degli input deve superare per "attivare" l'uscita $+1$.

Proprietà dell'iperpiano.

- Se $w_0 = 0$ l'iperpiano passa per l'**origine**; se $n > 2$ è un iperpiano di dimensione $n - 1$.
- **Libertà di scala**: moltiplicando $\mathbf{w}$ (bias compreso) per una costante $K > 0$ il confine non cambia.
- **$\mathbf{w}$ è ortogonale all'iperpiano**. Presi due punti $\mathbf{x}_a, \mathbf{x}_b$ sull'iperpiano, $\mathbf{w}^T\mathbf{x}_a + w_0 = 0$ e $\mathbf{w}^T\mathbf{x}_b + w_0 = 0$; sottraendo, $\mathbf{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 0$: $\mathbf{w}$ è ortogonale a ogni vettore che giace sull'iperpiano.
- Se esiste un iperpiano separatore, **ne esistono infiniti**, e quindi anche molti algoritmi per trovarli.
- In 2D l'equazione si riscrive $x_2 = -x_1\, w_1/w_2 - w_0/w_2$ (esercizio utile: disegnarla per diversi valori di $\mathbf{w}$).

Esempio — Terremoti o esplosioni nucleari?. Dati sismici: x_1 è la magnitudo delle onde di volume, x_2 quella delle onde di superficie. Un algoritmo di apprendimento trova il confine $-4{,}9 + 1{,}7x_1 - x_2 = 0$. Per un nuovo evento $(6, 3)$:

$$h(6, 3) = \text{sign}(-4{,}9 + 1{,}7 \cdot 6 - 1 \cdot 3) = \text{sign}(2{,}3) = +1 \ \Rightarrow\ \text{esplosione nucleare.}$$

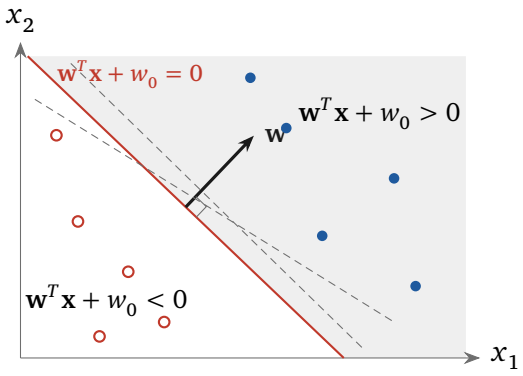
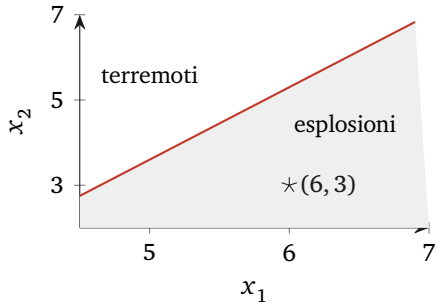


Figura 4.3. Un problema linearmente separabile: l'iperpiano $\mathbf{w}^T\mathbf{x} + w_0 = 0$ (rosso) separa i positivi (pieni) dai negativi (vuoti), $\mathbf{w}$ è ortogonale all'iperpiano e punta verso la regione positiva; anche altri iperpiani (tratteggiati) separano correttamente i dati.



Esempio — Filtro antispam. Si vuole $h(\text{mail}) = +1$ per lo spam e -1 altrimenti. Le feature $\phi_k(\text{mail})$ sono ad esempio la presenza (0/1) di parole o frasi ("free money") o la lunghezza del messaggio: è la rappresentazione *bag of words*. Ogni peso w_k esprime il contributo della sua feature: positivo per "free money", negativo per ".edu". La regola è $h_{\mathbf{w}}(\mathbf{x}) = \text{sign}\big(\sum_k w_k \phi_k(\mathbf{x})\big)$: se la combinazione pesata è positiva, è spam.

4.5.2 Il problema di apprendimento per la classificazione

Dati l esempi $(\mathbf{x}_p, y_p)$ con $y_p \in \{0, 1\}$ o $\{-1, +1\}$, si cerca $\mathbf{w}$ che minimizzi

$$R_{\text{emp}} = \frac{1}{l} \sum_{p=1}^{l} L\big(h(\mathbf{x}_p), y_p\big).$$

Usare direttamente la loss 0/1 su $\text{sign}(\mathbf{w}^T\mathbf{x})$ rende il problema difficile: la funzione è **costante a tratti**, quindi il gradiente è nullo quasi ovunque e non indica dove muoversi; si finisce in problemi combinatori.

Sostituire la loss con una funzione liscia. Si minimizza invece l'errore quadratico applicato all'**uscita lineare** $\mathbf{w}^T\mathbf{x}$ (non a $h(\mathbf{x})$, che contiene il segno):

$$E(\mathbf{w}) = \sum_{p=1}^{l} \big(y_p - \mathbf{x}_p^T\mathbf{w}\big)^2 = \|\mathbf{y} - X\mathbf{w}\|^2.$$

Con target $y_p = 1$, minimizzare $(1 - \mathbf{x}_p^T\mathbf{w})^2$ spinge $\mathbf{x}_p^T\mathbf{w}$ verso 1, quindi a valori positivi: nessun errore di classificazione. Con $y_p = -1$ lo spinge verso -1. Le due loss sono minimizzate nella

stessa zona (Figura 4.4). La funzione è quadratica, quindi un minimo esiste sempre (non necessariamente unico).

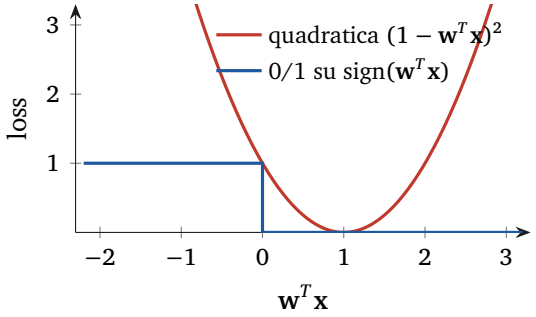


Figura 4.4. Loss 0/1 e loss quadratica per un esempio con target $y = 1$: entrambe sono minime nella zona $\mathbf{w}^T\mathbf{x} > 0$, ma solo la seconda ha un gradiente utile.

Lo stesso approccio vale identico per la regressione, quindi entrambi gli algoritmi visti — equazioni normali e discesa del gradiente — si applicano senza modifiche. In sintesi:

- il modello si **addestra** sul TR con LMS applicato a $\mathbf{w}^T\mathbf{x}$;
- si **usa** per classificare applicando la soglia: $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x})$;
- si **valuta** con l'errore di classificazione (loss 0/1), non solo con l'MSE:

$$\text{errore medio} = \frac{1}{l}\sum_{p=1}^{l} L\big(h(\mathbf{x}_p), d_p\big) = \frac{\#\text{errori}}{l}, \qquad \text{accuratezza} = \frac{l - \#\text{errori}}{l}.$$

4.5.3 Un esempio: due classi nel piano

Si considerino 200 punti in $\mathbb{R}^2$, 100 per ciascuna di due classi, generati da una distribuzione sconosciuta (Figura 4.5). Codificando le classi come 0 e 1 e addestrando con LMS, il confine $\{\mathbf{x} \mid \mathbf{x}^T\mathbf{w} = 0{,}5\}$ è lineare e lascia molti punti dal lato sbagliato. È una cattiva soluzione? Dipende da come sono stati generati i dati:

- **scenario 1**: ogni classe è generata da una gaussiana con componenti scorrelate, stessa varianza e medie diverse. La regola lineare è **quasi ottima**: la sovrapposizione tra le classi è inevitabile, dovuta al rumore;
- **scenario 2**: ogni classe è una miscela di 10 gaussiane strette. Il modello lineare è **troppo rigido** e servono modelli più flessibili.

I dati della figura (usati in tutto il resto delle note) sono generati come nello scenario 2.

Minimi quadrati e verosimiglianza. I minimi quadrati corrispondono al criterio di **massima verosimiglianza** se gli errori sperimentali hanno distribuzione normale.

4.5.4 Il bias induttivo del modello lineare

- **Language bias**: H è l'insieme delle funzioni lineari, che può essere molto restrittivo.
- **Search bias**: ricerca ordinata, guidata dalla minimizzazione dei minimi quadrati. Un metodo diverso, ad esempio uno che restringa i valori dei parametri, darebbe soluzioni con proprietà diverse (in particolare rispetto alla generalizzazione: è la regolarizzazione).

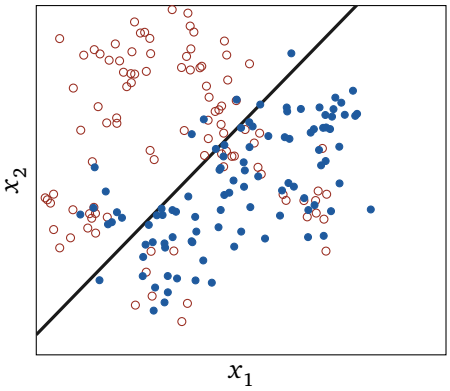


Figura 4.5. Il problema a due classi (classe 1 piena, classe 0 vuota) e il confine lineare ai minimi quadrati $\mathbf{x}^T\mathbf{w} = 0{,}5$. Su questi dati l'errore di classificazione è 0,205 sul training e 0,246 su un test set di 10 000 punti.

4.5.5 Classificazione multi-classe

Con $K > 2$ classi gli approcci semplici sono:

- codifica **1-of-K** delle classi e un modello lineare per ciascuna componente;
- **OVA** (*one-vs-all*): K classificatori binari, ciascuno distingue una classe da tutte le altre; vince la classe del classificatore con l'uscita più alta;
- **AVA** (*all-vs-all*, *one-vs-one*): un classificatore per ogni coppia di classi, $K(K-1)/2$ in tutto, ciascuno addestrato su meno dati; vince la classe con la massima somma di uscite o con più voti.

Con molte classi può verificarsi il **mascheramento** (una classe "nascosta" dalle altre). Esistono modelli con uscite multiple native e altri classificatori lineari noti, come la **Linear Discriminant Analysis** e la **regressione logistica** (che modella direttamente $P(y \mid \mathbf{x})$).

4.6 Limiti: la separabilità lineare

Se la funzione target di una regressione non è lineare, una retta è una soluzione troppo povera (il caso $M = 1$ del § 5.2). Per la classificazione il limite ha un nome preciso.

> **Definizione — Separabilità lineare.** Due insiemi di punti in $\mathbb{R}^n$ sono **linearmente separabili** se esiste un iperpiano (di dimensione $n-1$) che li separa completamente; nel piano, una singola retta.

Il confine di decisione lineare dà soluzioni esatte **solo** per insiemi linearmente separabili. Il modello lineare può rappresentare ad esempio le **congiunzioni**: $x_1 \wedge x_2 \wedge x_4$ si realizza con $1 \cdot x_1 + 1 \cdot x_2 + 0 \cdot x_3 + 1 \cdot x_4 \geq 2{,}5$ (vero solo se i tre input valgono 1), l'AND di due variabili con $x_1 + x_2 \geq 1{,}5$. Ma non tutto è separabile (Figura 4.6):

- **3 punti**: per ogni assegnazione delle etichette esiste un iperpiano separatore, **se non sono allineati**. Se sono allineati, l'etichettatura con il punto centrale di classe diversa non è separabile;
- **4 punti**: esiste un'etichettatura che nessun classificatore lineare separa, lo **XOR** ($00 \mapsto 0$, $01 \mapsto 1$, $10 \mapsto 1$, $11 \mapsto 0$), con i positivi su una diagonale e i negativi sull'altra.

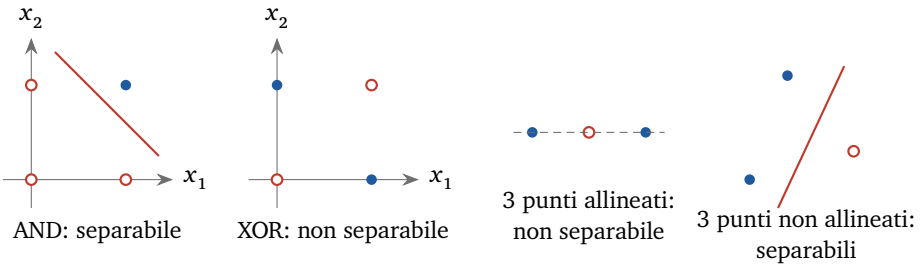


Figura 4.6. Limiti della separazione lineare (pieno = classe 1, vuoto = classe 0).

Contare quante etichettature di un insieme di punti un modello riesce a separare è esattamente l'idea della VC-dimension (capitolo 7): una retta nel piano separa tutte le etichettature di 3 punti non allineati, ma non di 4.

4.7 Espansione in basi lineare

Nei modelli parametrici **"lineare" si riferisce al modo in cui compaiono i coefficienti w**, non alla forma della funzione. Si possono usare input trasformati come $x, x^2, x^3, \ldots$ per modellare relazioni non lineari mantenendo **la stessa macchina di apprendimento** (i minimi quadrati), come nella regressione polinomiale

$$h_{\mathbf{w}}(x) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j.$$

Definizione — Linear Basis Expansion. Un'**espansione in basi lineare** (LBE) è un modello

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^{K} w_k \, \phi_k(\mathbf{x}),$$

dove le $\phi_k : \mathbb{R}^n \to \mathbb{R}$ sono trasformazioni **fissate** dell'input: il vettore di input viene aumentato con nuove variabili derivate da $\mathbf{x}$.

Esempi di ϕ: rappresentazioni **polinomiali** ($\phi(\mathbf{x}) = x_j^2$, $\phi(\mathbf{x}) = x_j x_i$, ...); trasformazioni non lineari di singoli input ($\log x_j$, $\sqrt{x_j}$); trasformazioni di più input ($\phi(\mathbf{x}) = \|\mathbf{x}\|$); spline. Ad esempio $h(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_3 \log x_2 + w_4 \log x_3 + w_5 \, x_2 x_3 + w_0$. Tipicamente il numero di parametri diventa $K > n$. Il modello è lineare nei parametri (e nelle ϕ), non in $\mathbf{x}$: lo stesso algoritmo vale per la regressione e per la classificazione (applicando la soglia a $\sum_k w_k \phi_k(\mathbf{x})$).

Attenzione — Pro e contro della LBE. **Pro**: il modello diventa più espressivo e rappresenta relazioni più complesse.
Contro:

- con molte funzioni di base si rischia facilmente l'**overfitting**: servono metodi di controllo della complessità, intesa come flessibilità e non come costo computazionale (capitolo 5);
- la **maledizione della dimensionalità**: il volume dello spazio cresce così in fretta che i dati diventano sparsi e non bastano più a sostenere la complessità del modello (§ 6.6.4);
- le ϕ sono **fissate prima** di vedere i dati, e resta aperta la domanda su quali scegliere. L'alternativa sono modelli adattivi, non lineari nei parametri, che imparano le ϕ (le reti neurali, § 11.2.2), o che le gestiscono in modo efficiente (le SVM con i kernel, § 13.3).

Punti chiave

- LMS: $E(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2$, gradiente $\partial E/\partial w_j = -2\sum_p \delta_p x_{p,j}$.
- Soluzione diretta: $(X^T X)\mathbf{w} = X^T\mathbf{y}$, cioè $\mathbf{w} = X^+\mathbf{y}$ (SVD). Soluzione iterativa: $\mathbf{w} \leftarrow \mathbf{w}+\eta\Delta\mathbf{w}$ (delta rule), batch o on-line.
- Classificazione: si addestra con LMS su $\mathbf{w}^T\mathbf{x}$, si classifica con $\text{sign}(\mathbf{w}^T\mathbf{x})$.
- Solo problemi linearmente separabili hanno una soluzione esatta; lo XOR non lo è.
- LBE: lineare nei parametri, non nell'input; più flessibilità, ma rischio di overfitting.

Domande di verifica

1. Formulare il problema di apprendimento per un modello lineare con LMS, per regressione e classificazione.
2. Ricavare la retta ai minimi quadrati nel caso univariato; derivare il gradiente e le equazioni normali.
3. Descrivere la discesa del gradiente; differenza tra batch e on-line; ruolo di η.
4. Spiegare la delta rule come regola di correzione dell'errore.
5. Perché per la classificazione si usa l'errore quadratico su $\mathbf{w}^T\mathbf{x}$ invece della loss 0/1?
6. Cos'è la separabilità lineare? Perché lo XOR non è linearmente separabile?
7. Cos'è la linear basis expansion? Pro e contro.

Capitolo 5

Overfitting e regolarizzazione

*Un modello che si adatta perfettamente ai dati di training può predire male i dati nuovi. Il capitolo definisce l'overfitting, lo osserva in un caso di studio (il fitting di polinomi, che è una espansione in basi lineare) e introduce lo strumento principale per controllare la complessità di un modello: la regolarizzazione di Tikhonov (*ridge regression*,* weight decay*), con le varianti L^1 ed* elastic net*.*

5.1 Generalizzazione e overfitting

> **Definizione — Ipotesi dell'apprendimento induttivo.** *Qualsiasi ipotesi h che approssima bene f sugli esempi di training approssimerà bene f anche sulle istanze nuove, mai viste.*

Questa ipotesi non è sempre vera, ed è proprio qui che nasce l'overfitting.

> **Definizione — Overfitting.** Un learner fa **overfitting** se restituisce un'ipotesi $h \in H$ con errore vero (di generalizzazione, *rischio*) R ed errore empirico (di training) E, quando esiste un'altra $h' \in H$ con
>
> $$E' > E \qquad \text{ma} \qquad R' < R :$$
>
> h' si adatta *peggio* ai dati di training ma è *migliore* sui dati nuovi. L'**underfitting** è la situazione opposta: il modello è troppo semplice per cogliere l'andamento della funzione target, e l'errore è alto già sul training.

Il punto critico diventa quindi la **stima dell'accuratezza**, che si affronta in modo **teorico** (Statistical Learning Theory, capitolo 7) o **empirico** (errori su training e test, cross-validation, capitolo 9).

5.2 Un caso di studio: il fitting polinomiale

Si usa un problema di regressione con un modello parametrico: lo spazio delle ipotesi sono i **polinomi di grado** M (una LBE con $\phi_j(x) = x^j$, § 4.7)

$$h_{\mathbf{w}}(x) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j,$$

la cui complessità cresce con M. La funzione target è $\sin(2\pi x)$ e i target dei $l = 10$ esempi contengono **rumore gaussiano**, per cui i campioni non stanno esattamente sulla curva vera. Il task è artificiale e irrealistico (una sola variabile, funzione target nota) e serve solo a visualizzare i concetti. I parametri si trovano minimizzando la somma degli errori quadratici

$$E(\mathbf{w}) = \sum_{p=1}^{l} \big(y_p - h_{\mathbf{w}}(x_p)\big)^2,$$

con le equazioni normali del § 4.3 applicate alle feature $(1, x, x^2, \dots, x^M)$.

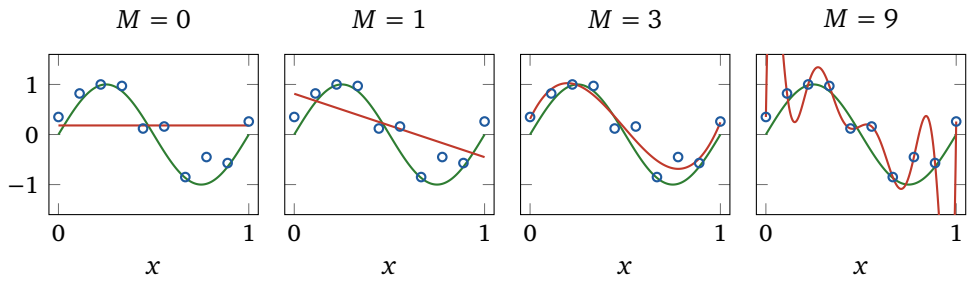


Figura 5.1. Fitting con polinomi di grado $M = 0, 1, 3, 9$ (rosso) dei 10 campioni rumorosi (blu) della funzione vera $\sin(2\pi x)$ (verde).

5.2.1 Effetto del grado del polinomio

- $M = 0$ (costante) e $M = 1$ (retta): il modello è troppo semplice e non coglie l'andamento della funzione target: **underfitting**.
- $M = 3$: la flessibilità in più è utile e la curva approssima bene il seno.
- $M = 9$: con 10 punti e 10 parametri il polinomio passa **esattamente** per tutti i dati, $E(\mathbf{w}) = 0$ sul training. Ma oscilla violentemente tra i punti e rappresenta malissimo la funzione vera: il modello è troppo complesso e **impara anche il rumore**: **overfitting**.

5.2.2 Errore di training e di test al variare di M

Per confrontare modelli e dataset si usa il **Root-Mean-Square error**

$$E_{RMS} = \sqrt{\frac{1}{l}\sum_{p=1}^{l}\left(y_p - h_{\mathbf{w}^*}(x_p)\right)^2}$$

dove $\mathbf{w}^*$ sono i pesi del modello addestrato (equivale a $\sqrt{2E(\mathbf{w}^*)/l}$ se E è definito con un fattore $\frac{1}{2}$ davanti alla somma). La divisione per l permette di confrontare dataset di dimensione diversa, la radice riporta l'errore sulla scala del target.

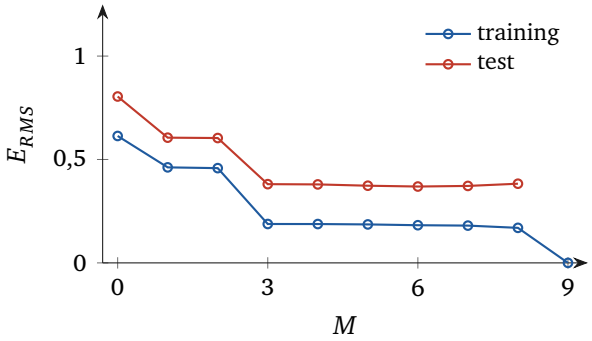


Figura 5.2. E_{RMS} di training e di test (1000 punti nuovi) al variare del grado M: a sinistra underfitting, a destra overfitting (per $M = 9$ l'errore di test esce dal grafico).

L'errore di training **decresce sempre** all'aumentare di M, perché un modello più flessibile può adattarsi meglio ai dati. L'errore di test prima decresce, poi resta stabile e infine **esplode** per $M = 9$: è la firma dell'overfitting. Anche i coefficienti ottimi $\mathbf{w}^*$ sono rivelatori (Tabella 5.1, dall'esempio originale di questo esperimento).

Tabella 5.1. Coefficienti ottimi $\mathbf{w}^*$ dei polinomi di grado M.

	$M=0$	$M=1$	$M=3$	$M=9$
w_0^*	0,19	0,82	0,31	0,35
w_1^*		−1,27	7,99	232,37
w_2^*			−25,43	−5321,83
w_3^*			17,37	48568,31
w_4^*				−231639,30
w_5^*				640042,26
w_6^*				−1061800,52
w_7^*				1042400,18
w_8^*				−557682,99
w_9^*				125201,43

Pesi enormi = overfitting. Nel polinomio di grado 9 i coefficienti diventano **enormi** e di segno alterno: si compensano a vicenda per far passare la curva esattamente per ogni punto, producendo oscillazioni fortissime. Penalizzare i pesi grandi è quindi un modo per controllare la complessità: è la regolarizzazione (§ 5.3).

5.2.3 Effetto della quantità di dati

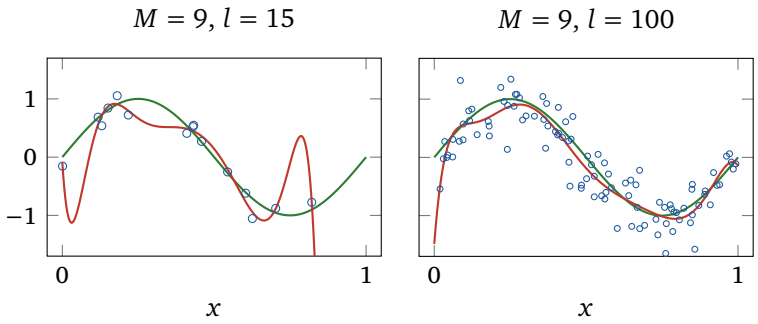


Figura 5.3. Polinomio di grado 9 addestrato su $l = 15$ e $l = 100$ esempi: con più dati l'overfitting si riduce.

A parità di modello ($M = 9$), con 15 esempi il polinomio oscilla ancora ma molto meno, e con 100 esempi segue quasi perfettamente il seno (Figura 5.3). **Con più dati si possono usare modelli più complessi** senza cadere nell'overfitting.

Cosa mostra l'esempio. La capacità di generalizzazione (misurata come rischio o errore di test) va studiata in relazione a **errore di training**, **complessità del modello** e **numero di dati**. In generale, al crescere della complessità l'errore di training decresce sempre, mentre quello di test decresce e poi risale (Figura 5.4). La teoria che lega i tre elementi è la Statistical Learning Theory (capitolo 7).

Esempio — Rumore o complessità?. Il rumore è la causa dell'overfitting, o lo è il compromesso di complessità? Si può avere overfitting anche con dati perfettamente puliti: un polinomio di grado molto alto che interpola pochi punti di una funzione liscia può oscillare molto tra un

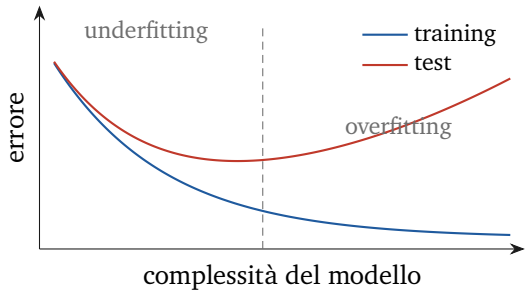


Figura 5.4. Andamento tipico (qualitativo) degli errori di training e di test al crescere della complessità del modello.

punto e l'altro. Il problema è la **complessità eccessiva rispetto ai dati disponibili**. Per lo stesso motivo un errore di training nullo non implica una buona soluzione.

5.3 Regolarizzazione di Tikhonov

Tra i molti modi di controllare la complessità c'è il *coefficient shrinkage*, il "restringimento" dei coefficienti. La **ridge regression** (**regolarizzazione di Tikhonov**) aggiunge all'errore un termine che **penalizza i pesi grandi**:

$$\text{Loss}(\mathbf{w}) = \underbrace{\sum_{p=1}^{l} \left(y_p - \mathbf{x}_p^T \mathbf{w}\right)^2}_{\text{termine d'errore } (R_{\text{emp}})} + \underbrace{\lambda \, \|\mathbf{w}\|^2}_{\text{penalità}}, \qquad \|\mathbf{w}\|^2 = \sum_j w_j^2. \tag{5.1}$$

$\lambda > 0$ è l'**iperparametro di regolarizzazione**, scelto in fase di **model selection** (capitolo 9). Il modello risultante è più "liscio" (*smooth*): si favoriscono modelli con pesi piccoli o nulli, che usano di fatto meno termini, cioè meno complessi.

Attenzione — Loss ed errore non sono più sinonimi. Da qui in poi **loss** indica la funzione obiettivo usata per l'addestramento (errore più penalità) ed **errore** E il solo termine sui dati, che misura la qualità del modello. Nei risultati si riporta l'errore, perché è la misura utile per chi usa il modello.

Cambia l'obiettivo dell'apprendimento. Non si cerca più *solo* il modello che si adatta meglio ai dati, ma **il modello più semplice che si adatta bene ai dati**. La loss è una bilancia tra due piatti:

- **adattamento ai dati** (errore di training): spinge a seguire i dati e minimizzare i residui;
- **semplicità del modello** (penalità $\|\mathbf{w}\|^2$): punisce i pesi grandi, rendendo il modello più liscio e più capace di generalizzare;

e λ decide quanto pesa la penalità.

5.3.1 Soluzione

Soluzione diretta della ridge regression.

$$\mathbf{w} = (X^T X + \lambda I)^{-1} X^T \mathbf{y},$$

e per $\lambda > 0$ la matrice $X^T X + \lambda I$ è **sempre invertibile**.

Dimostrazione. In forma matriciale $\text{Loss}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\, \mathbf{w}^T\mathbf{w}$. Il gradiente del primo termine è $-2X^T(\mathbf{y} - X\mathbf{w})$ (§ 4.3); quello del secondo è $2\lambda\mathbf{w}$, perché $\partial(\lambda \sum_j w_j^2)/\partial w_j = 2\lambda w_j$. Imponendo il gradiente nullo:

$$-2X^T\mathbf{y} + 2X^T X\mathbf{w} + 2\lambda\mathbf{w} = \mathbf{0} \iff (X^T X + \lambda I)\mathbf{w} = X^T\mathbf{y}.$$

Per ogni $\mathbf{v} \neq \mathbf{0}$ vale $\mathbf{v}^T(X^T X + \lambda I)\mathbf{v} = \|X\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0$: la matrice è definita positiva, quindi invertibile. È anche un vantaggio numerico rispetto alle equazioni normali semplici. ■

Con la **discesa del gradiente** si segue ancora il gradiente negativo della loss. Poiché la derivata della penalità rispetto a w_j è $2\lambda w_j$, la regola diventa

$$\boxed{\mathbf{w}_{new} = \mathbf{w} + \eta\,\Delta\mathbf{w} - 2\lambda\mathbf{w}} \tag{5.2}$$

dove $\eta\,\Delta\mathbf{w}$ è il passo sul solo termine d'errore e la penalità è moltiplicata direttamente per λ. È la tecnica del **weight decay**: anche con gradiente d'errore nullo, a ogni passo ogni peso si riduce di una frazione del suo valore.

5.3.2 Il compromesso governato da λ

- λ **piccolo**: conta soprattutto il termine d'errore; il modello resta troppo complesso (pesi di norma alta), con rischio di **overfitting**.
- λ **grande**: conta soprattutto la penalità; l'errore sui dati può crescere troppo, verso l'**underfitting**.

Il vantaggio è una **realizzazione concreta del controllo della complessità**, facile da implementare e di applicabilità generale (la stessa penalità si usa nelle reti neurali, § 12.6). Spingendo i pesi verso valori piccoli si ottiene un modello con VC-dimension più bassa (§ 7.4), e il compromesso è regolato da **un solo parametro**.

5.3.3 Effetto sul polinomio di grado 9

Senza regolarizzazione ($\lambda = 0$) il polinomio di grado 9 aveva errore di training nullo e overfitting fortissimo. Con un λ adatto ($\ln\lambda = -18$, cioè $\lambda \approx 1{,}5 \cdot 10^{-8}$) il modello è **peggiore sul training** ma molto migliore sui dati nuovi; con λ troppo alto ($\ln\lambda = 0$, $\lambda = 1$) si finisce in underfitting (Figura 5.5 e Figura 5.6). I coefficienti (Tabella 5.2) mostrano come la penalità "doma" i pesi enormi del caso non regolarizzato.

5.4 Altre forme di regolarizzazione

- **Ridge** (norma $\|\cdot\|_2$): penalizza il quadrato dei pesi e tende a rendere **tutti** i pesi più piccoli.

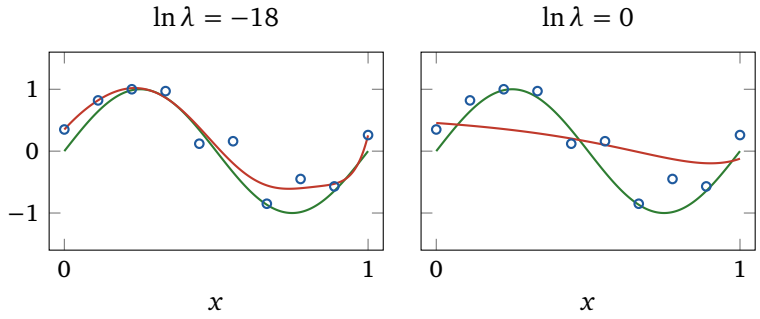


Figura 5.5. Polinomio di grado 9 regolarizzato sugli stessi 10 punti della Figura 5.1: con $\ln \lambda = -18$ la curva è liscia e segue la funzione vera; con $\ln \lambda = 0$ è troppo piatta.

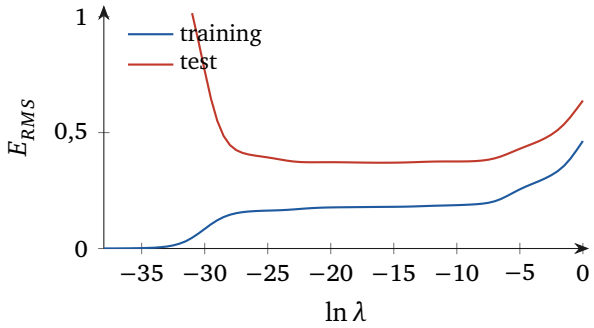


Figura 5.6. E_{RMS} di training e di test del polinomio di grado 9 al variare di $\ln \lambda$: a sinistra (regolarizzazione debole) overfitting, a destra underfitting, al centro il buon compromesso.

- **Lasso** (norma $\|\cdot\|_1$, penalità $\lambda \sum_j |w_j|$): tende a portare **alcuni pesi esattamente a zero**, lasciandone altri grandi: è una forma di **feature selection** e produce modelli più semplici. Il valore assoluto rende però la loss non differenziabile in zero, e servono tecniche di ottimizzazione non differenziabile (sub-gradiente, approssimazioni lisce della norma).
- **Elastic net**: usa entrambe le norme.

> **Perché L^1 azzera i pesi e L^2 no.** Il gradiente di λw^2 è $2\lambda w$: diventa piccolissimo quando w è vicino a zero, quindi la spinta verso zero si attenua e il peso resta piccolo ma non nullo. Il (sub-)gradiente di $\lambda |w|$ è invece $\lambda \operatorname{sign}(w)$: una spinta **costante** verso zero, anche per pesi già piccoli, che li porta esattamente a zero se il termine d'errore non li "difende". Geometricamente (Figura 5.7) le curve di livello della norma L^1 hanno **spigoli sugli assi**, dove una coordinata è nulla: la soluzione del problema vincolato cade tipicamente su uno spigolo.

Altri miglioramenti (opzionali):

- **data augmentation**: aggiungere rumore agli input insegna al modello a ignorare variazioni irrilevanti (robustezza e regolarizzazione); l'*oversampling* aiuta con dataset sbilanciati;
- **input derivati**: si usa un piccolo numero di nuove variabili, combinazioni lineari degli input (*Principal Component Regression*, *Partial Least Squares*).

Punti chiave

- Overfitting: esiste h' con errore di training maggiore ma rischio minore.
- Nel fitting polinomiale l'errore di training cala sempre con M, quello di test ha forma a U;

Tabella 5.2. Coefficienti del polinomio di grado 9 per tre valori di λ (esempio originale).

	$\ln\lambda = -\infty$ ($\lambda = 0$)	$\ln\lambda = -18$	$\ln\lambda = 0$
w_0^*	0,35	0,35	0,13
w_1^*	232,37	4,74	−0,05
w_2^*	−5321,83	−0,77	−0,06
w_3^*	48568,31	−31,97	−0,05
w_4^*	−231639,30	−3,89	−0,03
w_5^*	640042,26	55,28	−0,02
w_6^*	−1061800,52	41,32	−0,01
w_7^*	1042400,18	−45,95	−0,00
w_8^*	−557682,99	−91,53	0,00
w_9^*	125201,43	72,68	0,01

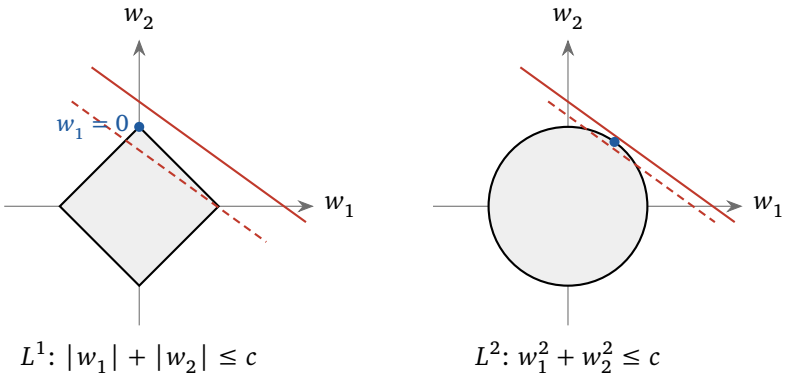


Figura 5.7. Perché L^1 produce soluzioni sparse. In rosso le curve di livello del termine d'errore: la soluzione regolarizzata è il punto in cui la curva di livello più bassa raggiungibile tocca la regione ammessa (grigia). Con L^1 il contatto avviene su un vertice del rombo ($w_1 = 0$); con L^2 in un punto generico del cerchio.

più dati permettono modelli più complessi.

- Ridge: Loss $= \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|^2$; soluzione $(X^TX + \lambda I)^{-1}X^T\mathbf{y}$; weight decay $\mathbf{w} \leftarrow \mathbf{w} + \eta\Delta\mathbf{w} - 2\lambda\mathbf{w}$.
- λ piccolo → overfitting, λ grande → underfitting; si sceglie con la model selection.
- L^1 azzera alcuni pesi (sparsità), L^2 li riduce tutti.

Domande di verifica

1. Definire formalmente l'overfitting.
2. Descrivere l'esempio del fitting polinomiale: cosa succede al variare di M e di l?
3. Perché un errore di training nullo non implica una buona soluzione?
4. Scrivere la loss di Tikhonov, la soluzione diretta e la regola di weight decay; ruolo di λ.
5. Differenza tra regolarizzazione L^1 e L^2.

Capitolo 6

K-nearest neighbors

Il K-nearest neighbors (K-NN) è l'estremo opposto del modello lineare: non costruisce un'ipotesi globale ma risponde, per ogni nuovo punto, con una stima locale basata sugli esempi più vicini. Il capitolo ne descrive l'algoritmo, il compromesso tra underfitting e overfitting governato da k, il legame con il classificatore ottimo di Bayes, il bias induttivo e i limiti, in particolare la maledizione della dimensionalità.

6.1 Apprendimento eager e lazy

Rispetto al momento in cui avviene l'apprendimento si distinguono due approcci:

- **eager** ("avido"): si analizzano i dati di training e si costruisce un'**ipotesi esplicita**, come nel modello lineare;
- **lazy** ("pigro"): si memorizzano i dati e si aspetta un punto di test; solo allora si costruisce un'ipotesi ad hoc per classificare quel punto.

6.2 L'algoritmo

Algoritmo — 1-nearest neighbor.

1. Memorizzare i dati di training $\langle \mathbf{x}_p, y_p \rangle$, $p = 1, \dots, l$.
2. Dato un input $\mathbf{x}$, trovare l'esempio di training più vicino

$$i(\mathbf{x}) = \arg\min_p d(\mathbf{x}, \mathbf{x}_p),$$

   ad esempio con la distanza euclidea $d(\mathbf{x}, \mathbf{x}_p) = \sqrt{\sum_{t=1}^{n} (x_t - x_{p,t})^2} = \|\mathbf{x} - \mathbf{x}_p\|$.
3. Restituire $y_{i(\mathbf{x})}$.

Il 1-NN è **molto flessibile**: sul training set commette **zero errori**, perché ogni punto è il vicino più prossimo di se stesso. Il confine di decisione non è lineare ed è molto irregolare, forse inutilmente rumoroso.

Il 1-NN usa implicitamente il **diagramma di Voronoi**: ogni cella contiene i punti dello spazio più vicini a un certo esempio che a qualsiasi altro, e i lati delle celle sono i punti equidistanti da due esempi. Il 1-NN assegna a ogni cella l'etichetta del suo esempio, quindi il suo confine di decisione è formato dai lati che separano celle di classi diverse (Figura 6.1).

6.2.1 K vicini

Un modo naturale di classificare un nuovo punto è guardare i suoi k vicini e farne la media:

$$\text{avg}_k(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} y_i,$$

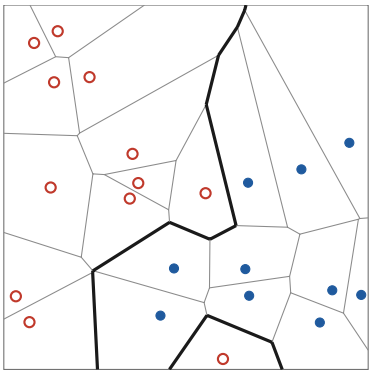


Figura 6.1. Diagramma di Voronoi di 22 esempi di due classi: in grigio i lati tra celle della stessa classe, in nero quelli tra celle di classi diverse, che formano il confine di decisione del 1-NN.

dove $N_k(\mathbf{x})$ è l'insieme dei k esempi più vicini a $\mathbf{x}$ secondo la distanza d. Se nell'intorno di $\mathbf{x}$ una classe domina chiaramente, è probabile che anche $\mathbf{x}$ vi appartenga: la regola di classificazione è il **voto a maggioranza** tra i membri di $N_k(\mathbf{x})$. Con target in $\{0, 1\}$

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathrm{avg}_k(\mathbf{x}) > 0{,}5 \\ 0 & \text{altrimenti;} \end{cases}$$

per la **regressione** si usa direttamente la media dei target dei k vicini. Aumentare k "liscia" la decisione, il che è utile con dati rumorosi (Figura 6.2).

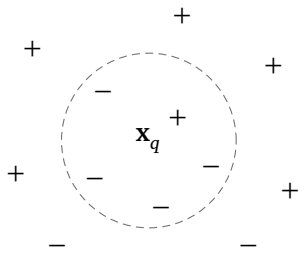


Figura 6.2. Il vicino più prossimo di $\mathbf{x}_q$ è positivo, quindi il 1-NN risponde "+"; tra i 5 vicini (nel cerchio) prevalgono i negativi, quindi il 5-NN risponde "−".

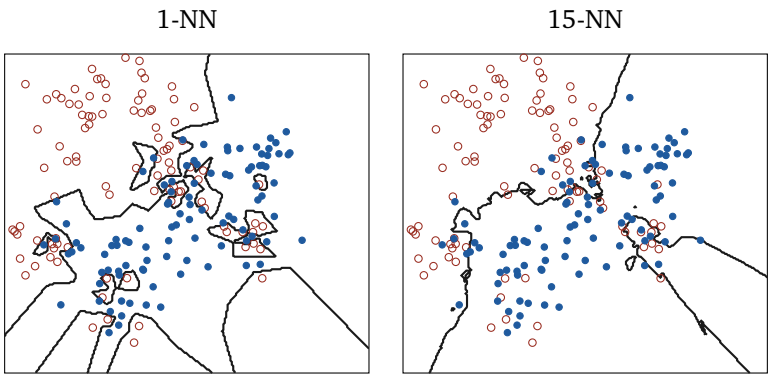


Figura 6.3. Confini di decisione del 1-NN e del 15-NN sul problema a due classi del § 4.5.3.

Con $k = 15$ il modello è ancora molto flessibile ma commette qualche errore sul training; il confine è meno irregolare e si **adatta alle densità locali** delle classi (Figura 6.3).

6.2.2 Varianti

Multi-classe: si restituisce la classe più frequente tra i k vicini,

$$h(\mathbf{x}) = \arg\max_{v} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} 1_{v,y_i}, \qquad 1_{v,y_i} = \begin{cases} 1 & \text{se } v = y_i \\ 0 & \text{altrimenti.} \end{cases}$$

Distanza pesata: i vicini più vicini contano di più,

$$h(\mathbf{x}) = \arg\max_{v} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \frac{1_{v,y_i}}{d(\mathbf{x}, \mathbf{x}_i)^2},$$

e se $d(\mathbf{x}, \mathbf{x}_i) = 0$ per qualche $\mathbf{x}_i$ si restituisce direttamente y_i.

6.3 Il comportamento al variare di k

Anche nel K-NN ritroviamo il **compromesso tra underfitting e overfitting**, governato da k, tra due estremi:

- $k = 1$: il caso **estremamente flessibile** (errore di training nullo, confine frastagliato, rischio di overfitting);
- $k = l$ (tutti i dati): un modello **rigidissimo**, che risponde con un'unica media per tutti gli input (in classificazione, la classe più frequente nel training set).

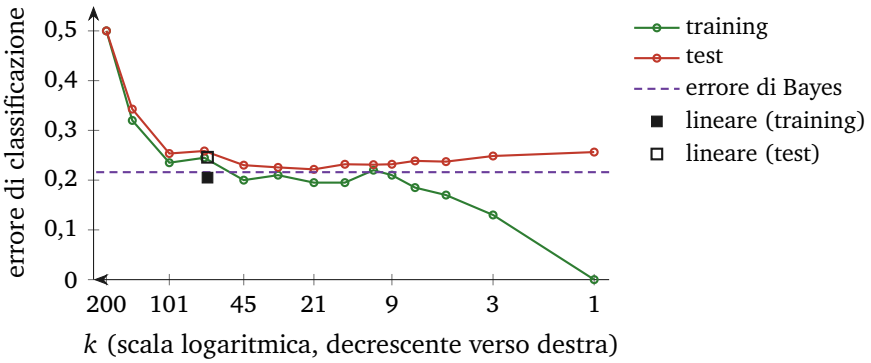


Figura 6.4. Errore di training e di test del K-NN sul problema a due classi (200 esempi di training, 10 000 di test). Da $k = l$ (rigido, a sinistra) a $k = 1$ (flessibile, a destra) l'errore di training scende fino a zero, quello di test ha forma a U, con il minimo per $k = 21$. I quadratini sono il modello lineare (posto in corrispondenza dei suoi 3 parametri, $l/k \approx 3$), la linea tratteggiata l'errore di Bayes (0,216).

È lo stesso andamento del fitting polinomiale al crescere del grado (Figura 5.2) e del bound della SLT (Figura 7.3): qui è k, letto da destra verso sinistra (cioè l/k), a fare da controllo della complessità, e i valori intermedi danno il miglior compromesso (Figura 6.4). Si potrebbe pensare che il K-NN abbia un solo parametro (k), ma realisticamente ha circa l/k **parametri effettivi**: è come se dividesse lo spazio in l/k regioni, ognuna con la sua media.

6.4 K-NN contro modello lineare

Il K-NN non costruisce un'ipotesi globale valida per tutte le istanze: **non c'è un modello da addestrare**, ma bisogna memorizzare gli esempi. Fa stime **locali**, con funzioni localmente costanti,

Tabella 6.1. I due estremi a confronto.

	Lineare	K-NN
Flessibilità	rigido (**bassa varianza**)	flessibile (**alta varianza**)
Timing	eager	lazy
Tipo	parametrico	instance-based
Parametri	$n + 1$ (3 nell'esempio)	circa l/k parametri effettivi

invece di un'approssimazione lineare globale: è un metodo lazy, basato sulla memoria, sulle istanze e sulle distanze. Con k piccolo bastano pochi punti per cambiare il confine: la flessibilità si paga.

6.5 Il classificatore ottimo di Bayes

Definizione — Classificatore ed errore di Bayes. Se si conosce la densità $P(\mathbf{x}, y)$, la scelta ottima è assegnare la classe più probabile:

$$h(\mathbf{x}) = \arg\max_{v} P(v \mid \mathbf{x}), \qquad v \in \{C_1, \dots, C_K\}.$$

Il suo tasso d'errore, il **Bayes rate**, è il **minimo errore raggiungibile** data la distribuzione dei dati.

Il K-NN approssima direttamente questa soluzione: il voto a maggioranza in un intorno è una stima di $\arg\max_v P(v \mid \mathbf{x})$ con due rilassamenti:

1. la probabilità condizionata **in un punto** diventa la probabilità condizionata **in un intorno** del punto (approssimazione locale);
2. le probabilità sono stimate con le **proporzioni** nel campione di training.

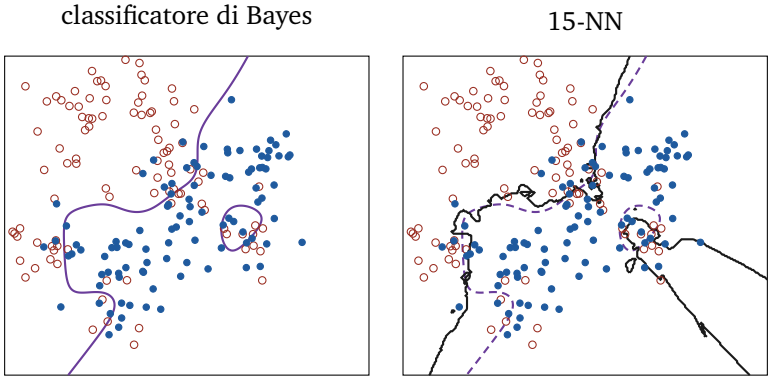


Figura 6.5. Il confine ottimo di Bayes (viola), calcolato conoscendo la densità generatrice, e il confine del 15-NN (nero): il secondo è vicino al primo, coerentemente con il minimo dell'errore di test della Figura 6.4.

6.6 Bias induttivo e limiti del K-NN

6.6.1 Il bias induttivo

1. **La distanza assunta** stabilisce quali esempi sono simili: si assume che la classificazione di un punto sia simile a quella dei vicini *secondo la metrica scelta*. Si possono usare metriche diverse dall'euclidea; dati simbolici richiedono metriche ad hoc, come la **distanza di Hamming** tra stringhe della stessa lunghezza (il numero di posizioni in cui differiscono). Questo bias è spesso sottovalutato, ma è cruciale.
2. **Località**: approssimazione locale, con un'assunzione di *smoothness* locale (da confrontare con il bias del deep learning, § 15.2.1).

6.6.2 Scala delle variabili

La scelta della scala dipende dalla conoscenza del dominio. Se le variabili devono contribuire allo stesso modo bisogna fare attenzione ai loro range: **riscalare i dati** (ad esempio a media zero e varianza unitaria) **equivale a cambiare la metrica**, e può cambiare del tutto quale sia il vicino più prossimo (Figura 6.6). Il K-NN è fragile anche rispetto a un pre-processing elementare.

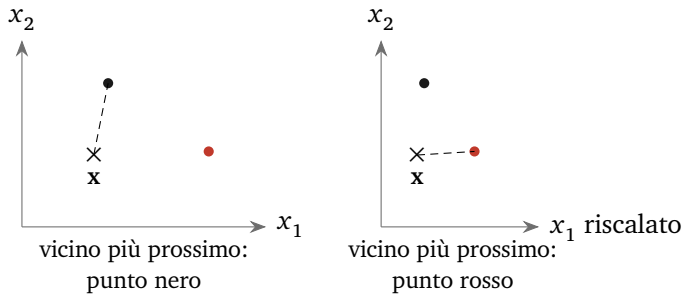


Figura 6.6. Dimezzando la scala di x_1 cambia il vicino più prossimo di **x**.

6.6.3 Costo computazionale e interpretabilità

Il K-NN costruisce l'approssimazione locale **per ogni nuovo esempio**: il costo è **spostato nella fase di predizione**. Per ogni input si calcolano le distanze da tutti gli esempi memorizzati (tempo proporzionale a l), anche se esistono algoritmi di ricerca di prossimità ad hoc, come l'indicizzazione degli esempi o la ricerca **approssimata** dei vicini, che rinuncia a trovare sempre il vicino esatto in cambio di una ricerca molto più veloce. Anche il costo in **spazio** è alto (tutto il training set). I modelli K-NN offrono infine poca interpretazione, soggettiva e dipendente dalla metrica.

6.6.4 La maledizione della dimensionalità

Il K-NN funziona bene se per ogni **x** si trova un insieme significativo di dati vicini, cioè con un **campionamento denso**. Con molte variabili di input fallisce spesso per la **curse of dimensionality**, che ha tre manifestazioni rilevanti per la generalizzazione.

1. In alta dimensione è difficile trovare punti vicini. Si considerino dati uniformi nel cubo unitario di $\mathbb{R}^n$ e un sottocubo che contenga una frazione r dei dati. Poiché il volume è lato^n, il lato deve essere

$$\text{lato} = r^{1/n}.$$

- Per catturare il 10% dei dati: in 2D basta un lato $0{,}1^{1/2} \approx 0{,}32$ (il 32% del range di ogni variabile); in 10D serve $0{,}1^{1/10} \approx 0{,}8$, l'**80% del range** di ogni coordinata. Anche per l'1% del volume in 10D serve il 63% del range: l'intorno non è più "locale" e i vicini non sono più simili.
- Viceversa, fissato il lato a 0,3: si cattura il 30% dei dati in 1D, il 9% in 2D, il 2,7% in 3D, circa lo 0,0006% in 10D. Quel lato può non bastare a trovare k dati, a meno di usare k piccolo (e quindi rischiare overfitting: la varianza della stima aumenta).

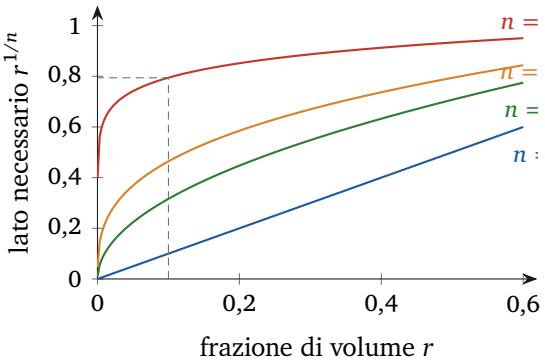


Figura 6.7. Lato del sottocubo necessario per catturare una frazione r del volume del cubo unitario, al variare della dimensione n: in 10 dimensioni il 10% del volume richiede un lato di circa 0,8.

2. Bassa densità di campionamento. La densità di campionamento è proporzionale a $l^{1/n}$: se 100 punti bastano a stimare una funzione in $\mathbb{R}^1$, per un'accuratezza simile in $\mathbb{R}^{10}$ ne servono $100^{10} = 10^{20}$.

3. Feature irrilevanti. Se il target dipende solo da poche delle molte feature (ad esempio 2 su 20), la somiglianza tra esempi può essere dominata dalle feature irrilevanti, e il "vicino" trovato non è davvero simile. Il problema cresce con la dimensione. Rimedi: **pesare le feature** secondo la loro rilevanza (allungando gli assi lungo alcune dimensioni; i pesi si cercano con una costosa model selection o con altri metodi) oppure fare **feature selection**, riducendo la dimensione dell'input.

6.7 Scelte di progetto

- La **metrica** d (euclidea, Hamming, Manhattan, pesi sulle feature, …): spesso è la chiave del successo di un'applicazione.
- Il valore di k, che controlla underfitting e overfitting.
- Spesso conviene selezionare un **sottoinsieme dei dati** (un insieme di prototipi, ad esempio con un clustering) e un **sottoinsieme delle feature**.

Estensioni ad altri modelli locali: *kernel smoothers*, regressione lineare locale, metodi a prototipi, *case-based reasoning*.

> **Cosa manca al K-NN.** Il K-NN non costruisce un vero modello: non estrae regolarità né sintetizza conoscenza, e non permette di "dimenticare" gli esempi dopo l'apprendimento. L'obiettivo dei capitoli successivi sono modelli **compatti** come la LTU (tutta la conoscenza in pochi parametri) ma **flessibili** come il K-NN, con un adeguato controllo della complessità: le reti neurali e le SVM.

Punti chiave

- K-NN: voto a maggioranza (o media) dei k esempi più vicini; metodo lazy, locale, basato sulla memoria.
- $k = 1$: errore di training nullo, alta varianza; $k = l$: una sola media, rigido. Circa l/k parametri effettivi; errore di test a U in k.
- Il K-NN approssima il classificatore di Bayes, $\arg\max_v P(v \mid \mathbf{x})$, in un intorno.
- Bias induttivo: metrica e località. Limiti: scala delle variabili, costo in predizione, maledizione della dimensionalità.

Domande di verifica

1. Descrivere il K-NN e confrontarlo con il modello lineare.
2. Come cambiano errore di training e di test al variare di k?
3. Che cos'è il classificatore ottimo di Bayes e che legame ha con il K-NN?
4. Qual è il bias induttivo del K-NN?
5. Spiegare la maledizione della dimensionalità.

Parte III

Generalizzazione: teoria e stima

Capitolo 7

Statistical Learning Theory

La Statistical Learning Theory (SLT) dà una risposta formale alla domanda centrale del corso: quando l'errore sui dati di training è una buona stima dell'errore sui dati futuri? Il capitolo definisce rischio e rischio empirico, misura la complessità di uno spazio delle ipotesi con la VC-dimension, presenta il bound sul rischio e il principio della Structural Risk Minimization, da cui nascono le SVM.

7.1 Rischio e rischio empirico

Si vuole approssimare una funzione sconosciuta $f(\mathbf{x})$, con target $d = f(\mathbf{x})$ + rumore. Idealmente si vorrebbe minimizzare la **funzione di rischio**, l'errore vero calcolato su *tutti* i dati possibili:

$$R = \int L\big(d, h(\mathbf{x})\big)\, dP(\mathbf{x}, d), \tag{7.1}$$

dove $P(\mathbf{x}, d)$ è la distribuzione (sconosciuta) dei dati e L una loss, ad esempio $L(h(\mathbf{x}), d) = (d - h(\mathbf{x}))^2$. Si cerca $h \in H$ che minimizzi R. Si dispone però solo di un dataset finito $TR = \{(\mathbf{x}_p, d_p)\}_{p=1}^{l}$, e si minimizza il **rischio empirico** (errore di training):

$$R_{\text{emp}}(h, TR) = \frac{1}{l} \sum_{p=1}^{l} \big(d_p - h(\mathbf{x}_p)\big)^2. \tag{7.2}$$

È il principio induttivo di **Empirical Risk Minimization** (ERM). La domanda cruciale è: **si può usare R_{emp} per approssimare R?** Per rispondere serve una misura della complessità di H.

Le alternative per stimare l'errore atteso sono di due tipi:

- **analitiche**: criteri AIC e BIC (*Akaike/Bayesian Information Criterion*, limitati ai modelli lineari nei parametri), MDL (*Minimum Description Length*), **SRM con la VC-dimension** (questo capitolo);
- **per ricampionamento**: cross-validation e bootstrap (capitolo 9).

7.2 La VC-dimension

Per un bound analitico serve misurare la **complessità** di H (capacità, potere espressivo, flessibilità) anche quando H è **infinito**, come nei modelli lineari con pesi reali: non si può usare $|H|$. L'idea di Vapnik e Chervonenkis è misurare **il numero di istanze distinte che H riesce a discriminare completamente**: nel caso della classificazione, il numero massimo di punti che si possono classificare senza errori *per tutte le possibili etichettature*.

Sia X lo spazio delle istanze e si considerino N punti (attenzione: qui N è il numero di punti considerati, che può differire dal numero di dati l). Con due classi esistono 2^N possibili **dicotomie** (etichettature dei punti con -1 o $+1$); una dicotomia è **rappresentata** in H se esiste $h \in H$ che la realizza.

Definizione — Shattering. H **frammenta** (*shatters*) un insieme di punti se e solo se può rappresentare **tutte** le possibili dicotomie su di esso: per ogni etichettatura esiste un'ipotesi $h \in H$ consistente con essa (errore zero).

Definizione — VC-dimension. La **VC-dimension** di una classe di funzioni H è la **cardinalità massima di un insieme di punti** (in una qualche configurazione) che può essere frammentato da H. $\mathrm{VC}(H) = p$ significa che

- H frammenta **almeno un** insieme di p punti;
- H **non** frammenta **nessun** insieme di $p + 1$ punti.

Se si possono frammentare insiemi arbitrariamente grandi, $\mathrm{VC}(H) = \infty$.

Attenzione — Asimmetria della definizione. Per il limite inferiore basta mostrare **una** configurazione frammentabile; per il limite superiore bisogna mostrare che **nessuna** configurazione di $p + 1$ punti lo è.

7.2.1 La VC-dimension delle rette nel piano

Teorema — Rette nel piano. *Per gli iperpiani separatori (LTU) in* $\mathbb{R}^2$*,* $h(\mathbf{x}) = \mathrm{sign}(\mathbf{w}^T\mathbf{x} + w_0)$*, vale* $\mathrm{VC}(H) = 3$.

Dimostrazione. $\mathrm{VC}(H) \geq 3$**.** Si prendano 3 punti non allineati. Le loro $2^3 = 8$ dicotomie sono tutte separabili da una retta (Figura 7.1): quando i tre punti hanno la stessa etichetta basta una retta che li lasci tutti dalla stessa parte; altrimenti un punto ha etichetta diversa dagli altri due, ed essendo i punti non allineati esiste una retta che isola quel vertice del triangolo dagli altri due. Non tutte le configurazioni di 3 punti funzionano (tre punti allineati con quello centrale di classe diversa non sono separabili), ma ne basta una.

$\mathrm{VC}(H) < 4$**.** Dati 4 punti qualsiasi del piano, si distinguono due casi.

- I 4 punti formano un quadrilatero convesso: le due diagonali si incrociano. Si assegnano le estremità di una diagonale alla classe $+1$ e quelle dell'altra alla classe -1.
- Un punto è interno al triangolo formato dagli altri tre: si assegna al punto interno una classe e agli altri tre la classe opposta.
- (Caso degenere) Tre dei punti sono allineati: si assegna al punto centrale una classe diversa da quella dei due estremi, come nella Figura 4.6.

In entrambi i casi l'etichettatura non è separabile da una retta: nel primo è uno XOR (i segmenti che uniscono i punti della stessa classe si incrociano, Figura 7.2); nel secondo ogni semipiano che contiene i tre vertici contiene anche il triangolo, e quindi il punto interno. Nessun insieme di 4 punti è frammentato. ■

Teorema — VC-dimension degli iperpiani. *La VC-dimension della classe degli iperpiani separatori (LTU) in uno spazio n-dimensionale è* $n + 1$.

È un valore massimo: vincolando il modello (ad esempio limitando la norma dei pesi, come nelle SVM, § 13.1.5) può diminuire.

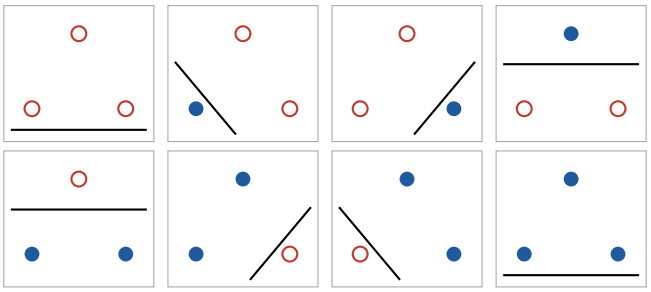


Figura 7.1. Le $2^3 = 8$ dicotomie di tre punti non allineati (pieno $= +1$, vuoto $= -1$), ciascuna con una retta che la realizza: le rette frammentano questi tre punti.

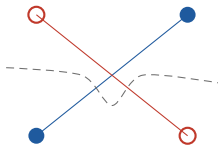


Figura 7.2. Con 4 punti esiste sempre un'etichettatura non separabile da una retta: qui i segmenti che uniscono i punti della stessa classe si incrociano (XOR) e serve un confine non lineare.

7.2.2 Altri esempi

Tabella 7.1. VC-dimension di alcune classi di ipotesi.

Classe H	VC
Soglie su $\mathbb{R}$, $\text{sign}(w_1 x + w_0)$	2
Intervalli su $\mathbb{R}$	2
Cerchi centrati nell'origine in 2D, $\text{sign}(\mathbf{x}^T\mathbf{x} - b)$	1
Cerchi centrati nell'origine, $\text{sign}(k\,\mathbf{x}^T\mathbf{x} - b)$	2
Cerchi in 2D (raggio e centro liberi)	3
Cerchi in 2D (raggio fisso, centro libero)	3

Per i cerchi centrati nell'origine con il solo parametro b, il cerchio include i punti vicini all'origine: con due punti a distanze diverse non si può etichettare "+" il più lontano e "−" il più vicino, quindi VC $= 1$. Aggiungendo k si può scambiare l'interno con l'esterno, e la VC-dimension sale a 2.

7.2.3 VC-dimension e numero di parametri

La VC-dimension è legata al numero di parametri liberi, ma **non coincide** con esso:

- nell'esempio dei cerchi, classi con lo **stesso** numero di parametri hanno VC-dimension diverse, e viceversa;
- si possono aggiungere parametri **ridondanti** senza cambiare nulla;
- esistono modelli con **un solo parametro e VC-dimension infinita**: l'esempio classico è $\text{sign}(\sin(\alpha x))$, che con un α abbastanza grande frammenta qualsiasi insieme di punti opportunamente scelti;
- per il **nearest neighbor** la VC-dimension è **infinita**: il 1-NN classifica correttamente qualunque etichettatura del training set.

La VC-dimension può essere difficile da calcolare per modelli con parametri non lineari.

7.3 Il bound sul rischio

Teorema — VC-bound. *Con probabilità almeno* $1-\delta$*, per ogni* $h \in H$ *(con* $\mathrm{VC} < N$*, dove* N *è il numero di dati):*

$$\underbrace{R[h]}_{\text{rischio}} \;\le\; \underbrace{R_{\text{emp}}[h] + \varepsilon(\mathrm{VC}, N, \delta)}_{\text{rischio garantito}} \tag{7.3}$$

dove ε *è la* ***VC-confidence****. Ad esempio, per la loss 0/1:*

$$\varepsilon(\mathrm{VC}, N, \delta) = \sqrt{\frac{\mathrm{VC}\left(\ln \frac{2N}{\mathrm{VC}} + 1\right) - \ln \frac{\delta}{4}}{N}}.$$

δ è la **confidenza**: con $\delta = 0{,}01$ il bound vale con probabilità 0,99. Esistono formulazioni diverse per diverse classi di funzioni e task, scritte in generale come $R \le R_{\text{emp}} + \varepsilon\left(\frac{1}{l}, \mathrm{VC}, \frac{1}{\delta}\right)$ o $R \le R_{\text{emp}} + \varepsilon\left(\frac{\mathrm{VC}}{l}, -\frac{\ln \delta}{l}\right)$. La VC-confidence **cresce con la VC-dimension** e **decresce con il numero di dati**: a meno di logaritmi dipende dal rapporto VC/N, quindi raddoppiare i dati ha lo stesso effetto di dimezzare la complessità. Il bound permette di **stimare l'errore sui dati futuri** usando solo l'errore di training e la VC-dimension, e ε si può calcolare **prima** dell'apprendimento.

Il bound spiega underfitting e overfitting.

- **Più dati** (l alto): ε diminuisce e il bound si avvicina a R: il rischio empirico diventa una buona stima di quello vero.
- **Modello troppo semplice** (VC bassa): ε piccolo ma R_{emp} alto: **underfitting**.
- **Modello troppo complesso** (VC alta, l fissato): R_{emp} basso ma ε alto, e con esso il bound su R: **overfitting**.

In parole semplici: si può approssimare bene f dagli esempi purché si abbiano abbastanza dati e la complessità del modello sia adatta al problema. Bisogna adattarsi ai dati abbastanza da evitare l'underfitting, ma non troppo, per evitare l'overfitting.

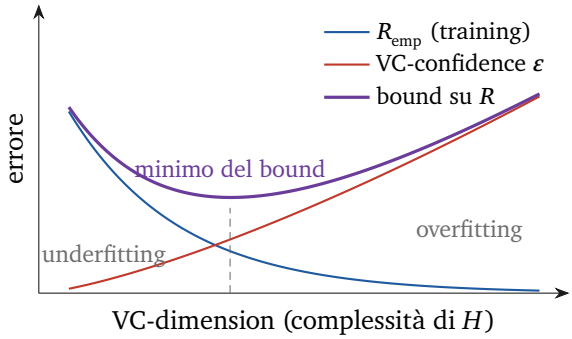


Figura 7.3. A numero di dati fissato: il rischio empirico decresce con la VC-dimension, la VC-confidence cresce; la loro somma, il bound su R, ha un minimo nel punto di miglior compromesso (andamento qualitativo).

Esempio — Ritrovare h e h'. Nella definizione di overfitting (§ 5.1), h ha errore di training minore ma rischio maggiore di h'. Sul grafico della Figura 7.3: h sta **a destra del minimo** (VC più alta, R_{emp} più basso, bound più alto); h' sta vicino al minimo.

Attenzione — Bound "nel caso peggiore". I bound devono valere per tutte le funzioni di H e tutti i training set, tranne una frazione δ: sono *worst case* e spesso molto larghi. La VC-confidence può essere **centinaia di volte** più grande dell'effetto di overfitting osservato empiricamente.

Una conseguenza pratica. Per molte classi ragionevoli (ad esempio gli approssimatori lineari) la VC-dimension è **lineare nel numero di parametri liberi**. Quindi per apprendere "bene" serve un numero di esempi **lineare nella VC-dimension**, cioè nel numero di parametri: è una giustificazione teorica di euristiche come "servono almeno k esempi per parametro".

7.4 Structural Risk Minimization

Definizione — Structural Risk Minimization. La **SRM** usa la VC-dimension come parametro di controllo e, invece di minimizzare solo R_{emp}, **minimizza il bound**, cioè la somma di errore di training e VC-confidence. Si definisce una **struttura annidata** di spazi delle ipotesi ordinati per VC-dimension (finita)

$$H_1 \subseteq H_2 \subseteq \cdots \subseteq H_n, \qquad \text{VC}(H_1) \leq \ldots \leq \text{VC}(H_n),$$

e si sceglie lo spazio (e in esso l'ipotesi) con il **miglior bound sul rischio vero**.

La SRM formalizza il **controllo della complessità**: un compromesso tra complessità (VC-dimension) e accuratezza sul training (*fitting*). Esempi di strutture annidate:

- reti neurali con numero crescente di **unità nascoste** (circa), ma anche con numero crescente di **epoche**, perché conta anche la grandezza dei pesi (§ 11.2.4);
- polinomi di **grado** M crescente;
- valori crescenti di c con il vincolo $\|\mathbf{w}\| < c$, cioè la **regolarizzazione** (controllata equivalentemente da λ: aumentare λ equivale a spostarsi verso spazi più piccoli);
- alberi di decisione con numero crescente di nodi (circa).

Gli **iperparametri importanti** di un modello sono proprio quelli legati alla VC-dimension.

7.4.1 A cosa serve il bound

- Dà un **fondamento teorico** al ML, indipendente dai dettagli del modello e dell'algoritmo: il rischio si può limitare analiticamente, il controllo della complessità ha un ruolo preciso, e la scelta ottima della complessità sulla struttura minimizza il rischio atteso (il **principio induttivo della SRM**). Spiega anche la differenza tra il ML e l'ottimizzazione pura: nel ML l'obiettivo non è minimizzare l'errore sui dati, ma sui dati *futuri*.
- Come **stima dell'errore** è usato raramente:
  - può essere molto largo, ma resta utile per la **model selection**, dove conta la **dimensione relativa** (il *ranking*) degli errori;
  - è un upper bound probabilistico valido per tutte le funzioni della classe, quindi si può cercare nella classe;
  - la VC-dimension di classi specifiche è difficile da calcolare;
  - il bound grezzo può non essere adeguato per una valutazione affidabile (**model assessment**); si studiano bound più stretti.
- Indica una direzione per sviluppare **nuovi modelli** guidati dalla SRM.

7.4.2 Due approcci pratici alla SRM

1. **Fissare la complessità** (quindi la VC-confidence) e **minimizzare l'errore di training**. È l'approccio delle reti neurali: alcune euristiche di addestramento realizzano già una SRM **implicita** (l'early stopping), oppure la si realizza con la **regolarizzazione di Tikhonov**, minimizzando R_{emp} più un termine di complessità.
2. **Fissare l'errore di training** e **minimizzare automaticamente la VC-confidence**. È l'idea delle **SVM** (capitolo 13).

Attenzione — Oltre il compromesso classico. Fenomeni più recenti, tipici delle reti profonde sovra-parametrizzate (*double descent*, *benign overfitting*), sembrano sfidare questa visione; sono discussi nel § 15.5.

Punti chiave

- ERM: si minimizza R_{emp} al posto di R; la domanda è quanto R_{emp} approssimi R.
- VC-dimension: il massimo numero di punti che H frammenta; $n + 1$ per gli iperpiani in $\mathbb{R}^n$; non coincide con il numero di parametri (infinita con un solo parametro, infinita per il NN).
- VC-bound: $R \leq R_{\text{emp}} + \varepsilon(\text{VC}, N, \delta)$, con ε crescente in VC e decrescente in N.
- SRM: su una struttura annidata $H_1 \subseteq H_2 \subseteq \ldots$ si sceglie lo spazio con il bound minimo. Due realizzazioni: complessità fissa + minimo R_{emp} (Tikhonov), R_{emp} fisso + minima VC-confidence (SVM).

Domande di verifica

1. Differenza tra rischio R e rischio empirico R_{emp}; cos'è il principio ERM?
2. Definire shattering e VC-dimension; dimostrare che la VC-dimension delle rette nel piano è 3.
3. La VC-dimension coincide con il numero di parametri liberi? Esempi.
4. Scrivere il VC-bound e spiegare come giustifica underfitting e overfitting.
5. Cos'è la Structural Risk Minimization? Esempi di strutture annidate. Limiti pratici del bound.

Capitolo 8

Bias, varianza ed ensemble

La SLT spiega underfitting e overfitting con la complessità del modello e il numero di dati. La decomposizione bias-varianza dà un punto di vista complementare: il training set è una *delle possibili realizzazioni dei dati, e l'errore atteso si scompone in bias, varianza e rumore. Da qui si arriva agli ensemble (bagging, boosting), che sfruttano la riduzione della varianza, e a qualche cenno sulla feature selection.*

8.1 L'idea

Training set diversi, estratti dalla stessa distribuzione, producono ipotesi diverse. L'errore atteso in un punto $\mathbf{x}$ (mediato sui training set) si scompone in tre parti:

- **bias**: la discrepanza tra la funzione vera e la predizione $h(\mathbf{x})$ *media*. È alto se H è troppo piccolo, cioè se il modello è troppo rigido;
- **varianza**: la variabilità della risposta del modello al variare del training set, dovuta a un'eccessiva flessibilità;
- **rumore**: le etichette contengono un errore casuale (per uno stesso $\mathbf{x}$ ci possono essere più valori di d).

Si studia la regressione con target y e loss quadratica.

> **Esempio — 20 punti e 50 rette.** Sia $y = x + 2\sin(1{,}5x) + \varepsilon$ con ε rumore gaussiano di media zero (varianza 0,2), e si adatti ai dati una retta $h(x) = w_1 x + w_0$ ai minimi quadrati. Due fatti:
>
> 1. la classe di ipotesi (rette) commette per questa f un **errore sistematico**;
> 2. a seconda del dataset, i parametri trovati **sono diversi**: ripetendo con 50 dataset di 20 punti si ottengono 50 rette diverse (Figura 8.1).

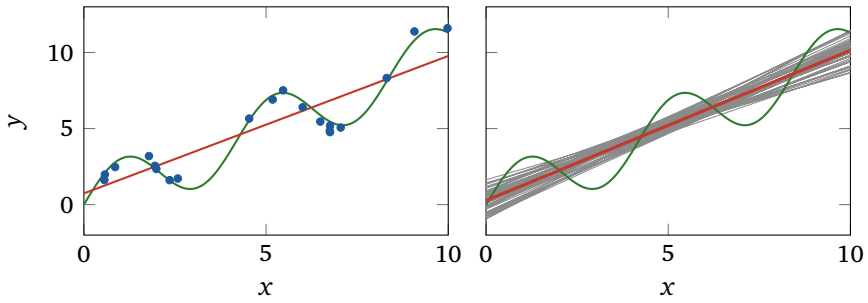


Figura 8.1. A sinistra un dataset di 20 punti, la funzione vera (verde) e la retta adattata (rosso). A destra le 50 rette ottenute da 50 dataset diversi (grigio) e la loro media $\bar{h}$ (rosso): la distanza tra $\bar{h}$ e la funzione vera è il bias, la dispersione delle rette attorno a $\bar{h}$ la varianza.

8.2 La decomposizione

Si assume che i dati siano estratti in modo **i.i.d.** da un'unica distribuzione P. Per un punto $\mathbf{x}$ arbitrario si vuole calcolare l'**errore di predizione atteso**

$$\mathbb{E}_P\big[(y - h(\mathbf{x}))^2\big],$$

dove y è il target di $\mathbf{x}$ che potrebbe comparire in un dataset e l'aspettativa è **su tutti i training set** estratti secondo P: a ogni training set corrisponde un'ipotesi h diversa (e un y diverso). Sia

$$\bar{h}(\mathbf{x}) = \mathbb{E}_P[h(\mathbf{x})]$$

la **predizione media** in $\mathbf{x}$ delle ipotesi addestrate su dati estratti da P.

Teorema — Decomposizione bias-varianza.

$$\boxed{\mathbb{E}_P\big[(y - h(\mathbf{x}))^2\big] = \underbrace{\mathbb{E}_P\big[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\big]}_{\text{varianza}} + \underbrace{\big(\bar{h}(\mathbf{x}) - f(\mathbf{x})\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_P\big[(y - f(\mathbf{x}))^2\big]}_{\text{rumore}^2 = \sigma^2}} \tag{8.1}$$

Dimostrazione. 1. **Si sviluppa il quadrato** e si usa la linearità del valore atteso:

$$\mathbb{E}_P\big[(y - h(\mathbf{x}))^2\big] = \mathbb{E}_P\big[h(\mathbf{x})^2 - 2y\,h(\mathbf{x}) + y^2\big] = \mathbb{E}_P\big[h(\mathbf{x})^2\big] + \mathbb{E}_P\big[y^2\big] - 2\,\mathbb{E}_P[y\,h(\mathbf{x})].$$

2. **Termine misto.** y è il target di un dato nuovo, mentre h dipende solo dal training set: le due variabili sono indipendenti, quindi $\mathbb{E}_P[y\,h(\mathbf{x})] = \mathbb{E}_P[y]\,\mathbb{E}_P[h(\mathbf{x})] = \mathbb{E}_P[y]\,\bar{h}(\mathbf{x})$. Inoltre $\mathbb{E}_P[y] = \mathbb{E}_P[f(\mathbf{x}) + \varepsilon] = f(\mathbf{x})$, perché il rumore ha media zero. Il termine misto vale $-2f(\mathbf{x})\bar{h}(\mathbf{x})$.
3. **Primo termine**, con il lemma della varianza (§ 1.3) applicato a $Z = h(\mathbf{x})$:

$$\mathbb{E}_P\big[h(\mathbf{x})^2\big] = \mathbb{E}_P\big[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\big] + \bar{h}(\mathbf{x})^2.$$

4. **Secondo termine**, con il lemma applicato a $Z = y$, la cui media è $f(\mathbf{x})$:

$$\mathbb{E}_P\big[y^2\big] = \mathbb{E}_P\big[(y - f(\mathbf{x}))^2\big] + f(\mathbf{x})^2.$$

5. **Si raccoglie**:

$$\begin{aligned}\mathbb{E}_P\big[(y - h(\mathbf{x}))^2\big] &= \mathbb{E}_P\big[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\big] + \underbrace{\bar{h}(\mathbf{x})^2 - 2f(\mathbf{x})\bar{h}(\mathbf{x}) + f(\mathbf{x})^2}_{=(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2} + \mathbb{E}_P\big[(y - f(\mathbf{x}))^2\big] \\ &= \mathrm{Var}[h(\mathbf{x})] + \mathrm{Bias}[h(\mathbf{x})]^2 + \mathbb{E}_P[\varepsilon^2],\end{aligned}$$

e $\mathbb{E}_P[\varepsilon^2] = \sigma^2$ è la varianza del rumore. ∎

In parole: **errore di predizione atteso = varianza + bias^2 + rumore^2**.

- **Bias** $(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2$: discrepanza tra la funzione vera e la predizione *media* sui training set. È un **errore sistematico**, dovuto a uno spazio H troppo piccolo o a un modello troppo rigido.
- **Varianza**: variabilità della risposta del modello sui diversi training set, dovuta a una **flessibilità eccessiva**.
- **Rumore**: anche la soluzione ottima sbaglia; è **irriducibile**, non dipende dal modello e rappresenta una tolleranza σ sulla risposta.

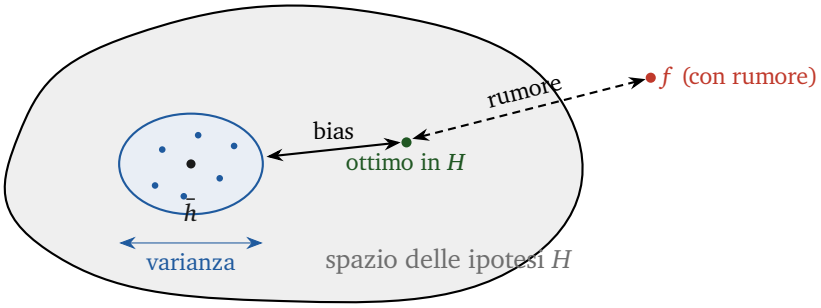


Figura 8.2. Vista grafica della decomposizione: training set diversi producono soluzioni diverse (punti blu, la cui dispersione è la varianza); la loro media $\bar{h}$ dista dalla miglior soluzione ottenibile (bias); la soluzione ottima stessa dista dai dati a causa del rumore.

Esempio — L'analogia delle freccette. Il centro del bersaglio è la funzione vera e ogni freccetta un modello addestrato su un training set diverso (Figura 8.3). **Alto bias e bassa varianza** corrisponde all'**underfitting**, **basso bias e alta varianza** all'**overfitting**. L'analogia non è perfetta per il ML, ma aiuta.

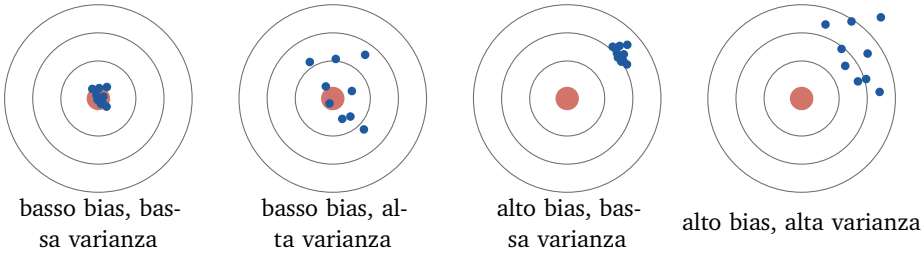


Figura 8.3. L'analogia delle freccette: il centro è la funzione vera, ogni punto un modello addestrato su un training set diverso.

8.3 Bias, varianza e regolarizzazione

Con la loss regolarizzata (§ 5.3)

$$\text{Loss}(\mathbf{w}) = \sum_p \big(d_p - o(\mathbf{x}_p)\big)^2 + \lambda \|\mathbf{w}\|^2$$

(l'intercetta è tipicamente esclusa dalla penalità), variando λ si ottengono soluzioni meno regolarizzate (complesse, λ basso) o più regolarizzate (semplici, λ alto). Nell'esperimento della Figura 8.4 si estraggono 25 dataset da 25 punti dalla sinusoide $\sin(2\pi x)$ con rumore, e su ciascuno si addestra un modello con 24 funzioni di base gaussiane per tre valori di λ.

- λ **alto**: **alto bias, bassa varianza**. La penalità domina, il modello dipende poco dai dati ed è troppo rigido.
- λ **medio**: bias più basso, varianza più alta: si dipende di più dal particolare training set.
- λ **molto basso**: **basso bias, alta varianza**. L'errore di test di un singolo modello può essere alto (per la varianza), ma *in media* le ipotesi sono corrette.

Un modello **sovra-regolarizzato** ha bias alto, uno **sotto-regolarizzato** varianza alta. Il minimo di bias^2 + varianza coincide circa con il minimo dell'errore di test (Figura 8.5).

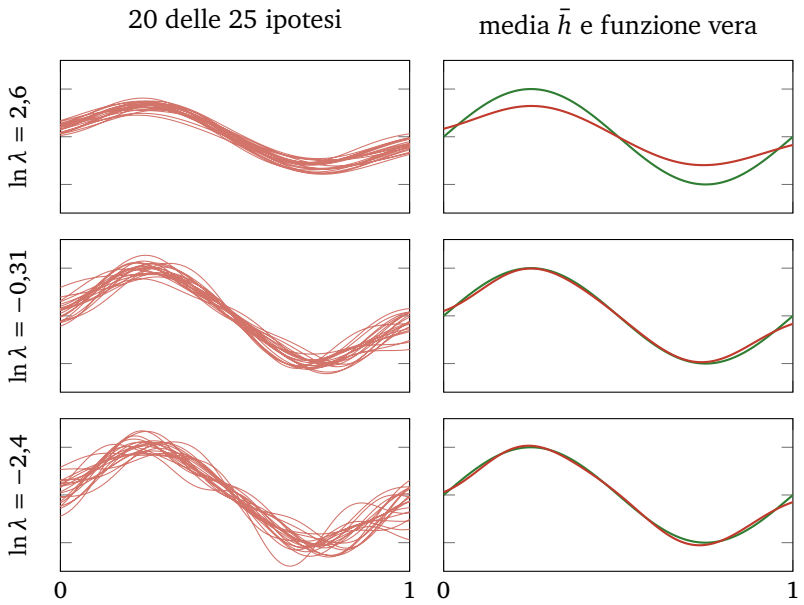


Figura 8.4. Effetto di λ su bias e varianza. A sinistra le ipotesi apprese da dataset diversi, a destra la loro media (rosso) e la funzione vera (verde). Con λ alto (in alto) le curve si somigliano ma la media è lontana dalla sinusoide; con λ basso (in basso) le curve sono molto diverse ma la media coincide quasi con la sinusoide.

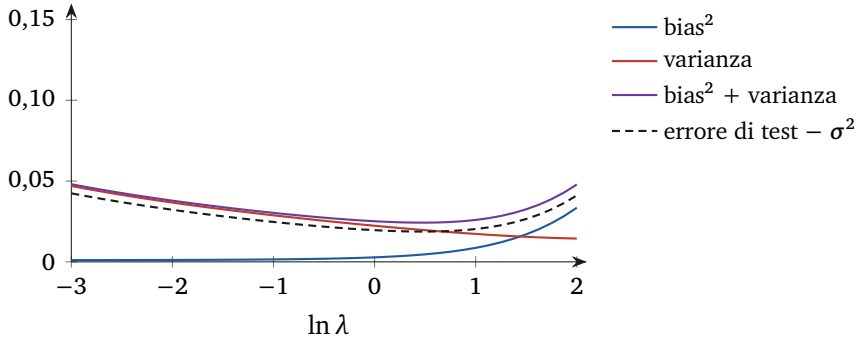


Figura 8.5. Bias^2, varianza e loro somma al variare di $\ln\lambda$, stimati su 100 dataset; la curva tratteggiata è l'errore di test medio meno la varianza del rumore ($\sigma^2 = 0{,}09$). Attenzione: qui la complessità è più alta a **sinistra**, al contrario dei grafici usuali.

Il collegamento con la SLT. Bias e varianza sono un'altra faccia del compromesso tra adattamento ai dati e complessità: il bias corrisponde all'incapacità del modello di rappresentare la funzione (alto R_{emp}, underfitting), la varianza alla sensibilità ai dati specifici (alta VC-confidence, overfitting). Una conseguenza pratica per la valutazione: a parità di complessità l'errore di test varia molto da un training set all'altro, soprattutto per i modelli complessi, e una singola misura può essere "fortunata" o "sfortunata" (capitolo 9).

8.4 Ensemble learning

L'idea degli **ensemble** è combinare le risposte di **più modelli**. Nello schema più semplice (a voto):

- **regressione**: comitato a media semplice, $o(\mathbf{x}) = \frac{1}{K}\sum_{i=1}^{K} h_i(\mathbf{x})$;
- **classificazione**: voto tra molti classificatori (purché diano risposte **diversificate**), oppure si classifica dopo aver mediato le uscite continue.

In alternativa (**stacking**) il combinatore è a sua volta un modello di ML.

Teorema — Il comitato non è peggiore della media dei suoi membri. *Per una loss convessa come l'errore quadratico, l'errore del comitato è minore o uguale alla media degli errori dei singoli modelli:*

$$\text{loss}\big(\mathbb{E}_i[h_i]\big) \leq \mathbb{E}_i\big[\text{loss}(h_i)\big]$$

(è la disuguaglianza di Jensen).

Esempio — Due modelli. Target $t = 5$. Due modelli che predicono 2 e 4: media degli errori $\frac{(5-2)^2+(5-4)^2}{2} = 5$, errore del comitato ($o = 3$) pari a $(5-3)^2 = 4$. Con predizioni 6 e 4: media degli errori $\frac{1+1}{2} = 1$, errore del comitato $(5-5)^2 = 0$.

8.4.1 Bagging

Il **bagging** (*bootstrap aggregating*) addestra K modelli su **sottoinsiemi diversi** del training set, ottenuti con il **bootstrap** (ricampionamento con reinserimento), e ne combina le risposte (media in regressione, voto in classificazione). Il legame con la decomposizione è diretto (si ricordi il caso "λ basso"): modelli ad **alta varianza e basso bias** funzionano bene *in media*. La media delle h riduce la varianza senza aumentare il bias. Inoltre il bagging tende ad aumentare il **margine** di un classificatore: la media di K curve di separazione tende a stare "nel mezzo".

8.4.2 Boosting

Se i modelli commettono gli **stessi errori**, combinarli non serve. Il **boosting** (ad esempio AdaBoost) li differenzia di proposito:

1. addestra i classificatori **in sequenza**, concentrandosi sugli **errori**: il secondo dà più peso alle istanze sbagliate dal primo (ad esempio sono più probabili nel suo training set), e così via;
2. combina i risultati con un **voto pesato**, dando più peso ai classificatori con errore più basso.

Se non lo si ferma, il boosting impara a classificare correttamente **tutte** le istanze di training usando **weak learner** (classificatori appena migliori del caso, con errore $< 1/2$ nel caso binario), e costruisce in modo incrementale modelli complessi. Può migliorare molto learner deboli, riduce il bias senza aumentare la varianza (resiste all'overfitting) e, come il bagging, tende a produrre separazioni a margine ampio. Però **soffre con dati rumorosi** (che vengono pesati sempre di più) e non risolve di per sé il problema underfitting/overfitting.

8.5 Feature selection (cenni)

Trovare il sottoinsieme delle feature più informative porta grandi benefici: **riduzione della dimensionalità** (con effetti sulla VC-dimension e sulla maledizione della dimensionalità), filtraggio di informazione irrilevante e rumore, **interpretabilità**. Però:

- è **computazionalmente difficile**: i sottoinsiemi possibili sono moltissimi e ogni valutazione richiede di riaddestrare il modello;
- si usa tipicamente una **ricerca euristica** (greedy, algoritmi genetici o altre tecniche), che può includere o meno il learner;
- i risultati sono molto sensibili al metodo di validazione: attenzione al **feature subset selection bias**, cioè all'errore di fare la selezione sull'intero dataset (§ 9.3).

Punti chiave

- $\mathbb{E}_P[(y - h(\mathbf{x}))^2] = \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + \sigma^2$; il rumore è irriducibile.
- Modelli rigidi (o molto regolarizzati): alto bias, bassa varianza; modelli flessibili (poco regolarizzati): basso bias, alta varianza.
- Un comitato non è peggiore della media dei suoi membri. Bagging: media di modelli ad alta varianza, che riduce la varianza. Boosting: learner deboli in sequenza, concentrati sugli errori.

Domande di verifica

1. Derivare la decomposizione bias-varianza per la loss quadratica.
2. Cosa rappresentano bias, varianza e rumore? Quale componente è irriducibile?
3. Come variano bias e varianza con il parametro di regolarizzazione λ?
4. Perché un comitato non è peggiore della media dei suoi membri?
5. Differenza tra bagging e boosting; perché il bagging riduce la varianza?

Capitolo 9

Validazione

*La validazione stima empiricamente l'errore di generalizzazione, per due scopi distinti: scegliere il modello (*model selection*) e stimarne il rischio (*model assessment*). Il capitolo presenta hold-out e K-fold cross-validation, la ricerca degli iperparametri, gli schemi formali che combinano selezione e valutazione, gli errori più frequenti e le misure d'errore per classificazione e regressione. Gli aspetti specifici delle reti neurali (epoche, early stopping, inizializzazioni) sono nel § 12.9.*

9.1 Perché validare

La prestazione di un sistema di ML è la sua **accuratezza predittiva** su dati nuovi. Le prestazioni sui dati di training danno una valutazione **troppo ottimistica**: all'inizio dell'addestramento c'è troppo bias (underfitting, errore di training alto), alla fine troppa varianza (errore di training basso ma overfitting). Inoltre, a parità di complessità, l'errore di test varia molto da un training set all'altro (§ 8.3). Se un iperparametro θ (implicito o esplicito) regola la complessità del modello, si vuole il valore di θ che minimizza l'errore di test: servono metodi per **stimare l'errore atteso** di un modello o di una classe di modelli. Oltre ai metodi analitici (§ 7.1) ci sono i metodi **empirici**, basati sul ricampionamento dei dati: **cross-validation** e **bootstrap**.

9.2 Model selection e model assessment

Definizione — I due obiettivi della validazione. Dopo aver addestrato i modelli sul training set:

- **model selection**: stimare le prestazioni (errore di generalizzazione) di diversi modelli per **scegliere il migliore**; include la ricerca dei migliori **iperparametri** (grado del polinomio, numero di unità di una rete, λ, η, …). *Restituisce un modello.*
- **model assessment**: scelto il modello finale, **stimarne l'errore di predizione** (rischio) su dati di test nuovi, come misura della sua qualità. *Restituisce una stima.*

Attenzione — Regola d'oro. Tenere separati gli obiettivi e usare insiemi di dati separati in ogni fase. Se il test set viene usato per scegliere il modello, la stima finale non è più affidabile (è ottimistica).

9.2.1 Hold-out

Nel mondo ideale si avrebbero un grande training set, un grande validation set e un test set esterno molto grande. Se i dati sono sufficienti, la tecnica di base, l'**hold-out**, divide il dataset in tre insiemi **disgiunti** (ad esempio 50% TR, 25% VL, 25% TS):

- **training set (TR)**: per adattare il modello, cioè minimizzare il R_{emp} della SLT [*training*];
- **validation set (VL)**, o *selection set*: per scegliere il modello migliore tra modelli e configurazioni di iperparametri diversi [*model selection*];
- **test set (TS)**: per stimare l'errore di generalizzazione del modello finale, cioè stimare R [*model assessment*]; non si usa mai per scegliere o regolare h.

TR e VL insieme formano il **development** o **design set**, usato per costruire il modello finale (Figura 9.1).

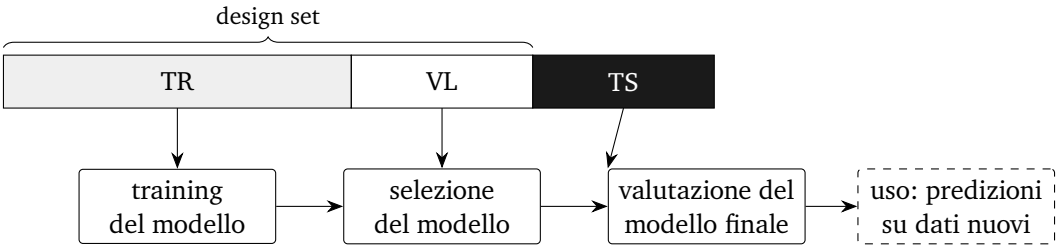


Figura 9.1. Ruolo di TR, VL e TS: sviluppo del modello (training e selezione), stima del rischio sul test set, uso del modello rilasciato.

Attenzione — Due precisazioni.

1. La stima fatta sul VL serve **solo** alla model selection: non è una buona stima per l'assessment.
2. I risultati sul TS **non** si possono usare per la model selection. Se lo si fa, quello non è più un test set: è un validation set.

Usare il test set in un ciclo di progettazione ripetuto (provo, guardo il test, cambio, riprovo) è **model selection**, non una valutazione: non si potrebbe fare lo stesso sugli esempi futuri, e l'errore sul test così usato è una **stima troppo ottimistica**. È come un esercizio d'esame di cui si sono viste le soluzioni. Da qui il concetto di **blind test set** (test cieco), usato nelle competizioni.

9.3 Un controesempio istruttivo

Esempio — Il target casuale. Si hanno 20–30 esempi con **1000 variabili di input** a valori casuali e un target **casuale** 0/1. Si seleziona il modello che usa **una sola** variabile di input, quella che per puro caso "indovina" il target al 99% (o 100%) su qualsiasi successiva suddivisione in TR, VL e TS. Risultato perfetto? No: la vera accuratezza è il **50%**, perché il target è casuale e nessun modello può fare meglio di una moneta. Su un test set davvero nuovo la stessa variabile darebbe accuratezze del tutto casuali (ad esempio 100%, 33%, 66% su tre test set diversi).

Con 1000 variabili casuali e soli 20 esempi è quasi certo che una variabile coincida per caso con il target sulla maggior parte degli esempi; poiché la scelta è fatta guardando **tutti** i dati, quella variabile "funziona" su ogni partizione successiva. Gli errori sono due:

(1) la stima dell'errore su TR e VL **non è** una buona stima del rischio;

(2) usare **l'intero dataset** per la selezione delle feature o del modello **pregiudica la stima** (*subset selection bias*): il test set è stato usato *implicitamente* all'inizio. Il test set va separato **prima** di qualsiasi model selection, **feature selection compresa**.

Attenzione. Qui si discute la **correttezza della stima**, non la possibilità di risolvere il task. La K-fold CV e le altre tecniche **non** risolvono il problema se la selezione è fatta prima della suddivisione.

9.4 K-fold cross-validation

L'hold-out richiede dati sufficienti per tre partizioni significative (quanti dipende dalla complessità del modello e dal rapporto segnale/rumore) e dipende dalla **particolare partizione** scelta. Con pochi dati sfrutta male l'informazione disponibile.

> **Definizione — K-fold cross-validation.** Si divide il dataset D in K sottoinsiemi mutuamente esclusivi $D_1, \dots, D_K$ (i **fold**); per ogni i si addestra su $D \setminus D_i$ e si valuta su D_i; si combinano i K risultati.

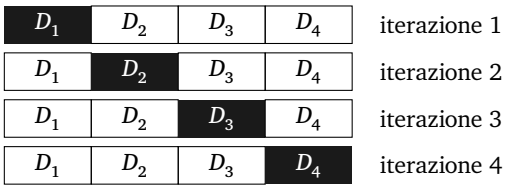


Figura 9.2. 4-fold cross-validation: a turno ogni fold (nero) fa da insieme di validazione (o di test) e gli altri da training.

Così tutti i dati vengono usati sia per l'addestramento sia per la valutazione. Non si ottiene un **unico modello** (ce n'è uno per fold), ma si ottiene anche una **varianza** dei risultati sui fold: il risultato si riporta come **media ± deviazione standard**. Si può usare sia per il validation set sia per il test set. Le questioni aperte sono il numero di fold (3, 5, 10, fino al **leave-one-out**, con $K = l$), il costo computazionale e la combinazione con altri schemi (§ 9.6.3).

Il termine "cross-validation" indica sia un **approccio** alla model selection basato sulla stima diretta dell'errore tramite ricampionamento (anziché analitica), sia le **implementazioni** che stabiliscono come dividere i dati, come la K-fold CV. Un'altra tecnica di ricampionamento è il **bootstrap**: campionamento casuale **con reinserimento**, ripetuto per ottenere sottoinsiemi diversi di validazione o di test.

9.5 Ricerca degli iperparametri

Gli **iperparametri** sono i parametri non appresi direttamente dall'algoritmo di apprendimento; si fissano cercando nello spazio degli iperparametri, tramite model selection su un validation set.

9.5.1 Grid search

La **grid search** esaustiva genera tutti i candidati di una griglia di valori. Ad esempio con numero di unità e λ:

Unità \ λ	0,1	0,01	0,001
1 unità	Res1	Res4	Res7
10 unità	Res2	Res5	Res8
100 unità	**Res3**	Res6	Res9

Se il risultato migliore sul validation set è Res3, vince la configurazione (100 unità, $\lambda = 0{,}1$). La ricerca va **automatizzata** ed è facile da **parallelizzare** (le prove sono indipendenti). Il costo è il prodotto cartesiano dei valori, $(\#\text{valori})^{\#\text{iperparametri}}$: con 3 valori per 2 iperparametri sono $3^2 = 9$ prove, con 5–6 iperparametri da 10 valori diventa proibitivo. Per questo:

- si possono fissare in una fase preliminare gli iperparametri che risultano poco rilevanti (con cautela, § 9.7);
- si usano **grid search annidate**: prima una ricerca **grossolana** su tutte le combinazioni (ad esempio con valori a crescita esponenziale) per trovare le regioni buone, poi ricerche **più fini** su intervalli sempre più piccoli, sempre considerando **tutti** gli iperparametri significativi insieme.

9.5.2 Alternative

La grid search è molto utile per **osservare il comportamento** del modello al variare degli iperparametri, ma scala male. Le alternative sono la **random search** e la ricerca **automatica** (approcci bayesiani, evolutivi, metodi come Hyperband), verso l'*AutoML*; le librerie ne includono molte, ma non vanno usate acriticamente.

> **Perché la random search funziona.** Se solo uno di due iperparametri conta davvero, una griglia 10×10 ne prova appena **10 valori distinti** (le altre 90 prove ripetono gli stessi valori variando l'iperparametro inutile), mentre 100 prove casuali ne provano **100** (Figura 9.3). La ricerca casuale permette anche di fissare il **budget** di prove indipendentemente dal numero di iperparametri.

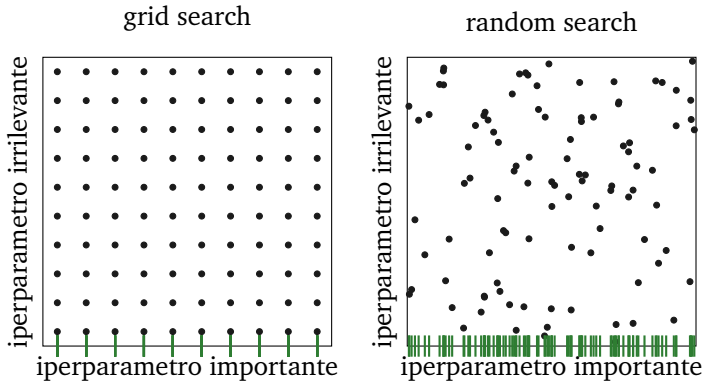


Figura 9.3. 100 prove con grid search e con random search: in verde, sull'asse orizzontale, i valori distinti provati per l'iperparametro importante (10 contro 100).

9.6 Schemi formali

Una notazione precisa evita ambiguità e mostra dove verrebbero commessi gli errori del controesempio. Gli schemi valgono per la selezione tra configurazioni di iperparametri θ ma anche tra **modelli diversi** (θ può indicare il tipo di modello).

Definizione — Notazione.

- **Dati**: $Z_1, \dots, Z_l$ campione i.i.d. di una distribuzione sconosciuta $p(z)$; $D_r = \{z_1, \dots, z_r\}$ è un insieme di r elementi.
- **Task**: classificazione, $Z = (X, Y) \in \mathbb{R}^n \times \{-1, +1\}$; regressione, $Z = (X, Y) \in \mathbb{R}^n \times \mathbb{R}$.
- **Loss**: classificazione $L((x, y), h) = 0$ se $h(x) = y$, 1 altrimenti; regressione $L((x, y), h) = (h(x) - y)^2$.
- **Rischio atteso** e ipotesi ottima: $R(h) = \mathbb{E}_Z[L(z, h)] = \int_Z L(z, h)\, p(z)\, dz$, $h^* = \arg\min_{h \in H} R(h)$.

- **Rischio empirico** su un insieme finito: $R_{\text{emp}}(h, D_r) = \frac{1}{r}\sum_{i=1}^{r} L(z_i, h)$.

Attenzione. Qui R_{emp} è inteso in senso generale, come **surrogato di** R su un insieme finito, usato per il training, la selezione o la valutazione a seconda della partizione. Il R_{emp} della SLT (l'errore di training) è, in questa notazione, $R_{\text{emp}}(h, D_{TR})$.

Prima si identifica l'obiettivo: il **miglior modello** ottenibile dal dataset (model selection), le **prestazioni attese** di un modello ottenuto minimizzando il rischio empirico (model assessment), o **entrambi**.

9.6.1 Model selection

Algoritmo — Model selection con hold-out.

1. Scegliere uno spazio delle ipotesi con iperparametro θ; dividere D_l in $D_{TR} = \{z_1, \dots, z_{tr}\}$ e $D_{VL} = \{z_{tr+1}, \dots, z_{tr+vl}\}$, con $tr + vl = l$.
2. Per ogni valore θ_m [grid search]:
   - $h^*_{\theta_m}(D_{TR}) = \arg\min_{h \in H_{\theta_m}} R_{\text{emp}}(h, D_{TR})$ [training];
   - stimare $R(h^*_{\theta_m})$ con $R_{\text{emp}}(h^*_{\theta_m}, D_{VL}) = \frac{1}{vl}\sum_{z_i \in D_{VL}} L\big(z_i, h^*_{\theta_m}(D_{TR})\big)$.
3. Scegliere $\theta^*_m = \arg\min_{\theta_m} R(h^*_{\theta_m})$ [miglior risultato sul VL].
4. Restituire $h^*(D_l) = \arg\min_{h \in H_{\theta^*_m}} R_{\text{emp}}(h, D_l)$ [riaddestramento su tutti gli l dati].

La stima di R sul VL serve solo alla scelta e **non viene restituita** come stima del rischio: farlo sarebbe l'errore (1) del controesempio.

Algoritmo — Model selection con K-fold CV.

1. Scegliere uno spazio delle ipotesi con iperparametro θ; dividere D_l in K parti distinte e uguali $D_1, \dots, D_K$; sia $\bar{D}_k$ il complementare di D_k.
2. Per ogni valore θ_m [grid search]:
   - per ogni parte D_k [ciclo sui fold]: $h^*_{\theta_m}(\bar{D}_k) = \arg\min_{h \in H_{\theta_m}} R_{\text{emp}}(h, \bar{D}_k)$ [training], e stima di $R(h^*_{\theta_m}(\bar{D}_k))$ con $R_{\text{emp}}(h^*_{\theta_m}(\bar{D}_k), D_k) = \frac{1}{|D_k|}\sum_{z_i \in D_k} L\big(z_i, h^*_{\theta_m}(\bar{D}_k)\big)$;
   - stima di $R(h^*_{\theta_m}(D_l))$ con la media $\frac{1}{K}\sum_k R(h^*_{\theta_m}(\bar{D}_k))$.
3. Scegliere $\theta^*_m = \arg\min_{\theta_m} R(h^*_{\theta_m}(D_l))$ [migliore media di validazione].
4. Restituire $h^*(D_l) = \arg\min_{h \in H_{\theta^*_m}} R_{\text{emp}}(h, D_l)$ [riaddestramento].

Attenzione — Ogni iterazione riparte da zero. Addestramento e stima sono locali a ciascuna suddivisione. Se ad esempio si usasse il modello addestrato nello split 1 per valutare lo split 2, si userebbe come validazione dello split 2 una parte (D_2) già usata per addestrare nello split 1.

9.6.2 Model assessment

Algoritmo — Model assessment con hold-out.

1. Dividere D_l in D_{TR} e D_{TS}, con $tr+ts=l$. Il test set va separato adesso, non dopo: altrimenti si commette l'errore (2) del controesempio.
2. $h^*(D_{TR}) = \arg\min_{h\in H} R_{\text{emp}}(h, D_{TR})$ [training; può includere una model selection interna su D_{TR}].
3. Stimare $R(h^*(D_{TR}))$ con $R_{\text{emp}}(h^*(D_{TR}), D_{TS}) = \frac{1}{ts}\sum_{z_i\in D_{TS}} L\big(z_i, h^*(D_{TR})\big)$ [errore di test].

Algoritmo — Model assessment con K-fold CV.

1. Dividere D_l in K parti distinte e uguali.
2. Per ogni parte D_k: $h^*(\bar{D}_k) = \arg\min_{h\in H} R_{\text{emp}}(h, \bar{D}_k)$ [training, eventualmente con model selection], e stima di $R(h^*(\bar{D}_k))$ con $R_{\text{emp}}(h^*(\bar{D}_k), D_k)$ [errore di test della iterazione].
3. Stimare $R(h^*(D_l))$ con la media $\frac{1}{K}\sum_k R(h^*(\bar{D}_k))$.

Con $K=l$ si ha la **leave-one-out CV**. Non si violano i principi: i risultati di test non vengono mai usati per addestrare o selezionare, in nessuna iterazione.

9.6.3 Combinare selezione e valutazione

Quando servono sia il modello migliore sia la stima del suo rischio si combinano gli schemi.

Algoritmo — Hold-out TR–VL–TS.

1. Dividere D_l in D_{TR}, D_{VL}, D_{TS}.
2. Per ogni θ_m: addestrare $h^*_{\theta_m}(D_{TR})$ e calcolare $R_{\text{emp}}(h^*_{\theta_m}(D_{TR}), D_{VL})$.
3. Scegliere θ^*_m con il miglior risultato sul VL.
4. Riaddestrare $h^*(D_{TR}\cup D_{VL})$ con θ^*_m (eventualmente usando una parte dei dati come VL per l'early stopping).
5. Stimare R con $\frac{1}{ts}\sum_{z_i\in D_{TS}} L\big(z_i, h^*(D_{TR}\cup D_{VL})\big)$.

Algoritmo — K-fold CV per la selezione e hold-out per il test.

1. Dividere D_l in D_{TR} (il design set) e D_{TS}.
2. Per ogni θ_m: stimare $R(h^*_{\theta_m}(D_{TR}))$ con una K-fold CV su D_{TR} (errore medio di validazione sui fold).
3. Scegliere θ^*_m con la stima migliore; riaddestrare $h^*(D_{TR})$.
4. Stimare R con l'errore su D_{TS}.

Un esempio completo di questo schema: si divide il dataset in TR e TS; per ogni cella di una grid search (ad esempio $\lambda = 0{,}1$ e 20 unità) si esegue un'intera K-fold CV dentro il TR e si sceglie la configurazione con il miglior errore medio di validazione; si riaddestra sull'intero TR e si valuta sul TS esterno.

K-fold esterna per il test, hold-out interno per la selezione. In ogni iterazione della K-fold esterna un fold fa da test e il resto viene diviso (a caso, con shuffling) in TR e VL per la model selection (Figu-

ra 9.4). Questo processo **non fornisce un modello finale**: ogni riga può selezionare un modello diverso, quindi si ottiene solo una stima del rischio della **classe** di modelli considerata.

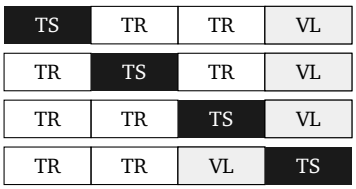


Figura 9.4. K-fold CV esterna per il test, con divisione interna TR/VL per la model selection (il riaddestramento non è mostrato).

Double (nested) K-fold CV.

Algoritmo — Double K-fold CV.

1. Dividere D_l in K parti distinte e uguali.
2. Per ogni parte D_k [ciclo esterno]:
   - scegliere $h^*(\bar{D}_k)$ con una **K-fold CV interna** su $\bar{D}_k$ (con K' eventualmente diverso da K) [ciclo interno];
   - stimare $R(h^*(\bar{D}_k))$ con $R_{\text{emp}}(h^*(\bar{D}_k), D_k)$ [errore di test di ogni split esterno].
3. Stimare $R(h^*(D_l))$ con la media sui fold di test.

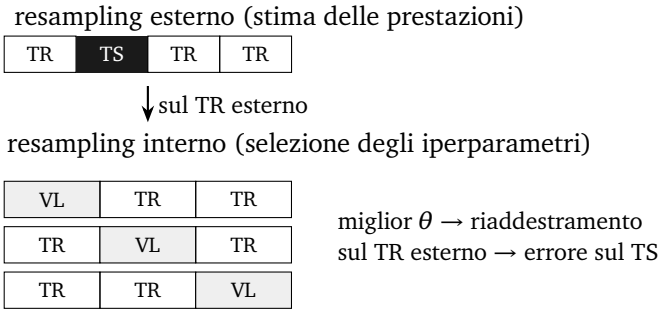


Figura 9.5. Double CV: per ogni split esterno (in alto, uno dei quattro), una CV interna sul training esterno seleziona gli iperparametri; il "test set" della CV interna è in realtà un validation set.

Anche la double CV **non dà un modello finale unico**, perché ogni ciclo esterno può scegliere iperparametri diversi: dà una stima del rischio della classe di modelli (cioè dell'algoritmo con la sua procedura di selezione), con la sua **varianza** sui fold. Se serve un modello finale si esegue una **model selection separata** (hold-out o K-fold CV) con riaddestramento. Non si violano le regole: la stima di test è già stata fatta e i risultati di test non vengono mai usati per la selezione. Non è quindi vero in assoluto che "il test viene sempre per ultimo": qui avviene *prima* dell'ultima model selection. La double CV sfrutta meglio tutti i dati sia per la selezione sia per la valutazione, al prezzo di un costo computazionale elevato.

> **Si può riaddestrare su tutto D_l?.** Sì: il riaddestramento finale su tutti i dati (TR+VL+TS) non usa i risultati del test per alcuna scelta. Il modello finale avrà in generale prestazioni pari o migliori della stima (ha più dati), che resta valida come stima, leggermente pessimistica, del rischio della procedura. L'importante è che il test non abbia influenzato nessuna decisione.

9.7 Errori frequenti e buone pratiche

9.7.1 Riprogettare in base al test

Riprogettare il modello in base al risultato sul test **non è corretto** (a meno di disporre di nuovi dati per un nuovo test set). Ripetendo il procedimento sugli stessi dati si può ottenere qualunque risultato: il modello si adatta a quel dataset, e **non si sta più stimando il rischio**. Non bisogna regolare il predittore (a mano o automaticamente) sulla base dell'errore di test, perché non si potrebbe fare sugli esempi futuri; la valutazione su un test interno resta utile per **verificare il proprio approccio**. Per fare un buon modello serve una buona model selection; per stimare le prestazioni reali serve una stima rigorosa sul test.

9.7.2 La cross-validation può andare in overfitting

Un uso intensivo della CV può portare a overfitting. Con 20–30 record e 1000 attributi si provano 1000 modelli di regressione lineare con un solo attributo ciascuno: il migliore dei 1000 sembra buono sul validation set, ma lo sarebbe sembrato anche con un output **puramente casuale**. Oppure: approssimando un target casuale con una K-fold CV per il test, dopo aver scelto le feature per massima correlazione con il target, si trova un errore del 3% invece del vero 50%, perché la selezione è stata fatta su **tutti** gli esempi. Rimedi:

- tenere da parte un **test set aggiuntivo** prima di qualsiasi model selection, e verificare che il modello migliore funzioni anche su di esso;
- per la selezione con CV, usare "CV + test" o la **double CV** (rifacendo la selezione per ogni fold);
- per la valutazione con CV, fare la **feature selection dentro ogni fold**: si ottiene la stima corretta, circa il 50% sul task casuale.

9.7.3 Selezione sequenziale degli iperparametri

> **Attenzione.** Selezionare gli iperparametri **uno alla volta**, in sequenza, introduce un **bias legato all'ordine**: ad esempio scegliere il miglior η con 10 unità e poi usarlo anche con 100 unità.

Il procedimento corretto:

1. prove iniziali per determinare gli **intervalli** giusti (ad esempio guardando la stabilità del training) e gli iperparametri più **influenti** (ad esempio guardando la varianza dei risultati);
2. ricerca su una griglia con **tutte le combinazioni** degli iperparametri;
3. eventuali griglie annidate, da grossolane a fini, sempre con **tutti gli iperparametri significativi insieme**, per tenere conto degli effetti incrociati.

Bisogna però ragionare e selezionare gli aspetti rilevanti, senza affidarsi solo ad approcci automatici costosi.

9.7.4 Quale cross-validation

La **5- o 10-fold CV** è un buon compromesso tra bias e varianza della stima, ed è raccomandata rispetto alla leave-one-out. L'hold-out semplice è molto usato (a volte il test set è dato separatamente), ma richiede attenzione con pochi dati.

Tabella 9.1. Confronto tra le tecniche (con R fold).

	Svantaggi	Vantaggi
Hold-out (test set)	varianza: stima inaffidabile delle prestazioni future (fortuna/sfortuna)	economico
Leave-one-out	costoso; comportamenti anomali	non spreca dati
10-fold	spreca il 10% dei dati; 10 volte più costoso dell'hold-out	spreca solo il 10%; 10 volte più costoso invece di l volte
3-fold	spreca più dati della 10-fold	leggermente meglio dell'hold-out
R-fold con $R = l$	identico al leave-one-out	

9.7.5 Esempi di model selection

- **Modelli diversi**: si sceglie la classe di modelli con il miglior errore di CV (ad esempio 10-fold), la si riaddestra su tutti i dati e quello è il modello da usare. Si guarda **solo l'errore di validazione**, non quello di training né una loro media.
- **Numero di unità nascoste**: l'errore di training decresce sempre con il numero di unità, quello di CV ha un minimo (in pratica si provano valori come 10, 20, 30, …).
- k **del K-NN**: si usa la **leave-one-out**, perché per i metodi non parametrici costa quanto una normale predizione (non c'è un modello da addestrare). Non c'è garanzia che un ottimo locale di k rispetto alla LOOCV sia globale (la relazione può essere irregolare); per esplorare molti valori si usano valori a crescita esponenziale ($k = 1, 2, 4, 8, 16, \dots$) o una ricerca *hill-climbing*.

9.7.6 Campionamento e pochi dati

- **Stratificazione**: in ogni partizione (TR e TS, o i fold) ogni classe è rappresentata circa nelle stesse proporzioni del dataset completo.
- Controllare la composizione dei fold: l'**ordine dei dati** può avere un significato (dati ordinati per classe o per tempo).
- **Hold-out o CV ripetuti** con campionamenti casuali diversi, mediando i risultati: utile soprattutto per **confrontare modelli diversi**.
- Con pochissimi dati, evitare (o tenere in conto) classi o feature mancanti nel training, classi speciali non campionate in TR o TS, outlier che influenzano la media dei risultati, la selezione di soli casi "facili".
- Anche un **blind test set** può essere fuorviante se proviene da una distribuzione diversa, è misurato con scale o tolleranze diverse, non è pre-elaborato come i dati di training o richiede **estrapolazione** (dati fuori range).

9.8 Misure d'errore

9.8.1 Classificazione

Per un classificatore binario si confrontano classe reale e classe predetta nella **matrice di confusione**:

Reale \ Predetto	Positivo	Negativo
Positivo	TP (veri positivi)	FN (falsi negativi)
Negativo	FP (falsi positivi)	TN (veri negativi)

- **Accuratezza** $= \dfrac{TP + TN}{\text{totale}}$: frazione di esempi classificati correttamente;
- **Sensibilità** (*True Positive rate*, *recall*) $= \dfrac{TP}{TP + FN}$: quanti positivi veri vengono riconosciuti;
- **Specificità** (*True Negative rate*) $= \dfrac{TN}{FP + TN} = 1 - FPR$: quanti negativi veri vengono riconosciuti;
- **Precisione** $= \dfrac{TP}{TP + FP}$: quanti dei predetti positivi sono davvero positivi.

Un falso positivo equivale a un **falso allarme**.

Attenzione — L'accuratezza da sola può ingannare.

- Nella classificazione binaria, un'accuratezza del 50% è quella di un predittore **casuale**.
- Con **dati sbilanciati** (ad esempio 99% di positivi) il classificatore banale che risponde sempre "positivo" ha il 99% di accuratezza senza aver imparato nulla.

La **curva ROC** mostra il True Positive rate (sensibilità) in funzione del False Positive rate (1−specificità) al variare della **soglia di decisione** del classificatore (Figura 9.6). La diagonale corrisponde al classificatore casuale; le curve migliori salgono rapidamente verso l'angolo in alto a sinistra e hanno un'**AUC** (*Area Under the Curve*) più alta: il classificatore ideale ha AUC = 1, quello casuale 0,5.

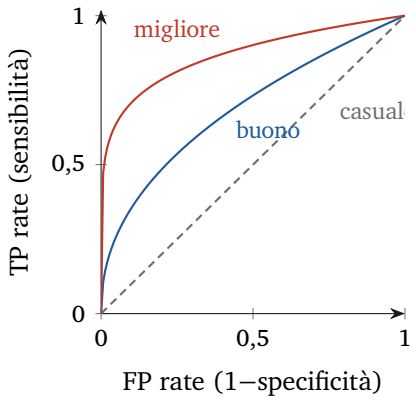


Figura 9.6. Curve ROC: la diagonale è il classificatore casuale (AUC = 0,5); più la curva sale verso l'angolo in alto a sinistra, maggiore è l'area sottesa.

9.8.2 Regressione

Con residuo $r_i = y_i - o_i$ (y target, o uscita del modello):

- **MSE** $= \text{mean}_i[r_i^2]$ e la sua radice, **RMS** (o S);
- **errore assoluto medio** (MAE) $= \text{mean}_i[|r_i|]$;
- **errore assoluto massimo** $= \max_i |r_i|$;
- **coefficiente di correlazione** R e **coefficiente di determinazione** R^2, ad esempio $R = \sqrt{1 - S^2/S_y^2}$, con $S_y^2 = \text{mean}_i[(y_i - \bar{y})^2]$ varianza del target: misura il grado di dipendenza lineare tra y e o, in $[0, 1]$, dove 1 è il meglio;
- grafici delle **uscite contro i target**; test statistici e analisi di significatività.

Punti chiave

- Model selection (restituisce un modello) e model assessment (restituisce una stima): obiettivi separati, insiemi separati.
- Il risultato sul TR non stima quello su VL e TS; il risultato sul VL non stima quello sul TS.
- Il test set si separa prima di ogni scelta, feature selection compresa.
- K-fold CV: media $\pm$ deviazione standard sui fold; ogni iterazione riparte da zero.
- Schemi combinati: TR–VL–TS, CV + test, double CV (stima del rischio della classe, nessun modello unico).

Domande di verifica

1. Differenza tra model selection e model assessment; perché servono insiemi separati?
2. Descrivere il controesempio del target casuale: quali sono i due errori?
3. Cos'è la grid search? Come se ne riduce il costo? Vantaggi della random search.
4. Scrivere gli algoritmi di model selection e di model assessment con K-fold CV.
5. Descrivere la double (nested) CV. Restituisce un modello?
6. La cross-validation può andare in overfitting? Come evitarlo?
7. Definire accuratezza, sensibilità, specificità, precisione; cos'è la curva ROC?

Parte IV

Reti neurali

Capitolo 10

Dal neurone al Perceptron

Questo capitolo introduce l'unità di base delle reti neurali, il neurone artificiale, e il suo caso più semplice, il Perceptron (un'unità a soglia). Si vede che reti di unità a soglia rappresentano qualunque funzione booleana grazie alla ri-rappresentazione nello strato nascosto, si studia l'algoritmo di apprendimento del Perceptron con la dimostrazione del suo teorema di convergenza, lo si confronta con l'LMS, e si introducono le unità sigmoidali, che rendono possibile la discesa del gradiente anche con uscite non lineari.

10.1 Le reti neurali come strumento di ML

Le reti neurali si studiano con due obiettivi: **modellare sistemi biologici** e processi di apprendimento (neuroscienze, scienze cognitive), dove conta il realismo biologico; oppure **costruire sistemi di ML efficaci**, dove contano le proprietà computazionali e algoritmiche, anche a scapito del realismo. Nel corso interessa il secondo aspetto: una rete neurale artificiale (ANN) realizza una funzione $h(\mathbf{x})$ con proprietà speciali. In particolare le reti neurali

- apprendono da esempi;
- sono **approssimatori universali** (§ 11.2.3): approcci flessibili per funzioni arbitrarie, anche non lineari;
- gestiscono rumore e dati incompleti, con prestazioni che **degradano gradualmente**;
- trattano dati reali e discreti, per regressione e classificazione;
- non sono un singolo modello ma un **paradigma** (nella classe degli approcci sub-simbolici).

L'idea di fondo è il **connessionismo**: comportamenti complessi che **emergono** dall'interazione di molte unità computazionali semplici interconnesse.

10.1.1 L'ispirazione biologica

Il cervello umano ha più di 10^{10} neuroni, ciascuno con 10^4–10^5 connessioni. Un neurone risponde in circa un millisecondo, eppure si riconosce un'immagine in circa 0,1 secondi, cioè in al massimo un centinaio di passi seriali: il cervello esegue una computazione **massicciamente parallela**. Un neurone riceve segnali dai **dendriti**, che ne modificano il potenziale; superata una **soglia**, emette un impulso (*spike*) che si propaga lungo l'**assone** fino alle **sinapsi**, dove passa ai neuroni successivi. Le sinapsi possono essere **eccitatorie** o **inibitorie** e la loro forza cambia con l'apprendimento (**plasticità**): nel modello artificiale la forza sinaptica diventa un **peso**.

> **Definizione — Apprendimento hebbiano.** Una sinapsi si rafforza quando gli input del neurone (e le uscite correlate) si ripetono: lo stimolo in ingresso rinforza la sinapsi, quindi **i pesi si avvicinano agli input**.

10.2 Il neurone artificiale

Un'**unità** (nodo, neurone) riceve input da sorgenti esterne o da altre unità. Ogni connessione ha un **peso** w, parametro libero modificato dall'apprendimento. L'unità i calcola

$$net_i(\mathbf{x}) = \sum_j w_{ij}\, x_j, \qquad o_i(\mathbf{x}) = f\big(net_i(\mathbf{x})\big), \tag{10.1}$$

dove net_i è l'**input netto** (che include il bias w_{i0} tramite l'input costante $x_0 = 1$) e f è la **funzione di attivazione** (Figura 10.1).

> **Attenzione — Notazione dei pesi.** w_{ij} è il peso **dell'unità** i, cioè della connessione **dall'input (o unità)** j **verso l'unità** i: il primo indice è l'unità che riceve. Alcune fonti e librerie usano la convenzione opposta.

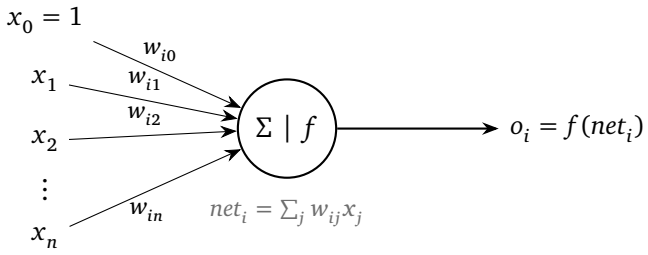


Figura 10.1. L'unità i: somma pesata degli input (bias incluso tramite $x_0 = 1$) e funzione di attivazione.

Tre funzioni di attivazione classiche (Figura 10.2): **lineare** (identità), $h(\mathbf{x}) = \sum_i w_i x_i$; **a soglia** (gradino), che dà il **Perceptron**, cioè la LTU del § 4.5; **logistica** (sigmoide), trattata nel § 10.7.

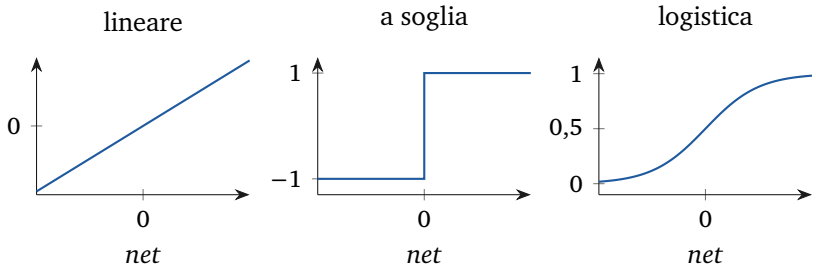


Figura 10.2. Funzione di attivazione lineare, a soglia (Perceptron) e logistica.

10.3 Reti di unità a soglia e funzioni booleane

Il primo modello di neurone (McCulloch e Pitts) ha due stati, attivo (1) e non attivo (0); ogni sinapsi è caratterizzata da un numero reale w, positivo per le connessioni **eccitatorie** e negativo per le **inibitorie**; un neurone si attiva quando la somma dei pesi provenienti dai neuroni attivi, più un bias, supera zero. Con input e output binari è sostanzialmente la LTU. Con $net = \mathbf{w}^T\mathbf{x} = \sum_{i=0}^{n} w_i x_i$ e uscita 1 se $net > 0$, 0 (o -1) altrimenti:

- **AND**: $w_1 = w_2 = 1$, $w_0 = -1{,}5$ (serve che entrambi gli input valgano 1 per superare 1,5);
- **OR**: $w_1 = w_2 = 1$, $w_0 = -0{,}5$ (basta un input a 1);
- **NOT**: $w_1 = -1$, $w_0 = 0{,}5$ (per $x_1 = 0$ il net vale $0{,}5 > 0$, per $x_1 = 1$ vale $-0{,}5$).

10.3.1 Lo XOR con due strati

Lo XOR non è linearmente separabile (§ 4.6): nessuna singola unità a soglia lo rappresenta. Si può però scomporre in funzioni separabili:

$$x_1 \oplus x_2 = x_1\bar{x}_2 + \bar{x}_1 x_2 = \bar{h}_1 \cdot h_2, \qquad h_1 = x_1 \cdot x_2 \text{ (AND)}, \quad h_2 = x_1 + x_2 \text{ (OR)}.$$

XOR come composizione di AND e OR. Con $a = x_1$, $b = x_2$, $h_1 = a \wedge b$ e $h_2 = a \vee b$ vale $\bar{h}_1 \wedge h_2 = a \oplus b$.

Dimostrazione. Per De Morgan $\bar{h}_1 = \overline{a \wedge b} = \bar{a} \vee \bar{b}$. Distribuendo:

$$\bar{h}_1 \wedge h_2 = (\bar{a} \vee \bar{b}) \wedge (a \vee b) = (\bar{a} \wedge a) \vee (\bar{a} \wedge b) \vee (\bar{b} \wedge a) \vee (\bar{b} \wedge b) = (\bar{a} \wedge b) \vee (a \wedge \bar{b}) = a \oplus b,$$

perché $\bar{a} \wedge a$ e $\bar{b} \wedge b$ sono sempre falsi. ■

Si ottiene una rete a **due strati**: due unità nascoste calcolano h_1 (AND) e h_2 (OR), e l'unità di uscita calcola $\bar{h}_1 \wedge h_2$ con pesi -1 da h_1, $+1$ da h_2 e bias $-0{,}5$ (Figura 10.3).

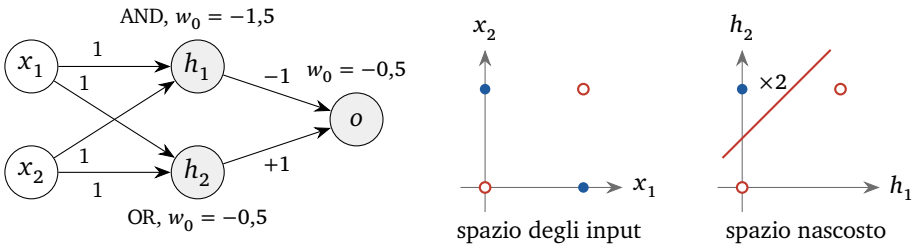


Figura 10.3. Lo XOR con una rete a due strati. Nello spazio degli input i punti non sono separabili; nello spazio delle unità nascoste (h_1, h_2) i due positivi $(0,1)$ e $(1,0)$ finiscono nello stesso punto $(0,1)$ e il problema diventa linearmente separabile (retta rossa $h_2 = h_1 + 0{,}5$).

Lo strato nascosto come rappresentazione. Lo strato nascosto produce una **ri-rappresentazione interna** degli input che rende facile il compito dello strato di uscita. Questa composizione si estende su **molti strati di astrazione** e, a differenza di qui (dove i pesi sono scelti a mano), nelle reti neurali queste rappresentazioni **si apprendono**. È il fattore chiave delle reti neurali, alla base del *representation learning* e del deep learning (capitolo 15).

Reti di unità a soglia.

- Le unità a soglia rappresentano AND, OR, NOT (quindi anche NAND, NOR): **ogni funzione booleana** può essere rappresentata da una rete di unità a soglia.
- **Due livelli** bastano (più livelli possono essere più efficienti, § 15.2).
- Un singolo strato non basta, per il limite della separabilità lineare.

Per ora però non c'è alcun algoritmo di apprendimento, nemmeno per una singola unità.

10.4 Apprendere con una singola unità

Per una singola unità ci sono due famiglie di metodi:

1. **Adaline** (*Adaptive Linear Neuron*): durante il training l'unità è **lineare** e si usa l'LMS, in forma diretta o con la discesa del gradiente (capitolo 4). Serve per regressione e, con la soglia, classificazione. È l'approccio che si estende agli MLP.
2. **Perceptron**: durante il training l'unità è **non lineare** (con soglia). Solo classificazione.

Per costruire un classificatore lineare si hanno quindi tre algoritmi: LMS diretto, LMS a gradiente e l'algoritmo del Perceptron (e più avanti le SVM).

10.4.1 L'algoritmo del Perceptron

L'obiettivo è **minimizzare il numero di esempi classificati male**, cioè trovare $\mathbf{w}$ con $\text{sign}(\mathbf{w}^T\mathbf{x}) = d$ per ogni esempio. L'algoritmo è **on-line**.

Algoritmo — Perceptron Learning Algorithm.

1. Inizializzare i pesi (a zero o a piccoli valori casuali); scegliere un learning rate $\eta \in (0, 1)$.
2. Finché non vale la condizione di arresto (ad esempio i pesi non cambiano più), per ogni esempio $(\mathbf{x}, d)$ con $d \in \{+1, -1\}$:
   - calcolare $out = \text{sign}(\mathbf{w}^T\mathbf{x})$;
   - se $out = d$, **non cambiare i pesi**;
   - se $out \neq d$, aggiornare $\mathbf{w}_{new} = \mathbf{w} + \eta\, d\, \mathbf{x}$.

In pratica si aggiunge $\eta\mathbf{x}$ se $\mathbf{w}^T\mathbf{x} \leq 0$ e $d = +1$ (falso negativo) e si sottrae $\eta\mathbf{x}$ se $\mathbf{w}^T\mathbf{x} > 0$ e $d = -1$ (falso positivo). In forma equivalente

$$\mathbf{w}_{new} = \mathbf{w} + \tfrac{1}{2}\eta\,(d - out)\,\mathbf{x},$$

perché $d - out$ vale 0 se la classificazione è corretta e ± 2 se è sbagliata.

Attenzione — Differenza con l'LMS. La formula somiglia a quella dell'LMS, ma qui $out = \text{sign}(\mathbf{w}^T\mathbf{x})$ contiene il **segno**, mentre nell'LMS l'errore è calcolato sull'uscita lineare $\mathbf{w}^T\mathbf{x}$.

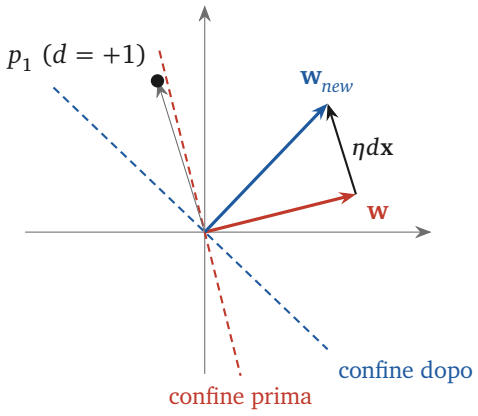


Figura 10.4. Aggiornamento del Perceptron: p_1 ha target $+1$ ma sta dal lato negativo del confine (rosso). Sommando $\eta d\mathbf{x}$ a $\mathbf{w}$, il vettore dei pesi si sposta verso p_1 e il nuovo confine (blu) lo classifica correttamente.

Geometricamente (Figura 10.4) $\mathbf{w}$ punta verso la regione positiva; se un esempio con $d = +1$ è classificato male, $\mathbf{w}$ viene spostato un po' nella sua direzione (somma vettoriale $\mathbf{w} + \eta d\mathbf{x}$); se un esempio con $d = -1$ è classificato male, $\mathbf{w}$ viene allontanato da esso.

Due letture della regola.

- $\mathbf{w}_{new} = \mathbf{w} + \eta\, d\, \mathbf{x}$ è un **apprendimento hebbiano**: $\mathbf{w}$ si muove verso $\mathbf{x}$.
- $\mathbf{w}_{new} = \mathbf{w} + \eta\,(d - out)\,\mathbf{x} = \mathbf{w} + \eta\,\delta\,\mathbf{x}$ è una regola di **correzione dell'errore** (delta rule): la modifica di un peso è proporzionale al prodotto del segnale d'errore per il segnale di input della connessione.

Il calcolo è facile perché il segnale d'errore δ è **direttamente misurabile**: si conosce la risposta desiderata per l'unità. Nelle reti con strati nascosti questo non sarà più vero (§ 11.3).

Il Perceptron **rappresenta** confini lineari, quindi risolve i problemi linearmente separabili. Il teorema seguente garantisce che riesce anche a **impararli**: il Perceptron apprende tutto ciò che può rappresentare.

10.5 Il teorema di convergenza del Perceptron

Teorema — Convergenza del Perceptron. *Se il problema è **linearmente separabile**, l'algoritmo del Perceptron converge, classificando correttamente tutti gli esempi, in un **numero finito di passi**, qualunque sia il punto di partenza (la soluzione finale non è unica e dipende dal punto di partenza). Se il problema non è separabile l'algoritmo può essere instabile e sviluppare **cicli**, con pesi non necessariamente ottimali.*

Nella dimostrazione si omette la ^{T} nei prodotti scalari.

Passo 1: ridursi a esempi tutti positivi. Siano $(\mathbf{x}_i, d_i)$, $i = 1, \dots, l$, gli esempi, con $d_i \in \{+1, -1\}$. Se il problema è separabile esiste una soluzione $\mathbf{w}^*$ con

$$d_i\,(\mathbf{w}^*\mathbf{x}_i) \geq \alpha, \qquad \alpha = \min_i d_i\,(\mathbf{w}^*\mathbf{x}_i) > 0.$$

Definendo $\mathbf{x}'_i = d_i\mathbf{x}_i$, si ha $\mathbf{w}^*\mathbf{x}'_i = d_i(\mathbf{w}^*\mathbf{x}_i) \geq \alpha$: **$\mathbf{w}^*$ risolve il problema originale se e solo se risolve il problema** $(\mathbf{x}'_i, +1)$ con tutti i target positivi (il ragionamento vale nei due versi, perché $d_i^2 = 1$). Basta quindi dimostrare il teorema per un problema con **soli esempi positivi**: "ribaltare" gli esempi negativi non cambia nulla.

Passo 2: i pesi dopo q errori. Si assume $\mathbf{w}(0) = \mathbf{0}$, $\eta = 1$ e si definisce $\beta = \max_i \|\mathbf{x}_i\|^2$. Con soli esempi positivi la regola diventa $\mathbf{w}(j) = \mathbf{w}(j-1) + \mathbf{x}_{i_j}$ a ogni errore, dove i_j è l'indice del j-esimo esempio classificato male (lo stesso esempio può comparire più volte). Dopo q errori

$$\mathbf{w}(q) = \sum_{j=1}^{q} \mathbf{x}_{i_j}.$$

Idea. Si trovano un **limite inferiore** a $\|\mathbf{w}(q)\|^2$ che cresce come q^2 e un **limite superiore** che cresce come q. Poiché una funzione quadratica supera prima o poi una lineare, i due limiti sono compatibili solo per q fino a un valore massimo: il numero di errori, e quindi di aggiornamenti, è finito (Figura 10.5).

Passo 3: limite inferiore. Moltiplicando $\mathbf{w}(q)$ per $\mathbf{w}^*$ e usando $\mathbf{w}^*\mathbf{x}_{i_j} \geq \alpha$ per ogni termine:

$$\mathbf{w}^*\mathbf{w}(q) = \sum_{j=1}^{q} \mathbf{w}^*\mathbf{x}_{i_j} \geq q\,\alpha.$$

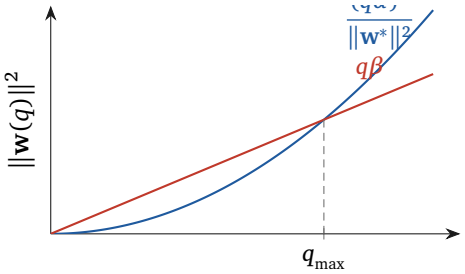


Figura 10.5. $\|\mathbf{w}(q)\|^2$ deve stare tra il limite inferiore quadratico (blu) e il limite superiore lineare (rosso): oltre $q_{\max}$ le due condizioni non possono valere insieme.

Per la disuguaglianza di Cauchy-Schwarz $\|\mathbf{a}\|^2\|\mathbf{b}\|^2 \geq (\mathbf{ab})^2$, con $\mathbf{a} = \mathbf{w}^*$ e $\mathbf{b} = \mathbf{w}(q)$:

$$\|\mathbf{w}^*\|^2 \, \|\mathbf{w}(q)\|^2 \geq \big(\mathbf{w}^*\mathbf{w}(q)\big)^2 \geq (q\alpha)^2 \quad \Longrightarrow \quad \|\mathbf{w}(q)\|^2 \geq \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2}.$$

Passo 4: limite superiore. Con lo sviluppo (1.2):

$$\|\mathbf{w}(q)\|^2 = \|\mathbf{w}(q-1) + \mathbf{x}_{i_q}\|^2 = \|\mathbf{w}(q-1)\|^2 + 2\,\mathbf{w}(q-1)\,\mathbf{x}_{i_q} + \|\mathbf{x}_{i_q}\|^2.$$

Poiché $\mathbf{x}_{i_q}$ è stato classificato male (è il q-esimo errore), $\mathbf{w}(q-1)\,\mathbf{x}_{i_q} \leq 0$; eliminando questo termine non positivo

$$\|\mathbf{w}(q)\|^2 \leq \|\mathbf{w}(q-1)\|^2 + \|\mathbf{x}_{i_q}\|^2,$$

e iterando fino a $\mathbf{w}(0) = \mathbf{0}$:

$$\|\mathbf{w}(q)\|^2 \leq \sum_{j=1}^{q} \|\mathbf{x}_{i_j}\|^2 \leq q\,\beta.$$

Passo 5: conclusione. Mettendo insieme i due limiti

$$q\,\beta \;\geq\; \|\mathbf{w}(q)\|^2 \;\geq\; \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2} \quad \Longrightarrow \quad \boxed{q \leq \frac{\beta\,\|\mathbf{w}^*\|^2}{\alpha^2}}$$

Il numero di errori, e quindi di aggiornamenti, è limitato da una costante: l'algoritmo converge in un numero finito di passi. ■

> **Interpretazione del bound.** Il numero di passi cresce con β (quanto sono "grandi" gli esempi) e decresce con α^2, che misura quanto il problema è "ben separato" (il margine minimo della soluzione $\mathbf{w}^*$). Problemi con classi vicine all'iperpiano separatore richiedono più passi. L'idea di **margine** è centrale nelle SVM (capitolo 13).

10.6 Perceptron contro LMS

Le due regole hanno la stessa forma, $\mathbf{w}_{new} = \mathbf{w} + \eta\,\delta\,\mathbf{x}$, ma

- l'LMS è derivato dal gradiente **senza** soglia, minimizzando l'errore dell'unità lineare: $\delta = d - \mathbf{w}^T\mathbf{x}$;
- il Perceptron usa l'uscita con soglia: $\delta = d - \text{sign}(\mathbf{w}^T\mathbf{x})$.

Una conseguenza importante: **l'LMS può classificare male anche problemi linearmente separabili**. L'LMS non minimizza il numero di errori della LTU ma l'errore quadratico, e modifica i pesi **anche per gli esempi già classificati correttamente**: un esempio corretto ma lontano dall'iperpiano (con $\mathbf{w}^T\mathbf{x}$ molto maggiore di 1) produce comunque un errore quadratico grande che "tira" l'iperpiano (Figura 10.6).

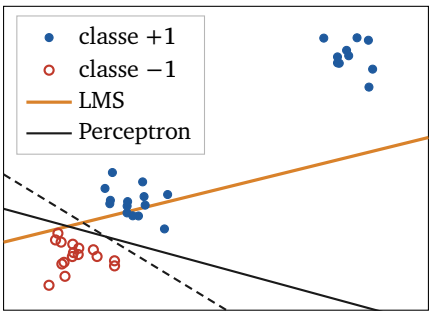


Figura 10.6. Un problema separabile: la soluzione ai minimi quadrati (arancione) è attirata dal gruppo lontano di esempi positivi e classifica male 4 esempi; il Perceptron (nero, due inizializzazioni diverse) li separa tutti.

Tabella 10.1. Algoritmo del Perceptron e LMS a confronto.

Perceptron	LMS
minimizza le classificazioni errate ($out = \text{sign}(\mathbf{w}^T\mathbf{x})$)	minimizza $E(\mathbf{w})$ con $out = \mathbf{w}^T\mathbf{x}$
converge sempre, per problemi separabili, in un numero finito di passi, a un classificatore perfetto	convergenza **asintotica**, anche per problemi non separabili
non converge se il problema non è separabile	non sempre zero errori di classificazione, anche su problemi separabili
difficile da estendere a reti di unità	**estendibile alle reti** con l'approccio a gradiente

L'ultima riga è decisiva: per costruire reti neurali serve un approccio basato sul **gradiente**, che richiede funzioni di attivazione differenziabili.

10.7 Unità sigmoidali

Definizione — Sigmoide logistica. La **funzione logistica** è una funzione "a schiacciamento" (*squashing*) con valori continui in $(0, 1)$:

$$f_\sigma(x) = \frac{1}{1 + e^{-ax}},$$

dove a è la **pendenza** (*slope*). È una versione **liscia e differenziabile della funzione soglia**. La versione simmetrica, con valori in $(-1, +1)$, è la **tangente iperbolica**:

$$f_{symm}(x) = 2f_\sigma(x) - 1 = \tanh(ax/2).$$

La relazione con la tangente iperbolica si verifica direttamente: $2f_\sigma(x) - 1 = \frac{2-(1+e^{-ax})}{1+e^{-ax}} = \frac{1-e^{-ax}}{1+e^{-ax}}$, e moltiplicando numeratore e denominatore per $e^{ax/2}$ si ottiene $\frac{e^{ax/2}-e^{-ax/2}}{e^{ax/2}+e^{-ax/2}} = \tanh(ax/2)$.

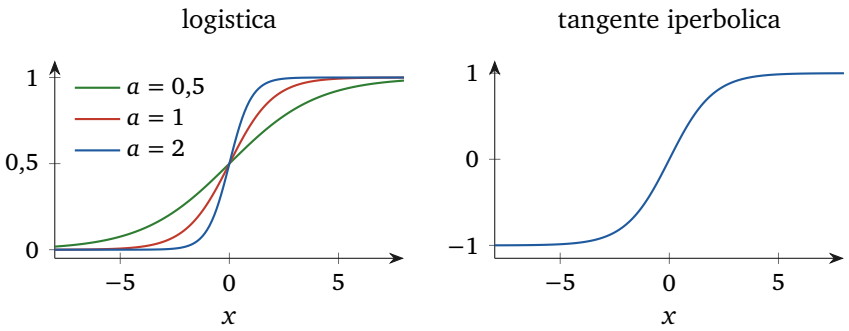


Figura 10.7. La logistica per tre valori della pendenza a (sinistra) e la tangente iperbolica (destra).

Vicino a zero la sigmoide è quasi lineare (**semi-linearità**). Per $a \to 0$ tende a una funzione **lineare** molto piatta, per $a \to \infty$ alla funzione **a gradino**. Per la classificazione, con la logistica un'uscita $\geq 0{,}5$ corrisponde alla classe positiva e $< 0{,}5$ a quella negativa (per la tanh la soglia è 0). La soglia si può spostare (studiando l'effetto su falsi positivi e negativi, ad esempio con una curva ROC) e si può definire una **zona di rifiuto** attorno alla soglia per evitare decisioni fragili.

Altre funzioni di attivazione.

- **Funzioni a base radiale** (RBF): $f(x) = e^{-ax^2}$ applicata a $\|\mathbf{w} - \mathbf{x}\|$, una gaussiana centrata sui pesi (**reti RBF**).
- **Softmax**: per uscite multiple (§ 12.8).
- **Neuroni stocastici**: uscita $+1$ con probabilità $P(net)$, -1 altrimenti (macchine di Boltzmann).
- Approssimazioni **lineari a tratti** della tanh, per calcoli efficienti.
- **ReLU** (*Rectified Linear Unit*): $f(x) = \max(0, x)$, la scelta predefinita nei modelli profondi (§ 15.6.2); **softplus**, la sua approssimazione liscia: $f(x) = \ln(1 + e^x)$.

Molte funzioni di attivazione hanno prestazioni simili in pratica.

10.7.1 Derivate

La derivata dell'identità è 1; quella della funzione a gradino **non è definita** in zero ed è nulla altrove, ed è esattamente il motivo per cui non si può usare con la discesa del gradiente. Per le sigmoidi (con $a = 1$):

$$\boxed{\frac{df_\sigma(x)}{dx} = f_\sigma(x)\big(1 - f_\sigma(x)\big), \qquad \frac{d\tanh(x)}{dx} = 1 - \tanh^2(x)} \tag{10.2}$$

e con a generico $\frac{df_\sigma}{dx} = a\, f_\sigma(x)\big(1 - f_\sigma(x)\big)$.

Derivazione per la logistica. $f_\sigma(x) = (1 + e^{-x})^{-1}$, quindi con la regola della catena

$$f'_\sigma(x) = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = f_\sigma(x)\big(1 - f_\sigma(x)\big),$$

perché $\frac{e^{-x}}{1+e^{-x}} = 1 - \frac{1}{1+e^{-x}}$. La derivata si calcola dal valore della funzione stessa. ■

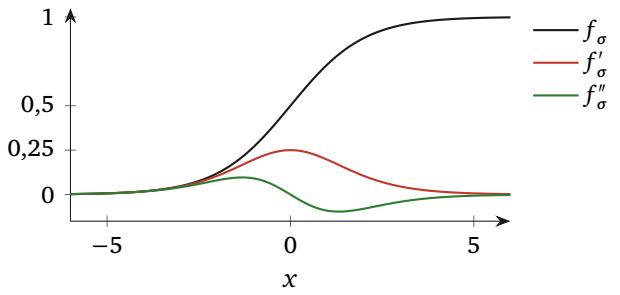


Figura 10.8. La logistica, la sua derivata prima (massima in zero, dove vale 0,25) e la derivata seconda.

10.7.2 LMS con un'unità sigmoidale

Poiché la logistica è una soglia differenziabile, si può derivare un algoritmo LMS come per l'unità lineare, passando da $o(\mathbf{x}) = \mathbf{x}^T\mathbf{w}$ a $o(\mathbf{x}) = f_\sigma(\mathbf{x}^T\mathbf{w})$ e minimizzando

$$E(\mathbf{w}) = \sum_{p=1}^{l} \big(d_p - f_\sigma(\mathbf{x}_p^T\mathbf{w})\big)^2.$$

Per un esempio p, con $net_p = \mathbf{x}_p^T\mathbf{w}$ e $o_p = f_\sigma(net_p)$, la regola della catena attraverso le variabili intermedie o_p e net_p dà

$$\frac{\partial E_p}{\partial w_j} = \frac{\partial E_p}{\partial o_p} \cdot \frac{\partial o_p}{\partial net_p} \cdot \frac{\partial net_p}{\partial w_j} = -2\,(d_p - o_p) \cdot f'_\sigma(net_p) \cdot x_{p,j},$$

e sommando sugli esempi

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^{l} \big(d_p - o(\mathbf{x}_p)\big) f'_\sigma\big(net(\mathbf{x}_p)\big)\, x_{p,j} = -2 \sum_{p=1}^{l} \delta_p\, x_{p,j}.$$

Questa derivata risponde alla domanda: *quanto una variazione di w_j influenza la variazione di E?* La discesa del gradiente è identica a quella per l'unità lineare, con un nuovo delta:

$$\boxed{\mathbf{w}_{new} = \mathbf{w} + \eta\,\delta_p\,\mathbf{x}_p, \qquad \delta_p = \big(d_p - o(\mathbf{x}_p)\big) f'_\sigma\big(net(\mathbf{x}_p)\big)} \tag{10.3}$$

È ancora una regola di correzione dell'errore, con alcune differenze:

- la pendenza a influenza l'ampiezza del passo;
- f'_σ è **massima per *net* vicino a 0** (zona quasi lineare), dove i delta possono essere grandi;
- f'_σ è **minima nei casi saturi** (f vicino a 0 o a 1): delta piccoli, pesi che cambiano lentissimamente. Bisogna evitare una **saturazione prematura**, ad esempio **partendo con pesi piccoli**.

> **Un ponte tra LMS e Perceptron.** Quando un esempio è classificato correttamente "con decisione" (uscita satura vicino al target), $f'_\sigma \approx 0$ e la correzione è quasi nulla, proprio come nel Perceptron, che non corregge gli esempi corretti. Con f_σ si approssima quindi non solo la LTU, ma anche l'algoritmo del Perceptron.

Punti chiave

- Neurone: $o = f(\sum_j w_j x_j)$; w_{ij} va dall'unità j all'unità i.
- Le unità a soglia rappresentano AND, OR, NOT; lo XOR richiede due strati, grazie alla ri-rappresentazione nello strato nascosto.
- Perceptron: $\mathbf{w} \leftarrow \mathbf{w} + \eta d\mathbf{x}$ solo sugli errori; converge in al più $\beta\|\mathbf{w}^*\|^2/\alpha^2$ errori se il problema è separabile.
- L'LMS può sbagliare su problemi separabili, ma si estende alle reti con il gradiente.
- $f'_\sigma = f_\sigma(1 - f_\sigma)$; per un'unità sigmoidale $\delta = (d - o)f'_\sigma(net)$.

Domande di verifica

1. Descrivere il neurone artificiale e le principali funzioni di attivazione.
2. Come si rappresentano AND, OR, NOT con un'unità a soglia? Come si risolve lo XOR?
3. Scrivere l'algoritmo di apprendimento del Perceptron.
4. Enunciare e dimostrare il teorema di convergenza del Perceptron.
5. Differenze tra Perceptron e LMS; perché l'LMS può sbagliare su problemi separabili?
6. Derivare la regola di aggiornamento per una singola unità sigmoidale.

Capitolo 11

Multi-Layer Perceptron e backpropagation

Componendo unità non lineari in strati si ottiene il Multi-Layer Perceptron (MLP). Il capitolo lo descrive come rete di unità e come funzione flessibile, lo interpreta come espansione in basi adattiva, ne discute il potere espressivo (approssimazione universale, VC-dimension, bias induttivo) e deriva per intero l'algoritmo che lo addestra: la backpropagation, che risolve il problema del credit assignment *per le unità nascoste.*

11.1 Il Multi-Layer Perceptron

Un MLP si può vedere in due modi (Figura 11.1):

- **come rete di unità interconnesse**, organizzate in **strati** (*layer*): lo **strato di input** è solo la sorgente dell'input $\mathbf{x}$ (carica il pattern, senza calcolare *net* né f); lo **strato nascosto** (*hidden*) proietta sullo strato di uscita o su un altro strato nascosto; lo **strato di uscita** produce $h(\mathbf{x})$;
- **come funzione flessibile**, composta da funzioni non lineari annidate. Una rete a due strati (uno nascosto e uno di uscita) calcola

$$h(\mathbf{x}) = f_k\Big(\sum_j w_{kj}\, f_j\Big(\sum_i w_{ji}\, x_i\Big)\Big), \tag{11.1}$$

  dove x_i sono le variabili di input, w i parametri liberi e f funzioni non lineari (sigmoidi). Si confronti con il modello lineare $h(\mathbf{x}) = \sum_i w_i x_i$: la rete è una composizione di funzioni lineari e non lineari.

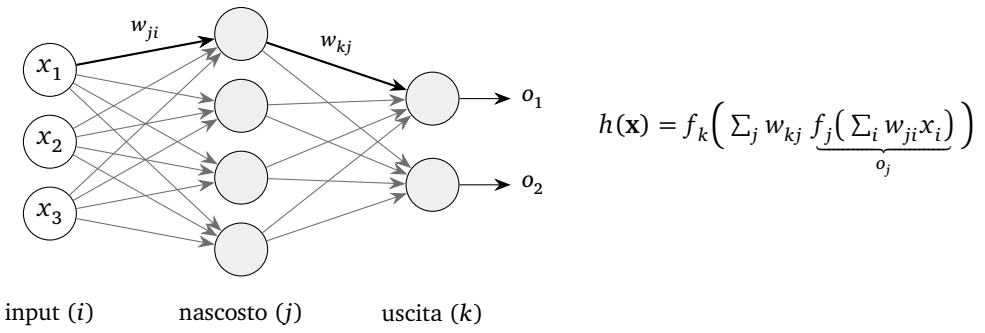


Figura 11.1. Le due viste di una rete feedforward con uno strato nascosto: rete di unità (sinistra) e funzione annidata (destra). Ogni unità nascosta e di uscita calcola *net* e f.

Una rete si descrive con il tipo di **unità** (net e funzione di attivazione), l'**architettura** (numero di unità, topologia, numero di strati) e l'**algoritmo di apprendimento**. L'MLP standard a due strati è **completamente connesso**; si possono avere più strati nascosti e connessioni che "saltano" strati.

11.1.1 Notazione uniforme

Per una generica unità t (nascosta, j, o di uscita, k):

$$net_t(\mathbf{x}) = \sum_u w_{tu}\, o_u, \qquad o_t(\mathbf{x}) = f_t\big(net_t(\mathbf{x})\big), \tag{11.2}$$

dove u indica una generica sorgente di input (un'unità di input i o nascosta j). Caricando il pattern nello strato di input si usa il simbolo o sia per gli input sia per le uscite delle unità: l'input all'unità t attraverso la connessione w_{tu} è o_u. Quindi:

- per i pesi w_{ji} (input → nascosta) l'input è $o_i = x_i$, la componente del pattern;
- per i pesi w_{kj} (nascosta → uscita) l'input è o_j, l'uscita dell'unità nascosta j;
- w_{t0} è il **bias** di ogni unità t, con input costante $o_0 = 1$;
- unità nascosta: $o_j = f_j\big(\sum_i w_{ji} x_i\big)$; unità di uscita: $o_k = f_k\big(\sum_j w_{kj} o_j\big)$.

11.1.2 Elaborazione feedforward

L'elaborazione di un pattern procede dall'input all'uscita:

1. si carica il pattern $\mathbf{x}$ nello strato di input;
2. si calcolano le uscite delle unità del primo strato nascosto, poi del secondo, e così via;
3. si calcolano le uscite dello strato di uscita, cioè $h(\mathbf{x})$;
4. si può calcolare l'errore (il delta) in uscita.

Le reti **feedforward** hanno un'unica direzione input → output. Le **reti ricorrenti** aggiungono connessioni di feedback e quindi una memoria delle computazioni passate (capitolo 18).

11.2 Perché le reti neurali sono flessibili

11.2.1 Task

Lo spazio delle ipotesi è lo **spazio continuo di tutte le funzioni** rappresentabili assegnando valori ai pesi di un'architettura data. Con unità di uscita **sigmoidali** la rete fa **classificazione**; con uscite **lineari** fa **regressione**. Con più unità di uscita si ottengono regressione multipla o classificatori multi-classe (ad esempio tre uscite che valgono 0,2; 0,7; 0,1 per tre classi).

11.2.2 La rete come espansione in basi adattiva

Nella (11.1) ogni termine

$$\phi_j(\mathbf{x}, \mathbf{w}) = f_j\Big(\sum_i w_{ji}\, x_i\Big)$$

può essere visto come calcolato da un'unità nascosta oppure come una **funzione di base** di una LBE (§ 4.7). Il confronto è istruttivo:

- **LBE**: $h(\mathbf{x}) = \sum_j w_j\, \phi_j(\mathbf{x})$ (regressione) o $h(\mathbf{x}) = f\big(\sum_j w_j \phi_j(\mathbf{x})\big)$ (classificazione), con le ϕ **fissate a priori**: il modello è lineare nei parametri;
- **rete neurale**: $h(\mathbf{x}) = f_k\big(\sum_j w_{kj}\, \phi_j(\mathbf{x}, \mathbf{w})\big)$, dove le ϕ **dipendono dai pesi** e quindi **si adattano ai dati** durante l'addestramento.

> **Basi adattive.** Le reti neurali sono un approccio a **funzioni di base adattive**: ogni unità nascosta calcola una nuova **feature derivata non lineare**, appresa dai dati. La capacità rappresentativa del modello è legata alla presenza di uno strato nascosto con attivazioni non lineari, che trasforma l'input nella **rappresentazione interna** della rete; l'apprendimento definisce una rappresentazione adatta al task, che permette di estrarre le statistiche di ordine superiore utili per approssimare la funzione target.

Le unità nascoste devono essere non lineari. Un MLP con unità nascoste **lineari** equivale a una rete a **un solo strato**.

Dimostrazione. Con f identità nelle unità nascoste e in uscita:

$$h(\mathbf{x}) = \sum_j w_{kj} \sum_i w_{ji} x_i = \sum_i \Big(\sum_j w_{kj} w_{ji} \Big) x_i = \sum_i w'_i x_i, \qquad w'_i = \sum_j w_{kj} w_{ji},$$

cioè ancora un modello lineare: la composizione di funzioni lineari è lineare. ■

Il prezzo della non linearità è che il modello è **non lineare nei parametri**: l'addestramento diventa un problema di **ottimizzazione non lineare e non convesso**.

11.2.3 Approssimazione universale

Teorema — Approssimazione universale. *Una rete con **un solo strato nascosto** (con attivazioni logistiche) e uscita lineare può approssimare arbitrariamente bene **ogni funzione continua** su un ipercubo, purché abbia abbastanza unità nascoste: data f e dato $\varepsilon > 0$ esiste h tale che $|f(\mathbf{x}) - h(\mathbf{x})| < \varepsilon$ per ogni $\mathbf{x}$ dell'ipercubo. Più in generale un MLP può approssimare arbitrariamente bene ogni mappatura input-output, con abbastanza unità nascoste.*

Attenzione — È un teorema di esistenza. Il teorema dice che la rete **esiste**, ma non fornisce né l'**algoritmo** per trovarla né il **numero di unità** necessario. Restano quindi due questioni: come apprendere (la backpropagation) e come scegliere l'architettura (model selection, § 12.7).

11.2.4 Potere espressivo e VC-dimension

Il potere espressivo di una rete dipende dal **numero di unità** e dalla loro configurazione. La VC-dimension dipende dal numero di parametri (in un MLP completamente connesso circa il numero di input per il numero di unità nascoste, più le unità nascoste per le uscite), ma anche dal **valore** dei pesi:

- pesi nulli: VC-dimension minima;
- pesi piccoli: si lavora nella parte lineare delle sigmoidi, VC-dimension piccola;
- pesi grandi: modello più complesso.

Poiché l'addestramento parte da pesi vicini a zero e i pesi crescono durante il training, la rete è uno **spazio delle ipotesi di dimensione variabile**: la VC-dimension effettiva cresce durante l'apprendimento. Su questo si basa l'early stopping (§ 1.).

Il teorema di approssimazione universale dice che uno strato nascosto basta, ma non che basti un numero *piccolo* di unità: esistono funzioni che con un solo strato richiederebbero un numero esponenziale di unità e con più strati molte meno (§ 15.2).

11.2.5 Il bias induttivo

Per le reti addestrate con backpropagation il bias induttivo è legato alla **smoothness**: piccole variazioni dell'input producono piccole variazioni dell'output (ad esempio derivata prima localmente limitata). È un'assunzione molto comune e ragionevole: se la funzione target non è liscia (un generatore di numeri casuali) la generalizzazione è comunque impossibile.

11.3 Verso l'algoritmo di apprendimento

11.3.1 Il problema del credit assignment

Il Perceptron ha un algoritmo di apprendimento ma non rappresenta tutte le funzioni booleane; una rete di unità a soglia le rappresenta tutte ma serve un algoritmo per addestrarla. Il problema è il **credit assignment**: quanta "responsabilità" dell'errore attribuire alle unità nascoste? Per le unità di uscita si conosce la risposta desiderata e quindi l'errore; per le unità nascoste **non si sa quale dovrebbe essere la loro uscita**, quindi il segnale d'errore non è direttamente misurabile. In più il modello è non lineare nei pesi. Il problema è risolto dall'algoritmo di **backpropagation**.

> **Definizione — Loading problem.** Data una rete e un insieme di esempi, esiste un insieme di pesi per cui la rete è consistente con gli esempi? (Si tratta di "caricare" i dati di training nei parametri liberi.) Il loading problem è **NP-completo**: non è noto un algoritmo polinomiale. In pratica però le reti si addestrano in tempi ragionevoli con la backpropagation, senza garanzia di ottimalità.

11.3.2 L'idea: estendere la discesa del gradiente

La discesa del gradiente si estende agli MLP purché **loss e funzioni di attivazione siano differenziabili**: bisogna trovare il delta di **ogni** unità. Il problema resta lo stesso: dati l esempi $(\mathbf{x}_p, d_p)$ e una loss (ad esempio l'errore quadratico), trovare $\mathbf{w}$ che minimizzi $E(\mathbf{w}) = R_{\text{emp}}$ calcolandone il gradiente. La backpropagation

- è semplice grazie alla **forma composizionale** del modello;
- usa solo quantità **locali** a ogni unità, preservandone la modularità;
- è **efficiente**: costa $O(\#W)$ invece di $O(\#W^2)$, grazie alla fattorizzazione dei delta.

11.4 La backpropagation

> **Attenzione — Come studiare la derivazione.** La derivazione va saputa **rifare**, non memorizzare: l'obiettivo è saperla ricalcolare per loss, attivazioni o architetture diverse.

11.4.1 Il problema

La rete di riferimento è un MLP completamente connesso con indici i (input), j (nascoste), k (uscita; K unità di uscita) e training set $TR = \{(\mathbf{x}^1, \mathbf{d}^1), \dots, (\mathbf{x}^l, \mathbf{d}^l)\}$. Ogni unità di input carica la componente del pattern: $o_i = x_i$. L'**errore totale** è

$$E_{tot} = \sum_p E_p, \qquad E_p = \frac{1}{2} \sum_{k=1}^{K} (d_k - o_k)^2, \tag{11.3}$$

dove E_p è l'errore sull'esempio p su tutte le uscite; il fattore $\frac{1}{2}$ serve solo a semplificare la derivata. D'ora in poi l'indice p è omesso: o_k sta per $o_k(\mathbf{x}_p)$ e d_k per $d_{p,k}$.

> **Definizione — Il problema della backpropagation.** Stimare il **contributo delle unità nascoste all'errore** in uscita, cioè calcolare per ogni peso della rete $\partial E_{tot}/\partial w$: la **regola delta generalizzata** (*Generalized Delta Rule*).

L'obiettivo è trovare i pesi che minimizzano E_{tot} scendendo lungo la sua superficie seguendo il gradiente. A differenza del modello lineare la superficie **non è convessa** (Figura 11.2).

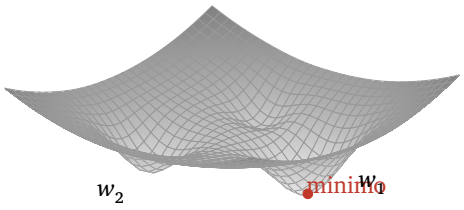


Figura 11.2. Una superficie d'errore non convessa nello spazio di due pesi (andamento qualitativo): più minimi locali, e il punto di arrivo della discesa dipende dal punto di partenza.

Algoritmo — Backpropagation (schema iterativo).

Inizializzare tutti i pesi $\mathbf{w}$ e η; calcolare le uscite delle unità ed E_{tot}.
Finché $E_{tot} > \epsilon$ (o un altro criterio di arresto):

1. per ogni peso w: $\Delta w = -\dfrac{\partial E_{tot}}{\partial w}$ (passo 1: calcolo del gradiente)
2. $w_{new} = w + \eta\,\Delta w + \dots$ (passo 2: regola di aggiornamento)
3. ricalcolare le uscite ed E_{tot}.

Bisogna distinguere il **calcolo del gradiente** (passo 1, ciò che fa la backpropagation) dalla **regola di aggiornamento** (passo 2: standard, con momentum, Quickprop, R-prop, …; capitolo 12). Per il passo 1:

$$\Delta w = -\frac{\partial E_{tot}}{\partial w} = -\sum_p \frac{\partial E_p}{\partial w} \overset{\text{def}}{=} \sum_p \Delta_p w.$$

11.4.2 La derivazione

Il gradiente per un peso generico. Si consideri un peso generico w_{tu}, che porta all'unità generica t l'input proveniente dall'unità generica u (che può essere un i o un j). Poiché $net_t = \sum_s w_{ts}\, o_s$ e $o_t = f_t(net_t)$, la regola della catena attraverso net_t dà

$$\Delta_p w_{tu} = -\frac{\partial E_p}{\partial w_{tu}} = \underbrace{-\frac{\partial E_p}{\partial net_t}}_{\delta_t} \cdot \underbrace{\frac{\partial net_t}{\partial w_{tu}}}_{o_u} = \delta_t\, o_u, \tag{11.4}$$

perché $\frac{\partial net_t}{\partial w_{tu}} = \frac{\partial \sum_s w_{ts} o_s}{\partial w_{tu}} = o_u$: tutti i termini della somma sono costanti rispetto a w_{tu} tranne quello con $s = u$. Si è definito il **delta dell'unità** t

$$\delta_t = -\frac{\partial E_p}{\partial net_t},$$

che si espande ancora con la regola della catena passando per $o_t = f_t(net_t)$:

$$\delta_t = -\frac{\partial E_p}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot \frac{\partial o_t}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot f_t'(net_t). \tag{11.5}$$

Resta da calcolare $-\partial E_p/\partial o_t$, e qui si distinguono due casi.

Caso 1: unità di uscita ($t = k$). L'uscita o_k compare direttamente in E_p:

$$-\frac{\partial E_p}{\partial o_k} = -\frac{\partial \, \frac{1}{2} \sum_{r=1}^{K} (d_r - o_r)^2}{\partial o_k} = (d_k - o_k),$$

perché solo il termine $r = k$ dipende da o_k, con derivata $\frac{1}{2} \cdot 2(d_k - o_k) \cdot (-1)$. Quindi

$$\boxed{\delta_k = (d_k - o_k) \, f'_k(net_k)} \tag{11.6}$$

È il delta della singola unità sigmoidale (§ 10.7.2): l'errore è misurabile perché si conosce il target.

Caso 2: unità nascosta ($t = j$). Un'unità nascosta non ha target e o_j non compare direttamente in E_p. Però o_j influenza E_p **attraverso tutte le unità di uscita** a cui è collegata, perché ogni net_k dipende da o_j. La regola della catena con una somma su tutte le k dà

$$-\frac{\partial E_p}{\partial o_j} = \sum_{k=1}^{K} \underbrace{-\frac{\partial E_p}{\partial net_k}}_{\delta_k} \cdot \frac{\partial net_k}{\partial o_j} = \sum_{k=1}^{K} \delta_k \, w_{kj},$$

poiché $\frac{\partial net_k}{\partial o_j} = \frac{\partial \sum_s w_{ks} o_s}{\partial o_j} = w_{kj}$. Quindi

$$\boxed{\delta_j = \Big(\sum_{k=1}^{K} \delta_k \, w_{kj} \Big) f'_j(net_j)} \tag{11.7}$$

> **Il punto cruciale.** Il termine $-\partial E_p / \partial net_k$ è **già stato calcolato**: è δ_k. Il delta di un'unità nascosta si ottiene **propagando all'indietro** i delta delle unità di uscita, pesati con gli stessi pesi w_{kj} usati in avanti: l'errore "risale" la rete, da cui il nome *back-propagation*. Questo risolve il credit assignment: la "colpa" di un'unità nascosta è la somma delle colpe delle unità a cui contribuisce, pesata da quanto contribuisce (Figura 11.3).

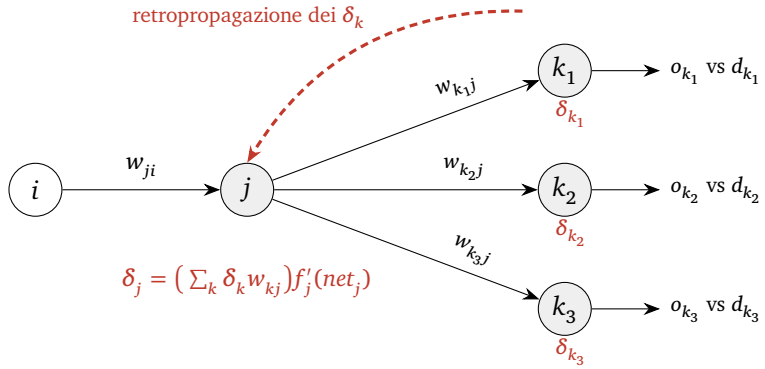


Figura 11.3. Retropropagazione dei delta: l'influenza dell'unità nascosta j sull'errore si calcola dai delta delle unità a cui è collegata e dai pesi w_{kj}. Ogni termine coinvolge solo unità e pesi adiacenti.

Lo stesso risultato si può ottenere derivando direttamente la definizione di E_p (per esercizio), ma la forma vista è più istruttiva perché mostra il flusso delle derivate parziali **sulla rete**.

Tabella 11.1. La backpropagation per una rete a due strati: $\Delta_p w_{tu} = \delta_t\, o_u$.

Peso	Aggiornamento	Delta
w_{kj} (nascosta → uscita)	$\Delta_p w_{kj} = \delta_k\, o_j$	$\delta_k = (d_k - o_k)\, f'_k(net_k)$
w_{ji} (input → nascosta)	$\Delta_p w_{ji} = \delta_j\, o_i$	$\delta_j = \left(\sum_k \delta_k w_{kj}\right) f'_j(net_j)$

11.4.3 Sintesi delle formule

δ_t è il **segnale d'errore** disponibile all'unità t e o_u l'input a t proveniente da u. La derivazione vale per **qualsiasi numero di strati nascosti**: i delta di un'unità arrivano da tutte le unità a cui è connessa,

$$\delta_j = \left(\sum_{h \in \text{successori}(j)} \delta_h\, w_{hj} \right) f'_j(net_j), \tag{11.8}$$

e la regola di aggiornamento per ogni esempio è $w_{tu}^{new} = w_{tu} + \eta\, \delta_t\, o_u$. Usando $\Delta_p w$ esempio per esempio si ottiene la versione **on-line**; sommando i contributi di tutti gli esempi prima di aggiornare, la versione **batch**.

Algoritmo — Ciclo di addestramento completo.

1. **Fase forward**: calcolare le uscite di tutte le unità, strato per strato.
2. Calcolare errori e delta nello strato di uscita.
3. **Fase backward**: propagare i delta dallo strato di uscita agli strati nascosti.
4. Aggiornare i pesi.
5. Ricominciare, fino al criterio di arresto.

Attenzione — Non dimenticare i bias. Calcolo e aggiornamento valgono anche per i **bias** di tutte le unità: con input costante $o_0 = 1$ e bias w_{t0}, si ha $\Delta w_{t0} = \delta_t \cdot 1$.

11.4.4 Proprietà e interpretazione

- **Calcoli solo locali**: ogni unità usa solo informazioni delle unità vicine (sotto e sopra); il resto arriva indirettamente con il flusso dei delta. Eppure la modifica di un peso ha un **effetto globale** sulla rete: l'influenza complessa di ogni w_{tu} sull'errore globale è stata **scomposta** in componenti semplici, i delta locali.
- **Processo di diffusione**: una propagazione locale dai "genitori" dell'unità all'unità e dall'unità ai "figli".
- **Efficienza (la fattorizzazione dei delta)**: il delta δ_t di ogni unità si calcola **una sola volta** e si riusa per **tutti** i suoi pesi w_{tu}. Il costo è proporzionale al numero di pesi, non al suo quadrato: con 10^9 pesi sono 10^9 operazioni invece di 10^{18}.

Tre livelli di lettura.

1. I singoli **passaggi matematici**.
2. Il significato delle derivate parziali come **variazioni locali** di una quantità rispetto a un'altra.
3. Il **quadro generale**: la scomposizione serve a trovare i delta di ogni livello, scomponendo l'errore *sulla rete* che si ha davanti.

Attenzione — Terminologia. Non si dice "rete backprop": la rete neurale è il **modello**, la backpropagation è un **algoritmo di addestramento** basato sul calcolo del gradiente attraverso la rete.

Esempio — Un passo di backpropagation. Rete con un input $x = 1$, un'unità nascosta sigmoidale e un'unità di uscita lineare, senza bias; pesi $w_{ji} = 0{,}5$, $w_{kj} = 1$, target $d = 1$, $\eta = 0{,}1$.

- **Forward**: $net_j = 0{,}5$, $o_j = \sigma(0{,}5) \approx 0{,}622$; $o_k = 1 \cdot 0{,}622 = 0{,}622$.
- **Delta di uscita** (lineare, $f' = 1$): $\delta_k = (1 - 0{,}622) \cdot 1 = 0{,}378$.
- **Delta nascosto**: $\delta_j = (\delta_k w_{kj})\, \sigma'(net_j) = 0{,}378 \cdot 0{,}622 \cdot (1 - 0{,}622) \approx 0{,}089$.
- **Aggiornamenti**: $\Delta w_{kj} = \eta\, \delta_k\, o_j \approx 0{,}1 \cdot 0{,}378 \cdot 0{,}622 \approx 0{,}0235$; $\Delta w_{ji} = \eta\, \delta_j\, x \approx 0{,}0089$.

Entrambi i pesi aumentano, facendo crescere l'uscita verso il target.

Punti chiave

- MLP: $h(\mathbf{x}) = f_k\big(\sum_j w_{kj} f_j(\sum_i w_{ji} x_i)\big)$; una LBE con basi $\phi_j(\mathbf{x}, \mathbf{w})$ adattive.
- Unità nascoste lineari $\Rightarrow$ modello lineare. Un solo strato nascosto basta per approssimare funzioni continue (teorema di esistenza).
- La VC-dimension dipende anche dal valore dei pesi: spazio delle ipotesi di dimensione variabile.
- Backpropagation: $\Delta_p w_{tu} = \delta_t o_u$, $\delta_k = (d_k - o_k) f'_k$, $\delta_j = (\sum_k \delta_k w_{kj}) f'_j$; costo lineare nel numero di pesi.

Domande di verifica

1. Perché le unità nascoste devono essere non lineari?
2. In che senso una rete neurale è una LBE adattiva?
3. Enunciare il teorema di approssimazione universale e discuterne i limiti.
4. Cos'è il problema del credit assignment? E il loading problem?
5. Derivare la backpropagation per un MLP a due strati con errore quadratico. Come cambia con una loss diversa o con più strati nascosti?
6. Perché la backpropagation ha costo lineare nel numero dei pesi?

Capitolo 12

Addestrare una rete neurale

Con la backpropagation si sa calcolare il gradiente di ogni peso; restano le scelte che servono per addestrare davvero un MLP: inizializzazione, minimi multipli, on-line/batch/mini-batch, learning rate e momentum, criteri di arresto, overfitting e regolarizzazione, numero di unità (con il Cascade Correlation), rappresentazione di input e output, validazione specifica per le reti. Il capitolo si chiude con pregi e difetti delle reti neurali e con un banco di prova per verificare un'implementazione.

12.1 L'addestramento come percorso

> **Un'interpretazione utile.** La backpropagation è un **percorso nello spazio dei pesi**, sulla superficie della loss. Il percorso dipende dai dati, dal modello, dal **punto di partenza** (pesi iniziali), dalla **velocità** di convergenza e dal **punto finale** (regola di arresto). Tutte queste scelte sono un **controllo della ricerca** nello spazio delle ipotesi.

L'algoritmo di base resta la discesa del gradiente: pesi iniziali piccoli, $\Delta\mathbf{w} = -\partial E/\partial\mathbf{w}$ calcolato con la backpropagation (fase forward fino alle uscite, fase backward per i delta), $\mathbf{w}_{new} = \mathbf{w} + \eta\,\Delta\mathbf{w}$, ripetuto. Il modello è spesso **sovra-parametrizzato** e l'ottimizzazione è **non convessa** e potenzialmente instabile: da qui le questioni di questo capitolo. Gli **iperparametri** sono i valori da fissare per eseguire l'addestramento. Le tecniche segnate con **(!)** sono di uso comune e vanno usate in pratica; le altre sono varianti da conoscere. Le "regole empiriche" trovate in rete vanno evitate, a meno che non siano dimostrate in letteratura scientifica o se ne sia verificato sperimentalmente il vantaggio.

12.2 Inizializzazione e minimi multipli

> **(!) Inizializzazione.** Inizializzare i pesi con **valori casuali vicini a zero**, ad esempio nell'intervallo $[-0{,}7;\ +0{,}7]$ per dati standardizzati.

Da evitare:

- pesi **tutti nulli**: i delta delle unità nascoste sarebbero nulli (dipendono dai pesi verso l'uscita, (11.7)) e la rete non imparerebbe;
- pesi **grandi**: le sigmoidi partono **saturate**, con derivate quasi nulle e apprendimento lentissimo;
- pesi **tutti uguali**: le unità nascoste restano **simmetriche**, ricevono gli stessi aggiornamenti e calcolano tutte la stessa cosa. La casualità serve a **rompere la simmetria**.

Si può tenere conto del **fan-in** (numero di input di un'unità), ad esempio con un intervallo range $\cdot$ 2/fan-in, ma non se il fan-in è troppo grande e non per le unità di uscita (altrimenti i delta partono vicini a zero). Molto usata è l'inizializzazione di **Glorot e Bengio**: bias a 0 e pesi da una distribuzione uniforme in $[-1/a, +1/a]$ con $a = \sqrt{\text{fan-in}}$ (in varianti successive a dipende anche dal fan-out). Una rete molto piccola può dipendere fortemente dall'inizializzazione.

12.2.1 Minimi multipli

La loss **non è convessa**: ci sono molti minimi locali e il risultato dipende dai pesi iniziali.

> **(!) Più inizializzazioni.** Provare diverse configurazioni iniziali casuali (ad esempio 5–10 o più *trial*) e riportare la **media** e la **varianza** dei risultati. Come scegliere il modello finale tra le prove è discusso nel § 12.9; si possono anche usare tutti i modelli con una risposta **di comitato** (§ 8.4).

Fermarsi in un minimo locale con errore alto non è un grosso problema: lo si vede dall'errore finale e si può ricominciare; spesso un **"buon" minimo locale è sufficiente**. In ogni caso il minimo globale non è l'obiettivo:

1. nel ML **non si cerca né il minimo locale né quello globale di** R_{emp}, ma il minimo di R, che non si può calcolare; spesso ci si ferma in un punto che non ha nemmeno gradiente nullo;
2. la rete è uno spazio delle ipotesi di dimensione variabile (§ 11.2.4): durante l'addestramento l'errore di training scende verso il minimo mentre la rete diventa troppo complessa, e bisogna fermarsi **prima** di questa condizione di *overtraining*, indipendentemente dalla questione dei minimi.

12.3 On-line, batch e mini-batch

Con la notazione della backpropagation, per il peso w_{tu}:

$$\text{batch: } -\frac{\partial E}{\partial w_{tu}} = -\sum_{p=1}^{l} \frac{\partial E_p}{\partial w_{tu}} = \sum_{p=1}^{l} \delta_{p,t}\, o_{p,u}, \qquad \text{on-line: } -\frac{\partial E_p}{\partial w_{tu}} = \delta_{p,t}\, o_{p,u}.$$

Nel batch si somma su tutta l'epoca e poi si aggiorna (§ 4.4.3); nell'on-line si aggiorna dopo ogni esempio. Il batch dà una stima **più accurata** del gradiente. Nell'on-line il gradiente di un esempio è un'approssimazione **rumorosa** del gradiente complessivo (da cui il nome **stochastic gradient descent**, SGD): la discesa procede a zig-zag, e questo **può aiutare a sfuggire ai minimi locali**.

> **(!) Shuffling.** Nella versione on-line i pattern vanno presentati in **ordine diverso** a ogni epoca, di solito **mescolandoli casualmente**, per evitare una deriva sistematica della discesa.

Nel **mini-batch** si divide ogni epoca in parti: si sommano i gradienti di mb esempi ($1 < mb < l$) prima di aggiornare i pesi, e si ripete finché tutti i dati dell'epoca sono stati usati (Figura 12.1). Lo SGD con mini-batch usa il campionamento casuale (shuffling) per una stima non distorta del gradiente; poiché il costo di un aggiornamento dipende da mb e non da l, è un fattore chiave per scalare a dataset di milioni di esempi (con dati enormi o in streaming si può fare anche a meno delle epoche).

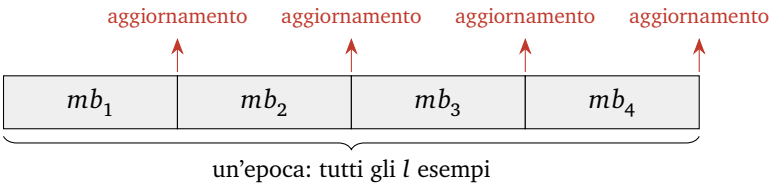


Figura 12.1. Un'epoca divisa in quattro mini-batch: i pesi si aggiornano dopo ogni mini-batch.

Tabella 12.1. On-line, mini-batch e batch.

On-line	Mini-batch (SGD)	Batch
non sfrutta il parallelismo tra esempi	compromesso; si adatta alla memoria della GPU	stima più accurata del gradiente
può essere instabile		più lento (aspetta tutti i dati)
il rumore può avere un effetto regolarizzante		

12.4 Il learning rate

- **Batch**: gradiente più accurato, si può usare un η più alto.
- **On-line**: più veloce ma più instabile, serve un η più piccolo.
- In generale un η **alto** è veloce ma può essere instabile, uno **basso** è stabile ma può essere troppo lento; con LMS si cerca tipicamente in $[0{,}01;\ 0{,}5]$.

(!) Guardare sempre la curva di apprendimento. Il grafico dell'errore di training durante l'addestramento mostra la **qualità** della discesa e la **velocità** di convergenza (il valore assoluto dell'errore dipende anche dalla capacità del modello). Si cerca una curva che scende rapidamente **e in modo liscio** (Figura 12.2); una curva irregolare può dipendere da batch troppo piccoli o da η troppo alto, e va evitata. Qui si guardano solo i fallimenti dell'ottimizzazione sul training, non la generalizzazione.

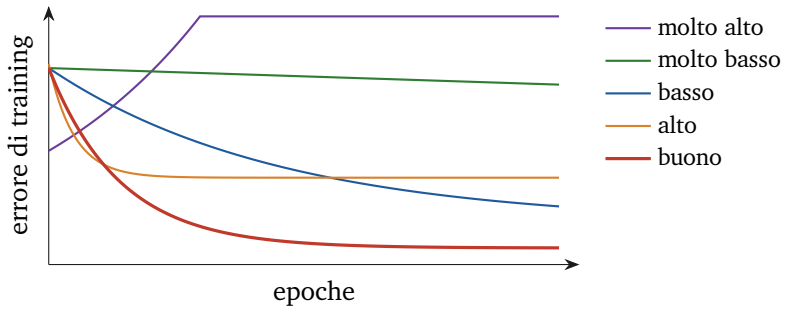


Figura 12.2. Effetto del learning rate sulla curva di training (andamenti qualitativi): con η molto alto la discesa diverge, molto basso resta quasi ferma, basso scende lentamente, alto scende in fretta ma si ferma presto; quello buono scende rapidamente e in modo liscio.

(!) Usare la media dei gradienti. Conviene usare la **media** dei gradienti sull'epoca (somma divisa per l): rende l'approccio uniforme rispetto al numero di dati (è il *least mean squares*) ed equivale a usare η/l. Confrontando on-line e batch, l'on-line richiede un η molto più piccolo (dello stesso fattore $1/l$) per essere comparabile; nel mini-batch, se si divide per mb, va ritarato η, oppure si divide sempre per l.

12.4.1 Momentum (!)

$$\Delta \mathbf{w}_{new} = -\eta \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} + \alpha \, \Delta \mathbf{w}_{old}, \qquad \mathbf{w}_{new} = \mathbf{w} + \Delta \mathbf{w}_{new} \tag{12.1}$$

con $0 < \alpha < 1$ (ad esempio 0,5–0,9); qui η è incluso in $\Delta\mathbf{w}$ e dopo ogni passo si salva $\Delta\mathbf{w}_{old} = \Delta\mathbf{w}_{new}$. È il metodo della **"palla pesante"**: a ogni passo si aggiunge una frazione dello spostamento precedente, come se i pesi avessero un'inerzia.

- **Più veloce nei plateau**: se il gradiente ha lo stesso segno in iterazioni consecutive, i passi si sommano e il passo effettivo cresce.
- **Smorza le oscillazioni**: nelle direzioni in cui il gradiente cambia segno i contributi si compensano (Figura 12.3).
- Per l'effetto inerzia permette di usare η più alti.

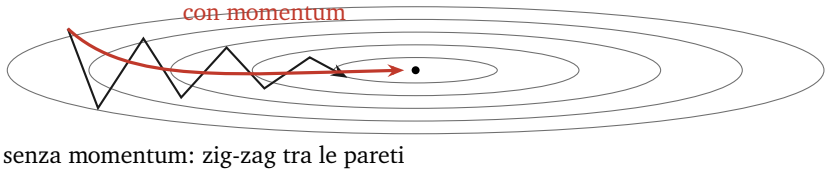


Figura 12.3. Una valle stretta della superficie d'errore (Hessiana mal condizionata): il gradiente puro rimbalza tra le pareti, il momentum procede lungo la valle verso il minimo.

Il momentum nasce per il batch. Nell'on-line $\Delta\mathbf{w}_{old}$ è quello dell'**esempio precedente**, e il momentum diventa una **media mobile dei gradienti passati** che liscia i gradienti stocastici dentro l'epoca (analogamente nel mini-batch): il significato è diverso dal batch, dove si combinano gradienti calcolati sugli stessi esempi in epoche diverse.

Nel **momentum di Nesterov** si applica prima il momentum ($\mathbf{w} + \alpha\Delta\mathbf{w}_{old}$), si valuta il gradiente in questo punto intermedio, e poi si applica il $\Delta\mathbf{w}$ complessivo (momentum più nuovo gradiente) ai pesi originali. Migliora la velocità di convergenza nel batch (non nel caso stocastico).

12.4.2 Learning rate variabile e adattivo

Con il mini-batch il gradiente **non tende a zero** vicino al minimo (per il rumore di campionamento), quindi un η fisso va evitato. Ad esempio lo si decresce linearmente fino all'iterazione τ,

$$\eta_s = \left(1 - \frac{s}{\tau}\right)\eta_0 + \frac{s}{\tau}\,\eta_\tau, \qquad s \le \tau,$$

e poi si usa $\eta = \eta_\tau$ costante e piccolo (ad esempio $\eta_\tau \approx 1\%$ di η_0, τ qualche centinaio di passi); η_0 si sceglie con il compromesso tra instabilità e blocco (prove preliminari o grid search). Gli algoritmi a **learning rate adattivo** lo regolano automaticamente, anche separatamente **per ogni parametro**: **AdaGrad**, **RMSProp**, **Adam** (molto popolare e spesso robusto con i valori di default, ma da usare con cautela); si combinano con il momentum. Nelle reti con molti strati η può dipendere anche dallo strato o dal fan-in.

12.4.3 Oltre lo SGD di base

Lo SGD è efficiente (lineare nel numero di parametri) ma dipende dal numero di dati e di epoche. Esistono molte euristiche: **Quickprop** (salta al minimo di una parabola che approssima localmente la loss), **R-prop** (usa solo il **segno** del gradiente, utile quando il gradiente svanisce negli strati profondi). I **metodi del secondo ordine** usano l'Hessiana (derivate seconde) per tenere conto della curvatura (metodo di Newton), ma l'Hessiana è costosissima; ne esistono versioni approssimate (Gauss-Newton, Levenberg-Marquardt, metodi *Hessian-free* come il gradiente coniugato, metodi quasi-Newton come BFGS).

> **Attenzione — Punti di sella.** Un punto di sella ha gradiente (quasi) nullo ma non è un minimo. In alta dimensione i punti di sella sono esponenzialmente più numerosi dei minimi locali. Il metodo di Newton cerca i punti a gradiente nullo e quindi è attratto dai punti di sella: può essere uno dei motivi per cui i metodi del secondo ordine non hanno sostituito la discesa del gradiente per le reti neurali, mentre la discesa del gradiente empiricamente riesce a sfuggire ai punti di sella.

Gli approcci di base (quelli segnati con (!)), applicati correttamente, di solito bastano: l'esperienza con tecniche note conta più di metodi sofisticati che non si sanno gestire. La tecnica di ottimizzazione conta solo se ci sono ostacoli all'addestramento: **la validazione conta più dell'ottimizzazione sul training set**, che non è l'obiettivo. Il punto di partenza è lo **SGD con momentum** (e regolarizzazione).

12.5 Criteri di arresto

- Il criterio di base è l'errore (errore medio $< \epsilon$): il migliore se si conosce la tolleranza dei dati (conoscenza dell'esperto), il che spesso non accade. Varianti: tolleranza massima invece che media; per la classificazione, il numero di errori.
- Criteri "interni": pesi che non cambiano più, gradiente quasi nullo (norma $< \epsilon$), errore che non diminuisce più in modo significativo in un'epoca (ad esempio meno dello 0,1%). Possono essere **prematuri** (ad esempio con η troppo piccolo), quindi si osservano k epoche (**patience**).
- **(!)** Fermarsi comunque dopo un numero eccessivo di epoche, per sfuggire a convergenze troppo lente, ma **non** fermarsi a un numero arbitrario di epoche fissato a priori (§ 12.9).
- Non necessariamente fermarsi con un errore di training molto basso: è il tema dell'overfitting.

12.6 Overfitting e regolarizzazione

Tipicamente **non** si vuole il minimizzatore globale di $R_{\text{emp}}(\mathbf{w})$, perché è probabilmente una soluzione in overfitting: è la differenza fondamentale rispetto all'ottimizzazione pura. Lo scopo è il **controllo della complessità**, tramite regolarizzazione (esplicita con una penalità, o implicita con l'early stopping) più la model selection per trovare il compromesso.

12.6.1 Come nasce l'overfitting in una rete

- Si parte con **pesi piccoli e casuali**.
- La mappatura input-output è quasi **lineare** (le sigmoidi lavorano nella zona lineare): il numero effettivo di parametri liberi, e la VC-dimension, è quasi quello di un'unità lineare.
- Man mano che l'ottimizzazione procede, le unità nascoste tendono a **saturare**, il numero effettivo di parametri e le non linearità aumentano: la VC-dimension cresce.

La curva della Figura 12.4 va letta insieme al bound della SLT. Gli strumenti sono tre:

1. **early stopping**: usare un **validation set** per decidere quando fermarsi, cioè quando l'errore di validazione inizia a salire. È un criterio vago: bisogna osservare più epoche prima di decidere (patience) ed eventualmente tornare indietro (*backtracking*). Poiché il numero effettivo di parametri cresce durante l'addestramento, **fermarsi prima equivale a limitare la complessità effettiva**;
2. **regolarizzazione** della loss, lo strumento principale;

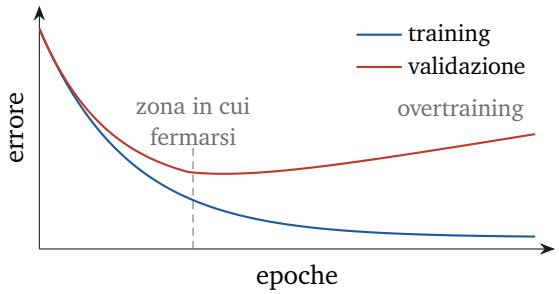


Figura 12.4. Comportamento tipico della backpropagation (qualitativo): l'errore di training decresce sempre, quello di validazione raggiunge un minimo e poi risale.

3. metodi di **pruning** (eliminazione di pesi o unità).

12.6.2 Regolarizzazione (!)

Si usa la penalità di Tikhonov (§ 5.3) su **tutti i pesi della rete**:

$$\text{Loss}(\mathbf{w}) = \sum_p \big(d_p - o(\mathbf{x}_p)\big)^2 + \lambda \|\mathbf{w}\|^2, \qquad \|\mathbf{w}\|^2 = \sum_i w_i^2,$$

con l'effetto di **weight decay** (5.2): si aggiunge circa $2\lambda w$ al gradiente di ogni peso (il 2 si può omettere). λ è in genere piccolo (ad esempio 0,01) e si sceglie con la model selection. Applicata a un modello lineare è la ridge regression; esistono penalità più sofisticate (ad esempio per la *weight elimination*). Nei risultati **(!)** si riporta l'errore (il termine sui dati), non la loss.

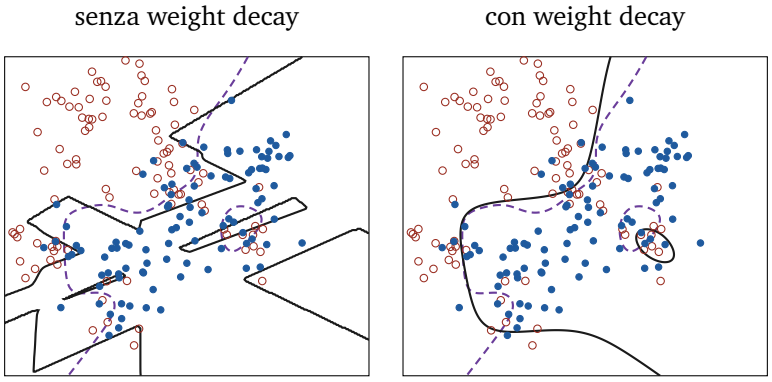


Figura 12.5. Una rete con 10 unità nascoste sul problema a due classi, senza e con weight decay (nero; tratteggiato il confine di Bayes). Con la regolarizzazione il confine è più regolare e l'errore di test scende da 0,253 a 0,229 (errore di Bayes 0,216).

Attenzione — La regolarizzazione non serve a stabilizzare la convergenza. La regolarizzazione serve a **controllare la complessità** del modello (la VC-dimension, legata al numero e al valore dei pesi), non a rendere più stabile la curva di apprendimento. Se la curva è instabile il problema è altrove (learning rate, dimensione dei batch, …).

Early stopping o regolarizzazione?. L'early stopping è un approccio **empirico**, che richiede un validation set e sacrifica una parte dei dati. La regolarizzazione nella loss è un approccio **fondato**: permette alla curva di validazione di seguire quella di training (purché λ non sia così alto da dare underfitting), e in quel caso non serve l'early stopping e ci si può fermare a convergenza. Si possono usare entrambi: l'early stopping non interviene se l'errore di validazione

non sale.

Alcuni dettagli:

- spesso il **bias** w_0 **è escluso** dal regolarizzatore (includerlo renderebbe i risultati dipendenti da traslazioni o scalature del target), oppure ha un proprio coefficiente;
- la regolarizzazione si applica tipicamente nella versione batch; nell'on-line e nel mini-batch la penalità viene applicata a ogni passo, cioè molte volte per epoca, e per un confronto equo con il batch si userebbe $\lambda \cdot mb/l$ (se λ si sceglie con la model selection, questo avviene automaticamente);
- un'altra tecnica di regolarizzazione è il **dropout** (§ 15.6.4).

12.6.3 Momentum e weight decay insieme

Con la loss regolarizzata, separando i due termini e usando η solo per il termine d'errore (come nel § 5.3), la regola con momentum diventa

$$\Delta w_{tu} = -\eta \frac{\partial \text{Loss}}{\partial w_{tu}} + \alpha \, \Delta w_{tu}^{old} \doteq \eta \, \delta_t \, o_u - \lambda \, w_{tu} + \alpha \, \Delta w_{tu}^{old}, \qquad w_{tu}^{new} = w_{tu} + \Delta w_{tu},$$

così η non moltiplica λ e cambiando l'uno non si cambia l'altro. Spesso il momentum include anche la parte di λ; per rendere **indipendenti tutti e tre gli iperparametri** η, α, λ si può separare il weight decay dalla memoria del momentum:

$$\Delta w_{tu} = \eta \, \delta_t \, o_u + \alpha \, \Delta w_{tu}^{old}, \qquad w_{tu}^{new} = w_{tu} + \Delta w_{tu} - \lambda \, w_{tu}.$$

Si divide per l solo il gradiente di E, dove c'è la somma sugli esempi.

12.7 Numero di unità nascoste

Il numero di unità è legato al controllo della complessità, alla dimensione dell'input e a quella del training set: in generale è un problema di **model selection**, e il numero di unità si sceglie insieme a λ con la cross-validation. Troppe poche unità danno underfitting, troppe possono dare overfitting; il numero può essere più alto se si usa un'adeguata regolarizzazione. Alternative alla ricerca manuale sono gli **approcci costruttivi**, che partono da una rete piccola e aggiungono unità, e i **metodi di pruning**, che partono da reti grandi ed eliminano pesi o unità.

12.7.1 Cascade Correlation

Il **Cascade Correlation** (CC) è l'approccio costruttivo più noto, per regressione e classificazione. Apprende **sia i pesi sia la topologia**: lo spazio delle ipotesi ha dimensione decisa dall'algoritmo, e a ogni passo si addestra una sola unità.

Algoritmo — Cascade Correlation.

1. Si parte da una rete N_0 **senza unità nascoste** e la si addestra. Se non risolve il problema si passa a N_1.
2. In N_1 si aggiunge un'unità nascosta i cui pesi in ingresso sono addestrati per **massimizzare la correlazione** tra la sua uscita e l'**errore residuo** della rete precedente.
3. Dopo l'addestramento i pesi in ingresso della nuova unità vengono **congelati** e si riaddestrano i pesi rimanenti (lo strato di uscita).

4. Se la rete non risolve il problema si aggiungono altre unità, ciascuna collegata a **tutti gli input e a tutte le unità nascoste precedenti** (da qui "a cascata").
5. Si continua finché l'errore residuo soddisfa il criterio di arresto.

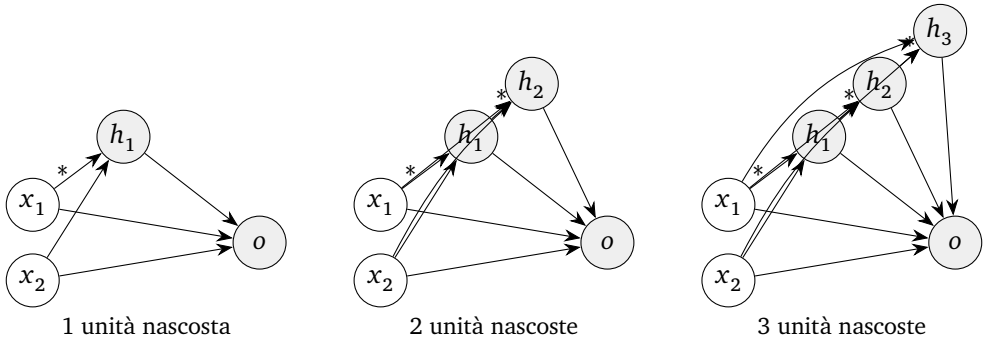


Figura 12.6. Crescita di una rete Cascade Correlation: ogni nuova unità riceve gli input e le uscite di tutte le unità nascoste precedenti; i pesi segnati con $*$ (in ingresso alle unità già inserite) restano congelati.

L'algoritmo alterna la minimizzazione dell'errore totale (LMS, ad esempio backpropagation sullo strato di uscita) e la **massimizzazione della covarianza** S tra l'uscita o_p della candidata e l'errore residuo $E_{p,k} = o_{p,k} - d_{p,k}$ di ogni uscita k:

$$S = \sum_k \Big| \sum_p (o_p - \bar{o})(E_{p,k} - \bar{E}_k) \Big|.$$

Il gradiente, usando $d|f|/dx = \text{sign}(f)\, df/dx$ e trascurando la dipendenza delle medie da o_p, è

$$\frac{\partial S}{\partial w_j} = \sum_k \text{sign}(S_k) \sum_p \big(E_{p,k} - \bar{E}_k\big) f'(net_{p,h})\, I_{p,j},$$

dove h è la candidata, S_k il termine k-esimo della somma e $I_{p,j}$ il suo input j-esimo; si fa **ascesa** del gradiente, perché si massimizza S.

- Il ruolo delle unità nascoste è **ridurre l'errore residuo**: ogni unità risolve un sotto-problema e diventa un **rilevatore di feature permanente**.
- **Pool di candidate**: la superficie di S ha molti massimi, quindi si addestra un gruppo di candidate e si sceglie la migliore.
- È un algoritmo **greedy**: converge facilmente e può trovare un numero minimo di unità, ma può andare in overfitting (serve regolarizzazione).
- La loss non deve essere necessariamente "max S" (per la regressione si può usare LMS).

12.8 Rappresentazione di input e output

Input. Il pre-processing ha un grande effetto.

- **Standardizzazione** (spesso utile): per ogni feature media 0 e deviazione standard 1, $(v - \text{media})/\text{dev. std.}$
- **Rescaling** in $[0, 1]$: $(v - \min)/(\max - \min)$.
- Input categorici: codifica **1-of-K** (§ 2.2.1).
- Dati mancanti: attenzione, **0 non significa "nessun input"** se 0 è nel range dei valori.

Output.

- **(!) Regressione**: unità di uscita **lineari**.
- **Classificazione**: una unità (binaria) o codifica 1-of-K con più uscite. Con la sigmoide si sceglie la soglia per assegnare la classe (eventualmente con zona di rifiuto); con 1-of-K vince l'uscita più alta. Spesso la **tanh** (simmetrica) apprende più in fretta. Come target si può usare 0,9 invece di 1 (e $-0{,}9$ o 0,1 invece di -1 o 0) per evitare la convergenza asintotica verso la saturazione; il range dei target deve comunque stare nel range delle uscite.
- **Softmax** per target 0/1 multi-classe: le uscite sommano a 1 e si interpretano come probabilità $p(\text{classe} = k \mid \mathbf{x})$:
$$o_k(\mathbf{x}) = \frac{e^{net_k}}{\sum_{j=1}^{K} e^{net_j}}.$$
- Come loss alternativa si può usare la **cross-entropy** (stima di massima verosimiglianza); per un'unità:
$$-\sum_{i \in TR} \Big\{ d_i \log\big(out(\mathbf{x}_i)\big) + (1 - d_i) \log\big(1 - out(\mathbf{x}_i)\big) \Big\}.$$

12.9 Validazione di una rete neurale

Gli schemi generali sono nel capitolo 9; per le reti ci sono tre questioni specifiche.

12.9.1 Non fissare il numero di epoche

> **Attenzione — Errore frequente.** Evitare di fermare l'addestramento dopo un **numero fisso e arbitrario di epoche**.

Se è piccolo ci si ferma **troppo presto** (underfitting, o si favoriscono i learning rate alti che richiedono meno epoche); se è grande **troppo tardi** (overfitting o tempo perso); lo stesso valore non può andare bene per tutte le configurazioni e le iterazioni di una K-fold CV; e non si saprebbe quale numero usare nel riaddestramento finale. Anche selezionare il numero di epoche con la model selection è impreciso. Con un sistema **ben regolarizzato** funzionano bene i criteri del § 12.5 (soglia sulla diminuzione dell'errore o sul gradiente), oppure l'early stopping se si può usare un validation set.

12.9.2 Early stopping e cross-validation

Il criterio di early stopping fa **parte della model selection** (non va applicato sul test set) e in linea di principio richiede un VL ogni volta che si addestra il modello. Il problema è scegliere il punto di arresto del **riaddestramento finale**, se ogni fold della CV si è fermato a un'epoca diversa:

- prendere la **media** delle epoche sui fold è rischioso: con più dati nel riaddestramento il punto di arresto ottimo cambia (ad esempio, in on-line, più dati per epoca significano più aggiornamenti per epoca e servono meno epoche);
- **molto meglio**: prendere la media dell'**errore di training** nel punto migliore di validazione e usarla come soglia nel riaddestramento, per raggiungere lo stesso livello di fitting;
- altrimenti serve un VL ogni volta: CV con early stopping attivo in ogni fold e, trovati i migliori iperparametri, riaddestramento sul dataset ricomposto usando ancora una parte dei dati come VL.

In alternativa all'early stopping si usano **modelli regolarizzati** con criteri di arresto standard; la regolarizzazione riduce comunque la dipendenza dall'early stopping.

12.9.3 Inizializzazioni diverse e modello finale

Inizializzazioni diverse portano a **modelli diversi**. Come scegliere il modello finale (scelta da dichiarare quando si riportano i risultati)?

- Si calcolano **media e varianza** dell'errore sulle prove: un'esecuzione qualsiasi cadrà in quell'intervallo, quindi il rischio è sotto controllo, a meno di una varianza alta (che è comunque un segnale negativo, § 8.2).
- Si usano **tutti** i modelli con un **ensemble** (media delle uscite o voto).
- Se se ne sceglie **uno**, è una **model selection** e serve un VL: il migliore (errore minimo sul VL), la mediana (scelta più cauta), oppure uno casuale (che non richiede un VL).
- Si possono scartare i casi "sfortunati" guardando solo l'errore di training.

12.10 Pregi e difetti delle reti neurali

Pregi.

- Metodo flessibile, che estende i metodi lineari alla stima di funzioni arbitrarie e costruisce un'**ipotesi esplicita** (a differenza del K-NN).
- Spazio delle ipotesi continuo e ricco, loss differenziabile: addestramento con la discesa del gradiente.
- Approssimatori universali; gestiscono rumore e dati incompleti con degrado graduale; dati continui e discreti, regressione e classificazione.
- Un paradigma ampio (anche apprendimento non supervisionato, memorie associative, approcci stocastici).
- **Rappresentazione distribuita** della conoscenza, compatta in matrici di pesi; tolleranza ai guasti; adattamento ad ambienti non stazionari con l'apprendimento on-line (*continual learning*, con il dilemma stabilità-plasticità).

Attenzione — Punti critici.

- **Black box**: è difficile interpretare la conoscenza acquisita (tema della *explainable AI*). Approcci: proiezione o clustering delle rappresentazioni interne, analisi di sensitività, *Layer-wise Relevance Propagation* (mappe di calore sulle parti dell'input rilevanti per la decisione), estrazione di regole simboliche.
- **Progettazione dell'architettura**: servono metodi costruttivi o di pruning, o approcci diversi (SVM).
- **Dipendenza dal processo di addestramento**: varianti e iperparametri influenzano la soluzione.
- Non ideali con **dati mancanti** (si usano rimozione, imputazione, …).

Le reti neurali sono indicate quando l'input è ad alta dimensione (discreto o reale), il task è di classificazione o regressione, i dati possono essere rumorosi, il tempo di addestramento non è critico, la forma della funzione target è sconosciuta, la leggibilità del risultato non è critica e il calcolo dell'uscita deve essere veloce.

12.11 Verificare un'implementazione: il benchmark MONK

Verificare la correttezza di un'implementazione è difficile: una rete spesso "funziona" (anche se male) nonostante piccoli errori. Un primo collaudo è il dataset **MONK**:

- 3 task di classificazione binaria piccoli e artificiali: una piccola rete con poche unità raggiunge accuratezze altissime (fino al 100%) in poco tempo;
- input: 6 variabili con 2, 3 o 4 valori simbolici ciascuna, quindi codifica **1-of-k** e **17 unità di input**;
- il test set include il training set (cattiva pratica, qui irrilevante); MONK3 contiene etichette rumorose e può andare in overfitting.

Consigli per le prime prove.

- Un'unità di uscita, con target e uscite coerenti ($0/1$ o $-1/+1$), ad esempio con tanh.
- **Codifica one-hot degli input**: dimenticarla è un errore frequentissimo.
- Batch con gradiente diviso per il numero di esempi: converge in qualche centinaio di epoche.
- Poche unità nascoste (2–5); provare intervalli di inizializzazione piccoli (o basati sul fan-in), perché con poche unità il risultato è sensibile all'inizializzazione.
- Con approcci on-line usare lo **shuffling**: gli esempi sono ordinati per classe.

Buoni risultati sul MONK **non garantiscono** la correttezza; risultati cattivi indicano **sicuramente** che codice o impostazioni vanno rivisti. Nelle curve conta la **forma** (lisce, convergenti), non i valori. Per verificare la backpropagation si può usare un esempio svolto passo per passo (attenzione se omette i bias) o una libreria affidabile come confronto.

Punti chiave

- Pesi iniziali piccoli e casuali; più inizializzazioni, riportando media e varianza.
- On-line (rumoroso, η piccolo, shuffling), batch (preciso), mini-batch (compromesso); gradiente medio.
- Momentum $\Delta\mathbf{w} = -\eta\nabla E + \alpha\Delta\mathbf{w}_{old}$: accelera nei plateau, smorza le oscillazioni.
- Overfitting: la VC-dimension effettiva cresce durante il training; rimedi: early stopping, regolarizzazione (weight decay), pruning.
- Uscite lineari per la regressione; sigmoide/tanh o softmax per la classificazione.
- Mai un numero fisso di epoche; l'early stopping è parte della model selection.

Domande di verifica

1. Come si inizializzano i pesi e perché?
2. Differenze tra on-line, batch e mini-batch.
3. Ruolo del learning rate; cos'è il momentum e perché funziona?
4. Serve trovare il minimo globale? Perché?
5. Come si manifesta l'overfitting in una rete neurale? Cos'è l'early stopping?
6. Scrivere la loss con weight decay e la regola di aggiornamento con momentum. La regolarizzazione migliora la stabilità della convergenza?
7. Descrivere il Cascade Correlation.

8. Come si rappresentano input e output per regressione e classificazione?
9. Perché non bisogna fermare l'addestramento dopo un numero fisso di epoche? Come si gestisce l'early stopping con la cross-validation?

Parte V

Support Vector Machines

Capitolo 13

Support Vector Machines e metodi kernel

Le Support Vector Machines (SVM) realizzano il secondo approccio alla Structural Risk Minimization: fissato l'errore di training, minimizzano automaticamente la VC-confidence scegliendo, tra tutti gli iperpiani separatori, quello a margine massimo. *Il capitolo procede in quattro passi: SVM lineare* hard margin *(con la risoluzione tramite moltiplicatori di Lagrange),* soft margin *per dati non separabili,* kernel *per confini non lineari, SVM per la regressione. Si chiude con una visione critica dell'uso pratico e con i metodi kernel in generale.*

Attenzione — Notazione. In questo capitolo N è il numero di esempi (indici i, j), m la dimensione dell'input e si scrive b invece di w_0 per il bias. Le parti segnate con * (i dettagli della risoluzione con i moltiplicatori di Lagrange) sono utili per completezza ma non indispensabili per usare il modello.

13.1 SVM lineare: il margine massimo

Una SVM è una **macchina lineare** (come la LTU) che **massimizza il margine di separazione** e così realizza una **Structural Risk Minimization**. Inizialmente (**hard margin**) si assume che il problema sia **linearmente separabile** e senza errori nei dati.

Dato il training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^{N}$ con $d_i \in \{-1, +1\}$, si cerca un iperpiano $\mathbf{w}^T\mathbf{x} + b = 0$ con

$$\mathbf{w}^T\mathbf{x}_i + b \geq 0 \;\text{ se } d_i = +1, \qquad \mathbf{w}^T\mathbf{x}_i + b < 0 \;\text{ se } d_i = -1.$$

$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ è la **funzione discriminante** e $h(\mathbf{x}) = \text{sign}(g(\mathbf{x}))$ l'ipotesi.

Definizione — Margine di separazione. Se l'iperpiano è alla stessa distanza dal più vicino esempio positivo e dal più vicino negativo, il **margine di separazione** ρ è il doppio della distanza tra l'iperpiano e il punto più vicino: una "zona di sicurezza" attorno al confine. L'**iperpiano ottimo** $\mathbf{w}_o^T\mathbf{x} + b_o = 0$ è quello che **massimizza il margine**.

Non tutti gli iperpiani separatori sono uguali: al variare dell'iperpiano varia il margine. Si vedrà che $\rho = 2/\|\mathbf{w}\|$, quindi

$$\text{massimizzare } \rho \iff \text{minimizzare } \|\mathbf{w}\|.$$

13.1.1 Rappresentazione canonica e support vector

Per la **libertà di scala** (moltiplicando $\mathbf{w}$ e b per una costante l'iperpiano non cambia) si possono riscalare $\mathbf{w}$ e b in modo che i punti più vicini all'iperpiano soddisfino $|g(\mathbf{x}_i)| = |\mathbf{w}^T\mathbf{x}_i + b| = 1$. Allora

$$\mathbf{w}^T\mathbf{x}_i + b \geq 1 \;\text{ se } d_i = +1, \qquad \mathbf{w}^T\mathbf{x}_i + b \leq -1 \;\text{ se } d_i = -1,$$

cioè, in forma compatta,

$$\boxed{d_i\,(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \qquad \forall i = 1, \ldots, N} \tag{13.1}$$

Definizione — Support vector. Un **support vector** $\mathbf{x}^{(s)}$ soddisfa il vincolo **con l'uguaglianza**: $d^{(s)}(\mathbf{w}^T\mathbf{x}^{(s)} + b) = 1$. I support vector sono i punti più vicini all'iperpiano, sul bordo del margine.

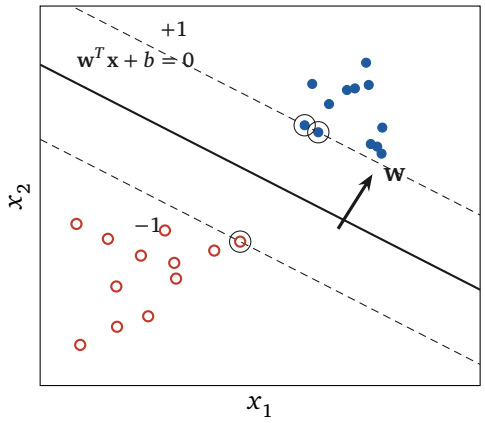


Figura 13.1. L'iperpiano a margine massimo (calcolato su questi dati). Le rette tratteggiate $\mathbf{w}^T\mathbf{x} + b = \pm 1$ delimitano il margine; i punti cerchiati sono i support vector.

13.1.2 Calcolo del margine

Distanza dall'iperpiano e margine. La distanza (con segno) di un punto $\mathbf{x}$ dall'iperpiano ottimo è $r = g(\mathbf{x})/\|\mathbf{w}_o\|$, e il margine vale

$$\rho = \frac{2}{\|\mathbf{w}_o\|}.$$

Dimostrazione. Poiché $\mathbf{w}_o$ è ortogonale all'iperpiano (§ 4.5), ogni punto si scrive come

$$\mathbf{x} = \mathbf{x}_p + r\,\frac{\mathbf{w}_o}{\|\mathbf{w}_o\|},$$

dove $\mathbf{x}_p$ è la proiezione di $\mathbf{x}$ sull'iperpiano e r la distanza (Figura 13.2). Valutando la funzione discriminante:

$$g(\mathbf{x}) = \mathbf{w}_o^T\mathbf{x}_p + b_o + r\,\mathbf{w}_o^T\frac{\mathbf{w}_o}{\|\mathbf{w}_o\|} = \underbrace{g(\mathbf{x}_p)}_{=0} + r\,\frac{\|\mathbf{w}_o\|^2}{\|\mathbf{w}_o\|} = r\,\|\mathbf{w}_o\|,$$

perché $\mathbf{x}_p$ sta sull'iperpiano. Quindi $r = g(\mathbf{x})/\|\mathbf{w}_o\|$. Per un support vector positivo $g(\mathbf{x}^{(s)}) = 1$, quindi la sua distanza dall'iperpiano è $1/\|\mathbf{w}_o\|$; essa è metà del margine, $\rho/2 = 1/\|\mathbf{w}_o\|$, da cui $\rho = 2/\|\mathbf{w}_o\|$. ∎

13.1.3 Il problema di ottimizzazione

Definizione — Problema primale — hard margin. Dato $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$, trovare $\mathbf{w}$ e b che minimizzano

$$\Psi(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w} \qquad \text{(cioè minimizzare } \|\mathbf{w}\|\text{)}$$

soggetto ai vincoli (zero errori di classificazione) $d_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ per ogni $i = 1, \dots, N$.

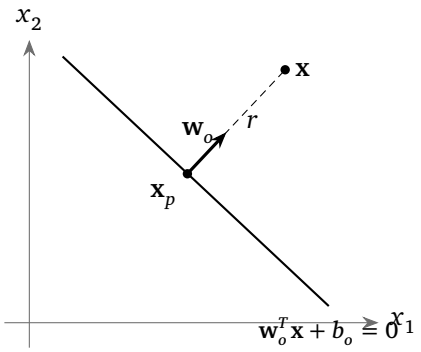


Figura 13.2. La distanza r tra un punto $\mathbf{x}$ e l'iperpiano ottimo: $\mathbf{x} = \mathbf{x}_p + r\,\mathbf{w}_o/\|\mathbf{w}_o\|$.

La funzione obiettivo è **quadratica e convessa** in $\mathbf{w}$ e i vincoli sono **lineari** in $\mathbf{w}$: è un problema di **programmazione quadratica** (QP), che si considera risolto da pacchetti esistenti. Interessa capire come è formulato, la relazione tra support vector e moltiplicatori, e la forma della soluzione. Risolverlo in questa forma scala con la dimensione dell'input m.

13.1.4 Soluzione con i moltiplicatori di Lagrange*

Si costruisce la **lagrangiana**

$$J(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{N} \alpha_i\big(d_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big), \tag{13.2}$$

con un **moltiplicatore di Lagrange** $\alpha_i \geq 0$ per ogni vincolo. J va minimizzata rispetto a $\mathbf{w}$ e b e massimizzata rispetto ad $\boldsymbol{\alpha}$: la soluzione è un **punto di sella** di J.

Condizioni di ottimalità. Derivando J e uguagliando a zero:

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{N} \alpha_i d_i \mathbf{x}_i = \mathbf{0} \qquad \Longrightarrow \qquad \mathbf{w} = \sum_{i=1}^{N} \alpha_i\, d_i\, \mathbf{x}_i, \tag{13.3}$$

$$\frac{\partial J}{\partial b} = -\sum_{i=1}^{N} \alpha_i d_i = 0 \qquad \Longrightarrow \qquad \sum_{i=1}^{N} \alpha_i\, d_i = 0. \tag{13.4}$$

Condizioni di Kuhn-Tucker. Nel punto di sella vale, per ogni i,

$$\alpha_i\big(d_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big) = 0.$$

Quindi: se $\alpha_i > 0$ allora $d_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$, cioè $\mathbf{x}_i$ è un **support vector**; se $\mathbf{x}_i$ non è un support vector allora $\alpha_i = 0$.

> **L'iperpiano dipende solo dai support vector.** Da (13.3), $\mathbf{w}_o = \sum_{i=1}^{N_s} \alpha_{o,i}\, d_i\, \mathbf{x}_i$, dove la somma si restringe agli N_s support vector. Tutti gli altri punti si potrebbero rimuovere senza cambiare la soluzione.

Il problema duale. Si sostituiscono le condizioni di ottimalità nella lagrangiana. Sviluppando (13.2):

$$J = \frac{1}{2}\,\mathbf{w}^T\mathbf{w} - \sum_i \alpha_i d_i\, \mathbf{w}^T\mathbf{x}_i - b\sum_i \alpha_i d_i + \sum_i \alpha_i.$$

1. Per (13.4) il terzo termine è nullo.
2. Per (13.3), $\mathbf{w}^T\mathbf{w} = \sum_i \sum_j \alpha_i\alpha_j d_i d_j\, \mathbf{x}_i^T\mathbf{x}_j$, e anche il secondo termine vale $\sum_i \alpha_i d_i \big(\sum_j \alpha_j d_j \mathbf{x}_j\big)^T \mathbf{x}_i = \sum_i \sum_j \alpha_i\alpha_j d_i d_j\, \mathbf{x}_i^T\mathbf{x}_j$.
3. Quindi $J = \frac{1}{2}S - S + \sum_i \alpha_i$, con $S = \sum_{i,j} \alpha_i\alpha_j d_i d_j\, \mathbf{x}_i^T\mathbf{x}_j$, e resta una funzione dei soli moltiplicatori.

Definizione — Problema duale — hard margin. Trovare $\{\alpha_i\}_{i=1}^N$ che massimizzano

$$Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i\alpha_j\, d_i d_j\, \mathbf{x}_i^T\mathbf{x}_j$$

soggetto a $\alpha_i \geq 0$ per ogni i e $\sum_{i=1}^N \alpha_i d_i = 0$.

Gli α si trovano con la programmazione quadratica o con approcci più efficienti (ad esempio **SMO**, *Sequential Minimal Optimization*). Il duale scala con il **numero di esempi** N, meno con la dimensione dell'input. Si noti che i dati compaiono **solo tramite prodotti scalari** $\mathbf{x}_i^T\mathbf{x}_j$: è la chiave dei kernel.

Algoritmo — Ricavare iperpiano e bias.

1. Risolvere il duale, ottenendo $\{\alpha_{o,i}\}$.
2. Calcolare $\mathbf{w}_o = \sum_{i=1}^N \alpha_{o,i}\, d_i\, \mathbf{x}_i$.
3. Calcolare $b_o = 1 - \mathbf{w}_o^T\mathbf{x}^{(s)}$ per un support vector positivo $\mathbf{x}^{(s)}$ (perché $\mathbf{w}_o^T\mathbf{x}^{(s)} + b_o = 1$), cioè $b_o = 1 - \sum_i \alpha_{o,i} d_i\, \mathbf{x}_i^T\mathbf{x}^{(s)}$.

Non serve conoscere esplicitamente $\mathbf{w}_o$: dato un pattern $\mathbf{x}$ si calcola

$$g(\mathbf{x}) = \sum_{i=1}^{N} \alpha_{o,i}\, d_i \underbrace{\mathbf{x}_i^T\mathbf{x}}_{\text{prodotto scalare}} + b_o$$

(la somma si restringe ai support vector) e si classifica con il segno di $g(\mathbf{x})$.

13.1.5 Perché il margine massimo generalizza

Su un problema separabile l'errore di training è **fissato** (a zero). Minimizzare $\|\mathbf{w}\|$ equivale allora a **minimizzare la VC-dimension**, e quindi la VC-confidence nel bound $R \leq R_{\text{emp}} + \varepsilon(\text{VC}, N, \delta)$ (§ 7.3).

Teorema — Vapnik. *Sia D il diametro della più piccola sfera che contiene i dati $\mathbf{x}_1, \ldots, \mathbf{x}_N$. Per la classe degli iperpiani separatori con margine ρ la VC-dimension è limitata da*

$$\text{VC} \leq \min\left(\left\lceil \frac{D^2}{\rho^2} \right\rceil,\ m_0\right) + 1,$$

dove m_0 è la dimensione dell'input.

Poiché $D^2/\rho^2 \propto \text{Raggio}^2\, \|\mathbf{w}\|^2$, restringendo la ricerca agli iperpiani a margine massimo la VC-dimension può essere **minore di** $m_0 + 1$ (quella degli iperpiani generici): più il margine è grande, più la VC-dimension è piccola, indipendentemente dalla dimensione dello spazio. L'iperpiano a margine massimo fornisce quindi

- una **soluzione unica** (a differenza del Perceptron) con **zero errori** (a differenza dell'LMS);
- una **SRM automatica**, che minimizza la VC-confidence dentro l'ottimizzazione, **senza iperparametri** nel caso separabile;
- la risoluzione con la programmazione quadratica vincolata invece della discesa del gradiente, con una forma duale che mette in luce i support vector e i prodotti scalari;
- una soluzione concentrata su esempi "selezionati", i support vector: un pregio, perché non dipende dai punti lontani dal confine; un difetto, perché i punti sul confine potrebbero essere i più rumorosi.

13.2 Soft margin: dati non separabili

Se almeno un punto viola la condizione $d_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ si passa al **soft margin**. Ammettere punti dentro il margine permette anche un **margine più ampio**: tolleranza al rumore e modello più semplice.

Si introducono le **variabili slack** $\xi_i \geq 0$ e i vincoli diventano

$$d_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \qquad \forall i.$$

- $\xi_i = 0$: il punto è fuori dal margine o sul bordo;
- $0 < \xi_i \leq 1$: il punto è dentro il margine ma classificato correttamente;
- $\xi_i > 1$: il punto è classificato male.

Un support vector soddisfa ora $d_i(\mathbf{w}^T\mathbf{x}_i + b) = 1 - \xi_i$. Il teorema di Vapnik **non vale più**, perché l'hard margin non ammette punti nel margine.

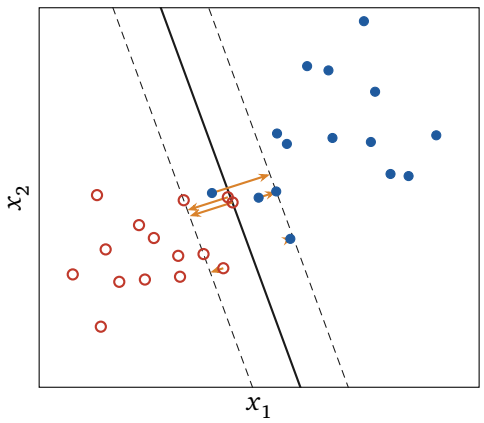


Figura 13.3. Soft margin (calcolato su questi dati con $C = 1$): le frecce arancioni partono dai punti con $\xi_i > 0$ e arrivano al bordo del margine della loro classe; la loro lunghezza è $\xi_i / \|\mathbf{w}\|$.

Definizione — Problema primale — soft margin. Trovare $\mathbf{w}$ e b che minimizzano

$$\Psi(\mathbf{w}, \boldsymbol{\xi}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\xi_i$$

soggetto a $d_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$ e $\xi_i \geq 0$ per ogni i.

C è un **iperparametro di regolarizzazione** (si perde la SRM completamente automatica): regola il compromesso tra minimizzazione del rischio empirico e del termine di capacità.

- C **basso**: molti errori di training ammessi, margine ampio, possibile **underfitting**.
- C **alto**: nessun errore ammesso, margine più piccolo, possibile **overfitting**. Per $C \to \infty$ si ritrova l'hard margin.

Definizione — Problema duale — soft margin*. Trovare $\{\alpha_i\}$ che massimizzano

$$Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j \, d_i d_j \, \mathbf{x}_i^T \mathbf{x}_j$$

soggetto a $\sum_i \alpha_i d_i = 0$ e $0 \le \alpha_i \le C$ per ogni i.

È **identico** al caso hard margin, tranne il vincolo superiore $\alpha_i \le C$. Le condizioni di Kuhn-Tucker diventano

$$\alpha_i \big(d_i (\mathbf{w}^T \mathbf{x}_i + b) + \xi_i - 1 \big) = 0, \qquad \mu_i \, \xi_i = 0,$$

dove i μ_i sono i moltiplicatori dei vincoli $\xi_i \ge 0$. Ne segue: se $0 < \alpha_i < C$ allora $\xi_i = 0$ e il punto è **sul bordo** del margine; se $\alpha_i = C$ allora $\xi_i \ge 0$ e il punto è **dentro** il margine (o dal lato sbagliato). La soluzione è $\mathbf{w}_o = \sum_i \alpha_{o,i} d_i \mathbf{x}_i$ e $b_o = d_j - \sum_i \alpha_{o,i} d_i \, \mathbf{x}_i^T \mathbf{x}_j$ per un esempio j con $0 < \alpha_j < C$ (o la media su tutti questi, per stabilità numerica); i moltiplicatori non nulli corrispondono ai support vector. L'uso è come prima: $h(\mathbf{x}) = \text{sign}\big(\sum_i \alpha_{o,i} d_i \, \mathbf{x}_i^T \mathbf{x} + b_o \big)$.

13.3 Kernel: SVM non lineari

13.3.1 Mappare in uno spazio delle feature

L'idea è mappare i dati dallo spazio degli input a uno **spazio delle feature** ad alta dimensione, dove diventano linearmente separabili (si ricordi la LBE, § 4.7).

Esempio — Da $\mathbb{R}^2$ a $\mathbb{R}^3$. Con $\Phi : \mathbb{R}^2 \to \mathbb{R}^3$, $(x_1, x_2)^T \mapsto (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)^T$ (i monomi di secondo grado), due classi separate da un'ellisse nello spazio di input diventano separabili da un **piano** nello spazio delle feature (Figura 13.4): l'equazione dell'ellisse $x_1^2/a^2 + x_2^2/c^2 = 1$ diventa $z_1/a^2 + z_3/c^2 = 1$, lineare nelle nuove coordinate.

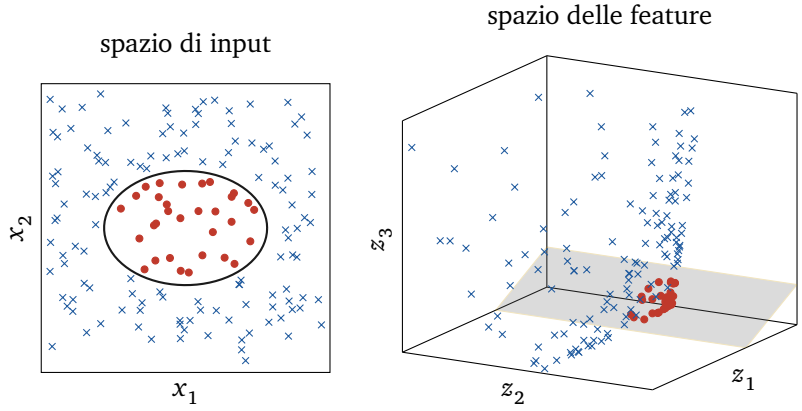


Figura 13.4. Con la mappa $\Phi(\mathbf{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$ le due classi, separate da un'ellisse nello spazio di input, diventano separabili da un piano (grigio) nello spazio delle feature.

La mappa dell'esempio è costruita apposta per il problema; trovare la trasformazione migliore **non è automatico** (a meno di conoscenza a priori), e spesso si usano trasformazioni generali. La

scelta di Φ è il punto critico delle SVM. Il procedimento è: (1) mappare i pattern in uno spazio delle feature ad alta dimensione; (2) cercare l'iperpiano ottimo in quello spazio. Il passo (1) è giustificato dal teorema seguente.

Teorema — Cover. *Un problema complesso di classificazione di pattern, proiettato in modo non lineare in uno spazio ad alta dimensione, ha più probabilità di essere linearmente separabile che in uno spazio a bassa dimensione, purché lo spazio non sia densamente popolato.*

Ad esempio, con una mappa deterministica, l esempi possono essere portati sui vertici di un simplesso di dimensione $l-1$ (il "triangolo" generalizzato): lì **ogni** partizione binaria degli esempi è linearmente separabile.

Spazi delle feature molto grandi possono però essere **computazionalmente infattibili** e portare a **overfitting** se non si controlla la complessità. L'approccio kernel gestisce lo spazio delle feature **implicitamente** e in un contesto regolarizzato, in cui la complessità è regolata dal **margine** e non dalla dimensione dello spazio. Con $\Phi : \mathbb{R}^{m_0} \to \mathbb{R}^{m_1}$ il training set diventa $\{(\Phi(\mathbf{x}_i), d_i)\}$ e l'iperpiano $\mathbf{w}^T\Phi(\mathbf{x}) + b = 0$; includendo il bias ($w_0 = b$, $\phi_0(\mathbf{x}) = 1$), $\mathbf{w}^T\Phi(\mathbf{x}) = 0$ è un'espansione lineare in basi in cui la dimensione dello spazio può essere grandissima. Con (13.3) nello spazio delle feature, $\mathbf{w} = \sum_i \alpha_i d_i \Phi(\mathbf{x}_i)$, e l'iperpiano diventa

$$\sum_{i=1}^{N} \alpha_i \, d_i \, \underbrace{\Phi^T(\mathbf{x}_i)\,\Phi(\mathbf{x})}_{\text{prodotto scalare}} = 0.$$

Il prodotto scalare è ora nello spazio delle feature, dove calcolare $\Phi(\mathbf{x})$ potrebbe essere intrattabile.

13.3.2 Il kernel trick

Sotto certe condizioni **non serve calcolare** $\Phi(\mathbf{x})$, e nemmeno conoscere lo spazio delle feature: basta una funzione che calcoli direttamente il prodotto scalare in quello spazio.

Definizione — Kernel. Un **kernel** (*inner product kernel*) è una funzione

$$k : \mathbb{R}^{m_0} \times \mathbb{R}^{m_0} \to \mathbb{R}, \qquad k(\mathbf{x}_i, \mathbf{x}) = \Phi^T(\mathbf{x}_i)\,\Phi(\mathbf{x}),$$

simmetrica: $k(\mathbf{x}_i, \mathbf{x}) = k(\mathbf{x}, \mathbf{x}_i)$.

Esempio — Il kernel dell'esempio. Con $\Phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$, per $\mathbf{x} = (x_1, x_2)^T$ e $\mathbf{y} = (y_1, y_2)^T$:

$$\Phi^T(\mathbf{x})\Phi(\mathbf{y}) = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (x_1y_1 + x_2y_2)^2 = (\mathbf{x}^T\mathbf{y})^2 = k(\mathbf{x}, \mathbf{y}).$$

Il prodotto scalare nello spazio delle feature (3D) si calcola **senza passare per la mappa**, direttamente dai pattern di input (2D).

I prodotti scalari tra le immagini degli esempi di training formano la **matrice kernel** (o **matrice di Gram**) $N \times N$, $K = \{k(\mathbf{x}_i, \mathbf{x}_j)\}_{i,j=1}^N$, simmetrica. Non ogni funzione calcola un prodotto scalare in qualche spazio: per il **teorema di Mercer** ciò vale per i kernel che producono matrici kernel **semidefinite positive** (con autovalori non negativi). Se k_1 e k_2 sono kernel, lo sono anche $k_1 + k_2$, αk_1 (con $\alpha > 0$) e k_1k_2.

13.3.3 SVM con kernel

Il primale nello spazio delle feature è il soft margin con $\Phi(\mathbf{x}_i)$ al posto di $\mathbf{x}_i$, ma $\mathbf{w}$ vive nello spazio delle feature e il problema può essere intrattabile. Il **duale** invece resta trattabile:

Definizione — Problema duale nello spazio delle feature. Trovare $\{\alpha_i\}$ che massimizzano

$$Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j \, d_i d_j \, \underbrace{k(\mathbf{x}_i, \mathbf{x}_j)}_{K_{ij}}$$

soggetto a $\sum_i \alpha_i d_i = 0$ e $0 \le \alpha_i \le C$.

Il kernel incorpora la mappa. La dipendenza da N (invece che dalla dimensione dello spazio) diventa un **vantaggio**, perché lo spazio delle feature può avere dimensione altissima o infinita.

Algoritmo — SVM con kernel.

1. Dato il training set $T = \{(\mathbf{x}_i, d_i)\}$, scegliere il parametro C e la funzione kernel k.
2. Calcolare la matrice kernel K.
3. Risolvere il problema di ottimizzazione (programmazione quadratica), trovando $\{\alpha_i\}$.
4. Calcolare il bias b dai moltiplicatori e dalla matrice kernel.
5. Per un nuovo pattern $\mathbf{x}$: $\mathbf{w}^T \Phi(\mathbf{x}) = \sum_i \alpha_i d_i \, k(\mathbf{x}, \mathbf{x}_i) + b$, senza mai calcolare $\mathbf{w}$, e classificare con il segno.

La soluzione è **sparsa** negli α: i moltiplicatori dei non-support vector sono nulli. In fase di test $h(\mathbf{x}) = \text{sign}\left(\sum_i \alpha_i d_i \, k(\mathbf{x}, \mathbf{x}_i) + b \right)$ e si memorizzano solo i support vector (che ricorda il K-NN).

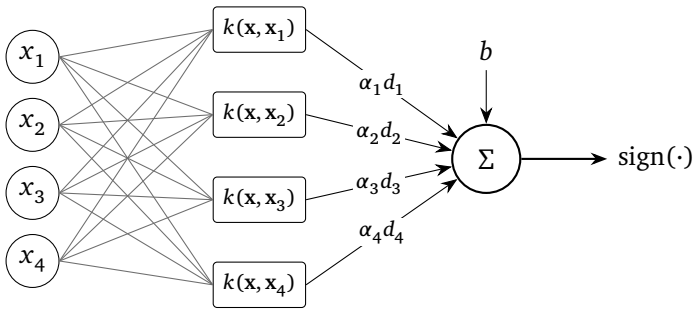


Figura 13.5. Vista “architetturale” di una SVM: lo strato nascosto ha tante unità quanti sono i support vector, e ciascuna calcola il kernel tra l’input e il proprio support vector.

Il numero di “unità nascoste” di questa vista è pari al **numero di support vector**, ma non sono unità di una rete neurale: in una rete le unità nascoste hanno **parametri adattivi**, qui i “centri” sono i support vector stessi e la funzione è fissata dal kernel (Figura 13.5).

Esempi di kernel.

- **Polinomiale**: $k(\mathbf{x}, \mathbf{x}_i) = (\mathbf{x}^T \mathbf{x}_i + 1)^p$, con p scelto dall’utente.
- **RBF** (*Radial Basis Function*, kernel gaussiano): $k(\mathbf{x}, \mathbf{x}_i) = \exp\!\left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}_i\|^2 \right)$, con σ^2 scelto dall’utente; si scrive anche con $\gamma = 1/(2\sigma^2)$. Con kernel molto “stretti” (σ piccolo) la risposta su $\mathbf{x}_i$ è solo d_i, come nel 1-NN.

- **Perceptron a due strati**: $k(\mathbf{x}, \mathbf{x}_i) = \tanh(\beta_0\, \mathbf{x}^T\mathbf{x}_i + \beta_1)$, con $\beta_0 > 0$ e $\beta_1 < 0$.

Attenzione. Il kernel polinomiale e l'RBF sono **sempre** kernel validi (prodotti scalari), mentre per il perceptron a due strati il teorema di Mercer vale solo per alcune scelte di β_0 e β_1. Un kernel RBF corrisponde a uno spazio delle feature di **dimensione infinita**.

13.4 SVM per la regressione

Si ha $d = f(\mathbf{x}) + v$, con f sconosciuta e rumore v indipendente da $\mathbf{x}$, e si stima d con un'espansione lineare di funzioni non lineari:

$$y = h(\mathbf{x}) = \mathbf{w}^T\Phi(\mathbf{x}), \qquad \mathbf{w} = (b, w_1, \dots, w_{m_1})^T, \quad \Phi(\mathbf{x}) = (1, \phi_1(\mathbf{x}), \dots, \phi_{m_1}(\mathbf{x}))^T.$$

Definizione — Loss ε-insensitive.

$$L_\varepsilon(d, y) = \begin{cases} |d - y| - \varepsilon & \text{se } |d - y| \geq \varepsilon \\ 0 & \text{altrimenti.} \end{cases}$$

Gli errori più piccoli di ε **non costano nulla**: si forma un **tubo** di ampiezza ε attorno alla predizione e solo i punti fuori dal tubo contribuiscono alla loss (Figura 13.6). Di nuovo **solo i support vector guidano la soluzione**: nella classificazione erano i punti sul bordo o oltre il margine, qui i punti sul bordo o fuori dal tubo.

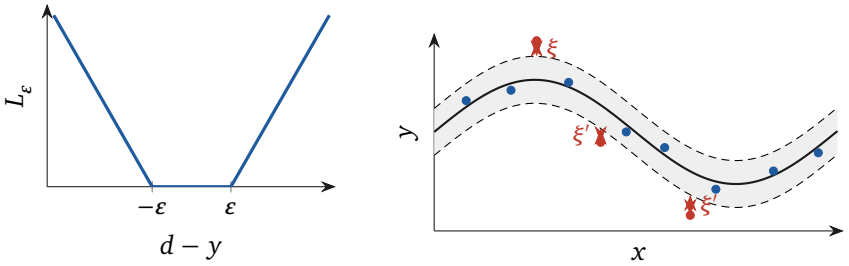


Figura 13.6. A sinistra la loss ε-insensitive. A destra il tubo di ampiezza ε attorno alla funzione stimata: solo i punti fuori dal tubo (rosso) hanno variabili slack non nulle.

Si introducono due insiemi di variabili slack non negative, ξ_i e ξ_i' (sopra e sotto il tubo), con $-\xi_i' - \varepsilon \leq d_i - \mathbf{w}^T\Phi(\mathbf{x}_i) \leq \varepsilon + \xi_i$.

Definizione — Problema primale — regressione. Minimizzare

$$\Psi(\mathbf{w}, \boldsymbol{\xi}, \boldsymbol{\xi}') = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}(\xi_i + \xi_i')$$

soggetto a $d_i - \mathbf{w}^T\Phi(\mathbf{x}_i) \leq \varepsilon + \xi_i$, $\mathbf{w}^T\Phi(\mathbf{x}_i) - d_i \leq \varepsilon + \xi_i'$, $\xi_i \geq 0$, $\xi_i' \geq 0$.

Definizione — Problema duale — regressione*. Trovare $\{\alpha_i\}$ e $\{\alpha_i'\}$ che massimizzano

$$Q(\boldsymbol{\alpha}, \boldsymbol{\alpha}') = \sum_{i=1}^{N} d_i(\alpha_i - \alpha_i') - \varepsilon\sum_{i=1}^{N}(\alpha_i + \alpha_i') - \frac{1}{2}\sum_{i,j=1}^{N}(\alpha_i - \alpha_i')(\alpha_j - \alpha_j')\, k(\mathbf{x}_i, \mathbf{x}_j)$$

soggetto a $\sum_i(\alpha_i - \alpha_i') = 0$ e $0 \leq \alpha_i, \alpha_i' \leq C$.

Dalla soluzione, con $\gamma_i = \alpha_i - \alpha'_i$,

$$\mathbf{w} = \sum_{i=1}^{N} \gamma_i \Phi(\mathbf{x}_i), \qquad h(\mathbf{x}) = \sum_{i=1}^{N} \gamma_i \Phi^T(\mathbf{x}_i)\Phi(\mathbf{x}) = \sum_{i=1}^{N} \gamma_i k(\mathbf{x}_i, \mathbf{x}),$$

e i **support vector** corrispondono ai γ_i non nulli. La procedura è la stessa della classificazione: scegliere C, ε e il kernel; calcolare K; risolvere il duale; calcolare il bias; la funzione stimata è una combinazione lineare di prodotti scalari in uno spazio delle feature che non serve conoscere.

13.5 Le SVM nella pratica

Pregi.

- **Regolarizzazione incorporata** nel problema di ottimizzazione (il margine) e approssimazione della SRM (il bound sulla VC-dimension è minimizzato dalla soluzione, nel caso hard margin senza kernel).
- **Problema convesso**: l'addestramento trova sempre il **minimo globale**, a differenza delle reti neurali.
- Un modello lineare, semplice e compatto, con un ricco insieme di funzioni di decisione non lineari grazie alla **trasformazione implicita delle feature**: un trattamento efficiente della LBE. Un insieme ricco non implica una VC-dimension alta, che dipende da margine, parametri del kernel e C.
- Il kernel trick si estende ad altri modelli (§ 13.6).

Attenzione — Difetti.

- Bisogna **scegliere il kernel e i suoi parametri**.
- È un algoritmo **batch**.
- La matrice kernel è $N \times N$: problemi molto grandi (oltre circa 20 000 esempi) erano difficili da risolvere esattamente con gli approcci standard; oggi esistono molte soluzioni, anche basate sulla discesa del gradiente.

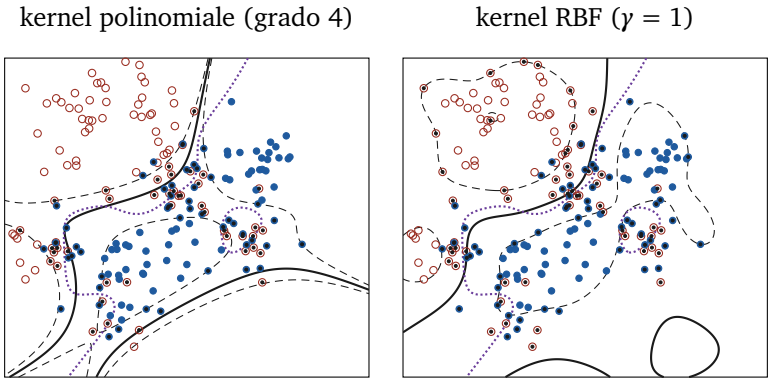


Figura 13.7. SVM ($C = 1$) sul problema a due classi: confine (nero), margini $g(\mathbf{x}) = \pm 1$ (tratteggio), support vector (punto nero al centro del simbolo) e confine di Bayes (viola). Polinomiale: errore di training 0,175, di test 0,237, support vector 46% dei dati. RBF: training 0,180, test 0,222, support vector 55%. Errore di Bayes 0,216.

Sul problema a due classi (Figura 13.7) il kernel radiale dà il risultato migliore, come ci si può aspettare visto che i dati provengono da miscele di gaussiane, e si comporta in modo simile alla

rete regolarizzata della Figura 12.5. I support vector sono i punti sul margine (slack nulla) o dalla parte sbagliata del margine (slack positiva).

> **Attenzione — I support vector possono essere tanti.** La teoria immagina pochi support vector, ma **non è sempre vero**: qui sono circa la metà dei dati. La sparsità della soluzione non è garantita.

13.5.1 Il ruolo degli iperparametri

Non esiste una teoria che garantisca che una data famiglia di SVM abbia alta accuratezza su un dato problema. Le proprietà del caso hard margin **non si estendono** direttamente al soft margin e ai kernel: C e il kernel possono portare a una VC-dimension anche **infinita**. Una SVM con kernel gaussiano di larghezza σ **abbastanza piccola** classifica correttamente un numero arbitrario di punti (come il 1-NN: contribuisce solo il support vector più vicino), quindi ha VC-dimension infinita, a meno di regolarizzare; con una larghezza **grande** contribuiscono tutti i support vector e si ottiene una sorta di media globale, con VC-dimension bassa. Controllando la larghezza del kernel si controlla la VC-dimension.

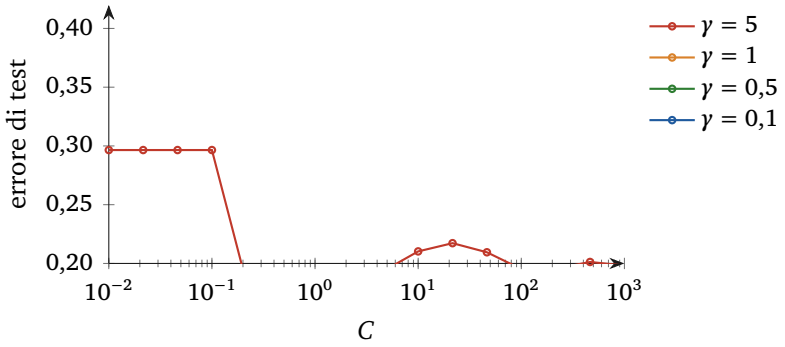


Figura 13.8. Errore di test di una SVM con kernel RBF in funzione di C per diversi valori di $\gamma = 1/(2\sigma^2)$, sul problema a due classi: il valore migliore di C cambia con γ.

Con kernel stretti (γ grande) conviene un C basso (più regolarizzazione); con γ piccoli l'ottimo si sposta verso C più alti (Figura 13.8). **Il miglior valore di C dipende dagli iperparametri del kernel**: vanno selezionati **insieme** (grid search), con una validazione rigorosa (validation e test set, oppure double CV), perché la loro regolazione può introdurre overfitting.

> **Attenzione — Luoghi comuni da sfatare.**
>
> 1. *"La SVM si può usare con i valori di default."* No: C e i parametri del kernel sono critici.
> 2. *"Scelgo la SVM perché non va in overfitting."* No: con soft margin e kernel la VC-dimension può essere altissima; l'overfitting dipende da C e dal kernel.
> 3. *"La SVM risolve la maledizione della dimensionalità."* No: gestisce bene l'alta dimensione dello **spazio delle feature**, non dello **spazio di input**.
> 4. *"La SVM trova da sola il numero di unità."* Il numero di support vector è determinato dalla soluzione, ma dipende da C e dal kernel, che vanno scelti, e non è detto che sia piccolo.

13.6 Metodi kernel

Il kernel trick si usa anche senza SVM: è la **kernelizzazione** di approcci tradizionali. Ogni volta che in un modello compare un **prodotto scalare** o una **misura di similarità**, lo si può sostituire con un

kernel, facendo operare il modello in un nuovo spazio implicito ad alta dimensione semplicemente specificando il kernel. È un'espansione in basi fissa e ad alta dimensione ma efficiente; la scelta delle basi è **modulare** (si cambia il kernel senza cambiare la macchina di apprendimento) e il quadro teorico è unificato.

Kernel e distanza. Un kernel induce una distanza nello spazio delle feature:

$$d_k(\mathbf{s},\mathbf{t})^2 = \|\Phi(\mathbf{s}) - \Phi(\mathbf{t})\|^2 = k(\mathbf{s},\mathbf{s}) - 2k(\mathbf{s},\mathbf{t}) + k(\mathbf{t},\mathbf{t}).$$

Dimostrazione. Con lo sviluppo (1.2): $\|\Phi(\mathbf{s}) - \Phi(\mathbf{t})\|^2 = \Phi(\mathbf{s})^T\Phi(\mathbf{s}) - 2\,\Phi(\mathbf{s})^T\Phi(\mathbf{t}) + \Phi(\mathbf{t})^T\Phi(\mathbf{t})$, e ogni prodotto scalare è un valore del kernel. ■

Il kernel è quindi una misura di **similarità**: oggetti simili (distanza piccola) danno un valore alto del kernel.

> **L'idea dei kernel.** Concentrarsi sulla rappresentazione dei **confronti tra coppie** di oggetti, invece che sulla rappresentazione di un singolo oggetto: in alcuni domini è più facile confrontare due oggetti che definire uno spazio di feature in cui rappresentarne uno. La conoscenza di dominio sta tutta nel kernel, **separata dal risolutore**.

Un kernel è buono quando raggruppa nello spazio delle feature gli oggetti della stessa classe, cioè riflette la similarità rilevante per il task; è cattivo quando la similarità che induce non ha a che fare con il task (ad esempio se tutti gli oggetti risultano ugualmente simili o ugualmente diversi). Una SVM è caratterizzata in gran parte dalla scelta del kernel, mentre nelle reti neurali le funzioni di base si **adattano** ai dati: la scelta del kernel migliore per un problema è un tema di ricerca (kernel adattivi). La progettazione di kernel si estende a oggetti non vettoriali, come stringhe, alberi e grafi (§ 19.4).

Punti chiave

- Vincoli canonici $d_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$, margine $\rho = 2/\|\mathbf{w}\|$: massimizzare il margine = minimizzare $\frac{1}{2}\mathbf{w}^T\mathbf{w}$.
- Duale: $\max \sum \alpha_i - \frac{1}{2}\sum \alpha_i\alpha_j d_i d_j\, \mathbf{x}_i^T\mathbf{x}_j$; $\mathbf{w} = \sum \alpha_i d_i \mathbf{x}_i$ dipende solo dai support vector ($\alpha_i > 0$).
- Teorema di Vapnik: VC $\leq \min(\lceil D^2/\rho^2 \rceil, m_0) + 1$; margine grande, VC piccola.
- Soft margin: slack ξ_i, penalità $C\sum \xi_i$, duale con $0 \leq \alpha_i \leq C$.
- Kernel $k(\mathbf{x},\mathbf{y}) = \Phi^T(\mathbf{x})\Phi(\mathbf{y})$ (Mercer: matrice semidefinita positiva); RBF di dimensione infinita.
- Regressione: loss ε-insensitive, $h(\mathbf{x}) = \sum \gamma_i k(\mathbf{x}_i, \mathbf{x})$.
- C e parametri del kernel vanno scelti insieme; la VC-dimension può essere infinita.

Domande di verifica

1. Definire il classificatore a margine massimo e ricavare $\rho = 2/\|\mathbf{w}\|$.
2. Come il margine massimo diventa un problema di programmazione quadratica? Cosa sono i support vector?
3. Perché la SVM approssima la SRM?
4. Come si gestiscono dati non separabili? Ruolo di C.

5. Come si ottengono confini non lineari? Cos'è il kernel trick? Perché permette di lavorare con espansioni in basi di dimensione altissima?
6. Come funziona la SVM per la regressione?
7. Perché la VC-dimension di una SVM con kernel RBF può essere infinita? Perché C e i parametri del kernel vanno selezionati insieme? La SVM risolve la maledizione della dimensionalità?
8. Cosa sono i metodi kernel? Che legame c'è tra kernel e similarità?

Parte VI

Modelli avanzati

Capitolo 14

Reti convoluzionali

Le reti convoluzionali (CNN) mostrano come si progetta una rete feedforward non *completamente connessa per un problema specifico, inserendo nell'architettura conoscenza a priori sui dati (le immagini). Il capitolo presenta la convoluzione come unità che scorre sull'input con pesi condivisi, il pooling, l'architettura complessiva con i suoi vantaggi, e i risultati che hanno fatto delle CNN il primo esempio di rete profonda.*

14.1 Il problema e l'idea

Nel riconoscimento di caratteri scritti a mano (ad esempio immagini 16×16 in scala di grigi) un primo approccio è un MLP standard con 256 input, uno per pixel: si ottiene un errore del 5–20%, con molti errori dovuti alla **mancanza di invarianza** a traslazioni, rotazioni e deformazioni. Una rete completamente connessa tratta ogni pixel come un input indipendente: non "sa" che pixel vicini sono correlati né che uno stesso tratto può comparire in posizioni diverse.

Conoscenza a priori nell'architettura.

- **Connessioni locali** (campi recettivi locali): ogni unità vede solo una piccola zona dell'immagine ed estrae **feature locali**.
- **Condivisione dei pesi** (*weight sharing*) tra unità diverse: la **stessa operazione** è applicata a parti diverse dell'input, perché le feature di un carattere possono comparire ovunque nell'immagine. Si riduce il numero di parametri liberi pur mantenendo una connettività complessa.

14.2 La convoluzione

Il nome viene dall'**operatore di convoluzione**, una media pesata di una funzione f con pesi dati da un'altra funzione g che scorre:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau.$$

g è non nulla solo su un intervallo limitato e agisce come una **finestra scorrevole**.

14.2.1 Convoluzione 1D

Un'unità con tre pesi che scorre su una sequenza di input calcola

$$out_t = \sum_{i=1}^{3} w_i\, x_{t+i-2};$$

ad esempio $o_2 = w_1x_1 + w_2x_2 + w_3x_3$ e $o_3 = w_1x_2 + w_2x_3 + w_3x_4$ (Figura 14.1). I pesi sono **condivisi** tra le copie dell'unità e si apprendono come al solito: l'unità è un **filtro adattivo**, e più unità diverse formano un **banco di filtri**. Questa rete si chiama anche *Time-Delay Neural Network*.

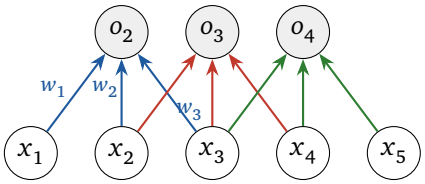


Figura 14.1. Convoluzione 1D: la stessa unità (stessi pesi) scorre sulla sequenza di input.

14.2.2 Convoluzione 2D

Su un'immagine si usa un **kernel** 2D, ad esempio 3×3 (il **campo recettivo locale** dell'unità), che si sposta di un pixel alla volta (**stride** 1); il **padding** gestisce i bordi. Il kernel scorre sull'immagine e produce una **feature map**, le feature estratte da quel filtro. In termini di reti neurali: si percorre l'immagine con **lo stesso neurone**, che usa connessioni locali (Figura 14.2).

a	b	c	d
e	f	g	h
i	j	k	l

input 3×4

w	x
y	z

kernel 2×2

$aw + bx + ey + fz$	$bw + cx + fy + gz$	$cw + dx + gy + hz$
$ew + fx + iy + jz$	$fw + gx + jy + kz$	$gw + hx + ky + lz$

feature map 2×3

Figura 14.2. Convoluzione 2D di un input 3×4 con un kernel 2×2 (stride 1, senza padding): ogni elemento della feature map è la somma dei prodotti del kernel con una porzione 2×2 dell'input; in blu la prima posizione.

Attenzione — Convoluzione o cross-correlazione. La convoluzione 2D di un'immagine I con un kernel K è $S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)\, K(i-m, j-n)$. Molte librerie implementano in realtà la **cross-correlazione**, $S(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$, come nella Figura 14.2 (i dettagli non sono richiesti).

Con **stride** maggiore di 1 il kernel "salta" pixel (ad esempio due alla volta): è un **sotto-campionamento** e produce una feature map più piccola.

14.2.3 Una rete di filtri

I pesi di un'unità sono un **filtro addestrato** a rilevare certe feature dell'immagine. Il progetto dell'architettura prevede:

- un **campo recettivo piccolo e locale**: i filtri appresi rispondono al massimo a pattern **spazialmente locali**;
- **lo stesso filtro su tutta l'immagine**: le feature sono rilevate indipendentemente dalla posizione, e si ottiene l'**invarianza per traslazione**;
- **filtri appresi**: i filtri che nell'elaborazione delle immagini tradizionale erano progettati a mano vengono appresi, senza dipendere da conoscenza a priori e sforzo umano;
- **molti strati impilati**: le unità delle feature map rappresentano aree via via più grandi dell'immagine originale (Figura 14.3).

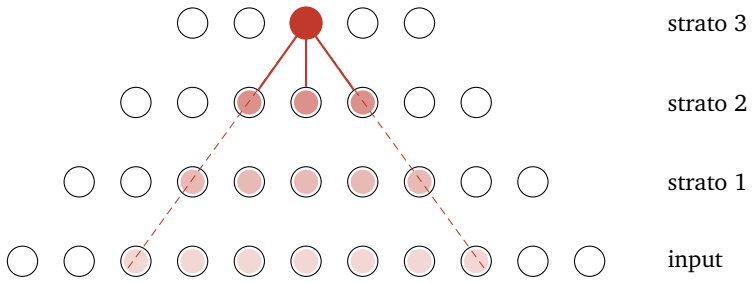


Figura 14.3. Crescita del campo recettivo con la profondità (caso 1D, kernel di ampiezza 3): un'unità dello strato 3 vede 3 unità dello strato 2 ma, indirettamente, 7 input.

14.3 Pooling

Il **pooling** riduce la feature map producendo un valore per un insieme rettangolare di posizioni invece che per ogni posizione: con un sotto-campionamento (stride > 1), con una **media** (semplice o pesata) o, più comunemente, con il **max pooling** (Figura 14.4). Il pooling rende la rappresentazione **approssimativamente invariante a piccole traslazioni**: il massimo (o la media) di una zona cambia poco se l'input si sposta di poco.

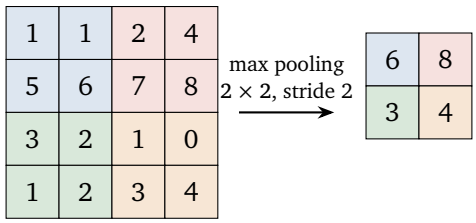


Figura 14.4. Max pooling con filtro 2×2 e stride 2: ogni blocco viene sostituito dal suo massimo.

14.4 La CNN nel complesso

Una CNN realizza con la condivisione dei pesi una finestra scorrevole di campi locali sul segnale, estesa alle immagini 2D e ripetuta su molti strati di feature map, alternata a operazioni di pooling; in cima ci sono di solito alcuni strati completamente connessi (con softmax per la classificazione). Il numero di filtri può crescere negli strati più alti; per un'immagine a colori (RGB) l'input ha tre canali, e la "profondità" del volume di ogni strato è il suo numero di feature map (Figura 14.5).

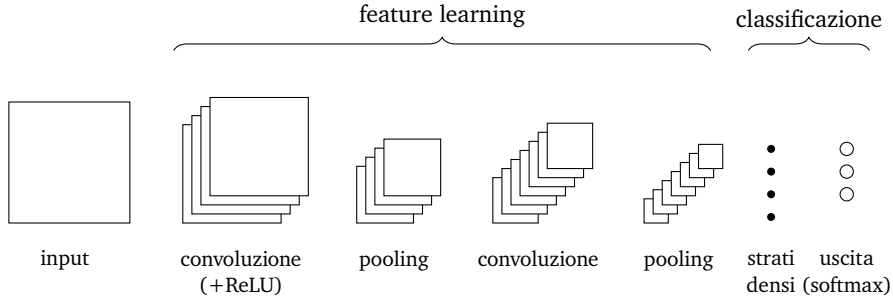


Figura 14.5. Schema di una CNN: convoluzioni e pooling alternati (estrazione delle feature) seguiti da strati completamente connessi (classificazione). Le feature map si rimpiccioliscono e aumentano di numero salendo.

I quattro ingredienti e i loro vantaggi.

1. **Connessioni locali** e
2. **pesi condivisi**: rilevano pattern locali in modo **invariante alla traslazione** e **riducono il numero di parametri liberi** pur mantenendo le connessioni, il che è una forma di **regolarizzazione**.
3. **Pooling**: riduce la dimensione della rappresentazione a ogni strato (una "piramide" di strati) e aiuta l'invarianza a piccoli spostamenti e distorsioni.
4. **Molti strati**: le operazioni 1 e 2 applicate a tutte le feature map producono un'**astrazione progressiva**, che compone primitive di basso livello (apprese una volta sola) nelle combinazioni utili agli strati successivi. Le unità degli ultimi strati hanno un campo recettivo che copre gran parte dell'immagine.

L'addestramento avviene con la **backpropagation**, con le euristiche note e qualche specializzazione (per i pesi condivisi, il pooling, …). Vista la dimensione tipica delle reti e dei dati, molti iperparametri si fissano **per esperienza**, perché una cross-validation su grandi spazi di iperparametri sarebbe troppo costosa.

14.5 Risultati

14.5.1 Pesi condivisi e connessioni locali

Su un problema di cifre manoscritte (circa 300 immagini di training) LeCun confrontò cinque reti: Net-1 senza strati nascosti, Net-2 completamente connessa con uno strato nascosto, Net-3 con connessioni locali 3×3, Net-4 e Net-5 con connessioni locali e pesi condivisi.

Tabella 14.1. Connessioni e pesi delle cinque reti.

Rete	Connessioni	Pesi
Net-1	2570	2570
Net-2	3214	3214
Net-3	1226	1226
Net-4	2266	1132
Net-5	5194	1060

Con i pesi condivisi il numero di **connessioni** può crescere mentre il numero di **pesi** (parametri liberi) resta basso. L'errore di training è nullo per tutte le reti, mentre l'accuratezza di test va da circa il 72% (Net-1) e l'88% (Net-2, Net-3) fino al 98% (Net-4) e quasi al 99% (Net-5, errore dell'1,6%).

14.5.2 Altri risultati

- Sul dataset MNIST (60 000 immagini di training, 10 000 di test) le reti profonde, convoluzionali e no, sono arrivate a errori di pochi decimi di punto percentuale. Anche un semplice MLP con molti strati e molte unità ottiene risultati di questo livello se si controlla la complessità generando **molte immagini deformate** per il training (*data augmentation*).
- AlexNet, una CNN profonda (cinque strati convoluzionali, alcuni seguiti da max pooling, e strati completamente connessi con softmax a 1000 classi, circa 60 milioni di parametri), migliorò

nettamente lo stato dell'arte nella classificazione di immagini (ImageNet). Per velocizzare l'addestramento usava unità **non saturanti** (ReLU) e le GPU; per ridurre l'overfitting negli strati densi usava il **dropout** (§ 15.6.4).

> **Attenzione — Un test set usato troppe volte.** Il test set di MNIST è stato usato così tante volte dalla comunità che miglioramenti dell'ordine dello 0,1% non sono più significativi: di fatto è diventato un validation set (§ 9.7).

Esistono molte architetture specializzate per le immagini, e si possono riusare le feature di reti **pre-addestrate** su grandi dataset (§ 15.3.2). Le implementazioni efficienti sfruttano le **GPU** tramite la rappresentazione a **tensori**: la moltiplicazione di matrici (cioè i prodotti scalari) si parallelizza molto bene, e molte operazioni di convoluzione si possono scrivere come un'unica moltiplicazione tra matrici a blocchi o con operazioni su tensori.

Un'alternativa al max pooling sono le **Capsule Network**: una "capsula" è un gruppo di neuroni che produce sia un parametro che indica se un'entità è presente, sia un vettore di **parametri di posa** (posizione, orientamento); al posto del pooling un meccanismo di **routing** instrada l'informazione di basso livello verso la capsula più adatta.

Punti chiave

- Convoluzione = la stessa unità (filtro adattivo) che scorre sull'input: connessioni locali e pesi condivisi.
- Kernel, stride, padding, feature map; il pooling (soprattutto max) riduce la mappa e dà invarianza a piccoli spostamenti.
- I pesi condivisi riducono i parametri liberi: è una regolarizzazione.
- Con la profondità il campo recettivo cresce e le feature diventano più astratte.

Domande di verifica

1. Quali sono le idee chiave delle CNN e cosa comporta ciascuna?
2. Cos'è la convoluzione in una rete neurale? Cosa sono kernel, stride, padding, feature map?
3. A cosa serve il pooling? Come funziona il max pooling?
4. Perché la condivisione dei pesi è una forma di regolarizzazione?
5. Come cresce il campo recettivo con la profondità?

Capitolo 15

Deep learning

Il deep learning (DL) usa reti con molti strati che apprendono rappresentazioni dei dati su più livelli di astrazione. Il capitolo discute perché la profondità può servire (risultati di no-flattening*, l'esempio della parità, il bias induttivo composizionale), i concetti di representation learning (pre-training, autoencoder, transfer learning) e di rappresentazione distribuita, alcuni risultati recenti e le tecniche che rendono addestrabili le reti profonde (ReLU, gradient clipping, batch normalization, dropout).*

15.1 Che cos'è il deep learning

> **Definizione — Deep learning.** Il **deep learning** permette a modelli computazionali composti da più strati di elaborazione di **apprendere rappresentazioni dei dati con più livelli di astrazione**.

Il DL è un **quadro generale** che include reti profonde (MLP con molti strati), reti convoluzionali, modelli generativi (*deep belief network*), reti ricorrenti e ricorsive, e si contrappone ai modelli **shallow** (superficiali, ad esempio con meno di 3 strati). Gli aspetti comuni sono:

- **più strati** di unità di elaborazione **non lineari**;
- apprendimento (supervisionato o non supervisionato) di **rappresentazioni delle feature** in ogni strato, con gli strati che formano una **gerarchia** da feature di basso livello a feature di alto livello;
- quindi, una rappresentazione **gerarchica, sparsa e distribuita** dei dati.

Il suo successo pratico dipende da tre fattori: **grandi dataset**, **potenza di calcolo** (le GPU) e **nuove idee** per facilitare l'addestramento.

15.1.1 Astrazione gerarchica

Come nello XOR (§ 10.3), lo strato nascosto crea una ri-rappresentazione che semplifica il compito dello strato successivo; il DL estende questa composizione su molti livelli.

- Si passa dal **feature engineering** (feature estratte a mano, per esperienza) al **feature learning**: si **apprendono rappresentazioni** dei dati, anche in modo non supervisionato o semi-supervisionato.
- I livelli di astrazione crescono con gli strati: un'immagine si può rappresentare come vettore di intensità dei pixel, oppure come insieme di bordi, regioni di forma particolare, ecc. **I bordi formano motivi, i motivi si assemblano in parti, le parti formano oggetti** (Figura 15.1).
- Funziona al meglio quando l'input ha una **struttura** (spaziale, temporale, …).

Il DL risolve una mappatura complicata **scomponendola in mappature semplici annidate**, e la nuova rappresentazione semplifica la classificazione nell'ultimo strato. Le feature si **apprendono dagli esempi**: con la backpropagation l'errore in uscita si propaga attraverso gli strati e le unità si specializzano nelle feature utili al task. I primi filtri di una rete per immagini estraggono tipicamente linee orizzontali o verticali, come un semplice filtro convolutivo.

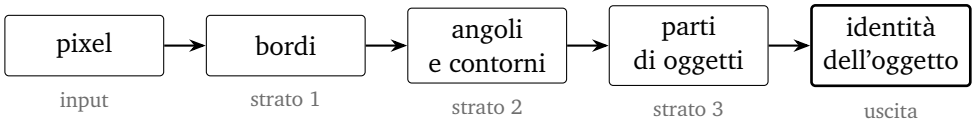


Figura 15.1. Una mappatura difficile (dai pixel all'identità dell'oggetto) scomposta in una serie di mappature semplici annidate, ciascuna descritta da uno strato.

15.1.2 Combinare per generalizzare

Le feature astratte si **riusano** ai livelli superiori e si **combinano** per generalizzare a casi mai visti: se la rete ha visto donne senza occhiali e uomini con occhiali, può rappresentare anche una donna con gli occhiali. Operazioni sulle rappresentazioni vettoriali apprese lo mostrano: "uomo con occhiali − uomo + donna = donna con occhiali". Lo stesso accade nel linguaggio:

- Rep(re) − Rep(maschio) + Rep(femmina) ≈ Rep(regina): la differenza tra "re" e "maschio" cattura il concetto di monarchia;
- Rep(Parigi) − Rep(Francia) + Rep(Polonia) ≈ Rep(Varsavia): la differenza cattura il concetto di capitale.

15.2 Servono davvero molti strati?

> **No-flattening.** *Una funzione che si rappresenta in modo compatto con un'architettura profonda può richiedere un'architettura molto grande per essere rappresentata con una insufficientemente profonda.*

Un circuito logico a **due strati** rappresenta **qualsiasi** funzione booleana in forma normale disgiuntiva (porte AND nel primo strato, eventualmente con input negati, e una porta OR nel secondo), ma con profondità due **la maggior parte delle funzioni booleane richiede un numero esponenziale di porte**.

> **Esempio — La funzione di parità.** La **parità dispari** estende lo XOR a N input: vale 1 se e solo se il numero di 1 tra gli N bit è dispari (si assumono disponibili anche i NOT).
>
> - **Con 2 strati** bisogna elencare tutte le configurazioni positive. Per 3 input: $x_1x_2x_3 + x_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$ (1 se l'input è 111, 100, 010 o 001), cioè 4 AND e 1 OR, 5 porte. In generale le configurazioni con un numero dispari di 1 sono metà delle 2^N, quindi servono $2^N/2$ porte AND più l'OR finale: $2^{N-1}+1$ porte, cioè 9 per $N = 4$ e **129** per $N = 8$, un numero **esponenziale**.
> - **Con** $\log N$ **strati** si usa un **albero binario completo** di XOR (Figura 15.2): un albero con N foglie ha $N - 1$ nodi interni, ognuno uno XOR da 3 porte (AND e OR come nel § 10.3), per un totale di $3(N - 1)$ porte, un numero **polinomiale**: 21 porte per $N = 8$ contro 129.

Questo vantaggio **non vale per tutte le classi di funzioni**: caratterizzare quelle che traggono un vantaggio (esponenziale) dalla profondità è un problema di ricerca aperto. Per le reti neurali i risultati sono di due tipi, entrambi **sull'efficienza** e non sull'espressività (per l'approssimazione universale basta uno strato):

1. **reti shallow**: esistono casi in cui un solo strato nascosto richiede un numero **esponenziale** di unità (rispetto alla dimensione n dell'input) o di pesi non nulli. Nel caso binario le funzioni su $\{0,1\}^n$ sono 2^{2^n}, e nel caso peggiore può servire un'unità per ogni configurazione di input

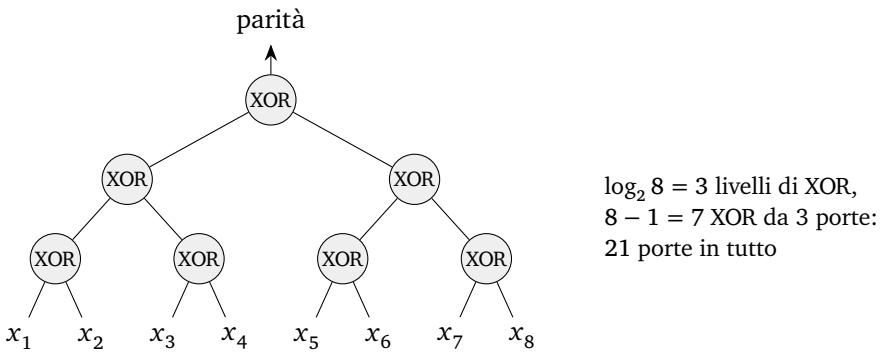


Figura 15.2. Parità di 8 bit con un albero di XOR: un numero di porte lineare in N, contro le $2^{N-1} + 1$ di un circuito a due strati.

da distinguere, come nella parità. Non è solo un problema di dimensione: significa anche che diventa **difficile apprendere con pochi esempi**, ed è questo il punto più rilevante per il ML;

2. **no-flattening delle reti profonde**: esistono famiglie di funzioni approssimabili in modo efficiente con profondità maggiore di d che richiedono un modello molto più grande (fino a un fattore esponenziale) se la profondità è al più d.

Non c'è però garanzia che un dato task abbia questa proprietà.

15.2.1 Composizionalità e bias induttivo

Le reti profonde sfruttano la **composizionalità delle rappresentazioni interne**: i concetti semplici di uno strato sono usati come **primitive** dallo strato successivo per rappresentare concetti più complessi, evitando la rappresentazione (e l'apprendimento) esplicita e combinatoria di tutte le combinazioni. Si imparano le singole parole o i sotto-pattern, non direttamente tutte le loro combinazioni.

> **Profondo non significa grande.** Le reti profonde non vanno viste come modelli *enormi*: sono modelli **compatti** rispetto a modelli superficiali potenzialmente molto più grandi (anche esponenzialmente) per lo stesso task. Meno unità e meno pesi aiutano anche l'apprendimento: permettono di generalizzare bene con **meno esempi**. Una piccola rete profonda può sostituire un'enorme rete superficiale.

Scegliere un modello profondo codifica una convinzione molto generale: che la funzione da apprendere sia una **composizione di funzioni più semplici** (o di fattori di variazione annidati). Se il task corrisponde a questo bias, la generalizzazione migliora proprio grazie ai molti strati: a parità di numero di parametri, reti più profonde generalizzano meglio, mentre aumentare i parametri senza aumentare la profondità non aiuta. Task tipicamente gerarchici sono le **immagini** (composizione di parti) e il **linguaggio**, ma non tutti i dati e i task. È comunque un bias molto più **generale** di vincoli ad hoc (le ϕ scelte a mano di una LBE, le feature ingegnerizzate); nulla però funziona senza assunzioni.

15.2.2 Maledizione della dimensionalità e bias

- Molti learner si basano su **approssimazioni locali** (un prior di smoothness locale): K-NN, kernel locali, alberi di decisione. Per distinguere $O(k)$ regioni richiedono $O(k)$ esempi, e in alta dimensione servono moltissimi esempi per coprire lo spazio (§ 6.6.4).

- Per distinguere un numero **esponenziale** di regioni (in k) con $O(k)$ esempi, cioè per generalizzare **non localmente**, servono ipotesi aggiuntive sulla distribuzione dei dati.
- Il DL sceglie un bias **generale**, legato alla composizione di funzioni: i dati sono generati dalla composizione di fattori, su più livelli. Quando l'ipotesi è vera si ottiene un guadagno potenzialmente esponenziale tra numero di esempi e numero di regioni distinguibili.
- La distribuzione delle immagini e dei testi reali sembra concentrata su una varietà (*manifold*) che occupa una piccola frazione del volume di tutte le immagini o i testi possibili.

Quanti strati e quante unità? Non c'è una risposta generale: è un problema di model selection. Le reti profonde usano spesso meno unità per strato, quindi meno parametri e meno dati per generalizzare, ma molti strati possono essere **più difficili da ottimizzare** (§ 15.6); con modelli così grandi l'esperienza conta molto nel fissare gli aspetti architetturali.

15.3 Representation learning

> **Definizione — Representation learning.** Il **representation learning** è l'insieme dei metodi che permettono a una macchina di ricevere dati grezzi e scoprire automaticamente le rappresentazioni necessarie per il rilevamento o la classificazione.

I metodi di DL sono metodi di representation learning con più livelli di rappresentazione, ma il concetto vale per molti modelli (le reti neurali in generale) e riguarda soprattutto i **dati grezzi** (immagini, testo).

- Molti compiti sono facili o difficilissimi **a seconda di come l'informazione è rappresentata**.
- Nel ML **una buona rappresentazione è quella che rende più facile il compito di apprendimento successivo**.
- Progettare le feature a mano è difficile ed equivale a scegliere a mano le $\phi(\mathbf{x})$ di una LBE; l'apprendimento supervisionato di un MLP produce invece una rappresentazione $\phi(\mathbf{x}, \mathbf{w})$ in ogni strato nascosto, appresa trovando il miglior $\mathbf{w}$.

Oltre all'addestramento completo di MLP e CNN, le rappresentazioni nascoste si possono **ottenere** con l'apprendimento non supervisionato (pre-training) e **sfruttare** per altri task (transfer learning).

15.3.1 Pre-training e autoencoder

Il **pre-training** strato per strato non supervisionato fu il primo approccio che rese possibile addestrare reti profonde (a parte CNN e RNN): si cattura la distribuzione dell'input strato per strato e si inizializza così la rete, invece di addestrarla subito *end-to-end* con la discesa del gradiente.

> **Definizione — Autoencoder.** Un **autoencoder** è una rete addestrata a **copiare il proprio input in uscita**. Ha uno strato nascosto $\mathbf{h}$ che descrive un **codice** dell'input, ed è composto da un **encoder** $\mathbf{h} = f(\mathbf{x})$ e da un **decoder** che produce la ricostruzione $\mathbf{r} = g(\mathbf{h})$. È apprendimento non supervisionato.

- **Undercomplete**: strato nascosto **più piccolo** dell'input; il vincolo architetturale forza la rete a catturare le feature più salienti (con decoder lineare e loss quadratica si ottiene la PCA).
- **Overcomplete**: strato nascosto **più grande** dell'input, con una regolarizzazione che impone sparsità, robustezza al rumore o altre proprietà, oltre alla banale capacità di copiare.

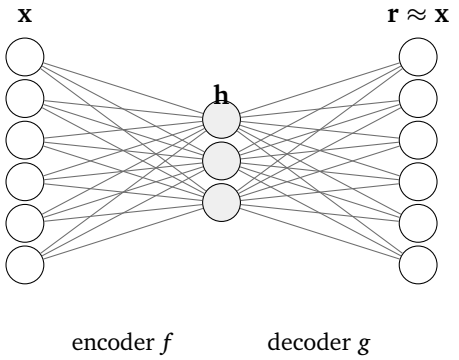


Figura 15.3. Un autoencoder *undercomplete*: lo strato nascosto, più piccolo dell'input, è costretto a catturare le feature più salienti per poter ricostruire l'input.

- **Denoising autoencoder**: si addestra a ricostruire l'input pulito da una versione corrotta (una buona rappresentazione si ottiene in modo robusto da un input corrotto ed è utile per recuperare l'input pulito).

Algoritmo — Pre-training strato per strato.

1. Addestrare il primo strato come **autoassociatore**, minimizzando l'errore di ricostruzione dell'input grezzo (bastano esempi non etichettati).
2. Usare le uscite delle unità nascoste come input di un altro strato, anch'esso addestrato come autoassociatore.
3. Ripetere fino al numero di strati desiderato.
4. Usare l'uscita dell'ultimo strato nascosto come input di uno strato supervisionato.
5. **Fine-tuning** di tutti i parametri rispetto al criterio supervisionato.

Il pre-training funziona sia come **inizializzazione** (si spera di portare i parametri in una regione da cui la discesa locale raggiunge un buon ottimo) sia come **regolarizzazione**, non nel senso di un modello più semplice ma di feature che catturano le regolarità della distribuzione degli input; riduce anche la **varianza** della stima. Oggi è largamente abbandonato per l'addestramento da zero, perché le tecniche moderne bastano per un buon addestramento end-to-end, ma resta utile dove permette di sfruttare grandi quantità di dati non etichettati (ad esempio l'NLP). Architetture collegate: gli **autoencoder variazionali** (lo spazio latente è una distribuzione), le architetture **encoder-decoder** per la traduzione, i **transformer** (§ 18.4.1), usati con il solo encoder (BERT) o il solo decoder per la generazione autoregressiva (GPT).

15.3.2 Transfer learning

Si usa la rappresentazione appresa da un modello per **migliorarne un altro**, assumendo che esistano feature utili in contesti o task diversi. Forme diverse: dall'autoencoder al classificatore; **multi-task learning** (stessi input, target diversi: una regolarizzazione implicita); **domain adaptation** (domini di input diversi con feature condivise); riuso di **modelli pre-addestrati** su dataset più grandi.

> **Esempio — Riusare una CNN pre-addestrata.** Una CNN addestrata su circa 1,2 milioni di immagini in 1000 categorie ha appreso rappresentazioni ricche, con primitive di basso livello condivise tra immagini diverse. Per un nuovo task si **sostituiscono gli ultimi strati** e si addestrano solo

quelli con poche immagini nuove, oppure si fa il **fine-tuning** di tutta la rete.

15.4 Rappresentazioni distribuite

Definizione — Rappresentazione distribuita. In una **rappresentazione distribuita** gli elementi (le feature) non sono mutuamente esclusivi, e le loro molte configurazioni corrispondono alle variazioni osservate nei dati.

Tabella 15.1. Rappresentazione simbolica (one-hot) e distribuita.

Simbolica (one-hot)	Distribuita
una sola unità vale 1, le altre 0	vettori di numeri reali (densi)
distanza sempre $\sqrt{2}$ tra concetti diversi	ogni concetto è l'insieme delle attivazioni di tutte le unità
	la similarità si può condividere tra più di due concetti; si condividono feature apprese; la distanza riflette il significato

Qui non si classifica, si **rappresenta**. L'apprendimento agisce sulla rappresentazione **interna**: una rappresentazione distribuita in **input** si usa solo se la conoscenza di dominio permette di fissarla (fissarla arbitrariamente può danneggiare il task); di solito si fornisce un input one-hot e si lascia che il modello sviluppi internamente la rappresentazione distribuita.

15.4.1 Contare la differenza

Quattro concetti: auto blu, bici blu, auto rossa, bici rossa. Con una rappresentazione one-hot servono 4 unità; con una distribuita ne bastano **2** (2^2 concetti), una per il colore e una per il tipo, e l'unità del "rosso" impara il rosso da immagini sia di auto sia di bici. In generale, con n feature a k valori una rappresentazione distribuita descrive k^n concetti, contro gli n di una one-hot: si può assegnare un codice a un numero **esponenziale** di regioni. Sono invece non distribuiti il K-NN (uno o pochi prototipi per input), i kernel RBF stretti (che si riducono al K-NN), le miscele di gaussiane, gli alberi di decisione (si attiva una sola foglia).

Attributi condivisi. Separando (*disentangling*) i concetti di colore e di tipo, si impara la distinzione auto/bici o il colore **senza vedere esempi di tutte le combinazioni**: avendo visto gli altri tre casi, si tratta facilmente la bici blu, che per una rappresentazione one-hot sarebbe del tutto nuova. Con le parole, una rappresentazione in cui "cane" e "gatto" condividono feature permette di trattarle in modo simile (le frasi con "gatto" informano quelle con "cane") e di imparare automaticamente che sono simili perché compaiono in contesti simili: è il **word embedding**. Nello spazio one-hot ogni parola è a distanza $\sqrt{2}$ da tutte le altre; nello spazio appreso parole di significato simile (nomi di paesi, anni) diventano vicine.

Il rovescio della medaglia è l'**interpretabilità**: una rappresentazione distribuita è meno facile da interpretare di una localista. Il DL sfrutta rappresentazioni distribuite **attraverso molti strati**: la composizionalità porta un ulteriore guadagno di efficienza statistica, per **due vantaggi potenzialmente esponenziali** rispetto ai modelli non distribuiti e superficiali.

15.5 Altri risultati recenti

- **Smoothness dalla struttura**: a parità di parametri, le reti più profonde impongono più smoothness di quelle superficiali (ogni strato lavora sulla superficie già liscia prodotta dal precedente): vincoli **impliciti** anziché espliciti.
- **Il puzzle del non-overfitting**: reti sovra-parametrizzate capaci di azzerare l'errore di training anche su etichette **casuali** generalizzano comunque bene sui dati reali. Una spiegazione è che la discesa del gradiente imponga una **regolarizzazione implicita**, convergendo verso soluzioni a **margine massimo**.
- **Double descent**: al crescere della dimensione del modello (o dei dati, o del tempo di addestramento) l'errore di test prima scende, poi sale (la classica U), raggiunge un picco attorno alla **soglia di interpolazione** (dove l'errore di training si azzera) e poi **scende di nuovo** (Figura 15.4). L'effetto è spesso evitato con una regolarizzazione accurata e non è ancora pienamente compreso; non riguarda necessariamente i modelli piccoli.
- **Ridondanza**: nei modelli profondi una gran parte dei pesi si può predire dagli altri o eliminare (pruning, quantizzazione) senza perdere accuratezza.
- **Lottery ticket hypothesis**: le buone prestazioni dipenderebbero da un'inizializzazione fortunata di una o più sotto-reti, e le reti grandi ne contengono esponenzialmente di più; riaddestrando la rete potata con la stessa inizializzazione si ottengono prestazioni simili. Il ruolo della casualità è il tema del capitolo 16.

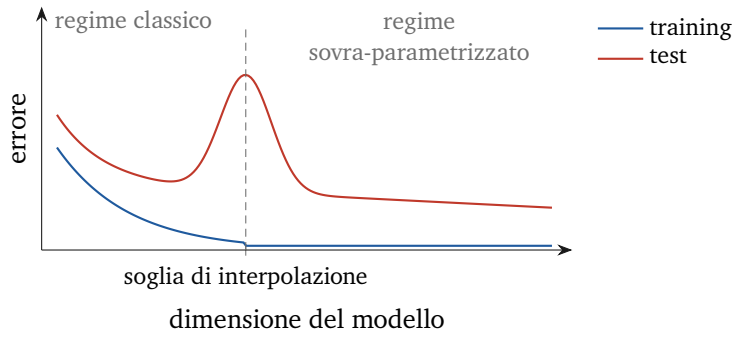


Figura 15.4. Il fenomeno della *double descent* (andamento qualitativo): dopo il picco alla soglia di interpolazione l'errore di test torna a scendere.

15.6 Tecniche

Molti strati possono essere difficili da ottimizzare: l'enfasi è sul miglioramento della discesa del gradiente, sulla regolarizzazione (le reti sono grandi) e sullo sfruttamento dei dati.

Tecniche chiave.

- **SGD con momentum** (mini-batch, decadimento di η) o **Adam** (§ 12.4).
- Attivazioni **ReLU** nelle unità nascoste.
- Loss di massima verosimiglianza (**cross-entropy**) con **softmax** in uscita (anche per evitare saturazione e gradienti piccoli).
- Regolarizzazione: early stopping, weight decay, **dropout**, **batch normalization**.

15.6.1 Vanishing ed exploding gradient

Retropropagando il gradiente attraverso molti strati, se i pesi sono piccoli il gradiente si **riduce esponenzialmente** (**vanishing gradient**), anche per via delle derivate delle sigmoidi; se sono grandi **cresce esponenzialmente** (**exploding gradient**). Le attivazioni come la tanh hanno derivate in $(0, 1]$, e la regola della catena moltiplica d di questi fattori per raggiungere gli strati vicini all'input: il delta decresce esponenzialmente con d e quegli strati si addestrano lentissimamente.

Schizzo formale. Se $\mathbf{h}_l = F_l(\mathbf{h}_{l-1})$ e $F = F_M \circ \ldots \circ F_1$, allora

$$\frac{\partial F}{\partial w_l} = \frac{\partial F_M}{\partial \mathbf{h}_{M-1}} \cdots \frac{\partial F_l}{\partial w_l}, \qquad \left|\frac{\partial F}{\partial w_l}\right| \approx \rho^{M-l+1} \text{ se } \left\|\frac{\partial F_i}{\partial \mathbf{h}_i}\right\| \approx \rho.$$

Con $\rho < 1$ il gradiente svanisce, con $\rho > 1$ esplode.

Gli approcci vanno da R-prop alle connessioni *shortcut*, alle reti randomizzate, alle ReLU.

Gradient clipping. La moltiplicazione ripetuta può creare **scogliere** (*cliff*) nella funzione di costo: derivate altissime che "catapultano" i pesi molto lontano. Il **gradient clipping** limita la norma del gradiente mantenendone la direzione:

$$\text{se } \|\mathbf{g}\| > v \text{ allora } \mathbf{g} \leftarrow \frac{v\,\mathbf{g}}{\|\mathbf{g}\|},$$

con v soglia sulla norma.

15.6.2 ReLU

Definizione — ReLU.

$$f(x) = \max(0, x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0. \end{cases}$$

- **Propagazione del gradiente** migliore attraverso molti strati: niente saturazione, perché $f'(x) = 1$ per ogni net positivo.
- **Attivazione sparsa**: in una rete inizializzata a caso circa metà delle unità nascoste è attiva.
- **Calcolo efficiente**: solo confronti, somme e prodotti.

La ReLU **non è differenziabile in 0**, ma nelle librerie si assume derivata 0 o 1: è un'approssimazione sicura, perché è improbabile valutare esattamente $f(0)$. Conviene iniziare con net positivi (ad esempio bias piccoli e positivi, 0,1), così che le unità siano inizialmente attive. Varianti come **ELU** ($\alpha(e^x - 1)$ per $x \leq 0$, x per $x > 0$) e **Leaky ReLU** ($0{,}01x$ per $x < 0$, x per $x \geq 0$) apprendono anche sulla parte sinistra (le unità non si "spengono") e hanno attivazione media più vicina a zero (Figura 15.5).

15.6.3 Batch normalization

La **batch normalization** normalizza, in ogni strato, le attivazioni del mini-batch a media zero e varianza unitaria, calcolandone le statistiche sul mini-batch; la trasformazione è inclusa nella backpropagation. Come normalizzare l'input è pratica standard, così si mantengono normalizzati i dati che fluiscono negli strati intermedi. Effetti: **regolarizzazione** (per il rumore introdotto dal mini-batch) e **apprendimento più rapido**.

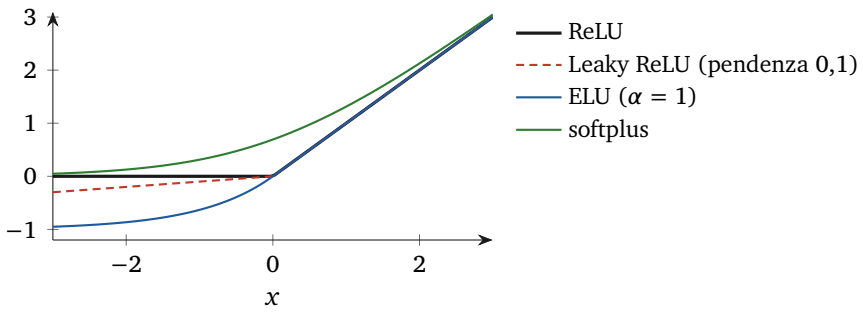


Figura 15.5. La ReLU e alcune varianti (la Leaky ReLU è disegnata con pendenza 0,1 invece di 0,01 per renderla visibile).

15.6.4 Dropout

Il **dropout** seleziona casualmente una sotto-rete a ogni passo di addestramento. Si spiega con due concetti noti: **ensemble** (un bagging implicito ed economico di sotto-reti) e **regolarizzazione**. Il bagging (§ 8.4) media modelli ad alta varianza e basso bias; il dropout sfrutta questo principio con un numero **esponenziale** di sotto-reti (Figura 15.6), avendo a test time **una sola rete**, e massimizza la diversità dell'ensemble evitando co-adattamenti tra le unità.

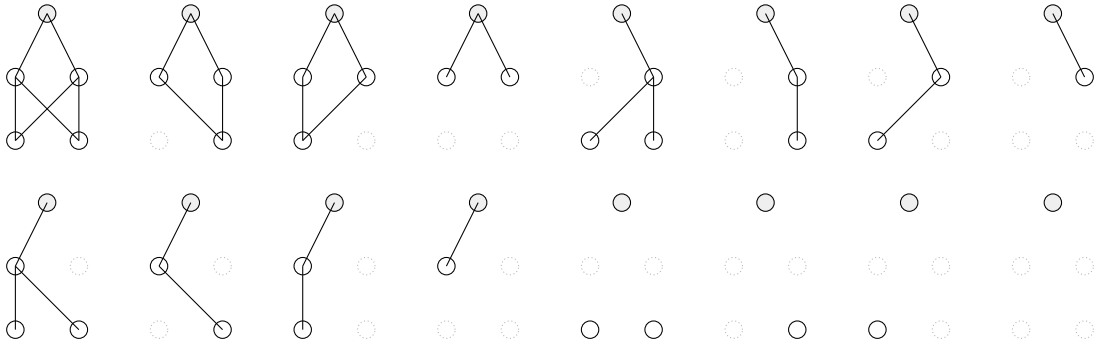


Figura 15.6. Le 16 sotto-reti di una rete con 2 input, 2 unità nascoste e un'uscita, ottenute rimuovendo sottoinsiemi delle unità non di uscita. Alcune non collegano più l'input all'uscita, ma in reti grandi è improbabile che non resti alcun percorso.

Algoritmo — Dropout.

- Ogni volta che si carica un esempio in un mini-batch si campiona una **maschera binaria** per tutte le unità di input e nascoste, indipendentemente per ogni unità; la probabilità di includere un'unità è un iperparametro (tipicamente 0,8 per l'input e 0,5 per le unità nascoste).
- Ogni unità è moltiplicata per la sua maschera: zero equivale a rimuoverla, e si ottiene una sotto-rete.
- Forward, backpropagation e aggiornamento procedono come al solito sulla sotto-rete.
- Le unità rimosse vengono poi reinserite con i loro pesi.

Solo una piccola frazione delle sotto-reti viene addestrata, ciascuna per un solo passo, ma la **condivisione dei parametri** porta anche le altre a buoni parametri. A test time si approssima la predizione dell'ensemble con la *weight scaling inference rule*: si moltiplicano i pesi per la probabilità di inclusione, così che l'uscita attesa di ogni unità sia la stessa dell'addestramento. L'effetto regolarizzante: riduce le interazioni tra le unità e le diversifica; riduce la varianza come il bagging;

inserisce **rumore strutturato** (ad esempio riconoscere un volto senza l'unità che rileva il naso); soprattutto regolarizza ogni unità a essere una feature buona **in molti contesti**. Per la regressione lineare equivale a un weight decay L^2 con un λ diverso per ogni input.

Altre tecniche: la regolarizzazione L^1 (§ 5.4), che elimina feature; le **Generative Adversarial Network** (GAN), due reti in competizione, una che **genera** candidati e una che li **valuta** distinguendoli dai dati veri.

Punti chiave

- DL: molti strati non lineari che apprendono una gerarchia di rappresentazioni distribuite.
- No-flattening: funzioni compatte con molti strati possono richiedere modelli esponenzialmente grandi con pochi strati (parità: $2^{N-1} + 1$ porte contro $3(N - 1)$).
- Bias induttivo: la funzione è una composizione di funzioni più semplici.
- Representation learning: pre-training (autoencoder), transfer learning. Rappresentazioni distribuite: k^n configurazioni, attributi condivisi.
- Tecniche: ReLU, cross-entropy con softmax, SGD con momentum/Adam, gradient clipping, batch normalization, dropout (ensemble implicito di sotto-reti).

Domande di verifica

1. Cosa si intende per deep learning? Quali sono gli aspetti comuni ai vari approcci?
2. Servono davvero molti strati? Discutere l'esempio della parità e i risultati di no-flattening.
3. Qual è il bias induttivo dei modelli profondi?
4. Cos'è il representation learning? Pre-training e transfer learning. Cos'è un autoencoder?
5. Rappresentazioni simboliche contro distribuite: vantaggi delle seconde.
6. Cos'è il vanishing gradient? Come si affronta?
7. Descrivere il dropout e spiegarne l'effetto regolarizzante.

Capitolo 16

Reti neurali randomizzate

E se lo strato nascosto di una rete non venisse addestrato affatto? Il capitolo mostra come la casualità sia una componente utile del ML e descrive le reti a pesi casuali, in cui si addestra solo lo strato di uscita con un metodo lineare. Il loro funzionamento si spiega con concetti già visti: espansione in basi, teorema di Cover, regolarizzazione.

16.1 La casualità nel Machine Learning

Un approccio **randomizzato** usa un certo grado di casualità come parte della sua strategia costruttiva. La casualità compare a diversi livelli:

- **elaborazione dei dati**: suddivisione (hold-out, K-fold CV), generazione di dati (bootstrap, boosting, bilanciamento delle classi);
- **apprendimento**: ordine degli esempi (shuffling), inizializzazione casuale dei pesi;
- **selezione degli iperparametri** (random search);
- **ensemble**: differenziazione casuale dei modelli (inizializzazioni diverse, campionamento dei training set nel bagging);
- **nel cuore del modello**: neuroni stocastici (*Restricted Boltzmann Machine*), il dropout, e le **Random Forest** (o *Extremely Randomized Trees*): ensemble di alberi di decisione, ciascuno costruito su un campione casuale del training set e con una scelta casuale delle variabili su cui dividere i nodi, che uniscono i vantaggi degli ensemble e della casualità.

16.2 Reti a pesi casuali

La domanda è se si possano sfruttare le architetture MLP (o ricorrenti) **senza lunghi cicli di addestramento** degli strati nascosti. Esiste una lunga tradizione di reti randomizzate (già nel Perceptron originale un'area di "proiezione" era connessa casualmente all'input e solo le risposte finali venivano apprese), oggi note con nomi come **RVFL** (*Random Vector Functional Link*) ed **ELM** (*Extreme Learning Machine*).

> **Definizione — Rete randomizzata.**
>
> - Uno (o più) **strati nascosti connessi casualmente**: i pesi sono **fissati dopo l'inizializzazione casuale** e non vengono addestrati.
> - Si addestra **solo lo strato di uscita** (**readout**), un modello lineare, con la pseudoinversa o la ridge regression.

Lo strato nascosto non addestrato immerge in modo **non lineare** l'input in uno **spazio delle feature ad alta dimensione**, dove il problema ha più probabilità di essere risolvibile linearmente: è una **LBE** con basi casuali, giustificata dal teorema di Cover (§ 13.3), la stessa proprietà sfruttata dai kernel. Il readout combina le feature con un modello lineare (Figura 16.1).

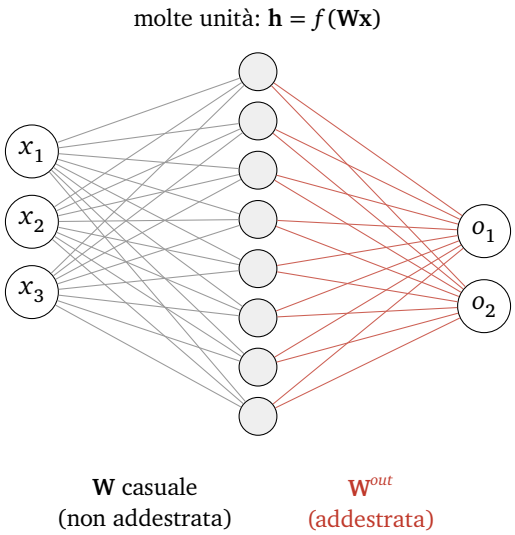


Figura 16.1. Rete feedforward randomizzata: lo strato nascosto, ampio e con pesi casuali fissi, realizza una grande espansione in basi casuale; solo il readout lineare (rosso) viene addestrato.

Algoritmo — Rete feedforward randomizzata.

1. **Costruzione**: riempire la matrice dei pesi nascosti $\mathbf{W}$ con valori casuali (ad esempio rumore gaussiano), usando **molte unità**.
2. **Apprendimento** del readout: dati l esempi $(\mathbf{x}_p, \mathbf{d}_p)$, con $\mathbf{H} = f(\mathbf{W}\mathbf{X})$ (le attivazioni nascoste di tutti gli esempi), si minimizza l'errore quadratico con il metodo diretto (§ 4.3):

   $$\mathbf{W}^{out} = \mathbf{H}^{+}\mathbf{d},$$

   spesso con **regolarizzazione** L^2, utile per l'alto numero di unità (§ 5.3):

   $$\mathbf{W}^{out} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^T\mathbf{d},$$

   oppure con L^1.
3. **Uso**: si calcolano le attivazioni nascoste e poi le uscite $\mathbf{o}(\mathbf{x}) = \mathbf{W}^{out} f(\mathbf{W}\mathbf{x})$, applicando eventualmente una soglia o una funzione non lineare per la classificazione.

I modelli di riferimento sono la RVFL, l'ELM e le reti RBF con centri casuali.

> **Perché funziona.** In una rete normale lo strato nascosto è un'espansione in basi **adattiva** $\phi_j(\mathbf{x}, \mathbf{w})$, con pesi appresi; qui è un'espansione in basi **casuale** ma molto **ampia**. Se le basi sono abbastanza numerose, il problema diventa (quasi) linearmente separabile nello spazio nascosto, e basta un modello lineare in uscita, addestrabile in un solo passo: il costo è quello di una regressione lineare regolarizzata.

16.2.1 Pro e contro

- Un numero elevato di unità può fornire un'espansione in basi **sufficiente** per il task, con basi teoriche (anche teoremi di approssimazione universale).
- Sono **estremamente efficienti**, perché l'addestramento è limitato all'ultimo strato.

- Esistono molte varianti: apprendimento incrementale aggiungendo neuroni (come il Cascade Correlation), pruning.
- Modelli con meno unità adattive, addestrate sul task, restano **più eleganti**: il dibattito tra **feature casuali** e **feature apprese** è aperto, e vale la pena conoscere entrambi gli approcci.
- Si estendono alle reti ricorrenti: le **Echo State Network** (§ 18.4.2).

Sono utili con i **big data**, per implementazioni minuscole su dispositivi con risorse limitate (sistemi embedded, reti di sensori, edge computing), nel **deep learning**, dove l'addestramento è particolarmente oneroso (reti profonde randomizzate come pile di moduli RVFL/ELM, CNN randomizzate), e nelle **reti ricorrenti**, dove l'addestramento è ancora più oneroso e il bias introdotto dalla dinamica ha solide basi teoriche.

> **Conclusione.** La casualità non è un nemico: è un'alleata, se la si controlla. Ritornano gli stessi concetti: teorema di Cover, espansione in basi lineare, regolarizzazione, rappresentazione delle feature.

Punti chiave

- La casualità compare nei dati, nell'apprendimento, nella selezione degli iperparametri, negli ensemble, nei modelli stessi.
- Rete randomizzata: strato nascosto ampio con pesi casuali fissi, readout lineare $\mathbf{W}^{out} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^T\mathbf{d}$.
- Funziona per il teorema di Cover: una LBE casuale molto ampia rende il problema quasi linearmente separabile.

Domande di verifica

1. A quali livelli compare la casualità nel ML?
2. Descrivere una rete feedforward randomizzata: costruzione, addestramento del readout, uso.
3. Enunciare il teorema di Cover e spiegarne il legame con le reti randomizzate e con i kernel.
4. Pro e contro delle feature casuali rispetto alle feature apprese.

Capitolo 17

Apprendimento non supervisionato: K-means e SOM

Il capitolo introduce l'apprendimento non supervisionato dal punto di vista del clustering come quantizzazione vettoriale. Si ricava il K-means (versione on-line e batch) come discesa del gradiente sull'errore di quantizzazione e si arriva alle Self-Organizing Map di Kohonen, che al clustering aggiungono la preservazione della topologia e quindi la visualizzazione dei dati.

17.1 Apprendimento non supervisionato

Nell'apprendimento non supervisionato il training set contiene solo dati **non etichettati** $\langle \mathbf{x} \rangle$ (§ 2.2.2). I task principali sono il **clustering**, la **riduzione di dimensionalità** (con visualizzazione e pre-elaborazione: PCA, Multi-Dimensional Scaling, Independent Component Analysis, …) e la modellazione della densità (non trattata). Un motivo pratico della sua importanza è che i **dati non etichettati costano molto meno** di quelli etichettati.

> **Definizione — Clustering.** Il **clustering** partiziona i dati in **cluster**, sottoinsiemi di dati simili: gli esempi di un cluster sono più simili tra loro che agli esempi di un altro cluster. Ogni cluster è rappresentato da un **centroide** (prototipo, vettore di riferimento, *codevector*); l'insieme dei centroidi è il **codebook**.

> **Attenzione — La metrica domina.** Come nel K-NN, è la **distanza assunta** a stabilire quali esempi sono simili, e i raggruppamenti dipendono da quella metrica. La distanza euclidea è popolare ma è **solo una scelta**, con implicazioni profonde (ad esempio si possono pesare di meno gli outlier usando 1 meno una gaussiana centrata sul prototipo).

17.2 Quantizzazione vettoriale

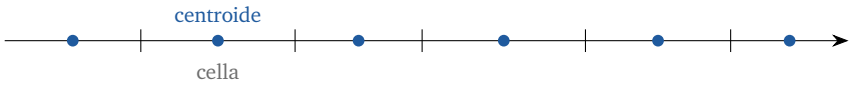
> **Definizione — Vector Quantization.** La **quantizzazione vettoriale** (VQ) codifica una varietà di dati $V \subseteq \mathbb{R}^n$ usando un insieme finito di vettori di riferimento (il codebook) $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_K)$, $\mathbf{w}_i \in \mathbb{R}^n$. Un vettore $\mathbf{x} \in V$ è descritto dal vettore di riferimento **vincente** (*best matching*) $\mathbf{w}_{i^*(\mathbf{x})}$, quello con distorsione $d(\mathbf{x}, \mathbf{w}_{i^*(\mathbf{x})})$ minima.

La VQ divide la varietà nei **poliedri di Voronoi** (§ 6.2, Figura 6.1)

$$V_i = \{\mathbf{x} \in V : \|\mathbf{x} - \mathbf{w}_i\| \leq \|\mathbf{x} - \mathbf{w}_j\| \ \forall j\},$$

e ogni $\mathbf{x}$ è descritto dal vettore di riferimento della sua cella. Il problema è l'inverso di quello del K-NN: **dato il dataset, trovare i centri** che minimizzano la loss. In generale è difficile: trovare il codebook ottimo è **NP-completo**.

> **Esempio — Quantizzazione in una dimensione.** Nell'elaborazione dei segnali la quantizzazione approssima un intervallo continuo di valori con un piccolo insieme di simboli discreti, commettendo un errore di quantizzazione (distorsione): ogni valore è sostituito dal centroide del suo intervallo.



Il clustering ha un obiettivo più generale (trovare raggruppamenti *interessanti*, spesso definiti implicitamente dalla procedura stessa e non dall'errore di quantizzazione minimo), ma la VQ fornisce un quadro utile e i suoi algoritmi sono molto usati per il clustering partizionale.

17.2.1 L'errore di quantizzazione

L'obiettivo è partizionare in modo ottimo una distribuzione sconosciuta in regioni approssimate da prototipi, cioè trovare un quantizzatore $\mathbf{x} \mapsto c(\mathbf{x}) = \mathbf{w}_{i^*(\mathbf{x})}$ da uno spazio continuo a uno discreto. Con la distorsione quadratica $d(\mathbf{x}, c(\mathbf{x})) = \|\mathbf{x} - c(\mathbf{x})\|^2$ (§ 2.3), la loss è la distorsione attesa (errore di ricostruzione o di quantizzazione):

$$E = \int \|\mathbf{x} - \mathbf{w}_{i^*(\mathbf{x})}\|^2 \, p(\mathbf{x}) \, d\mathbf{x},$$

dove $p(\mathbf{x})$ è la distribuzione degli input. In versione discreta, su l dati e K prototipi,

$$E = \sum_{i=1}^{l} \sum_{j=1}^{K} \|\mathbf{x}_i - \mathbf{w}_j\|^2 \, \delta_{winner}(i,j), \qquad \delta_{winner}(i,j) = \begin{cases} 1 & \text{se } j \text{ è il vincitore per } \mathbf{x}_i \\ 0 & \text{altrimenti.} \end{cases} \tag{17.1}$$

Minimizzare E rispetto ai $\mathbf{w}$ è il problema della VQ. L'integrando **non è differenziabile con continuità**, perché il vincitore cambia in modo discreto al variare di $\mathbf{x}$; le derivate si calcolano però **localmente**, fissando $\mathbf{x}$ dentro una cella di Voronoi.

17.3 K-means

17.3.1 Versione on-line

Regola del K-means on-line. La discesa del gradiente stocastica su (17.1) dà, per ogni esempio $\mathbf{x}_i$,

$$\Delta \mathbf{w}_{i^*} = \eta \, \delta_{winner}(i, i^*) \, (\mathbf{x}_i - \mathbf{w}_{i^*}) :$$

si sposta **solo il prototipo vincitore**, verso $\mathbf{x}_i$.

Dimostrazione. Per un esempio $\mathbf{x}_i$ fissato, il termine di E che dipende da $\mathbf{w}_j$ è $\|\mathbf{x}_i - \mathbf{w}_j\|^2 \delta_{winner}(i,j)$. Poiché $\|\mathbf{x}_i - \mathbf{w}_j\|^2 = \sum_t (x_{i,t} - w_{j,t})^2$, la derivata rispetto a $\mathbf{w}_j$ è $-2(\mathbf{x}_i - \mathbf{w}_j)\, \delta_{winner}(i,j)$: è nulla per tutti i prototipi tranne il vincitore. Muovendosi nel verso opposto al gradiente (e assorbendo il 2 in η) si ottiene la regola. ■

È l'algoritmo **K-means** nella versione on-line: una discesa del gradiente stocastica, quindi soggetta a **minimi locali** e dipendente dall'**inizializzazione**; il **numero di cluster** va fornito in anticipo.

17.3.2 Versione batch

Algoritmo — K-means batch.

1. Scegliere K centri: K esempi estratti a caso, o K punti casuali nella regione che contiene i dati.
2. Assegnare ogni esempio al centro più vicino (il **vincitore**):

$$i^*(\mathbf{x}) = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|^2, \qquad \|\mathbf{x} - \mathbf{w}_i\|^2 = \sum_{j=1}^{n} (x_j - w_{ij})^2.$$

3. Ricalcolare ogni centro come **media** (centroide) dei membri del suo cluster:

$$\mathbf{w}_i = \frac{1}{|\text{cluster}_i|} \sum_{\mathbf{x}_j \in \text{cluster}_i} \mathbf{x}_j.$$

4. Se il criterio di convergenza non è soddisfatto (nessuna o minima riassegnazione, minima diminuzione dell'errore quadratico), tornare al passo 2.

Dalla versione on-line alla batch. Sommando su tutti gli esempi $\mathbf{x}$ di un cluster C il passo on-line con $\eta = 1$ e mediando:

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \frac{1}{|C|} \sum_{\mathbf{x} \in C} (\mathbf{x} - \mathbf{w}_{old}) = \mathbf{w}_{old} + \frac{1}{|C|} \sum_{\mathbf{x} \in C} \mathbf{x} - \mathbf{w}_{old} = \frac{1}{|C|} \sum_{\mathbf{x} \in C} \mathbf{x},$$

cioè la media del cluster, il passo 3 dell'algoritmo batch. ■

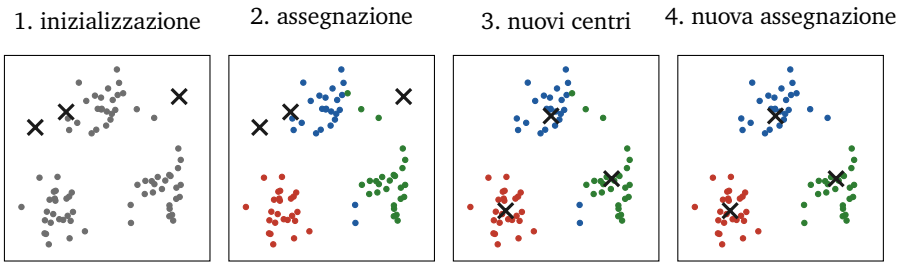


Figura 17.1. K-means batch con $K = 3$ (calcolato): inizializzazione dei centri ($\times$), assegnazione degli esempi al centro più vicino, ricalcolo dei centri come medie, nuova assegnazione.

17.3.3 Pregi e difetti

- È l'algoritmo più semplice e usato con un criterio d'errore quadratico: facile da implementare ed efficiente.
- Il numero K di cluster va fornito.
- I minimi locali di E rendono il risultato dipendente dall'inizializzazione, con prototipi solo sub-ottimi.
- Funziona molto bene per cluster **compatti e ipersferici**; con cluster di altre forme (ad esempio due spirali intrecciate) fallisce.
- **Nessuna visualizzazione**: non proietta i dati in uno spazio a dimensione minore, e l'indicizzazione dei prototipi è arbitraria, quindi la mappatura non è ordinata.

Per ridurre il problema dei minimi locali si usa una regola **soft-max**: si modificano **tutti** i centri, con un passo che decresce con la loro distanza da $\mathbf{x}$ (ad esempio il *maximum-entropy clustering*). Le SOM sono un'istanza di questa strategia, in cui però la vicinanza tra prototipi è definita su una **mappa ordinata**.

17.4 Self-Organizing Map

Le **Self-Organizing Map** (SOM, mappe di Kohonen) sono reti di neuroni disposti su una **griglia regolare a bassa dimensione** (di solito 2D): ogni unità è identificata dalle sue **coordinate** $\mathbf{r}_i$ sulla mappa, riceve **lo stesso input** $\mathbf{x}$ e ha un vettore di pesi $\mathbf{w}_i$ della stessa dimensione n dell'input (Figura 17.2).

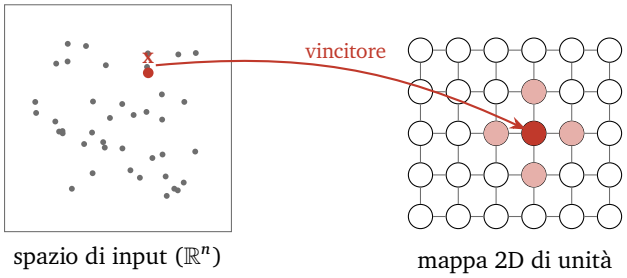


Figura 17.2. Una SOM: ogni unità della griglia ha un vettore di pesi nello spazio di input. Il vincitore è l'unità con i pesi più vicini all'input; i suoi vicini sulla mappa (rosa) vengono aggiornati con lui.

La SOM apprende una mappa dallo spazio di input alla griglia che **preserva la topologia**: unità vicine sulla mappa rispondono a input simili, e punti vicini nello spazio di input vengono mappati sulla stessa unità o su unità vicine. Si usa per il **clustering** (i cluster si individuano sulla mappa), per la **VQ** (l'input diventa il vettore dei pesi del vincitore), per la **compressione** (l'input diventa l'indice del vincitore) e per la **proiezione ed esplorazione** dei dati in due dimensioni.

L'apprendimento è **competitivo**: i neuroni competono per ogni input, il vincitore apprende di più, e gradualmente le unità si specializzano su insiemi diversi di input. Un analogo biologico è la corteccia somatosensoriale, in cui una mappa topologicamente ordinata del corpo associa neuroni adiacenti a parti del corpo vicine.

Algoritmo — SOM.

1. Inizializzare casualmente i pesi della mappa.
2. Estrarre un input $\mathbf{x}$.
3. **Fase competitiva**: trovare il vincitore, l'unità con $\mathbf{w}$ più simile a $\mathbf{x}$ (*winner-take-all*):
$$i^*(\mathbf{x}) = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|.$$
4. **Fase cooperativa**: aggiornare i pesi del vincitore **e delle unità a lui vicine sulla mappa**:
$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \eta(t)\, h_{i,i^*(\mathbf{x})}(t)\,\big[\mathbf{x} - \mathbf{w}_i(t)\big]. \tag{17.2}$$
5. Ripetere fino a convergenza.

La fase cooperativa è un aggiornamento hebbiano (i pesi si avvicinano all'input) modulato dalla **funzione di vicinato** h, che decresce con la distanza **sulla mappa** tra l'unità i e il vincitore, ad

esempio una gaussiana

$$h_{i,i^*}(t) = \exp\left(-\frac{\|\mathbf{r}_i - \mathbf{r}_{i^*}\|^2}{2\sigma^2(t)}\right),$$

con $\sigma(t)$ larghezza del vicinato (in pratica si usano anche vicinati di forma geometrica, rettangolare o esagonale, con un numero finito di unità). Il **learning rate** η e il **raggio** σ diminuiscono con le iterazioni: il vicinato è **ampio all'inizio** (ordinamento globale della mappa) e si restringe lentamente (convergenza e raffinamento locale).

> **L'emergere dell'ordine topologico.** I pesi non sono modificati indipendentemente ma **per sottoinsiemi topologicamente legati**: il vincitore e i suoi vicini sulla griglia ricevono aggiornamenti simili e quindi, dopo l'apprendimento, rispondono a input simili. Così l'informazione topologica viene trasmessa alla mappa (Figura 17.3). Una dimostrazione formale dell'ordinamento è però difficile.

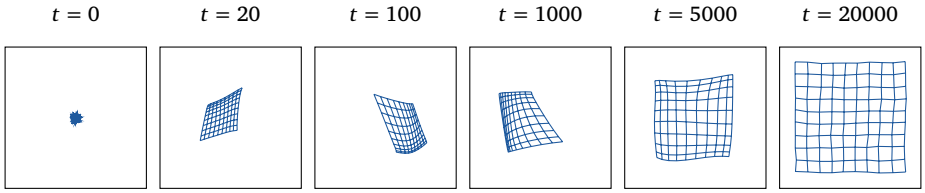


Figura 17.3. Addestramento di una SOM 10×10 su dati uniformi nel quadrato (calcolato): i vettori dei pesi, collegati secondo la griglia, partono concentrati al centro e si distendono progressivamente sulla distribuzione.

Tabella 17.1. K-means e SOM a confronto.

K-means	SOM
aggiorna solo il vincitore	aggiorna il vincitore e i suoi vicini sulla griglia, con un vicinato che si restringe
prototipi senza alcun ordine	prototipi su una mappa ordinata che preserva la topologia
nessuna visualizzazione	visualizzazione 2D dei dati
sensibile ai minimi locali	il vicinato ampio iniziale aiuta a evitarli

Con un vicinato di raggio zero la SOM si riduce al K-means on-line.

17.4.1 Visualizzazione

Sulla mappa si possono visualizzare la **densità** dei vettori di riferimento (colore proporzionale al numero di esempi assegnati a ogni unità) e le **distanze** tra i pesi di unità vicine, con la **U-matrix**: colori scuri indicano grandi distanze tra unità adiacenti (confini tra cluster), colori chiari piccole distanze (unità dello stesso cluster). Questo rende la SOM facilmente interpretabile. Ad esempio, addestrando una mappa 13×9 su 39 indicatori di benessere di 77 paesi, le zone chiare della U-matrix raggruppano paesi simili e le zone scure separano i gruppi, senza alcuna ipotesi a priori sulla forma dei cluster.

17.5 Valutare un clustering

La valutazione è spesso **soggettiva**: esistono pochi riferimenti oggettivi, tranne in sotto-domini ben definiti. Esistono misure **oggettive** come l'errore di quantizzazione, che però dipende dal numero di centroidi. Non si usa semplicemente l'etichetta di un task di classificazione sottostante (la *purity*): se si hanno le etichette, tanto vale usare un classificatore. Va distinta la qualità del clustering in sé dalla qualità dei risultati per il dominio. Altri metodi non supervisionati sono il clustering gerarchico (dendrogrammi), l'analisi delle componenti principali, le regole di associazione, ICA, Multidimensional Scaling, t-SNE.

Punti chiave

- VQ: codifica i dati con un codebook; loss = errore di quantizzazione $\sum \|\mathbf{x}_i - \mathbf{w}_{i^*}\|^2$.
- K-means on-line: $\Delta \mathbf{w}_{i^*} = \eta(\mathbf{x} - \mathbf{w}_{i^*})$ solo per il vincitore; batch: assegnazione e media.
- SOM: griglia di unità; fase competitiva (vincitore) e cooperativa (vincitore e vicini sulla mappa, con vicinato gaussiano che si restringe); preserva la topologia e permette la visualizzazione (U-matrix).

Domande di verifica

1. Definire clustering e quantizzazione vettoriale. Cos'è l'errore di quantizzazione?
2. Derivare la regola on-line del K-means e passare alla versione batch. Pregi e difetti del K-means.
3. Descrivere la SOM: architettura, fase competitiva, fase cooperativa, funzione di vicinato.
4. Perché la SOM preserva la topologia? Differenze tra K-means e SOM.
5. Come si visualizza una SOM?

Capitolo 18

Reti neurali ricorrenti

Le reti viste finora sono feedforward e lavorano su vettori. Le reti ricorrenti (RNN) aggiungono connessioni di feedback, che danno alla rete uno stato, cioè una memoria delle computazioni passate, e le permettono di elaborare sequenze di lunghezza variabile. Il capitolo presenta il dominio delle sequenze, l'unità ricorrente e i sistemi a transizione di stato, l'addestramento tramite unfolding*, e gli approcci collegati: transformer, Echo State Network, reti ricorsive per alberi.*

18.1 Dati sequenziali e trasduzioni

Servono modelli per sequenze quando l'uscita dipende dalla **storia degli input** (processi dinamici, controllo, serie temporali) o quando il dominio è fatto di **sequenze di lunghezza variabile** (linguaggio e parlato, sequenze biologiche come le proteine, dati di sensori). I dati **strutturati** (sequenze, alberi, grafi) generalizzano i dati piatti del § 2.2.1.

> **Definizione — Sequenze e trasduzioni.** I dati sono **sequenze** di vettori $\mathbf{l}_1, \mathbf{l}_2, \ldots$ con un ordine seriale (ad esempio il tempo) e lunghezza variabile; ogni elemento $\mathbf{l}_t$ è un vettore (ad esempio $\mathbf{l}_t = [1, 0, 1, 0{,}7]$). Il task è una **trasduzione**: una mappatura da una sequenza di input a un valore o a una sequenza di output.

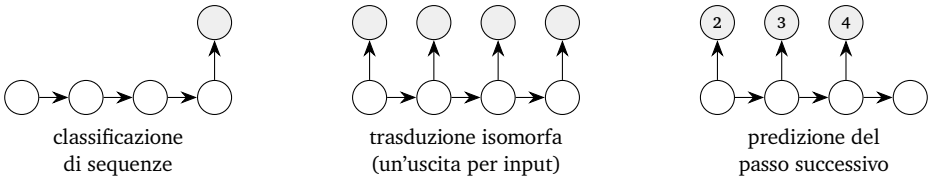


Figura 18.1. Tipi di trasduzione (input vuoti, uscite grigie): classificazione di sequenze (un'uscita alla fine), trasduzione isomorfa (un'uscita per ogni input), predizione dell'elemento successivo. Esiste anche la generazione di sequenze.

18.2 La memoria nelle reti neurali

18.2.1 Memoria finita: le IDNN

Un primo approccio, "spaziale", usa una memoria finita: le **Input Delay Neural Network** (della classe delle *Time-Delay NN*, § 14.2.1). Una **finestra scorrevole** di dimensione fissa sulla sequenza (un registro a scorrimento) viene data in input a un MLP. La dimensione della finestra va **nota in anticipo** e il numero di pesi **dipende da essa**; se la finestra contiene abbastanza informazione, basta un normale addestramento di MLP. Le CNN estendono questa idea alle immagini.

18.2.2 Unità ricorrenti

Un'**unità ricorrente** ha una connessione di feedback (*self-loop*) e usa sia l'input corrente sia l'informazione di **stato**:

$$x(t) = \tau\big(\mathbf{l}(t), x(t-1)\big) = f\big(\mathbf{w}^T\mathbf{l}(t) + \hat{w}\,x(t-1) + \theta\big), \qquad x(0) = 0, \tag{18.1}$$

dove $\mathbf{l}(t)$ è l'input al tempo t, f l'attivazione, $\hat{w}$ il **peso ricorrente**, θ il bias e $x(t-1)$ lo stato al passo precedente, fornito da un ritardo unitario q^{-1} (Figura 18.2). La parte nuova rispetto a un neurone normale è il termine $\hat{w}\,x(t-1)$. Lo **stato** riassume l'informazione passata (il **contesto**), e questa codifica del passato è **adattiva**, perché dipende dai pesi.

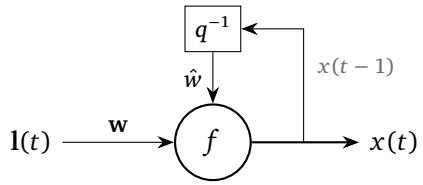


Figura 18.2. Un'unità ricorrente: il self-loop con ritardo unitario q^{-1} porta lo stato precedente come input aggiuntivo, pesato da $\hat{w}$.

Esempio — Contare gli 1. Una RNN con una sola unità che restituisce il numero di 1 ricevuti finora in una sequenza binaria: unità lineare con $w = 1$, $\hat{w} = 1$, $\theta = 0$, cioè $x(t) = l(t) + x(t-1)$. Con l'input $1, 0, 1, 1, 0, 1$ lo stato vale $1, 1, 2, 3, 3, 4$: **lo stato accumula la somma**. Una IDNN **non può farlo** per sequenze di lunghezza arbitraria, perché la sua memoria è limitata alla finestra.

18.2.3 Sistemi a transizione di stato

Definizione — Sistema a transizione di stato. Un modello ricorrente è un sistema

$$\begin{cases} \mathbf{x}(t) = \tau\big(\mathbf{x}(t-1), \mathbf{l}(t)\big) \\ \mathbf{y}(t) = g\big(\mathbf{x}(t), \mathbf{l}(t)\big) \end{cases} \qquad \mathbf{x}(0) = \mathbf{0},$$

dove τ è la **funzione di transizione di stato**, realizzata da una rete neurale, e g la funzione di uscita.

Lo stato al tempo t è una **codifica** (un *embedding*) di tutta la sotto-sequenza fino a t: $\mathbf{x}(1)$ codifica $\mathbf{l}_1$, $\mathbf{x}(2)$ codifica $\mathbf{l}_1\mathbf{l}_2$, e così via. Lo stato ha valori **continui** (a differenza degli automi o degli HMM, che hanno stati discreti) e può essere un vettore, per comporre una rete di unità ricorrenti.

18.2.4 La Simple RNN

Con molte unità ricorrenti nascoste si ottiene la **Simple RNN** (di Elman): ogni unità nascosta riceve l'input e gli stati precedenti di **tutte** le unità nascoste, e uno strato di uscita (eventuale) produce $\mathbf{y}(t)$. Per l'unità nascosta i

$$x_i(t) = f\Big(\sum_{j=1}^{m} w_{ij}\,l_j(t) + \sum_k \hat{w}_{ik}\,x_k(t-1) + \theta_i\Big), \qquad \text{cioè} \qquad \mathbf{x}(t) = f\big(W\mathbf{l}(t) + \hat{W}\mathbf{x}(t-1) + \boldsymbol{\theta}\big).$$

Anche questa rete è molto potente: è un **approssimatore universale** di sistemi dinamici non lineari ed è **Turing-equivalente**. I modelli ricorrenti si basano su tre assunzioni:

- **causalità**: l'uscita al tempo t_0 dipende solo dagli input ai tempi $t \leq t_0$ (necessaria e sufficiente per avere uno stato interno);
- **stazionarietà**: la funzione di transizione τ è **la stessa** in ogni istante, indipendentemente dalla lunghezza delle sequenze; è ciò che permette di elaborare dati di lunghezza variabile con un modello di dimensione fissa;
- **adattività**: le funzioni di transizione sono reti neurali con pesi liberi, **apprese dai dati**.

18.3 Apprendimento: l'unfolding

Si **srotola** (*unfolding*) nel tempo lo stesso modello lungo la sequenza di input: si costruisce una rete feedforward (la **rete di codifica**) equivalente alla RNN sui passi della sequenza, con **una replica del modello per ogni passo**; per la stazionarietà le repliche **condividono i pesi** (Figura 18.3).

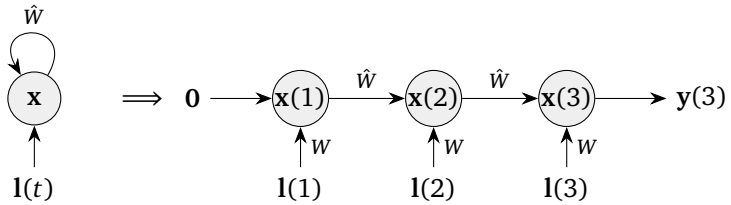


Figura 18.3. Unfolding di una RNN su tre passi: si ottiene una rete feedforward con tanti strati quanti passi, in cui tutte le repliche usano gli stessi pesi W e $\hat{W}$.

> **Il punto chiave.** Sulla rete srotolata si può applicare la **backpropagation**. Si ottiene una rete diversa per ogni sequenza, con **molti strati** (tanti quanti i passi): di fatto una rete **profonda** con pesi condivisi tra gli strati. Gli algoritmi **BPTT** (*Back-Propagation Through Time*) e **RTRL** (*Real-Time Recurrent Learning*) calcolano in modi diversi lo stesso gradiente sulla rete srotolata.

> **Attenzione — Dipendenze a lungo termine.** Poiché la rete srotolata è molto profonda, il **vanishing gradient** (§ 15.6) rende difficile apprendere **dipendenze a lungo termine**, quando l'uscita dipende da input molto lontani nel passato.

Modelli avanzati affrontano il problema: le **LSTM** (*Long Short-Term Memory*) usano unità "gate" che selezionano il flusso dell'informazione passata e del gradiente; sono nate per le dipendenze lunghe e non vanno usate acriticamente su serie brevi. Le **GRU** (*Gated Recurrent Units*) sono LSTM semplificate; le **RNN bidirezionali** considerano sia il contesto a sinistra sia quello a destra. Spesso si combinano reti profonde e RNN.

18.4 Approcci collegati

Altri approcci ai dati sequenziali sono gli Hidden Markov Model (che modellano la distribuzione di probabilità delle transizioni di stato), le SOM per sequenze, i modelli basati su distanze e i kernel per stringhe, i transformer e le RNN randomizzate.

18.4.1 Transformer e attenzione

Un **transformer** è un'architettura basata sul meccanismo di **attenzione** (multi-head, in parallelo), con encoder e/o decoder e reti feedforward. L'attenzione permette di **accedere a qualunque**

punto della sequenza: lo strato di attenzione pesa tutti gli stati secondo una **rilevanza appresa**, fornendo informazione anche su elementi molto lontani (utile ad esempio nella traduzione, dove un contesto lontano può determinare il significato di una parola). Tutti gli elementi vengono elaborati **simultaneamente**, calcolando pesi tra le loro rappresentazioni in strati successivi con unità a **prodotto scalare scalato** tra vettori *query* e vettori delle parole, che cercano le correlazioni più alte tra le parole (in "see that girl", "that" viene associato a "girl"). Si applica anche a porzioni di immagini (*vision transformer*).

18.4.2 Echo State Network

La macchina a stati si può sfruttare per codificare le sequenze anche **con pesi ricorrenti casuali e fissi**, e poi apprendere solo la mappatura di uscita: è il **Reservoir Computing**, l'estensione alle RNN delle reti randomizzate (capitolo 16).

Definizione — Echo State Network. Una **Echo State Network** (ESN) è composta da

- un grande **reservoir** di unità ricorrenti connesse in modo sparso, **non addestrate** dopo l'inizializzazione casuale;
- un **readout** feedforward di unità **lineari**, addestrato con metodi lineari efficienti (ad esempio la ridge regression).

Echo State Property: la funzione di transizione di stato deve essere **contrattiva** (si limita il raggio spettrale dei pesi del reservoir, garantendo la stabilità del sistema dinamico). Così lo stato dipende asintoticamente, come un'eco, **solo dalla storia degli input** e non dalle condizioni iniziali.

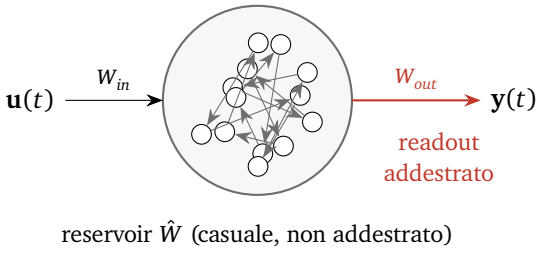


Figura 18.4. Una Echo State Network: input e reservoir con pesi casuali fissi, readout lineare addestrato.

Le ESN sono **molto efficienti**. Il reservoir agisce come una proiezione delle sequenze nello spazio degli stati con un'organizzazione **markoviana**: per la contrattività gli input recenti contano di più, e gli stati di due sequenze sono tanto più vicini quanto più è lungo il loro **suffisso comune** (ad esempio le stringhe `aaaaa` e `abbaa`, che finiscono entrambe con `aa`, hanno stati vicini). Questo rende facile apprendere task markoviani ed è un **bias architetturale** che vale anche per i modelli ricorrenti addestrati. Più reservoir si possono impilare (**DeepESN**): l'organizzazione a strati dà **scale temporali multiple**, con gli strati più alti concentrati sulle frequenze più basse.

18.4.3 Verso gli alberi: le reti ricorsive

L'approccio ricorrente si estende agli **alberi** con le **reti neurali ricorsive** (RecNN): la codifica procede **dal basso verso l'alto**, dalle foglie alla radice, e lo stato di ogni nodo dipende dalla sua etichetta e dagli stati dei suoi **figli** (la funzione di transizione riceve più stati). Le unità sono le stesse per tutti i nodi di un albero e per tutti gli alberi del dataset (condivisione dei pesi), e se la

struttura cambia la rete di codifica srotolata cambia di conseguenza (Figura 18.5). L'estensione ai grafi generali è l'argomento del capitolo 19.

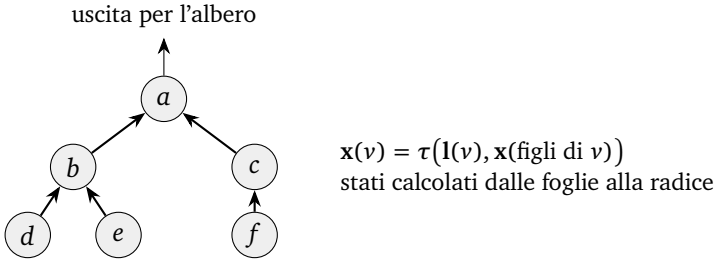


Figura 18.5. Una rete ricorsiva srotolata su un albero: lo stato di ogni nodo codifica il suo sottoalbero.

Punti chiave

- Unità ricorrente: $x(t) = f(\mathbf{w}^T \mathbf{l}(t) + \hat{w}\, x(t-1) + \theta)$; lo stato codifica la sotto-sequenza passata.
- Sistemi a transizione di stato; assunzioni di causalità, stazionarietà, adattività.
- Unfolding + backpropagation (BPTT, RTRL): rete profonda con pesi condivisi, vanishing gradient sulle dipendenze lunghe (LSTM, GRU).
- Transformer: attenzione su tutta la sequenza. ESN: reservoir casuale contrattivo e readout lineare. RecNN: alberi, codifica dalle foglie alla radice.

Domande di verifica

1. Perché servono modelli per dati sequenziali? Cos'è una trasduzione?
2. Differenza tra IDNN e RNN; perché una IDNN non può contare gli 1 in sequenze arbitrarie?
3. Scrivere l'equazione di un'unità ricorrente e di una Simple RNN.
4. Quali sono le assunzioni di causalità, stazionarietà e adattività?
5. Cos'è l'unfolding e come permette di usare la backpropagation?
6. Cos'è una Echo State Network e cos'è la Echo State Property?
7. Come si estendono le RNN agli alberi?

Capitolo 19

Apprendimento su dati strutturati e grafi

L'ultimo capitolo estende l'apprendimento ai grafi. Si formula il problema delle trasduzioni su grafi, si presenta il message passing *delle Deep Graph Networks nelle sue due famiglie (a strati e ricorsiva), si discutono i problemi legati alla profondità (over-smoothing, over-squashing, eterofilia) con alcuni strumenti di analisi, e si chiude con i kernel per strutture.*

19.1 Perché i grafi

I dati hanno spesso **relazioni**: astrazioni di immagini e comprensione di scene, social network e knowledge graph, alberi sintattici del linguaggio, reti di trasporto, piccole molecole (*drug design*) e reti biologiche, termini logici, reti cerebrali. Il corso ha seguito una progressione nel dominio di input: dai **vettori** (reti feedforward) alle **sequenze** (RNN) alle **strutture** (alberi, grafi). L'obiettivo qui è ottenere modelli **profondi ed efficienti** per i dati strutturati, combinando la stratificazione (più livelli di astrazione) con l'efficienza degli approcci randomizzati o incrementali.

19.1.1 I grafi considerati

Si considerano **grafi etichettati**: ogni **nodo** v ha un'**etichetta vettoriale** $\mathbf{l}(v)$ (ad esempio $[1, 0, 1, 0{,}7]$); gli archi possono avere un'etichetta ed essere orientati; possono esserci **cicli**. A è la **matrice di adiacenza** e $\mathcal{N}(v)$ l'insieme dei vicini di v.

19.1.2 Il problema della rappresentazione

Non esisteva un modo **sistematico**, valido per qualunque task, per estrarre feature da esempi strutturati: è un caso di representation learning (§ 15.3) esteso ai domini strutturati.

- Le rappresentazioni basate su **feature** scelte a mano sono incomplete o dipendenti dal task (ad esempio gli indici topologici in chimica).
- Le rappresentazioni di dimensione fissa (ad esempio la matrice di adiacenza come vettore) soffrono di sovra-dimensionamento o incompletezza (spreco con il padding, perdita di informazione), del problema dell'**allineamento** tra grafi diversi (quale nodo di un grafo corrisponde a quale dell'altro?) e dell'ordine dei nodi, che rende difficile generalizzare.

> **Portare i modelli ai dati.** Trattare la natura propria dei dati di input è la chiave per applicare con successo il ML. Invece di **portare i dati ai modelli** (trasformare grafi in vettori, alberi in sequenze, con problemi di allineamento e perdita di informazione), si **portano i modelli ai dati**.

> **Esempio — Molecole.** Nel QSPR/QSAR si vuole correlare la struttura chimica delle molecole con le loro proprietà, Proprietà $= T(\text{Struttura})$: un valore numerico (regressione) o "tossico sì/no" (classificazione). Le molecole non sono vettori: sono rappresentate più naturalmente da grafi di dimensione variabile, e si vuole predire direttamente dalla struttura.

19.2 Deep Graph Networks

19.2.1 Trasduzioni su grafi

Si vuole apprendere una **trasduzione** T da un dominio di grafi (di dimensione variabile) a uno spazio discreto o continuo, a partire da esempi $(\text{grafo}_i, \text{target}_i)$. Un encoder T_{enc} calcola un **embedding** $\mathbf{h}_v$ di ogni nodo (lo spazio latente); da qui

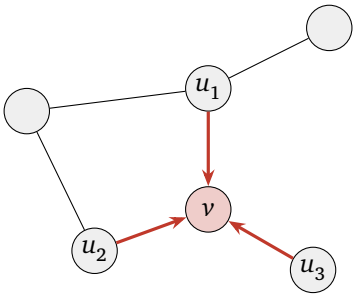
- un'uscita **per ogni nodo** dà i compiti **a livello di nodo** (trasduzioni struttura-struttura isomorfe, come la classificazione dei nodi);
- una funzione di **readout** che aggrega gli embedding dei nodi in un embedding del grafo $\mathbf{h}_g$ dà i compiti **a livello di grafo** (regressione o classificazione di grafi);
- esistono anche trasduzioni non isomorfe e compiti **sugli archi** (ad esempio la *link prediction*).

19.2.2 Il message passing

Il funzionamento di una **Deep Graph Network** (DGN) si descrive con il **message passing**: i nodi si scambiano messaggi per diventare consapevoli della struttura circostante (Figura 19.1).

1. Ogni nodo calcola un **messaggio**, funzione della propria etichetta, dei vicini e degli archi, e lo invia ai vicini.
2. I nodi **aggregano** i messaggi ricevuti con una funzione **invariante per permutazione** (l'ordine di arrivo non conta).
3. Ogni nodo **aggiorna** il proprio stato combinando i messaggi con i parametri liberi W.

Iterando il message passing l'informazione si **diffonde** da ogni nodo agli altri. Infine si emette un'uscita per ogni nodo, oppure per l'intero grafo tramite un'aggregazione invariante per permutazione delle rappresentazioni dei nodi, il **global pooling** (ad esempio somma o media).



1. ogni vicino $u \in \mathcal{N}(v)$ invia un messaggio
2. v aggrega i messaggi (somma, media: invariante per permutazione)
3. v aggiorna il proprio stato $\mathbf{h}_v$ con i pesi W

Figura 19.1. Il message passing su un nodo v: messaggi dai vicini, aggregazione, aggiornamento dello stato.

19.2.3 Il legame con le CNN

Sia le CNN sia le reti convoluzionali per grafi visitano ogni posizione dei dati con gli stessi pesi, ma in una **griglia** (un'immagine) i vicini di un pixel sono **ordinati e in numero fisso**, mentre in un **grafo** sono **non ordinati e in numero variabile**. Per questo sui grafi si limita il campo recettivo ai vicini, si condividono i pesi in modo **invariante per permutazione** e si estende il vicinato con la **stratificazione**.

19.2.4 La formula generale

$$\boxed{\mathbf{h}_v^{(l)} = \mathrm{AGG}_{W^{(l)}}\Big(\mathbf{L}_v,\ \mathbf{h}_v^{(l-1)},\ \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}\Big), \qquad l = 1, \dots, L} \tag{19.1}$$

dove AGG aggrega i messaggi dei vicini con un operatore invariante per permutazione (ad esempio una somma), $W^{(l)}$ sono i parametri liberi, $\mathbf{L}_v$ l'etichetta del nodo e $\mathcal{N}$ è dato dalla matrice di adiacenza. Applicare il calcolo a ogni nodo è una **visita parallela e non ordinata** del grafo. Alcune varianti:

- **GCN**: $\mathbf{h}^{(l)} = \mathrm{ReLU}\big(\hat{A}\,\mathbf{h}^{(l-1)}W^{(l)}\big)$, con $\hat{A}$ matrice di adiacenza normalizzata per grado;
- **GraphConv**: $\mathbf{h}^{(l)} = f\big(W_1^{(l)}\mathbf{h}^{(l-1)} + W_2^{(l)}A\,\mathbf{h}^{(l-1)}\big)$;
- **GAT**: aggregazione basata sull'attenzione.

Le DGN sono intrinsecamente profonde. Il processo non è locale: iterando sugli strati, la rappresentazione di un nodo include progressivamente l'informazione di contesto di nodi sempre più lontani (Figura 19.2). La **profondità serve a far crescere il contesto**.

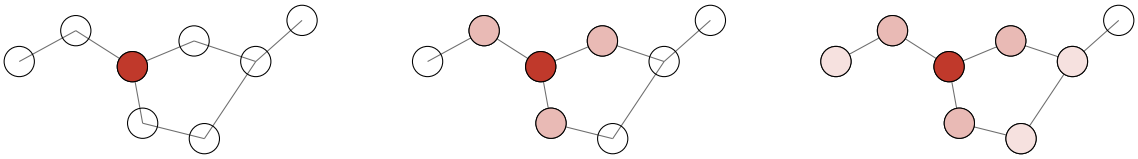


Figura 19.2. Crescita del contesto di un nodo (rosso) con gli strati di message passing: al primo strato lo stato dipende dal nodo stesso, poi dai vicini diretti, poi dai vicini dei vicini.

19.2.5 Approccio a strati: GCN e NN4G

Nelle reti convoluzionali per grafi (GCN, NN4G) **ogni iterazione del message passing è un nuovo strato** (con pesi propri), e il contesto di ogni nodo si **compone** attraverso gli strati: il campo recettivo si estende di strato in strato e addestrando il modello si impara come rappresentare il grafo (end-to-end per le GCN; strato per strato, in modo incrementale, per NN4G). **NN4G** calcola uno stato per ogni strato e ogni nodo:

$$h_v^{(1)} = f\Big(\sum_{j=0}^{|L_v|} \bar{w}_{1j}\, L_j(v)\Big),$$

$$h_v^{(l)} = f\left(\underbrace{\sum_{j=0}^{|L_v|} \bar{w}_{lj}\, L_j(v)}_{\text{etichetta di } v} + \underbrace{\sum_{j=1}^{l-1} \hat{w}_{lj} \sum_{u\in\mathcal{N}(v)} h_u^{(j)}}_{\text{vicini, strati precedenti}}\right), \quad l = 2, \dots, N.$$

Non è ricorsiva: ogni strato usa gli stati degli strati **precedenti**. Le unità si aggiungono una alla volta (in stile Cascade Correlation, § 12.7), con numero di strati automatico e senza vanishing gradient attraverso gli strati, e le uscite dei vari strati si combinano per produrre il risultato sul grafo.

19.2.6 Approccio ricorsivo: GNN e GraphESN

Negli approcci **ricorsivi** la diffusione è simile, ma l conta le **iterazioni sullo stesso strato** (pesi condivisi). È un **sistema dinamico**: si richiede la **convergenza a un punto fisso**, usato come embedding del grafo, e per questo la dinamica del message passing deve essere **contrattiva**. Nella **GNN** l'embedding è addestrato; nella **GraphESN** (estensione delle Echo State Network) la funzione di message passing **non è addestrata**: è un reservoir casuale di cui si controllano i parametri dinamici,

$$\mathbf{h}_v^{(l)} = \tanh\Big(\sum_{j=0}^{|L_v|} \bar{w}_{lj}\, L_j(v) + \sum_{u \in \mathcal{N}(v)} \hat{W}\, \mathbf{h}_u^{(l-1)} \Big),$$

iterata fino a convergenza; l'uscita per un grafo si ottiene con un global pooling X e un **readout lineare** addestrato (pseudoinversa o ridge regression): $y(g) = W_{out}\, X(\mathbf{h}(g))$.

19.3 Problemi aperti

19.3.1 Efficienza

L'addestramento end-to-end con backpropagation è costoso su grafi molto grandi. La GraphESN per la classificazione di nodi ottiene accuratezze pari o superiori a quelle di molti modelli addestrati end-to-end, con un'accelerazione fino a 10 volte, e scala oltre i 100 000 nodi. NN4G+ costruisce la rete **incrementalmente**, addestrando un'unità alla volta: uno strato si espande finché l'accuratezza di validazione migliora e si aggiungono strati finché migliora, ottenendo architetture adatte al task in minuti invece che ore.

19.3.2 I problemi della profondità

- **Over-smoothing**: l'accuratezza tipicamente cala oltre 4–5 strati, perché il message passing agisce come una **diffusione** e le rappresentazioni dei nodi negli strati profondi **collassano**, diventando tutte simili.
- **Over-squashing**: un **collo di bottiglia**. Il campo recettivo cresce **esponenzialmente** con la profondità, ma tutta quell'informazione va compressa in un embedding di dimensione fissa per nodo: la **sensibilità** della rappresentazione di un nodo all'input di nodi lontani, misurata dallo Jacobiano $\partial \mathbf{h}_v^{(L)} / \partial \mathbf{x}_u$, decresce esponenzialmente con il numero di strati.
- Per l'addestramento end-to-end, la difficoltà di **retropropagare il gradiente** attraverso molti strati.

Gli effetti sono l'**under-reaching** (non si ottiene il campo recettivo necessario per apprendere interazioni tra nodi lontani) e la difficoltà con l'**eterofilia**: nei grafi **omofili** i nodi vicini condividono per lo più la stessa classe, in quelli **eterofili** i nodi della stessa classe sono in genere lontani, e predizioni basate sui vicini immediati possono essere fuorvianti (Figura 19.3).

19.3.3 Strumenti di analisi

Modelli come NN4G (incrementale, senza propagare il gradiente attraverso gli strati) e GraphESN (nessun addestramento dell'embedding) permettono di **separare il bias del message passing dai problemi dell'addestramento end-to-end**. Nella GraphESN si controlla direttamente la norma spettrale $\|W\|$, cioè la costante di Lipschitz della funzione di message passing. Poiché

$$\left\| \frac{\partial \mathbf{h}_v^{(L)}}{\partial \mathbf{x}_u} \right\| \le \prod_{l=1}^{L} \|W^{(l)}\| \cdot \left(A^L\right)_{u,v},$$



omofilia alta omofilia bassa (eterofilia)

Figura 19.3. Grafi ad alta e bassa omofilia (il colore è la classe): i secondi sono difficili per il message passing standard.

imporre $\|W\| > 1$ impedisce che la sensibilità rispetto all'input svanisca esponenzialmente (over-squashing).

Un secondo strumento è l'**analisi spettrale**. La "frequenza" di un segnale su un grafo (etichette, embedding) è la sua variabilità tra nodi vicini, misurata dall'**energia di Dirichlet**

$$E(\mathbf{h}) = \sum_{v} \sum_{u \in \mathcal{N}_v} \|\mathbf{h}_v - \mathbf{h}_u\|^2,$$

che dice quanto il segnale varia sul grafo e, tramite la trasformata di Fourier su grafo (ottenuta dalla decomposizione del Laplaciano $L = D - A$, con D matrice diagonale dei gradi), come il segnale si distribuisce sulle frequenze: un segnale liscio ha bassa energia. I grafi eterofili hanno segnali ad **alta frequenza**, e i task eterofili richiedono di **preservare le alte frequenze**. Su un benchmark eterofilo (una rete di pagine Wikipedia), NN4G e GraphESN preservano meglio l'energia attraverso gli strati e mantengono l'accuratezza all'aumentare dei passi di message passing, mentre quella di GCN, GAT e GraphConv cala.

19.3.4 Estensioni

Grafi **dinamici** (relazioni o valori che evolvono nel tempo, trattati ad esempio con reservoir multistrato), **generazione** di grafi (apprendere la distribuzione $p(G)$ per campionarne di nuovi, ad esempio molecole), **spiegabilità** (individuare le sotto-strutture che determinano una predizione), **network quantification** (predire la prevalenza delle classi tra i nodi non etichettati), applicazioni in **bioinformatica**.

19.4 Kernel per strutture

I metodi kernel (§ 13.6) si estendono facilmente a sequenze, alberi e grafi, perché basta definire un prodotto scalare $k(x, x') = \langle \phi(x), \phi(x') \rangle$, cioè una **similarità** tra oggetti di qualsiasi tipo, con un embedding implicito in uno spazio euclideo.

- **Kernel marginalizzati**: la similarità tra due grafi si basa sui cammini di etichette comuni ottenuti con random walk; il kernel è il prodotto scalare dei vettori di conteggio, mediato su tutti i cammini (costo tipicamente quadratico nella dimensione del grafo).
- **Kernel di convoluzione**: si decompongono gli oggetti in **sotto-strutture** e si definisce il kernel combinando kernel tra le sotto-strutture.

Attenzione — Limiti dei kernel per strutture.

- **Efficienza**: la scomposizione in sotto-strutture rende il costo critico.
- **Adattività**: la similarità è **fissata prima dell'apprendimento**, quindi la codifica non è appresa (con il vantaggio di poter usare la conoscenza a priori sul problema).

- Per i domini strutturati non si conosce l'**universalità**: la scelta del kernel è critica, e calcolare un kernel **completo** per grafi è difficile almeno quanto il problema dell'**isomorfismo di grafi**.

Conclusione. Se i dati hanno relazioni (seriali, gerarchiche, reti), non conviene usare una rappresentazione piatta. Il deep learning su grafi si può fare anche **in modo efficiente**, senza un addestramento completamente end-to-end; la profondità nelle DGN è necessaria (fa crescere il contesto) ma causa problemi (over-smoothing, over-squashing, difficoltà di addestramento) che sono oggetto di ricerca.

Punti chiave

- Trasduzioni su grafi: livello di nodo, di grafo, di arco.
- Message passing: messaggio, aggregazione invariante per permutazione, aggiornamento; $\mathbf{h}_v^{(l)} = \mathrm{AGG}_{W^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)}\}_{u \in \mathcal{N}(v)})$.
- A strati (GCN, NN4G: ogni iterazione è uno strato) o ricorsivo (GNN, GraphESN: iterazioni contrattive fino al punto fisso).
- Over-smoothing, over-squashing, under-reaching, eterofilia.
- Kernel per strutture: marginalizzati, di convoluzione; similarità fissata a priori.

Domande di verifica

1. Perché trattare i dati strutturati direttamente invece di trasformarli in vettori?
2. Quali trasduzioni si possono definire sui grafi?
3. Descrivere il message passing e la formula generale delle DGN.
4. Differenza tra approcci a strati (GCN, NN4G) e ricorsivi (GNN, GraphESN).
5. Cosa sono over-smoothing e over-squashing? Cos'è l'eterofilia?
6. Pro e contro dei kernel per strutture.